Causal Econometrics with Julia

Xiang Ao

2026-10-16

Table of contents

Preface **1**
How to cite . . . 2

I. Identification **3**

1. Identification & Potential Outcomes **5**
1.1. Why Do We Need Tools for Causal Inference? . . . 5
1.2. Confounders . . . 5
1.3. Controlled and Uncontrolled Confounders . . . 7
1.4. Potential Outcome Setup . . . 7
1.5. ATE, ATT and ATU . . . 7
1.6. Identification: From Potential Outcome to Observed . . . 8
1.7. Unconfoundedness and Overlap . . . 9
1.8. Observational Data Flowchart . . . 9
1.9. Summary . . . 10

2. Five Estimands on One DGP **11**
2.1. The data-generating process . . . 11
2.2. The five estimands . . . 12
2.3. When the estimands differ . . . 15
2.4. Which estimand should you choose? . . . 16
2.5. Summary . . . 16

3. DAGs, ADMGs, Identification, and Estimation **17**
3.1. DAGs and ADMGs . . . 17
3.2. A DAG: Backdoor Adjustment . . . 18
3.3. Estimating the DAG Effect with CausalEstimate . . . 18
3.4. An ADMG: Front-Door Identification . . . 20
3.5. Estimating the ADMG Effect with CausalGraphs . . . 20
3.6. General ID Algorithm . . . 22
3.7. When the Graph Says No . . . 23
3.8. Practical Workflow . . . 24
3.9. Summary . . . 24

4. Applied Graph Workflow: Smoking Cessation **25**
4.1. Data . . . 25
4.2. Adjustment DAG . . . 26
4.3. Identification . . . 27
4.4. Estimation with CausalEstimate . . . 28
4.5. Sensitivity Graph . . . 29

4.6. Structured Baseline Graph . . . 31
4.7. Summary . . . 32

5. Sensitivity Analysis **33**
5.1. A simulated example . . . 33
5.2. Cinelli-Hazlett robustness value . . . 34
5.3. E-values . . . 38
5.4. Rosenbaum bounds for matched studies . . . 39
5.5. Reporting practice . . . 41
5.6. Summary . . . 41

II. Estimation **43**

6. Estimation: RA, IPW, AIPW and IPWRA **45**
6.1. Experimental Data . . . 45
6.2. Adjustment Formula . . . 45
6.3. Regression Adjustment . . . 46
6.4. Linear RA . . . 46
6.5. IPW . . . 48
6.6. Doubly Robust Estimators . . . 49
6.7. Summary . . . 51

7. Matching Estimators **53**
7.1. Assumptions . . . 53
7.2. A simulated example . . . 53
7.3. Estimating the propensity score . . . 54
7.4. Nearest-neighbour 1:1 matching . . . 54
7.5. Balance diagnostics . . . 55
7.6. Estimating the ATT on matched data . . . 56
7.7. Matching with replacement . . . 56
7.8. Matching vs weighting . . . 57
7.9. Entropy balancing . . . 57
7.10. Matching, weighting, and direct estimation . . . 59
7.11. When to use which method . . . 62
7.12. Summary . . . 62

8. Nonparametric Causal Methods **63**
8.1. Why Do We Need Nonparametric Methods? . . . 63
8.2. Influence Function Based Estimators . . . 64
8.3. TMLE . . . 71
8.4. DoubleML . . . 72
8.5. Summary . . . 73

9. Heterogeneous Treatment Effects with Machine Learning **75**
9.1. The data-generating process . . . 75
9.2. A wrapper for fitting random forests . . . 76
9.3. S-learner (“single”) . . . 78
9.4. T-learner (“two”) . . . 78

9.5. X-learner (Künzel et al. 2019) . . . 79
9.6. R-learner (Nie and Wager 2021) . . . 79
9.7. DR-learner (Kennedy 2020) . . . 80
9.8. Comparing all five estimators . . . 80
9.9. Best Linear Projection (BLP) . . . 81
9.10. GATES: sorted group ATEs . . . 82
9.11. Variable importance — which covariate drives heterogeneity? . . . 84
9.12. Policy learning: who should we treat? . . . 85
9.13. Causal forests with panel data . . . 86
9.14. Summary . . . 88

10. Continuous and Multivalued Treatments **89**
10.1. Setup . . . 89
10.2. Naive regression vs adjusted regression . . . 90
10.3. Generalized Propensity Score (Hirano-Imbens 2004) . . . 91
10.4. Doubly-robust dose-response estimation . . . 93
10.5. Multivalued treatments . . . 95
10.6. When to reach for these methods . . . 96
10.7. Summary . . . 97

11. Bayesian Causal Inference **99**
11.1. Setup . . . 99
11.2. Bayesian g-computation . . . 100
11.3. Why use BART/BCF in R instead? . . . 102
11.4. When to use Bayesian methods . . . 103
11.5. Connections . . . 103
11.6. Summary . . . 104

12. Distributional Treatment Effects **105**
12.1. Beyond the average . . . 105
12.2. Identifying counterfactual distributions . . . 107
12.3. Distributional DiD with Engression . . . 108
12.4. When QTE differs from ATT: a panel simulation . . . 109
12.5. Beyond QTE: posterior bands as distributional output . . . 112
12.6. Summary . . . 112

III. Designs **113**

13. Difference-in-Differences **115**
13.1. $T = 2$ Panel Data . . . 115
13.2. Parallel Trends Assumption . . . 116
13.3. No Anticipation Assumption . . . 116
13.4. DiD is Identified with PT and NA . . . 116
13.5. Advantages and Disadvantages of DiD . . . 117
13.6. Traditional DiD . . . 117
13.7. TWFE with Staggered Treatment Timing . . . 117
13.8. Goodman-Bacon Decomposition . . . 118

13.9. Wooldridge's ETWFE . . . 119
13.10Example 1: US Teen Employment . . . 120
13.11Example 2: Staggered DiD . . . 123
13.12Nonlinear ETWFE: Count and Binary Outcomes . . . 126
13.13Spatial Interference . . . 131
13.14Persistent Outcomes and Heterogeneous Dynamics . . . 132
13.15Synthetic Control and Synthetic DiD . . . 133
13.16Summary . . . 133

14. DiD with Continuous Treatment **135**
14.1. Two periods . . . 135
14.2. Staggered adoption . . . 137
14.3. Wooldridge's regression and the CGBS estimator . . . 139
14.4. Walmart and county retail employment . . . 140
14.5. Summary . . . 146

15. Synthetic Control, DiD, and TASC **147**
15.1. Synthetic Control . . . 147
15.2. Synthetic DiD . . . 148
15.3. Time-Aware Synthetic Control . . . 148
15.4. Example: California Proposition 99 . . . 149
15.5. Summary . . . 152

16. Randomization Inference for Synthetic Control **153**
16.1. The placebo test . . . 153
16.2. The MSPE ratio test . . . 156
16.3. Time placebo . . . 158
16.4. TASC: a model-based band . . . 159
16.5. How the two inference strategies compare . . . 160
16.6. Summary . . . 161

17. IV & Regression Discontinuity **163**
17.1. Instrumental Variables . . . 163
17.2. When 2SLS with Covariates Is *Actually* LATE . . . 169
17.3. Regression Discontinuity Design . . . 175
17.4. Summary . . . 182

18. Shift-Share Instrumental Variables **183**
18.1. The construction . . . 184
18.2. Two identification views . . . 184
18.3. Inference . . . 185
18.4. Simulation: build intuition . . . 185
18.5. BHJ shock-level inference . . . 189
18.6. Empirical: the China shock (Autor, Dorn, Hanson 2013) . . . 190
18.7. Summary . . . 191

19. Recentered Formula Instruments **193**
19.1. Formula instruments . . . 193
19.2. Shift-share as a formula instrument . . . 195

19.3. Simulation . . . 196
19.4. The China shock . . . 199
19.5. Spillovers: deworming . . . 204
19.6. Summary . . . 207

20. IV in Poisson with Fixed Effects **209**
20.1. The model and setup . . . 209
20.2. Control functions and GMM . . . 211
20.3. Estimators for endogenous regressors . . . 212
20.4. Why recommend generalized residuals over exact Terza . . . 217
20.5. The four estimators compared . . . 219
20.6. Practical guidance: an observable diagnostic workflow . . . 219
20.7. Implementation in Julia and Stata . . . 221
20.8. Summary . . . 224

IV. Longitudinal Causal Inference **225**

21. G-Methods for Time-Varying Treatments **227**
21.1. The time-varying confounding problem . . . 227
21.2. Parametric g-formula . . . 228
21.3. IPTW for time-varying treatments . . . 230
21.4. Marginal structural models . . . 231
21.5. Hernán-style g-estimation: when to use which . . . 231
21.6. A robustness check: g-formula with covariates only . . . 232
21.7. When to reach for these methods . . . 232
21.8. Summary . . . 233

V. Survival & Time-to-Event **235**

22. Survival Analysis with Causal Inference **237**
22.1. The censoring problem . . . 237
22.2. Simulated survival data . . . 238
22.3. Kaplan-Meier estimator . . . 238
22.4. IPW-adjusted survival . . . 239
22.5. Restricted Mean Survival Time . . . 241
22.6. Cox proportional hazards (regression adjustment) . . . 242
22.7. Doubly-robust survival estimation . . . 242
22.8. When to use these methods . . . 242
22.9. Summary . . . 243

VI. Mediation **245**

23. Causal Mediation Analysis **247**
23.1. Classical Mediation . . . 248
23.2. Causal Mediation . . . 249
23.3. NIE and NDE: Natural Direct and Indirect Effects . . . 254

23.4. IIE and IDE: Interventional Direct and Indirect Effects . . . 256
23.5. Summary . . . 259

VII. Causal Discovery **261**

24. Causal Discovery: Observed Variables **263**
24.1. What can be recovered from the data? . . . 263
24.2. Simulation . . . 264
24.3. CI Test Infrastructure . . . 265
24.4. PC Algorithm . . . 266
24.5. RSL-D Algorithm . . . 267
24.6. Comparison . . . 268
24.7. Monte Carlo Evaluation . . . 270
24.8. Real Data: Student Performance in PISA . . . 272
24.9. Summary . . . 279

25. Causal Discovery: Latent Variables **281**
25.1. Maximal Ancestral Graphs and PAGs . . . 281
25.2. Simulation with Hidden Variables . . . 282
25.3. FCI Algorithm . . . 283
25.4. L-MARVEL Algorithm . . . 286
25.5. Comparison . . . 287
25.6. Monte Carlo Evaluation . . . 288
25.7. Summary . . . 291
25.8. How Much Does Causal Discovery Help in Economics? . . . 292

26. From Graph to Estimate **295**
26.1. Backdoor: a-fixable effects . . . 295
26.2. Front-door: p-fixable effects . . . 296
26.3. When identification fails . . . 298
26.4. End-to-end: discover, identify, estimate . . . 299
26.5. What changes after identification . . . 299
26.6. Summary . . . 300

VIIIAppendix **301**

27. Julia Packages Used in This Book **303**
27.1. Working With The Environment . . . 303
27.2. Package Map . . . 303
27.3. Panelest.jl . . . 305
27.4. CausalEstimate.jl . . . 306
27.5. CausalGraphs.jl . . . 307
27.6. ETWFE in Panelest . . . 308
27.7. SynthDiD.jl . . . 308
27.8. TASC.jl . . . 309
27.9. RDRobust.jl . . . 309
27.10Lavaan.jl . . . 309

27.11Crumble.jl 310
27.12Practical Advice 310

References **313**

Preface

Most applied questions are causal. Did a program raise earnings? Did a drug reduce mortality? Did a policy change behavior? With experimental data, a difference in means can sometimes answer the question. With observational data, it almost never can. Causal econometrics is the business of saying what would have to be true for a number we can compute to mean the causal object we want.

This book is a working guide in Julia. It is for applied researchers who already know regression and probability, and who want to see the methods carried out end to end. Each chapter gives the identifying assumptions and then runs code on a real or simulated data set. Where it matters, the influence functions, score equations, and weighting schemes are written out so the link between the formula and the Julia call is visible.

Julia is not the usual choice for this material. R remains the default language of applied causal inference, and a companion R volume — Introduction to Causal Econometrics with Observational Data — covers the same ground. A messier working notebook, Topics on Econometrics and Causal Inference, holds the rougher posts that fed into both books. The Julia version exists because some of the heavier estimators — TMLE on large samples, distributional difference-in-differences, fully Bayesian g-computation — are uncomfortably slow in R, and because Julia's type system makes it possible to write small, focused estimation packages whose code is easy to read. Several such packages were written alongside this book and are used throughout: `CausalEstimate.jl` for unified TMLE and AIPW, `CausalGraphs.jl` for graph-based identification, `Lavaan.jl` for structural equation modeling, `Crumble.jl` for causal mediation, and a handful of smaller libraries for difference-in-differences, synthetic control, shift-share IV, and regression discontinuity. They are not requirements for following the text, but readers who want to see how an estimator is actually built will find the source short enough to read.

The book is organized in the order in which an applied project usually runs. Part I is identification. Part II is estimation. Part III is designs: difference-in-differences, synthetic control, instrumental variables, regression discontinuity, and shift-share IV. The remaining parts cover longitudinal settings, survival outcomes, mediation, and causal discovery. The appendix lists the Julia packages used in the examples.

The book is meant to be read at a desk with Julia running. Source files and datasets are available in the repository linked above. `Project.toml` lists the packages and `Manifest.toml` records the versions they resolve to. Several are the author's own unregistered packages, installed from local copies; the README lists them and explains how to set them up. The "Edit this page" link at the foot of each chapter goes directly to the corresponding `.qmd` file. Corrections and suggestions are welcome through the issues tracker.

A note on the code you see. Hidden setup chunks (package loading, helper definitions) execute when the book is rendered but are not shown. Some visible code blocks are marked `eval: false`: these are displayed for readability but not run, either because they only show which packages a chapter

uses or because they require a tool or package outside the book's default environment (these are flagged where they appear). A block marked `eval: false` therefore shows intended usage rather than executed output.

How to cite

Ao, Xiang. *Causal Econometrics with Julia*. https://xiangao.github.io/causal_econometrics_julia/.

```
@book{ao_causal_econometrics_julia,
  author = {Ao, Xiang},
  title  = {Causal Econometrics with Julia},
  url    = {https://xiangao.github.io/causal_econometrics_julia/}
}
```

Part I.

Identification

1. Identification & Potential Outcomes

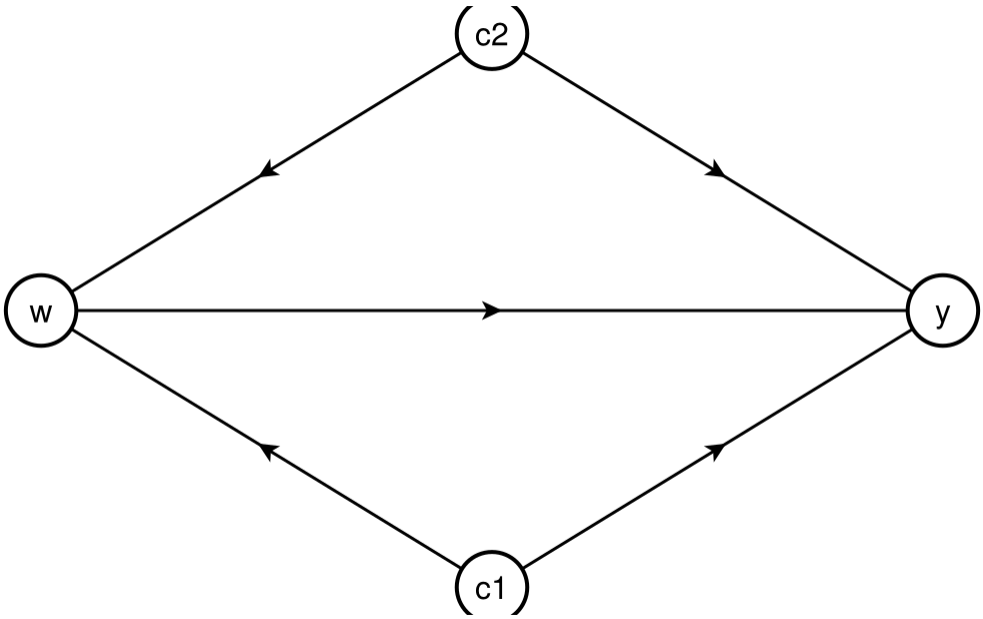
```
using CairoMakie
CairoMakie.activate!(type = "png")
using GraphMakie
using Graphs
using GeometryBasics
```

1.1. Why Do We Need Tools for Causal Inference?

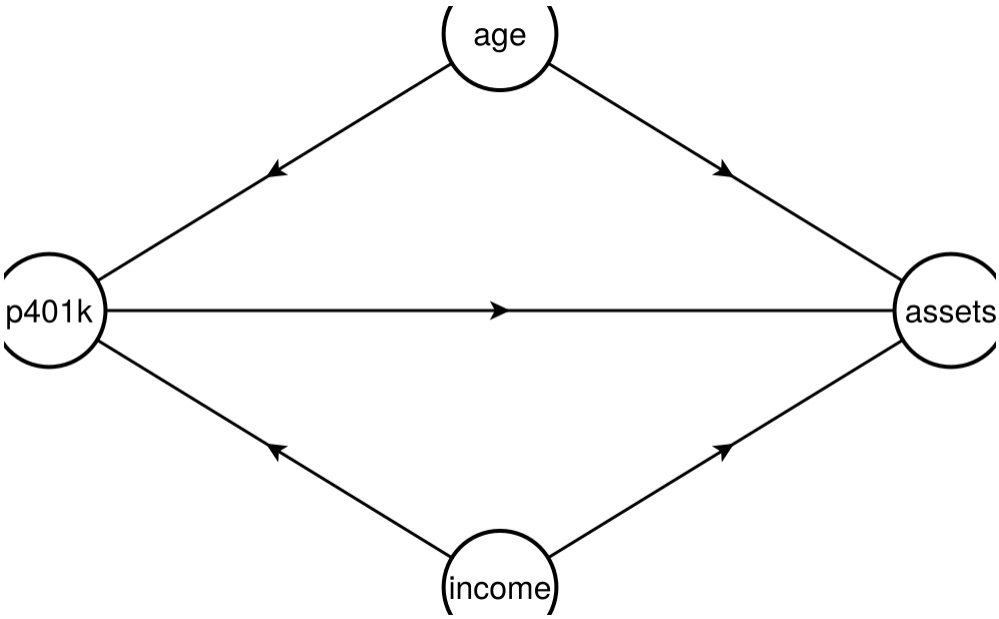
Association is not causation. We have known how to measure associations for a long time, but most econometric models are associational. The gap between the two arises from confounding.

1.2. Confounders

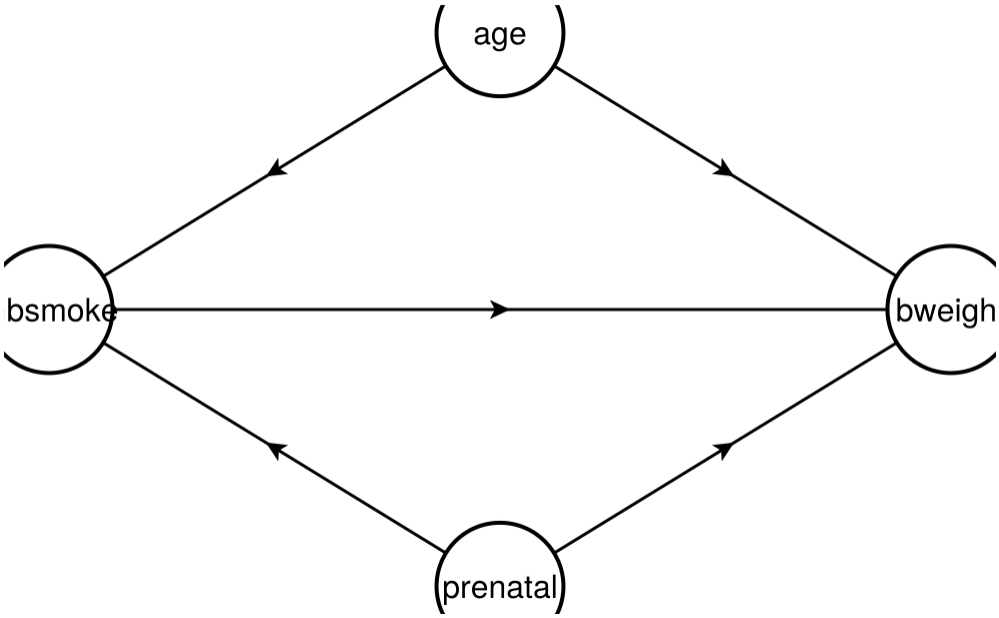
In this graph, w is the treatment, y the outcome, and c_1, c_2 are confounders. Both confounders affect both w and y, so the observed association between w and y may reflect confounding rather than a causal effect.



A concrete example: income and age may both drive 401(k) participation and assets. To identify the causal effect of participation on assets, we need to control for income and age.



Another example: prenatal care and maternal age may affect both smoking and birth weight. Even if smoking has no causal effect on birth weight, it will appear to because of these shared causes.



1.3. Controlled and Uncontrolled Confounders

If we can control for all confounders, correlation does imply causation. The two questions are: do we observe all confounders, and how do we control for them? In a randomized experiment neither question arises. In an observational study both do. If c_2 is unobserved, any estimate of the effect of w on y is biased.

1.4. Potential Outcome Setup

We study causal effects through the potential outcomes (counterfactual) framework. Each unit has two potential outcomes, $Y(1)$ and $Y(0)$, one for each treatment state. These are fixed before any assignment; the experiment tries to recover them.

The fundamental problem is that we never observe both: only the realized outcome $Y = WY(1) + (1 - W)Y(0)$ is seen. Individual treatment effects $\tau_i = Y_i(1) - Y_i(0)$ are not identifiable; only averages can be estimated.

We write W for treatment assignment and X for covariates.

1.5. ATE, ATT and ATU

The standard estimands are:

$$\tau_{ATE} = E[Y(1) - Y(0)] \tag{1.1}$$

$$\tau_{ATT} = E[Y(1) - Y(0)|W = 1] \tag{1.2}$$

$$\tau_{ATU} = E[Y(1) - Y(0)|W = 0] \tag{1.3}$$

$$\tau_{ATE} = \rho\tau_{ATT} + (1 - \rho)\tau_{ATU} \tag{1.4}$$

where $\rho = P(W = 1)$.

1.6. Identification: From Potential Outcome to Observed

We focus on τ_{ATT}. To identify it, we need:

$$E[Y(1)|W=1] \tag{1.5}$$

which is easy, because we observe it.

And we need

$$E[Y(0)|W=1] \tag{1.6}$$

To identify this, we'll need unconfoundedness:

$$E[Y(0)|W,X] = E[Y(0)|X] \tag{1.7}$$

and overlap:

$$\pi(x) = P(W=1|X=x) < 1 \tag{1.8}$$

(for the ATT only $\pi(x) < 1$ is needed; the ATE requires $0 < \pi(x) < 1$).

Unconfoundedness says that within each subgroup $X = x$, treated and control units have the same mean of $Y(0)$: $E[Y(0)|W=1, X=x] = E[Y(0)|W=0, X=x] = E[Y|W=0, X=x]$. Averaging over the covariate distribution of the treated,

$$E[Y(0)|W=1] = E\big[E[Y|W=0,X] \mid W=1\big], \tag{1.9}$$

which involves only observed data. Therefore ATT is identified:

$$\begin{aligned} \tau_{ATT} &= E[Y(1) - Y(0)|W=1] \\ &= E[Y|W=1] - E\big[E[Y|W=0,X] \mid W=1\big] \end{aligned} \tag{1.10}$$

Note the conditional-on-X assumption does NOT imply the marginal equality $E[Y(0)|W=1] = E[Y(0)|W=0]$: treated and control units have different covariate distributions, so the adjustment over X is essential. In a randomised experiment, unconfoundedness holds without conditioning on X (i.e. $E[Y(0)|W] = E[Y(0)]$); then $E[Y(0)|W=1] = E[Y(0)|W=0] = E[Y|W=0]$, the formula collapses to $E[Y|W=1] - E[Y|W=0]$, and a difference-in-means estimator is consistent.

$$\hat{\tau}_{ATT} = \bar{Y}_1 - \bar{Y}_0 \tag{1.11}$$

But in observational studies, most cases we cannot assume unconfoundedness. What can we do to justify unconfoundedness better?

1.7. Unconfoundedness and Overlap

Weaker Assumptions: Unconfoundedness (conditional independence, or ignorability), stated jointly for both potential outcomes since we are after the ATE here (as opposed to the ATT case above, which only required it for $Y(0)$):

$$E[Y(d)|W, X] = E[Y(d)|X] \quad \text{for } d \in \{0, 1\} \tag{1.12}$$

Overlap:

$$0 < \pi(x) = P(W = 1|X = x) < 1 \tag{1.13}$$

SUTVA (stable unit treatment value assumption): 1. No interference between units. No spillovers. 2. Consistency. No hidden versions of treatment or control. Treatment is clearly defined.

Within each subgroup of $X = x$, W is independent of $Y(0)$. With the overlap assumption, we ensure there is a comparable control unit for each treated unit.

However, these two criteria are often contradictory: we may need more covariates to satisfy conditional independence; meanwhile, if the dimension of covariates goes high, then it's likely in some partition we don't have control units.

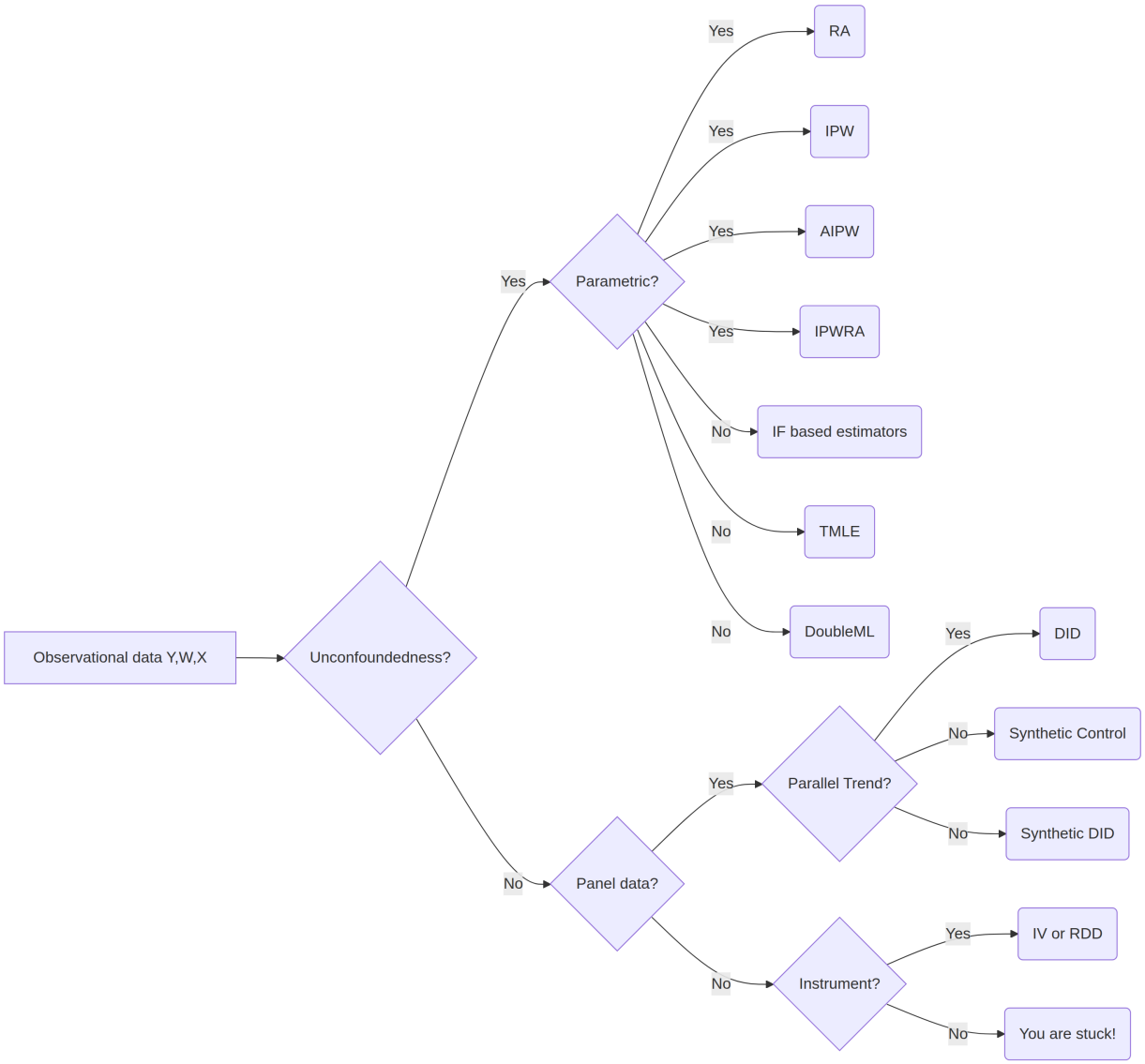
We need four assumptions for identification:

- Unconfoundedness (conditional independence, or ignorability)
- Overlap (positivity, common support)
- No interference
- Consistency

The last two together are SUTVA, the stable unit treatment value assumption (Rubin 1980).

1.8. Observational Data Flowchart

In observational studies, I summarise the decision what method to use for identification of causal effect:



1.9. Summary

A causal effect compares two potential outcomes, $Y(1)$ and $Y(0)$, and we only ever see one of them. So we target averages, the ATE, ATT and ATU, and identification means writing them in terms of observed data. Under unconfoundedness given the covariates X and overlap, the ATT equals the treated mean minus the control mean adjusted to the covariate distribution of the treated. Two more conditions, no interference and consistency, make the observed outcome equal to the right potential outcome.

When unconfoundedness is not credible, other designs replace it with their own assumptions: parallel trends for DiD, a factor structure for synthetic control, continuity at a cutoff for regression discontinuity, and a relevant and excludable instrument for IV. The flowchart maps these choices, and the rest of the book develops each one.

2. Five Estimands on One DGP

```
using DataFrames
using Distributions
using Random
using Statistics
using CairoMakie
CairoMakie.activate!(type = "png")
using GLM
```

Before choosing an estimator, we need to know the estimand. ATE, ATT, LATE, CATE and QTE are not five ways to estimate the same object. They are five different objects.

Here I use one simulated data set so we can calculate all of them. The simulation is useful because we observe both potential outcomes. In real data we do not, which is exactly why identification assumptions matter.

2.1. The data-generating process

Each person has an observed covariate X_i, an unobserved type U_i, and a binary treatment D_i. The instrument Z_i is a random offer. It changes the probability of treatment, but it has no direct effect on the outcome.

```
Random.seed!(42)
n = 20_000

X = rand(Uniform(0, 1), n)          # observed covariate, in [0,1]
U = rand(Normal(0, 1), n)           # latent type, unobserved
Z = rand(Bernoulli(0.5), n)         # random instrument

# Treatment assignment: depends on Z (the instrument) and U (selection on type)
# Higher U → more likely to take treatment regardless of Z (always-takers)
# Lower  U → less likely to take treatment regardless of Z (never-takers)
# Middle U → responsive to Z (compliers)
pD0 = @. clamp(0.10 + 0.20 * U, 0, 1)       # P(D=1 | Z=0, U)
pD1 = @. clamp(0.10 + 0.20 * U + 0.50, 0, 1)  # P(D=1 | Z=1, U) - adds 0.50

# ONE latent draw per unit, two thresholds: since pD1 >= pD0, this gives
# D1 >= D0 for every unit - monotonicity (no defiers) holds EXACTLY.
```

```
# (Drawing D0 and D1 with independent rand() calls would create defiers
#  by construction, even though pD1 > pD0.)
V  = rand(n)
D0 = V .< pD0   # potential treatment if not offered slot
D1 = V .< pD1   # potential treatment if offered slot
D  = ifelse.(Z .== 1, D1, D0)

# Heterogeneous treatment effect: depends on X
# Y(d) = baseline + d * tau(X) + 0.3 * U + noise
 (x) = 1.0 + 2.0 * x            # the true individual treatment effect function
Y0   = 0.5 .* X .+ 0.3 .* U .+ randn(n)
Y1   = Y0 .+  .(X)
Y    = ifelse.(D, Y1, Y0)

df = DataFrame(X=X, Z=Z, D=D, Y=Y)
first(df, 6)
```

	X	Z	D	Y
	Float64	Bool	Bool	Float64
1	0.173575	1	1	-1.64247
2	0.321662	1	1	1.29054
3	0.258585	1	1	2.52079
4	0.166439	1	1	-0.00756264
5	0.527015	1	1	4.47846
6	0.483022	0	0	1.91673

In the simulated data we have `Y0` and `Y1` for every observation. That lets us calculate the true estimands directly. With real data, each observation only reveals one potential outcome.

2.2. The five estimands

2.2.1. ATE — average treatment effect

$$\text{ATE} = \mathbb{E}[Y(1) - Y(0)] \tag{2.1}$$

ATE is the mean effect in the whole population. It compares the mean outcome if everyone were treated with the mean outcome if nobody were treated.

```
ate_true = mean(Y1 .- Y0)
@printf("ATE (population mean of Y1 - Y0) = %.3f\n", ate_true)

# What the integral over  (X) should be:  (1 + 2x) dx on [0,1] = 1 + 1 = 2
@printf("Theoretical ATE =  (1 + 2x)dx on [0,1] = 2.000\n")
```

```
ATE (population mean of Y1 - Y0) = 2.002
Theoretical ATE =  (1 + 2x)dx on [0,1] = 2.000
```

2.2.2. ATT — average treatment effect on the treated

$$\text{ATT} = \mathbb{E}[Y(1) - Y(0) \mid D = 1] \tag{2.2}$$

ATT is the mean effect for the units that actually took treatment. It differs from ATE when treatment selection is related to treatment-effect heterogeneity.

```
att_true = mean(Y1[D] .- Y0[D])
@printf("ATT (mean of Y1-Y0 conditional on D=1) = %.3f\n", att_true)
```

```
ATT (mean of Y1-Y0 conditional on D=1) = 2.006
```

Here ATT is close to ATE – and in fact the two are analytically identical in this DGP, exactly 2.0 in the population; the estimates above differ only by finite-sample simulation noise, not by any true gap between the estimands. The reason is mechanical: the individual treatment effect $\tau_i = 1 + 2X_i$ depends only on X_i, while selection into treatment depends on U_i (and the instrument Z_i below), and X_i is generated independently of U_i and Z_i. Since treatment-effect heterogeneity is unrelated to who selects into treatment or who complies with the instrument, averaging τ_i over the treated subpopulation or over compliers gives the same population average as over everyone – hence ATE = ATT = LATE = 2.0 exactly here. If selection (or compliance) also depended on X, these estimands would diverge, since ATE, ATT, and LATE are different averages of τ_i across different, non-identical subpopulations.

2.2.3. LATE — local average treatment effect (Imbens-Angrist)

$$\text{LATE} = \mathbb{E}[Y(1) - Y(0) \mid D(1) > D(0)] \tag{2.3}$$

LATE is the mean effect for compliers, the units whose treatment status is changed by the instrument. Under the usual IV assumptions, the Wald estimator identifies this effect, not the ATE. Identifying LATE by the Wald ratio additionally requires monotonicity (no defiers): $D(1) \geq D(0)$ for all i. This DGP imposes it by generating both potential treatments from one shared latent draw V_i with ordered thresholds ($D(z) = \mathbb{1}\{V_i < p_{Dz}\}$ and $p_{D1} \geq p_{D0}$), so no unit is pushed out of treatment by the offer. Raising the *probability* alone would not be enough: with independent draws for $D(0)$ and $D(1)$, about 3% of units would be defiers and the Wald ratio would no longer equal the complier mean.

```
compliers = (D1 .== 1) .& (D0 .== 0)
late_true = mean(Y1[compliers] .- Y0[compliers])
@printf("LATE (mean of Y1-Y0 conditional on complier status) = %.3f\n", late_true)
@printf("Share of compliers: %.3f\n", mean(compliers))

# Wald estimator from the data alone
wald = (mean(Y[Z .== 1]) - mean(Y[Z .== 0])) /
       (mean(D[Z .== 1]) - mean(D[Z .== 0]))
@printf("Wald IV estimate (should match LATE): %.3f\n", wald)
```

```
LATE (mean of Y1-Y0 conditional on complier status) = 2.004
Share of compliers: 0.460
Wald IV estimate (should match LATE): 2.004
```

With heterogeneous effects, IV averages over the people moved by the instrument. In this example the Wald estimate matches the complier mean because Z changes treatment only for that group.

2.2.4. CATE — conditional average treatment effect

$$\mathrm{CATE}(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x] \tag{2.4}$$

CATE keeps X in the estimand. In this DGP, $\mathrm{CATE}(x)$ is simply $\tau(x) = 1 + 2x$.

```
# Bin X and compute mean of (Y1 - Y0) in each bin
nbins = 20
edges = range(0, 1, length=nbins + 1)
centers = (edges[1:end-1] .+ edges[2:end]) ./ 2

cate_est = [mean((Y1 .- Y0)[(X .>= edges[k]) .& (X .< edges[k+1])])
            for k in 1:nbins]

# Label by the actual bin centers (bin 5 = [0.20,0.25) has center 0.225)
@printf("CATE at x=%.3f: estimated %.2f, true %.2f\n",
        centers[5],  cate_est[5],   (centers[5]))
@printf("CATE at x=%.3f: estimated %.2f, true %.2f\n",
        centers[17], cate_est[17],  (centers[17]))
```

```
CATE at x=0.225: estimated 1.45, true 1.45
CATE at x=0.825: estimated 2.65, true 2.65
```

2.2.5. QTE — quantile treatment effect

$$\mathrm{QTE}(q) = F^{-1}_{Y(1)}(q) - F^{-1}_{Y(0)}(q) \tag{2.5}$$

(We index quantiles by q to avoid a clash with the treatment-effect function $\tau(x)$.)

QTE compares the two marginal outcome distributions. It is the difference between a quantile under treatment and the same quantile under control. It does not follow the same person across the two potential outcomes.

```
qte_grid = 0.05:0.05:0.95
qte_est  = [quantile(Y1, q) - quantile(Y0, q) for q in qte_grid]

@printf("QTE(0.10) = %.3f\n", quantile(Y1, 0.10) - quantile(Y0, 0.10))
@printf("QTE(0.50) = %.3f\n", quantile(Y1, 0.50) - quantile(Y0, 0.50))
@printf("QTE(0.90) = %.3f\n", quantile(Y1, 0.90) - quantile(Y0, 0.90))
```

```
QTE(0.10) = 1.704
QTE(0.50) = 1.992
QTE(0.90) = 2.289
```

2.3. When the estimands differ

In the first DGP the scalar estimands are close because:

- X is uniform on [0, 1] (so ATE = average of over uniform $X = 2$)
- Treatment selection is on U (unobserved type), not X (which drives), so ATT ATE
- The instrument shifts compliers uniformly across X, so LATE ATE

If treatment selection depends on the effect modifier, the numbers separate. Here I make high-X units more likely to take treatment:

```
Random.seed!(7)
n = 20_000
X2 = rand(Uniform(0, 1), n)
U2 = rand(Normal(0, 1), n)
Z2 = rand(Bernoulli(0.5), n)
# High X → much more likely to take treatment (selection on X, the modifier)
pD0_2 = @. clamp(0.10 + 0.20 * U2 + 0.6 * X2, 0, 1)
pD1_2 = @. clamp(pD0_2 + 0.50, 0, 1)
V2    = rand(n)            # shared draw: monotonicity exact, as above
D0_2  = V2 .< pD0_2
D1_2  = V2 .< pD1_2
D2    = ifelse.(Z2 .== 1, D1_2, D0_2)
Y0_2  = 0.5 .* X2 .+ 0.3 .* U2 .+ randn(n)
Y1_2  = Y0_2 .+  .(X2)
Y2    = ifelse.(D2, Y1_2, Y0_2)

ate2  = mean(Y1_2 .- Y0_2)
att2  = mean(Y1_2[D2] .- Y0_2[D2])
late2 = mean((Y1_2 .- Y0_2)[(D1_2 .== 1) .& (D0_2 .== 0)])

@printf("Selection-on-X DGP:\n")
@printf("  ATE  = %.3f\n", ate2)
@printf("  ATT  = %.3f (now higher because treated have higher X → larger  )\n", att2)
@printf("  LATE = %.3f (average  (X) among compliers)\n", late2)
```

```
Selection-on-X DGP:
  ATE  = 1.999
  ATT  = 2.126 (now higher because treated have higher X → larger  )
  LATE = 1.917 (average  (X) among compliers)
```

Now ATT is larger than ATE. The treated group has higher X, and high X means a larger treatment effect. In this case reporting ATE or ATT changes the substantive answer.

2.4. Which estimand should you choose?

The estimand should come from the research question:

Policy question	Right estimand
"What if we treated everyone?"	ATE
"What did treating the currently-treated achieve?"	ATT
"What can the instrument tell us?" (e.g. policy expansion that's already in place)	LATE
"Who benefits most?"	CATE(x)
"Does the effect vary across the outcome distribution?"	QTE(q)
"What is the distribution of individual effects?"	Often unidentifiable; bounds required

It is fine for one paper to report more than one estimand, as long as each is named correctly. What is not fine is to report the coefficient that is easiest to estimate and call it "the causal effect."

2.5. Summary

- ATE, ATT and LATE are different averages of the individual treatment effect.
- CATE(x) keeps heterogeneity by covariates. QTE(q) describes changes in the outcome distribution.
- With heterogeneous effects, IV estimates LATE. It should not be described as ATE unless extra assumptions justify that interpretation. This is the clean case with no covariates. Once we add covariates linearly to 2SLS, even LATE needs a *rich covariates* condition that is rarely defended. See Blandhol et al. (2025) and the IV chapter section "When 2SLS with Covariates Is *Actually* LATE".
- Estimator choice comes after the estimand is fixed.

3. DAGs, ADMGs, Identification, and Estimation

```
using CausalGraphs
using CausalEstimate
import CausalGraphs: identify, make_graph, to_mermaid, markov_pillow, is_fix, is_p_fix, ID_alg
import CausalEstimate: estimate, confint, pvalue
using DataFrames
using Random
using Statistics
using Logging
```

The potential-outcomes chapter asked when an observed data set can tell us about $E[Y(1) - Y(0)]$. Graphs give one way to state the assumptions behind the answer. A graph records which variables are allowed to cause other variables, and whether some observed variables may share unobserved common causes. Once the graph is stated, identification is a graph problem. Estimation comes after that: estimate the functional that the graph identified.

I use `CausalGraphs.jl` for the graph and identification steps, then `CausalEstimate.jl` for back-door/a-fixable effects, and `CausalGraphs.estimate_causal` directly for p-fixable and nested-fixable effects. The order is the important part: graph first, identified functional second, estimator third.

3.1. DAGs and ADMGs

A directed acyclic graph, or DAG, has directed arrows such as `X -> A`. In causal work, an arrow means a direct causal relation is allowed by the model. Absence of an arrow is also a substantive assumption.

An acyclic directed mixed graph, or ADMG, adds bidirected arrows such as `A <-> Y`. A bidirected edge represents an unobserved common cause. This is useful because many econometric applications have variables we cannot measure: ability, preferences, latent health, firm quality, and so on.

The graph is not an estimator. It is a compact statement of assumptions. The workflow is:

1. write down a DAG or ADMG;
2. ask whether the target effect is identified;
3. estimate the identified functional from data.

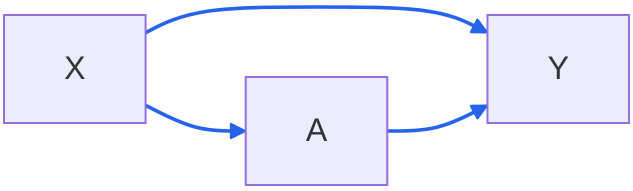
3.2. A DAG: Backdoor Adjustment

Start with a simple observational study. `A` is treatment, `Y` is the outcome, and `X` is an observed confounder:

```
g_dag = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)
```

```
ADMG([:X, :A, :Y], [(:X, :A), (:X, :Y), (:A, :Y)], Tuple{Symbol, Symbol}[], Dict{Symbol, Vecto
```

```
println("```{mermaid}")
println(to_mermaid(g_dag; direction = "LR"))
println("```")
```



The graph says `X` affects both treatment and outcome. It also says there is no unobserved common cause between `A` and `Y` after we condition on `X`. In this case the treatment is a-fixable, which corresponds to backdoor-style identification.

```
id_dag = identify(g_dag, :A, :Y)
(strategy = id_dag.strategy,
 markov_pillow = markov_pillow(g_dag, :A; treatment = :A))
```

```
(strategy = :a_fixable, markov_pillow = [:X])
```

The Markov pillow is the adjustment set used by the estimator. Under this graph,

$$E[Y(a)] = E_X\{E(Y \mid A = a, X)\}. \tag{3.1}$$

So the graph has translated a causal query into an observed-data functional.

3.3. Estimating the DAG Effect with CausalEstimate

Here is a small simulated data set where `X` confounds the treatment-outcome association.

```
Random.seed!(2026)

n = 400
X = randn(n)
pA = logistic.(-0.2 .+ 0.8 .* X)
A = Float64.(rand(n) .< pA)
Y = 1 .+ 1.5 .* A .+ 0.7 .* X .+ 0.3 .* randn(n)

df_dag = DataFrame(X = X, A = A, Y = Y)
first(df_dag, 5)
```

	X	A	Y
	Float64	Float64	Float64
1	-0.91813	1.0	1.76741
2	0.313593	1.0	2.77638
3	-1.23885	0.0	0.130173
4	0.546413	0.0	1.43362
5	0.114729	0.0	0.906974

Because the graph is a-fixable, `GraphID` routes to AIPW backdoor adjustment. The call below estimates the average contrast $E[Y(1) - Y(0)]$.

```
dag_res = estimate(
    ATE(outcome = :Y, treatment = :A),
    GraphID(graph = g_dag),
    AIPW(crossfit = 5),
    df_dag,
)

effect_table("Backdoor ACE", dag_res)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64	Float64	Float64
1	Backdoor ACE	1.465	1.265	1.666	0.1023

The graph chose the adjustment set (Markov pillow of `A`) and the estimator used that set. Access it via `dag_res.components[:adjustment_set]`. The DGP sets the coefficient on `A` to 1.5, so the point estimate recovers the true ACE within sampling noise.

`crossfit = 5` is the package default, and the number of folds matters more than it looks. With `crossfit = 2` each nuisance is fit on only half the sample, and the extra noise in the out-of-fold predictions propagates into *both* the point estimate and the influence-function variance — fewer folds means a noisier estimate reported with a wider interval, not a cheap approximation to the same answer. Two folds is the smallest value the package allows; it is not a sensible default. The next chapter measures the size of this effect on real data.

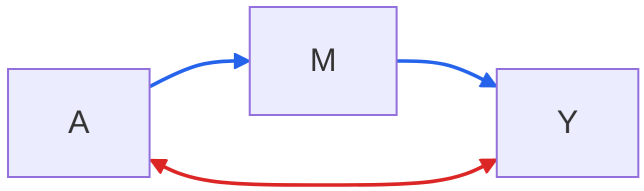
3.4. An ADMG: Front-Door Identification

Now suppose treatment and outcome share an unobserved common cause. A simple adjustment argument no longer works. The front-door ADMG has a mediator `M` that transmits the effect of `A` to `Y`, while `A` and `Y` are hidden-confounded:

```
g_fd = make_graph(
    vertices = [:A, :M, :Y],
    di_edges = [(:A, :M), (:M, :Y)],
    bi_edges = [(:A, :Y)],
)
```

```
ADMG([:A, :M, :Y], [(:A, :M), (:M, :Y)], [(:A, :Y)], Dict{Symbol, Vector{Symbol}}(), Dict{Symbo
```

```
println("```{mermaid}")
println(to_mermaid(g_fd; direction = "LR"))
println("```")
```



The bidirected edge `A <-> Y` means that ordinary backdoor adjustment is not available. But the graph is p-fixable, the ADMG generalization that includes front-door identification.

```
id_fd = identify(g_fd, :A, :Y)
(strategy = id_fd.strategy,
 a_fixable = is_fix(g_fd, :A),
 p_fixable = is_p_fix(g_fd, :A))
```

```
(strategy = :p_fixable, a_fixable = false, p_fixable = true)
```

The intuition is that `M` carries the causal effect of `A`, and `M` is not itself hidden-confounded with `A`. The graph lets us use mediator information to recover the causal effect even though `A` and `Y` are confounded.

3.5. Estimating the ADMG Effect with CausalGraphs

Simulate a front-door setting. The latent `U` is not included in the observed data; it is represented in the graph by `A <-> Y`.

```
Random.seed!(2027)

U = randn(n)
A_fd = Float64.(rand(n) .< logistic.(-0.1 .+ 0.8 .* U))
M = Float64.(rand(n) .< logistic.(-0.4 .+ 1.2 .* A_fd))
Y_fd = Float64.(rand(n) .< logistic.(-1 .+ 0.9 .* M .+ 0.7 .* U))

df_fd = DataFrame(A = A_fd, M = M, Y = Y_fd)
first(df_fd, 5)
```

	A	M	Y
	Float64	Float64	Float64
1	1.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	1.0
5	0.0	1.0	1.0

The DGP has a known truth. Under $do(A = a)$ the mediator is $M \sim \text{Bern}(\Lambda(-0.4 + 1.2a))$ while U keeps its marginal $N(0, 1)$ distribution, so

$$\tau = [\Lambda(0.8) - \Lambda(-0.4)]\ [g(-0.1) - g(-1.0)], \qquad g(c) = E_U[\Lambda(c + 0.7U)], \tag{3.2}$$

which evaluates to $0.2887 \times 0.1894 = 0.0547$. The naive contrast $E[Y \mid A = 1] - E[Y \mid A = 0]$ equals 0.158 in the population this DGP defines — almost three times the causal effect, because U raises both the chance of treatment and the outcome. That gap is what the front-door step removes. The table below reports the sample version of the naive contrast alongside the front-door estimate.

Since `identify()` returns `:p_fixable`, this is a front-door effect. `CausalEstimate.jl` currently handles backdoor (a-fixable) graphs; for p-fixable effects we call `CausalGraphs.estimate_causal` directly, which routes to NPS TMLE.

```
fd_res = with_logger(NullLogger()) do
    CausalGraphs.estimate_causal(
        a = [1.0, 0.0],
        data = df_fd,
        graph = g_fd,
        treatment = :A,
        outcome = :Y,
        n_iter = 80,
        cvg_criteria = 0.02,
        truncate_lower = 0.01,
        truncate_upper = 0.99,
    )
end

naive_fd = mean(df_fd.Y[df_fd.A .== 1]) - mean(df_fd.Y[df_fd.A .== 0])
```

```
vcat(
    DataFrame(estimand = ["Naive E[Y|A=1] - E[Y|A=0]"], estimate = [r4(naive_fd)],
              lower_95 = [missing], upper_95 = [missing], se = [missing]),
    effect_table("Front-door ACE", fd_res[:TMLE]),
    DataFrame(estimand = ["Truth"], estimate = [0.0547],
              lower_95 = [missing], upper_95 = [missing], se = [missing]);
    cols = :union,
)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64?	Float64?	Missing
1	Naive E[Y|A=1] - E[Y|A=0]	0.1014	*missing*	*missing*	*missing*
2	Front-door ACE	0.05239	0.02208	0.0827	*missing*
3	Truth	0.0547	*missing*	*missing*	*missing*

The naive contrast is far above the truth; the front-door estimate brackets it.

3.6. General ID Algorithm

Backdoor/a-fixable, p-fixable/front-door and nested-fixable are useful because they are easy to estimate. ADMGs can be more general. The Pearl-Shpitser ID algorithm asks the broader question:

```
Can P(Y | do(A)) be written using only the observed joint law P(V)?
```

The algorithm works recursively:

1. remove variables that are not ancestors of the outcome;
2. add interventions that are irrelevant after graph surgery;
3. split the remaining graph into bidirected districts;
4. identify each district as a factor of the observed law, if possible;
5. return a hedge witness when no such reduction is possible.

For the front-door graph, the general ID algorithm also succeeds:

```
id_alg_fd = ID_algorithm(g_fd, :A, :Y)
(identified = id_alg_fd.identified,
 expression = string(id_alg_fd.expression))
```

```
(identified = true, expression = "sum_{M} P(M | A) * sum_{A} P(A) * P(Y | A, M)")
```

The printed expression reuses `A` as the inner summation index; read it as $\sum_M P(M \mid A) \sum_{A'} P(Y \mid A', M) P(A')$, where the inner A' ranges over the marginal distribution of treatment and is distinct from the A being intervened on. The expression is symbolic. It is not yet an estimate. For finite-support variables, `CausalGraphs.estimate_causal` evaluates the symbolic functional by empirical probabilities and attaches a finite-support EIF standard error via the `:IDPlugin` key.

```
id_res = with_logger(NullLogger()) do
    estimate_id(
        a = [1.0, 0.0],
        data = df_fd,
        graph = g_fd,
        treatment = :A,
        outcome = :Y,
    )
end

effect_table("ID plug-in ACE", id_res[:IDPlugin];
             se = id_res[:IDPlugin].standard_error)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64	Float64	Float64
1	ID plug-in ACE	0.05346	0.02278	0.08413	0.01565

This plug-in estimator is useful for discrete examples and diagnostics. It is not a universal TMLE for every continuous ID functional.

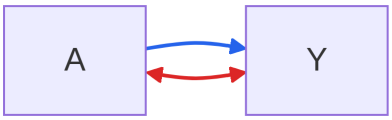
3.7. When the Graph Says No

The bow graph has both a direct causal edge and unobserved confounding between the same two variables:

```
g_bow = make_graph(
    vertices = [:A, :Y],
    di_edges = [(:A, :Y)],
    bi_edges = [(:A, :Y)],
)
```

```
ADMG([:A, :Y], [(:A, :Y)], [(:A, :Y)], Dict{Symbol, Vector{Symbol}}(), Dict{Symbol, Bool}(:A =>
```

```
println("```{mermaid}")
println(to_mermaid(g_bow; direction = "LR"))
println("```")
```



The effect of `A` on `Y` is not identified in this ADMG. The observed association mixes the direct causal effect with the latent common cause.

```
id_bow = identify(g_bow, :A, :Y)
id_alg_bow = ID_algorithm(g_bow, :A, :Y)

(route = id_bow.strategy,
 id_algorithm_identified = id_alg_bow.identified,
 hedge = id_alg_bow.hedge)
```

```
(route = :not_identified, id_algorithm_identified = false, hedge = (S = [:Y], F = [:A, :Y], tr
```

3.8. Practical Workflow

For applied work, a useful workflow is:

1. write the graph before looking at estimates;
2. use bidirected edges for latent common causes;
3. run `identify(graph, treatment, outcome)`;
4. if identified and a-fixable, call `estimate(ATE(...), GraphID(graph=...), AIPW(...), df)`; for p-fixable or nested-fixable effects, call `CausalGraphs.estimate_causal(...)`;
5. report the graph and the identification route along with the estimate.

3.9. Summary

A graph states the identifying assumptions before any estimation. A DAG says which variables may cause which. An ADMG adds bidirected edges for unobserved common causes. Identification then becomes a question about the graph. Backdoor adjustment works when an observed set blocks every confounding path. The front-door formula works when a mediator carries the whole effect and is free of the hidden confounder. The Pearl-Shpitser ID algorithm decides the general case and says when no route exists, as in the bow graph.

The order is graph first, identified functional second, estimator third. In the backdoor example, AIPW on the adjustment set returns 1.465 (0.102) against a true effect of 1.5. In the front-door example, the naive contrast is 0.1014. The front-door estimate is 0.0524, and its interval covers the true 0.0547.

4. Applied Graph Workflow: Smoking Cessation

```
using CausalGraphs
using CausalEstimate
import CausalGraphs: identify, make_graph, to_mermaid, markov_pillow
import CausalEstimate: estimate, confint, pvalue
using DataFrames
using DelimitedFiles
using Downloads
using Logging
using Random
```

The previous chapter used small graph examples. Here I apply the same graph-identification-estimation workflow to a real data set: the complete-case National Health and Nutrition Examination Survey I Epidemiologic Follow-up Study (NHEFS) data distributed through `causaldata` and mirrored by RDatasets.

The question is:

```
What is the effect of quitting smoking on weight change from 1971 to 1982?
```

The treatment is `qsmk`, an indicator for quitting smoking between baseline and follow-up. The outcome is `wt82_71`, weight change over the follow-up period. This is observational data. The effect is not identified by the data alone. It is identified only after we state the adjustment assumptions in the graph.

4.1. Data

For this example, keep the treatment, outcome, and baseline covariates that are commonly used in NHEFS smoking-cessation examples.

```
function read_rdatasets_csv(url, cols)
    local_file = Downloads.download(url; timeout = 120)
    x, header = readdlm(local_file, ',', Any, '\n'; header = true)
    raw = DataFrame(x, Symbol.(vec(header)))
    DataFrame((c => Float64.(raw[!, c]) for c in cols)...)
end

nhefs_url = "https://vincentarelbundock.github.io/Rdatasets/csv/causaldata/nhefs_complete.csv"
```

```
nhefs_cols = [
    :qsmk, :wt82_71,
    :sex, :race, :age, :school,
    :smokeintensity, :smokeyrs,
    :exercise, :active, :wt71,
]

nhefs = read_rdatasets_csv(nhefs_url, nhefs_cols);

(n = nrow(nhefs), variables = ncol(nhefs))
```

```
(n = 1566, variables = 11)
```

```
preview_table(nhefs)
```

	qsmk	wt82_71	sex	race	age	school	smokeintensity	smokeyrs	
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	0.0	-10.09	0.0	1.0	42.0	7.0	30.0	29.0	...
2	0.0	2.605	0.0	0.0	36.0	9.0	20.0	24.0	...
3	0.0	9.414	1.0	1.0	56.0	11.0	20.0	26.0	...
4	0.0	4.99	0.0	1.0	68.0	5.0	3.0	53.0	...
5	0.0	4.989	0.0	0.0	40.0	11.0	20.0	19.0	...

4.2. Adjustment DAG

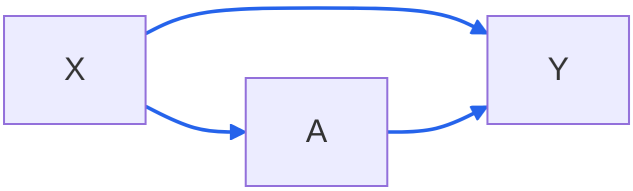
Start with the grouped graph

```
X -> A -> Y
X -----> Y
```

where `X` is the baseline covariate set, `A` is quitting smoking, and `Y` is later weight change. This graph says the baseline variables may affect both quitting and later weight change, and that there is no unmeasured common cause of quitting and later weight change after conditioning on those baseline variables.

```
conceptual_graph = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
);
```

```
println("```{mermaid}")
println(to_mermaid(conceptual_graph; direction = "LR"))
println("```")
```



For estimation, expand `X` into the observed NHEFS columns. The expanded graph is still an adjustment DAG: each baseline variable is allowed to point into quitting and into later weight change.

```
nhefs_vertices = [
    :sex, :race, :age, :school,
    :smokeintensity, :smokeyrs,
    :exercise, :active, :wt71,
    :qsmk, :wt82_71,
]

baseline = setdiff(nhefs_vertices, [:qsmk, :wt82_71])

nhefs_edges = vcat(
    [(w, :qsmk) for w in baseline],
    [(w, :wt82_71) for w in baseline],
    [(:qsmk, :wt82_71)],
)

nhefs_graph = make_graph(vertices = nhefs_vertices, di_edges = nhefs_edges);

(vertices = length(nhefs_vertices), directed_edges = length(nhefs_edges))
```

```
(vertices = 11, directed_edges = 19)
```

4.3. Identification

`CausalGraphs.identify` checks the graph before estimation. In this adjustment DAG, the quitting node is a-fixable. The Markov pillow is the sufficient adjustment set:

```
nhefs_id = identify(nhefs_graph, :qsmk, :wt82_71)

DataFrame(
    strategy = [string(nhefs_id.strategy)],
    markov_pillow = [join(string.(markov_pillow(nhefs_graph, :qsmk; treatment = :qsmk)), ", ")]
)
```

	strategy	markov_pillow
	String	String
1	a_fixable	sex, race, age, school, smokeintensity, smokeyrs, exercise, active, wt71

The graph therefore identifies the causal mean by the adjustment formula

$$E[Y(a)] = E_X\{E(Y \mid A = a, X)\}. \tag{4.1}$$

This formula is not a modeling claim by itself. It is the observed-data functional implied by the graph. Estimation still requires nuisance models for the outcome regression and treatment mechanism.

4.4. Estimation with CausalEstimate

The book keeps identification and estimation as separate steps. Here `CausalGraphs` supplies the graph result and `CausalEstimate` estimates the identified functional. Because the graph is a-fixable, `GraphID` routes to the AIPW average treatment effect machinery.

```
Random.seed!(2026)

nhefs_result = estimate(
    ATE(outcome = :wt82_71, treatment = :qsmk),
    GraphID(graph = nhefs_graph),
    AIPW(crossfit = 5),
    nhefs,
)

effect_table("Quit smoking vs continued smoking", nhefs_result)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64	Float64	Float64
1	Quit smoking vs continued smoking	3.452	1.986	4.918	0.748

Under the DAG, the estimate is the average effect of quitting smoking versus continuing smoking on weight change in the complete-case NHEFS population. The positive estimate is consistent with weight gain after quitting. The causal interpretation depends on the graph, especially the assumption that the measured baseline variables block the relevant backdoor paths.

Two settings deserve comment, because both move the number.

Folds. `crossfit = 5` is the package default and this chapter's seed makes the comparison reproducible. Dropping to `crossfit = 2` gives 3.866 with a standard error of 1.331 — a 95% interval of $[1.26, 6.48]$, more than five kilograms wide. Five folds gives 3.452 (0.748) and ten gives 3.312 (0.737). Two folds fits each nuisance on 783 observations, and the resulting noise in the out-of-fold predictions inflates the point estimate *and* the influence-function variance at the same time. Use the default, or more.

Nuisance estimation. The companion R chapter fits the same adjustment set by full-sample parametric nuisances (a linear outcome model interacted with treatment, a logistic propensity) and reports 3.293 with a standard error of 0.498. The propensity scores in this data set stay inside $[0.05, 0.75]$, so there is no overlap problem to explain the gap: the wider interval here is the variance price of cross-fitting flexible learners rather than trusting a correctly specified parametric model.

When the parametric model is right, it wins; the cross-fit estimator protects against its being wrong.

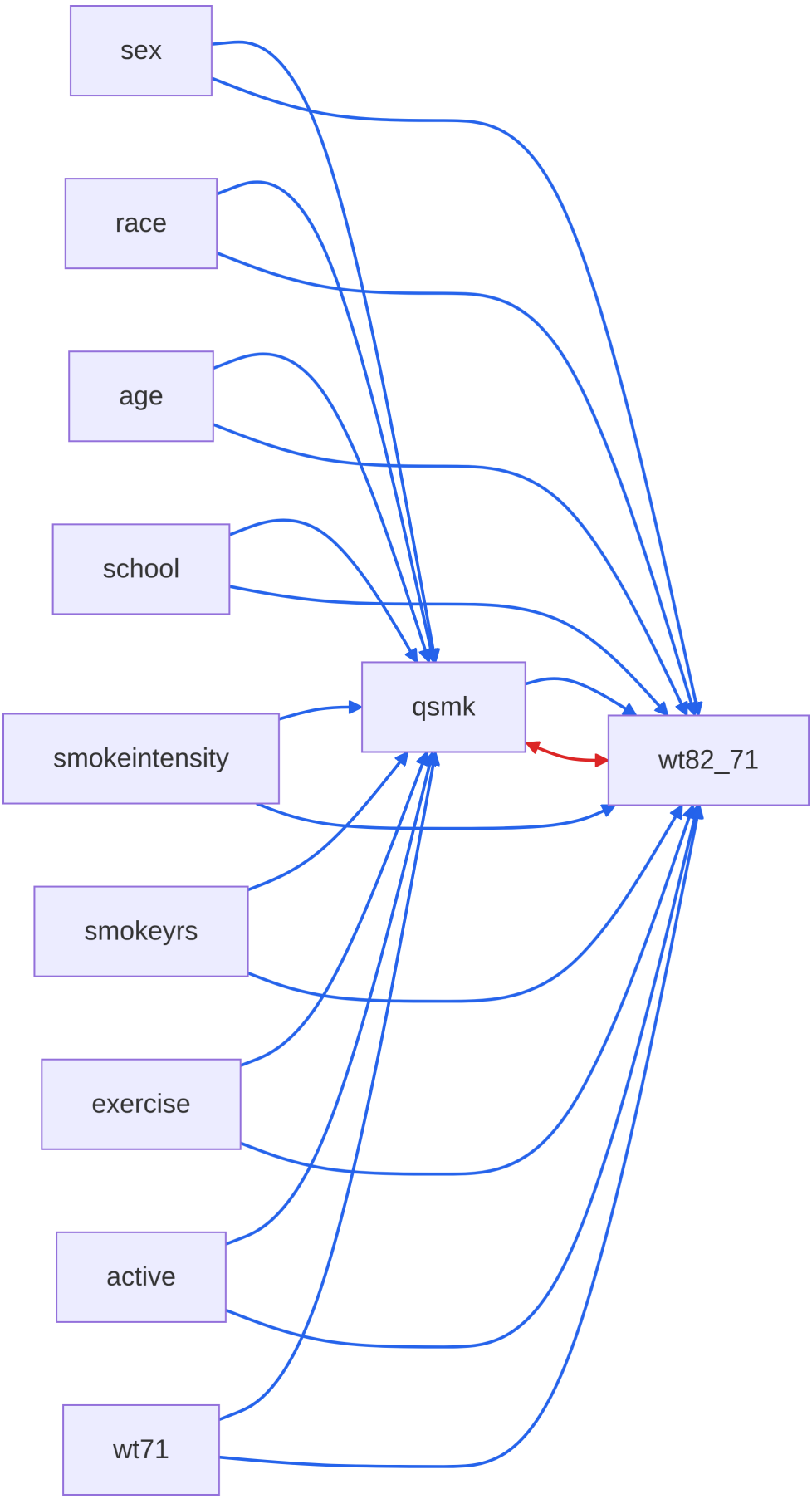
4.5. Sensitivity Graph

Now encode a different assumption: quitting and later weight change share an unobserved common cause even after conditioning on the measured baseline variables. In an ADMG this is a bidirected edge, `qsmk <-> wt82_71`.

```
sensitivity_graph = make_graph(
    vertices = nhefs_vertices,
    di_edges = nhefs_edges,
    bi_edges = [(:qsmk, :wt82_71)],
)

sensitivity_id = identify(sensitivity_graph, :qsmk, :wt82_71);
```

```
println("```{mermaid}")
println(to_mermaid(sensitivity_graph; direction = "LR"))
println("```")
```


sex
race
age
school
qsmk
smokeintensity
wt82_71
smokeyrs
exercise
active
wt71

```
DataFrame(strategy = [string(sensitivity_id.strategy)])
```

	strategy
	String
1	not_identified

The identifying conclusion changes. Under this graph, the observed NHEFS variables are not enough to identify the effect. This is why the graph has to come before the estimator. The same data can imply different causal answers under different assumptions.

4.6. Structured Baseline Graph

The simple adjustment graph is often the clearest starting point. A more detailed DAG can record baseline ordering: demographics may precede education, smoking history, activity, and baseline weight; those variables may then affect quitting and later weight change.

```
structured_edges = [
    (:age, :school), (:age, :smokeyrs), (:age, :wt71),
    (:age, :active), (:age, :exercise), (:age, :qsmk), (:age, :wt82_71),

    (:sex, :smokeintensity), (:sex, :smokeyrs), (:sex, :wt71),
    (:sex, :qsmk), (:sex, :wt82_71),

    (:race, :school), (:race, :smokeintensity),
    (:race, :qsmk), (:race, :wt82_71),

    (:school, :smokeintensity), (:school, :qsmk), (:school, :wt82_71),

    (:smokeyrs, :smokeintensity), (:smokeyrs, :qsmk), (:smokeyrs, :wt82_71),
    (:smokeintensity, :qsmk), (:smokeintensity, :wt82_71),

    (:exercise, :wt71), (:exercise, :qsmk), (:exercise, :wt82_71),
    (:active, :wt71), (:active, :qsmk), (:active, :wt82_71),
    (:wt71, :qsmk), (:wt71, :wt82_71),

    (:qsmk, :wt82_71),
]

structured_graph = make_graph(vertices = nhefs_vertices, di_edges = structured_edges);
structured_id = identify(structured_graph, :qsmk, :wt82_71)

DataFrame(
    strategy = [string(structured_id.strategy)],
    same_adjustment_set = [
        Set(markov_pillow(structured_graph, :qsmk; treatment = :qsmk)) == Set(baseline)
```

```
    ],
)
```

	strategy	same_adjustment_set
	String	Bool
1	a_fixable	1

The check returns `true`, and the reason is worth spelling out, because it is not an accident of this data set. Every arrow we added runs between baseline variables, and every baseline variable is pre-treatment and in the adjustment set. A back-door path from `qsmk` to `wt82_71` has to leave `qsmk` through an incoming arrow, travel among the baseline variables, and then enter `wt82_71` from one of them. That last baseline variable has an arrow pointing into the outcome, so on this path it is a non-collider, and we are conditioning on it. The path is blocked whatever the arrows upstream of it look like. The measured parents of `qsmk` are therefore still the full baseline, and the identifying functional is unchanged.

That does not make the detail useless. It records what we believe about the baseline ordering, which a reader may want to see. But each arrow is a causal assumption that can be wrong, and here none of them is needed to identify this estimand. The grouped adjustment graph asserts less and identifies the same thing.

4.7. Summary

We apply the graph-first workflow to the NHEFS question of how quitting smoking affects weight change from 1971 to 1982. The adjustment DAG lets the nine baseline covariates affect both quitting and weight change, and allows no unmeasured common cause beyond them. `dagitty` returns the full baseline as the adjustment set. AIPW with cross-fitted nuisance models on that set gives an effect of 3.45 kg, with a standard error of 0.75.

The causal reading rests on the graph. Adding one bidirected edge between quitting and weight change leaves no valid adjustment set. The number can still be computed, but nothing then justifies reading it as an effect. A more detailed ordering among the baseline variables changes nothing here, because every backdoor path ends at a baseline variable we condition on.

5. Sensitivity Analysis

```
using DataFrames
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

The previous chapters often treat identification as yes or no. Applied work is usually less clean. We may believe the adjustment set is good, but still worry about some remaining confounding. Sensitivity analysis asks a practical question: how strong would unmeasured confounding have to be to change the conclusion?

> **Related reading**: An extended R-based treatment of the Cinelli-Hazlett robustness value with `sensemakr` and benchmark interpretations appears in the companion blog chapter on sensitivity analysis of *Topics on Econometrics and Causal Inference*.

Here I use three tools:

1. **Cinelli-Hazlett (2020) omitted-variable bias bounds** — robustness values and benchmark comparisons for OLS regression estimates.
2. **E-values (VanderWeele-Ding 2017)** — for risk-ratio estimates, the minimum strength an unmeasured confounder would need.
3. **Rosenbaum bounds** — the critical odds ratio of hidden bias that would render a matched-sample test insignificant.

There is no standard Julia package for these, but the formulas are short.

5.1. A simulated example

Use a simulation so the truth is known:

```
Random.seed!(2026)
n   = 2000
X1  = randn(n)
X2  = randn(n)
U   = randn(n)              # unmeasured confounder
```

```
D   = @. Float64(rand() < 1 / (1 + exp(-(0.5 * X1 + 0.5 * X2 + 0.6 * U))))
Y   = @. 1.0 * D + 1.5 * X1 + 1.5 * X2 + 1.0 * U + 0.5 * randn()

df  = DataFrame(Y = Y, D = D, X1 = X1, X2 = X2, U = U)

# Pull the coefficient out BY NAME. Indexing positionally (`[end]`, `[2]`) is a
# standing invitation to report the wrong number: `[end]` on `Y ~ D + X1 + X2`
# is the coefficient on X2, not on D, and in this DGP X2's true coefficient
# (1.5) is close enough to the confounded D estimate that the mistake does not
# look like one.
coef_of(m, name) = coef(m)[findfirst(==(name), coefnames(m))]

@printf("True treatment effect: 1.000\n")
@printf("Naive (D only):        %.3f\n",
        coef_of(lm(@formula(Y ~ D), df), "D"))
@printf("Adjusted (D, X1, X2):  %.3f  (biased because U is unobserved)\n",
        coef_of(lm(@formula(Y ~ D + X1 + X2), df), "D"))
@printf("Oracle (D, X1, X2, U): %.3f\n",
        coef_of(lm(@formula(Y ~ D + X1 + X2 + U), df), "D"))
```

```
True treatment effect: 1.000
Naive (D only):        2.787
Adjusted (D, X1, X2):  1.497  (biased because U is unobserved)
Oracle (D, X1, X2, U): 0.966
```

The adjusted regression is still biased because U is unobserved.

5.2. Cinelli-Hazlett robustness value

The robustness value is the smallest partial R^2 that an unobserved confounder would need with treatment and with outcome, the same value on both, to bring the estimate to zero. It is an equal-strength threshold.

The closed-form expression depends only on the t-statistic of the treatment coefficient and the regression's degrees of freedom:

$$RV_q = \tfrac{1}{2}\left[\sqrt{f_q^4 + 4f_q^2} - f_q^2\right], \qquad f_q = q\,\frac{|t|}{\sqrt{df}}, \tag{5.1}$$

where $|t|$ is the absolute t-stat of the treatment coefficient, df is the residual degrees of freedom, and q is the fraction of the estimate we want explained away ($q = 1$ for full explanation).

```
"""
    robustness_value(t_stat, df; q=1.0)
Equal-strength threshold: the partial R², the same with treatment and with
outcome, that an unmeasured confounder needs to explain away a fraction `q` of
the estimate.
Cinelli & Hazlett (2020).
"""
function robustness_value(t_stat::Real, df::Real; q::Real=1.0)
    fq = q * abs(t_stat) / sqrt(df)
    return 0.5 * (sqrt(fq^4 + 4 * fq^2) - fq^2)
end

# Fit the adjusted regression and extract t-stat + dof
fit_adj   = lm(@formula(Y ~ D + X1 + X2), df)
t_adj     = coef(fit_adj)[2] / stderror(fit_adj)[2]
df_resid  = dof_residual(fit_adj)

rv  = robustness_value(t_adj, df_resid; q=1.0)
rv5 = robustness_value(t_adj, df_resid; q=1.0 - 1.96/abs(t_adj))   # q to reach significance

@printf("t-statistic on D:                %.2f\n", t_adj)
@printf("Residual df:                     %.0f\n", df_resid)
@printf("Robustness value (q=1):          %.4f  (= %.2f%% partial R²)\n",
        rv, 100rv)
@printf("RV to reach insignificance:      %.4f  (= %.2f%% partial R²)\n",
        rv5, 100rv5)
```

```
t-statistic on D:                29.24
Residual df:                     1996
Robustness value (q=1):          0.4745  (= 47.45% partial R²)
RV to reach insignificance:      0.4521  (= 45.21% partial R²)
```

The useful interpretation is comparative: compare the RV to the partial R^2 values of observed covariates.

5.2.1. Benchmarking against observed covariates

How strong are the observed covariates as a reference? Compute their partial R^2 contributions:

```
# Partial R² of a covariate: variance explained over a model that omits it
function partial_r2(full_fit, reduced_fit)
    ss_full     = sum(abs2.(residuals(full_fit)))
    ss_reduced  = sum(abs2.(residuals(reduced_fit)))
    1 - ss_full / ss_reduced
end
```

```
# Partial R² of X1 with the outcome (omit X1)
fit_noX1_Y = lm(@formula(Y ~ D + X2), df)
pR2_X1_Y   = partial_r2(fit_adj, fit_noX1_Y)

# Partial R² of X1 with the treatment (a separate regression)
fit_D_full = lm(@formula(D ~ X1 + X2), df)
fit_D_noX1 = lm(@formula(D ~ X2), df)
pR2_X1_D   = partial_r2(fit_D_full, fit_D_noX1)

@printf("Benchmark - X1:\n")
@printf("  Partial R² with Y: %.4f\n", pR2_X1_Y)
@printf("  Partial R² with D: %.4f\n", pR2_X1_D)
@printf("\nRobustness value:    %.4f\n", rv)
# A confounder with these two partial R² values biases the estimate by
# se * sqrt(df) * sqrt(R²_Y * R²_D / (1 - R²_D)) (Cinelli-Hazlett 2020, eq. 4).
# Comparing min(R²_Y, R²_D) with the RV is not enough: the RV is the equal-strength
# threshold, and an unequal pair can explain the estimate with one value below it.
bias_X1 = stderror(fit_adj)[2] * sqrt(df_resid) *
          sqrt(pR2_X1_Y * pR2_X1_D / (1 - pR2_X1_D))
@printf("\nBias from a confounder with X1's two partial R²: %.4f\n", bias_X1)
@printf("Would it explain the estimate (%.4f)? %s\n", abs(coef(fit_adj)[2]),
        bias_X1 >= abs(coef(fit_adj)[2]) ? "YES" : "NO")
```

```
Benchmark - X1:
  Partial R² with Y: 0.6209
  Partial R² with D: 0.0489

Robustness value:    0.4745

Bias from a confounder with X1's two partial R²: 0.4090
Would it explain the estimate (1.4975)? NO
```

The RV answers the equal-strength question only. A confounder can explain the estimate away with one partial R^2 below the RV if the other is large enough, because the bias depends on their product. So for a specific pair of strengths we use the bias formula, as the code does: an unmeasured confounder as strong as X_1 on both margins would or would not overturn the result.

Because this is a simulation we can check the machinery against the confounder that actually exists. U's partial R^2 is 0.787 with the outcome and 0.064 with the treatment. The two numbers straddle the robustness value, so the RV alone cannot say whether U explains the estimate away. The bias formula can, and we apply it below: U's bias is smaller than the estimate, so the verdict "not explained away" is correct — and it is correct even though the adjusted estimate is badly biased. Those are different questions, and the distinction is the main thing to take from this section: a robustness value asks whether a confounder could drive the estimate to *zero*, not whether the estimate is *right*. Here it cannot, and the estimate is still 50% too large.

Feeding U's true strengths into the Cinelli-Hazlett bias formula gives a bound of 0.531, so the bias-corrected estimate is $1.497 - 0.531 = 0.966$ against a true effect of 1.000. The bound is doing

its job: it is not a point correction one would apply in practice, since in real data the confounder's strengths are exactly what we do not know, but it confirms that the formula is calibrated rather than merely suggestive.

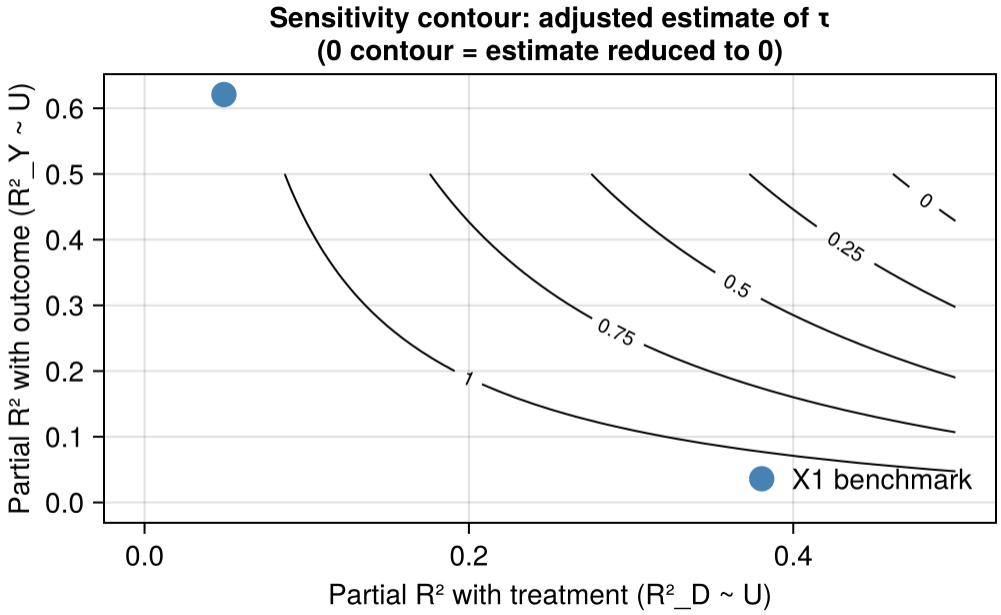
5.2.2. A sensitivity contour plot

The contour plot below shows, for each $(R^2_{D\sim U}, R^2_{Y\sim U})$ pair, the implied estimate of the treatment effect under that strength of unobserved confounding:

```
# Bias formula (Cinelli-Hazlett 2020, eqn 4):
# bias = se(^) * sqrt(df) * sqrt( R²_Y * R²_D / (1 - R²_D) )
# adjusted_estimate = ^ - bias (or + bias, the worst case is the larger absolute)
tau_hat = coef(fit_adj)[2]
se_hat  = stderror(fit_adj)[2]

grid = 0.0:0.01:0.5
# First index = R²_D (x axis), second = R²_Y (y axis), as contour! expects.
bias_matrix = [se_hat * sqrt(df_resid) * sqrt(r2y * r2d / max(1 - r2d, 1e-8))
               for r2d in grid, r2y in grid]
adjusted    = tau_hat .- bias_matrix    # worst-case (downward) bias

fig = Figure(size = (700, 500))
ax  = Axis(fig[1, 1],
           xlabel = "Partial R² with treatment (R²_D ~ U)",
           ylabel = "Partial R² with outcome (R²_Y ~ U)",
           title  = "Sensitivity contour: adjusted estimate of  \n" *
                    "(0 contour = estimate reduced to 0)")
contour!(ax, grid, grid, adjusted, levels = [0.0, 0.25, 0.5, 0.75, 1.0],
         labels = true, color = :black)
# Mark the X1 benchmark
scatter!(ax, [pR2_X1_D], [pR2_X1_Y], color = :steelblue, markersize = 18,
         label = "X1 benchmark")
axislegend(ax, position = :rb, framevisible = false)
fig
```



A confounder above and to the right of the zero contour would reduce the estimate to zero. The benchmark dot shows the strength of an observed covariate for comparison.

5.3. E-values

For risk-ratio estimates, VanderWeele and Ding (2017) propose a simpler-to-interpret sensitivity statistic. For an observed risk ratio $RR > 1$, the **E-value** is the minimum strength of association (on the risk-ratio scale) that an unobserved confounder would need to have with both treatment and outcome to fully explain the observed association:

$$\text{E-value} = RR + \sqrt{RR(RR-1)}. \tag{5.2}$$

For $RR < 1$, use $1/RR$ first and apply the formula.

```
"""
    evalue(rr; lo=missing, hi=missing)
Compute the E-value for a risk ratio and (optionally) its 95% confidence
interval. Returns (E_point, E_ci) where E_ci is the E-value needed to
shift the lower CI bound past 1 (or upper past 1 if rr < 1).
"""
function evalue(rr::Real; lo=missing, hi=missing)
    e_point = rr >= 1 ? rr + sqrt(rr * (rr - 1)) : begin
        rr_inv = 1 / rr
        rr_inv + sqrt(rr_inv * (rr_inv - 1))
    end
    e_ci = missing
```

```
    if !ismissing(lo) && !ismissing(hi)
        if rr >= 1 && lo > 1
            e_ci = lo + sqrt(lo * (lo - 1))
        elseif rr < 1 && hi < 1
            hi_inv = 1 / hi
            e_ci = hi_inv + sqrt(hi_inv * (hi_inv - 1))
        else
            e_ci = 1.0   # CI crosses 1, no E-value needed
        end
    end
    return (e_point = e_point, e_ci = e_ci)
end

# Example: a study reports RR = 1.40 [1.20, 1.62]
ev = evalue(1.40; lo = 1.20, hi = 1.62)
@printf("RR = 1.40 [1.20, 1.62]\n")
@printf("E-value (point): %.2f\n", ev.e_point)
@printf("E-value (CI):    %.2f\n", ev.e_ci)
```

```
RR = 1.40 [1.20, 1.62]
E-value (point): 2.15
E-value (CI):    1.69
```

The interpretation is on the risk-ratio scale. A hidden confounder with associations of this size with both treatment and outcome could explain away the result, and one weaker than this on both could not. Like the RV, it is an equal-strength threshold: a confounder stronger on one association can be weaker on the other.

5.4. Rosenbaum bounds for matched studies

For matched estimators (propensity score matching, full matching, optimal matching), **Rosenbaum bounds** (Rosenbaum 2002) parametrise hidden bias as Γ, the maximum ratio of treatment odds between the two units of a matched pair due to unobservables. $\Gamma = 1$ corresponds to within-pair randomisation; $\Gamma = 2$ means one unit's odds of receiving treatment can be up to twice the other's.

For each value of Γ, the upper-bound p-value of the Wilcoxon signed-rank test on matched-pair outcome differences is:

```
# Simulate matched-pair differences from a known DGP
Random.seed!(7)
n_pairs = 500
true_effect = 0.5
diffs = true_effect .+ 0.8 .* randn(n_pairs)

using Distributions: Normal, cdf
```

```
"""
    rosenbaum_pvalue(diffs, gamma)
Upper-bound one-sided p-value of the Wilcoxon signed-rank test on matched
pair differences, allowing for hidden bias of magnitude `gamma`  1.
"""
function rosenbaum_pvalue(diffs, gamma)
    abs_d = abs.(diffs)
    sgn   = sign.(diffs)
    rks   = sortperm(sortperm(abs_d))   # ranks of |d|
    # Under Γ, prob of positive sign is bounded by Γ/(1+Γ)
    p_pos = gamma / (1 + gamma)
    T_obs = sum(rks[sgn .> 0])
    mu    = p_pos * sum(rks)
    v     = p_pos * (1 - p_pos) * sum(rks .^ 2)
    z     = (T_obs - mu) / sqrt(v)
    return 1 - cdf(Normal(), z)
end

gammas = 1.0:0.25:5.0
pvals  = [rosenbaum_pvalue(diffs, g) for g in gammas]
result = DataFrame(Gamma = gammas, p_upper = pvals)

# Critical Γ: smallest Γ where p exceeds 0.05
critical_idx = findfirst(p -> p > 0.05, pvals)
critical_gamma = critical_idx === nothing ? Inf : gammas[critical_idx]

@printf("Critical Γ (p > 0.05): %.2f\n", critical_gamma)
@printf("\n%-10s %s\n", "Γ", "p (upper bound)")
for (g, p) in zip(gammas, pvals)
    @printf("%-10.2f %.4f\n", g, p)
end
```

```
Critical Γ (p > 0.05): 3.00

Γ          p (upper bound)
1.00       0.0000
1.25       0.0000
1.50       0.0000
1.75       0.0000
2.00       0.0000
2.25       0.0001
2.50       0.0013
2.75       0.0121
3.00       0.0599
3.25       0.1799
3.50       0.3725
```

```
3.75        0.5887
4.00        0.7698
4.25        0.8891
4.50        0.9534
4.75        0.9827
5.00        0.9942
```

If the critical Γ is close to 1, small hidden bias could overturn the matched-pair result. (The reported value is the first point *on the 0.25 grid* at which significance is lost; the actual crossing lies between it and the preceding grid point.)

5.5. Reporting practice

Most observational causal estimates should have some sensitivity analysis. A minimum report is:

Estimator family	What to report
Linear regression	Cinelli-Hazlett RV + benchmark comparison
Risk-ratio analysis	E-value (point + CI)
Matched / pair-based test	Critical Γ from Rosenbaum bounds

When the result is fragile, the next step is either to improve measurement and adjustment or to report partial-identification bounds.

5.6. Summary

- Sensitivity analysis asks how strong hidden confounding must be to change the result.
- Robustness values are useful for regression estimates.
- E-values are useful for risk ratios.
- Rosenbaum bounds are useful for matched pairs.
- The basic formulas are easy to implement directly in Julia.

Part II.

Estimation

6. Estimation: RA, IPW, AIPW and IPWRA

```
using CausalEstimate
using Panelest
using MLJ
using GLMNet
using DecisionTree
using Vcov
using ReadStatTables
using DataFrames
using Statistics
using GLM
using Downloads
using Random
using Printf
using RegressionTables
```

Once the causal effect is identified, we need an estimator. This chapter covers four: regression adjustment (RA), inverse probability weighting (IPW), augmented IPW (AIPW), and inverse-probability-weighted regression adjustment (IPWRA).

6.1. Experimental Data

With experimental data the assumptions hold by design. A difference-in-means estimator suffices (under randomization ATE = ATT = ATU):

$$\hat{\tau} = \bar{Y}_1 - \bar{Y}_0 \tag{6.1}$$

A t-test or a regression of Y on W and X both recover the same estimate.

6.2. Adjustment Formula

The adjustment formula connects causal and statistical quantities:

$$\begin{aligned} E[Y(1) - Y(0)] &= E[Y(1)] - E[Y(0)] \\ &= E_X[E[Y(1)|X]] - E_X[E[Y(0)|X]] \\ &= E_X[E[Y(1)|W=1,X]] - E_X[E[Y(0)|W=0,X]] \\ &= E_X[E[Y|W=1,X]] - E_X[E[Y|W=0,X]] \end{aligned} \tag{6.2}$$

The steps are: linearity of expectation, iterated expectations, unconfoundedness plus overlap, and consistency.

6.3. Regression Adjustment

We estimate $E[Y \mid W = w, X = x]$ by regression.

We define a conditional mean function:

$$\begin{aligned} \mu_0(X) &\equiv E[Y|W = 0, X] \\ &= E[Y(0)|W = 0, X] \\ &= E[Y(0)|W = 1, X] \end{aligned} \tag{6.3}$$

The RA estimators are:

$$\hat{\tau}_{ATT} = \frac{1}{n_1} \sum_{i:W_i=1} \{Y_i - \hat{\mu}_0(X_i)\} \tag{6.4}$$

$$\hat{\tau}_{ATE} = \frac{1}{n} \sum_{i=1}^{n} \{\hat{\mu}_1(X_i) - \hat{\mu}_0(X_i)\} \tag{6.5}$$

where $\hat{\mu}_0(X_i)$ is the predicted value of Y_i from a regression of Y on X for the control group.

Target:

$$\begin{aligned} \tau_{ATT} &= E[Y|W = 1] - E[Y(0)|W = 1] \\ &= E_X\{E[Y|W = 1, X] - \mu_0(X) \mid W = 1\} \\ &= E_X\{E[Y|W = 1, X] \mid W = 1\} - (\alpha_0 + E(X \mid W = 1)\beta_0) \end{aligned} \tag{6.6}$$

The last term is a linear form of $\mu_0(X)$. We can specify other forms, but the idea is to model it with some functional form. For parametric forms, we need to make sure extrapolation does not go out of control.

6.4. Linear RA

Regression Adjustment is basically an imputation estimator. While we observe $E[Y|W = 1, X]$, we model $E[Y(0)|W = 1]$, based on unconfoundedness and a functional form (say linear form). We estimate β_0 on the control sample, then get the expected values for the treated sample, for each value of X.

Implementation of linear RA is easy. We regress Y on X and W and their interaction. It's shown that we need to de-mean X to get the effect correct.

```julia
using CSV, DataFrames, Statistics, GLM

# Load the pension dataset from the hdm R package (Chernozhukov et al.)
# Treatment: p401 (eligibility for 401k), Outcome: net_tfa (net total financial assets)
pension = CSV.read("data/pension.csv", DataFrame)

# Calculate means for de-meaning (treated group means)
mean_inc_treated = mean(pension[pension.p401 .== 1, :inc])
mean_age_treated = mean(pension[pension.p401 .== 1, :age])

# For ATT: we de-mean X by deducting the mean of X in the treated group
# For ATE: we de-mean X by deducting the mean of X in the whole sample
data = DataFrames.transform(pension,
    :inc => (x -> x .- mean_inc_treated) => :inc_dm,
    :age => (x -> x .- mean_age_treated) => :age_dm
)

lm_ra = lm(@formula(net_tfa ~ p401 * (inc_dm + age_dm)), data)
show_regression_html(lm_ra)
```

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	2.410e+04	831.154	28.998	< 2e-16 ***
p401	1.416e+04	1397.304	10.134	< 2e-16 ***
inc_dm	0.772	0.030	25.670	< 2e-16 ***
age_dm	797.675	63.503	12.561	< 2e-16 ***
p401 & inc_dm	0.330	0.051	6.429	1.342e-10 ***
p401 & age_dm	815.335	133.260	6.118	9.810e-10 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

After demeaning the covariates, we include all interactions of demeaned covariates with treatment. Because the covariates are demeaned at *treated-group* means, the coefficient on W is the ATT (demeaning at whole-sample means would give the ATE instead).

6.4.1. Questions on RA

1. Why do we have to do interaction of W and X? What if we don't do it?
2. What if we don't de-mean X?

Answers:

1. It has been shown (Słoczyński 2022) that with heterogeneous treatment effect, and unequal divide between treated and control, we need to do the interaction term. Otherwise, we'll get biased estimates, for τ_{ATT} or τ_{ATE}. The intuition is that we need to model the difference between treated and control, and the difference is not constant across X.

2. If we don't de-mean X, then the coefficient on W is not the average treatment effect. We can still retrieve the ATT or ATE by calculating the marginal effect of W though.

6.5. IPW

Under unconfoundedness and overlap,

$$\begin{aligned} E[Y(w)] &= E[E[Y|W=w,X]] \\ &= \sum_x (\sum_y yP(y|w,x))P(x) \\ &= \sum_x \sum_y yP(y|w,x)P(x)\frac{P(w|x)}{P(w|x)} \\ &= \sum_x \sum_y yP(y,w,x)\frac{1}{P(w|x)} \\ &= \sum_x E[\mathbb{1}(W=w,X=x)Y]\frac{1}{P(w|x)} \\ &= E[\frac{\mathbb{1}(W=w)Y}{P(w|X)}] \end{aligned} \tag{6.7}$$

The IPW equation means that the expected value of the potential outcome is the weighted average of the observed outcome, where the weight is the inverse of the propensity score $P(w|x)$. $\mathbb{1}(W=w)$ is an indicator function, which is 1 if $W=w$ and 0 otherwise.

$$\begin{aligned} \tau_{ATE} &= E[Y(1)] - E[Y(0)] \\ &= E[\frac{WY}{\pi(X)}] - E[\frac{(1-W)Y}{1-\pi(X)}] \end{aligned} \tag{6.8}$$

The sample estimators are:

$$\hat{\tau}_{ATE,IPW} = \frac{1}{N}\sum_i \frac{W_i Y_i}{\hat{\pi}(X_i)} - \frac{1}{N}\sum_i \frac{(1-W_i)Y_i}{1-\hat{\pi}(X_i)} \tag{6.9}$$

$$\hat{\tau}_{ATT,IPW} = \bar{Y}_1 - \frac{1}{N_1}\sum_i \frac{\hat{\pi}(X_i)(1-W_i)Y_i}{1-\hat{\pi}(X_i)} \tag{6.10}$$

6.5.1. IPW Intuition

IPW removes confounding X by creating a pseudo-population in which X is independent of W. The intuition is that we can create a pseudo-population in which X is independent of W, by weighting the observations by the inverse of the propensity score. Once X is independent of W, X is not a confounder anymore. We can then estimate the effect of W on Y by comparing the weighted average of Y for $W=1$ and $W=0$.

Suppose there are two types of people, one type has high probability to be treated, the other one has low probability to be treated. Without adjusting for the propensity score, the treated group will be dominated by the first type, and the control group will be dominated by the second type. The difference between the treated and control group will be due to the difference in the composition of the two groups, not due to the treatment effect. By weighting the observations by the inverse of the propensity score, we can create a pseudo-population in which the two groups have the same composition. The difference between the treated and control group will be due to the treatment effect.

6.6. Doubly Robust Estimators

We can model both outcome and treatment assignment, and use both models to estimate the treatment effect. The estimator is called doubly robust because it is consistent if *either* the outcome regression *or* the propensity/treatment model is correctly specified — we only need one of the two to be right, not both.

6.6.1. AIPW

$$\mu_1^{AIPW} = E[\frac{W(Y-\mu_1(X))}{\pi(X)} + \mu_1(X)] \tag{6.11}$$

For the ATT, control units are weighted by the odds $\hat{\pi}(X_i)/(1-\hat{\pi}(X_i))$ rather than $1/(1-\hat{\pi}(X_i))$: this reweights the controls to the covariate distribution of the treated (one can check $\mathrm{E}[\frac{\pi(X)}{1-\pi(X)}(1-W)Y] = P(W=1)\,\mathrm{E}[Y(0)|W=1]$).

$$\mu_0^{AIPW} = E[\frac{(1-W)(Y-\mu_0(X))}{1-\pi(X)} + \mu_0(X)] \tag{6.12}$$

The two working models below follow the Stata manual's `teffects aipw` example, and they use different covariates: prenatal care (`prenatal1`) enters only the outcome model, mother's education (`medu`) only the propensity model; both use marital status, mother's age and first-baby status. Double robustness needs a common adjustment set. Here we take the union of these five covariates as sufficient. Then each model is correct only if the covariate it leaves out is irrelevant to it: education to birth weight given the others, or prenatal care to smoking.

```
using CSV, DataFrames, GLM, Statistics

# Load Cattaneo (2010) birthweight dataset
# Treatment: mbsmoke (maternal smoking), Outcome: bweight (birthweight in grams)
data = CSV.read("data/cattaneo2.csv", DataFrame)

# Outcome Model (mu)
mu = lm(@formula(bweight ~ mbsmoke * (prenatal1 + mmarried + mage + fbaby)), data)

# Propensity Score Model (pi)
pi_model = glm(@formula(mbsmoke ~ mmarried + mage + fbaby + medu), data, Binomial(), LogitLink
```

```
# Predict Potential Outcomes for treated and control assignments
data_treated = copy(data)
data_treated.mbsmoke .= 1
mm1 = GLM.predict(mu, data_treated)

data_control = copy(data)
data_control.mbsmoke .= 0
mm0 = GLM.predict(mu, data_control)

# Predict Propensity Scores
pi1 = GLM.predict(pi_model, data)

# Assemble data for final AIPW computation
data.w = data.mbsmoke
data.y = data.bweight
data.m1 = mm1
data.m0 = mm0
data.pi1 = pi1

# Compute Doubly Robust potential outcomes
data.mu1 = @. (data.w * (data.y - data.m1) / data.pi1 + data.m1)
data.mu0 = @. ((1 - data.w) * (data.y - data.m0) / (1 - data.pi1) + data.m0)

# Calculate treatment effect
data.tau = data.mu1 .- data.mu0
tau_aipw = mean(data.tau)
@printf "ATE (AIPW) = %.4g\n" tau_aipw
```

```
ATE (AIPW) = -233.8
```

6.6.2. IPWRA

The idea of IPWRA is to combine RA and IPW. Basically RA with IPW weights.

Implementation:

- Estimate propensity score. Get predicted probability of treated for each observation.
- Estimate outcome model: two separate weighted equations, one for treated and one for control, using $1/\hat{\pi}(X)$ as weights for the treated group and $1/(1-\hat{\pi}(X))$ for the control group. (A single weighted regression with full treatment-covariate interactions would give numerically identical fits — the normal equations are block-diagonal — so the two-equation form is a convenience, not a requirement.)
- Get predicted values for setting everyone treated. Get predicted values for setting everyone control. These are the two potential outcomes.
- Take difference. The average is the ATE.

```julia
using CSV, DataFrames, GLM, Statistics

# Load Cattaneo (2010) birthweight dataset
data = CSV.read("data/cattaneo2.csv", DataFrame)

# 1. Estimate propensity score
pi_model = glm(@formula(mbsmoke ~ mmarried + mage + fbaby + medu), data, Binomial(), LogitLink
data.pi1 = GLM.predict(pi_model, data)

# Split data into treated and control
data1 = subset(data, :mbsmoke => x -> x .== 1)
data0 = subset(data, :mbsmoke => x -> x .== 0)

# 2. Estimate outcome models using inverse probability weights
wts1 = 1 ./ data1.pi1
wts0 = 1 ./ (1 .- data0.pi1)

# GLM.jl lm accepts weights via the `wts` keyword argument
mu1 = lm(@formula(bweight ~ prenatal1 + mmarried + mage + fbaby), data1; wts=wts1)
mu0 = lm(@formula(bweight ~ prenatal1 + mmarried + mage + fbaby), data0; wts=wts0)

# 3. Predict potential outcomes for the whole sample using the respective models
data_treated = copy(data)
data_treated.mbsmoke .= 1
mm1 = GLM.predict(mu1, data_treated)

data_control = copy(data)
data_control.mbsmoke .= 0
mm0 = GLM.predict(mu0, data_control)

# 4. Take the difference
data.w = data.mbsmoke
data.y = data.bweight
data.m1 = mm1
data.m0 = mm0

data.tau = data.m1 .- data.m0
tau_ipwra = mean(data.tau)
@printf "ATE (IPWRA) = %.4g\n" tau_ipwra
```

```
ATE (IPWRA) = -233.2
```

6.7. Summary

Under unconfoundedness and overlap, the adjustment formula turns the causal effect into an average of observed conditional means. There are four ways to estimate it. Regression adjustment models

the outcome and imputes the missing potential outcome. IPW models treatment and reweights the sample so that the covariates no longer predict treatment. AIPW and IPWRA use both models, and they are consistent if either one is correctly specified.

In the 401(k) data, linear RA with covariates de-meaned at the treated means gives an ATT of \$14,160 (1,397). The interactions matter because the effect grows with income and age. In the birth-weight data, AIPW gives −234 g and IPWRA −233 g. The two use the same nuisance models, so they differ only in how the models are combined.

7. Matching Estimators

```
using DataFrames
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

Matching is a way to make treated and control observations more comparable. For each treated observation, we look for controls with similar covariates. Then we compare outcomes in this matched sample.

This does not solve endogeneity. Matching only helps with selection on observables. If treatment is selected on unobserved variables, matching will not fix the problem. Its value is that it makes overlap visible.

Julia does not yet have the same matching ecosystem as R's `MatchIt`, `cobalt`, and `WeightIt`. Here I implement the basic pieces directly: propensity-score matching, balance diagnostics, IPW, and entropy balancing. For full matching, CEM, CBPS, energy balancing, and better balance plots, see the R companion chapter.

> **Related reading**: The Matching and Weighting Part 1 and Treatment effects and matching chapters of *Topics on Econometrics and Causal Inference* walk through matching with `MatchIt` and Stata's `teffects` commands respectively.

7.1. Assumptions

Matching needs the same assumptions as other selection-on-observables methods:

1. **SUTVA** — no interference, no hidden treatment versions.
2. **Ignorability** — conditional on X, $D \perp (Y(0), Y(1))$.
3. **Overlap** — every x has positive probability of both treatments.

7.2. A simulated example

```
Random.seed!(42)
n  = 2000
X1 = randn(n)
X2 = randn(n)

# Treatment depends on covariates (confounding)
ps_true = @. 1 / (1 + exp(-(-0.5 + 0.6 * X1 + 0.4 * X2)))
D       = Float64.(rand(n) .< ps_true)

# Outcome: true treatment effect = 1, plus confounding through X1, X2
Y = @. 1.0 * D + 0.8 * X1 + 0.6 * X2 + 0.5 * randn()

df = DataFrame(Y = Y, D = D, X1 = X1, X2 = X2)
@printf("Naive difference in means: %.3f\n", mean(Y[D .== 1]) - mean(Y[D .== 0]))
@printf("True ATE: 1.000\n")
```

```
Naive difference in means: 1.713
True ATE: 1.000
```

7.3. Estimating the propensity score

```
ps_fit = glm(@formula(D ~ X1 + X2), df, Binomial(), LogitLink())
df.ps  = predict(ps_fit)

@printf("PS range: [%.3f, %.3f]\n", minimum(df.ps), maximum(df.ps))
```

```
PS range: [0.060, 0.927]
```

7.4. Nearest-neighbour 1:1 matching

```
"""
    nearest_neighbor_match(ps_treated, ps_control)
Return a vector of indices into the control group, matching each treated
unit to its nearest-neighbour control on PS. Sampling is without replacement.
"""
function nearest_neighbor_match(ps_treated, ps_control)
    n_t = length(ps_treated)
    avail = trues(length(ps_control))
    matches = zeros(Int, n_t)
    for (i, p) in enumerate(ps_treated)
        candidates = findall(avail)
```

```
        if isempty(candidates)
            break
        end
        diffs = abs.(ps_control[candidates] .- p)
        best  = candidates[argmin(diffs)]
        matches[i] = best
        avail[best] = false
    end
    matches
end

idx_T = findall(df.D .== 1)
idx_C = findall(df.D .== 0)

matched_C = nearest_neighbor_match(df.ps[idx_T], df.ps[idx_C])
matched_idx_C = idx_C[matched_C[matched_C .> 0]]
matched_idx_T = idx_T[1:length(matched_idx_C)]

@printf("Matched %d treated to %d controls\n",
        length(matched_idx_T), length(matched_idx_C))
```

```
Matched 818 treated to 818 controls
```

7.5. Balance diagnostics

The standardized mean difference before and after matching tells us whether the matched groups are balanced:

```
function smd(x_t, x_c)
    (mean(x_t) - mean(x_c)) / sqrt((var(x_t) + var(x_c)) / 2)
end

@printf("%-15s %12s %12s\n", "Covariate", "SMD before", "SMD after")
for v in [:X1, :X2]
    smd_before = smd(df[!, v][df.D .== 1], df[!, v][df.D .== 0])
    smd_after  = smd(df[!, v][matched_idx_T], df[!, v][matched_idx_C])
    @printf("%-15s %12.3f %12.3f\n", String(v), smd_before, smd_after)
end
```

```
Covariate         SMD before    SMD after
X1                     0.614        0.270
X2                     0.394        0.152
```

Standardized mean differences below 0.1 are usually treated as acceptable. If they remain large, the matching is not doing enough.

7.6. Estimating the ATT on matched data

After matching, the ATT is the simple difference in means:

```
att = mean(df.Y[matched_idx_T]) - mean(df.Y[matched_idx_C])
@printf("Matched ATT: %.3f  (true ATE = 1.000)\n", att)

# Standard error: paired-difference SE (treats matched pairs as paired)
diffs = df.Y[matched_idx_T] .- df.Y[matched_idx_C]
se    = std(diffs) / sqrt(length(diffs))
@printf("Paired-diff SE: %.3f\n", se)
@printf("95%% CI: [%.3f, %.3f]\n", att - 1.96 * se, att + 1.96 * se)
```

```
Matched ATT: 1.287  (true ATE = 1.000)
Paired-diff SE: 0.032
95% CI: [1.225, 1.350]
```

The matched estimate is still visibly biased (compare it to the truth of 1.0). The balance table above says why: with 818 treated units and only 1,182 controls, 1:1 matching without replacement exhausts the good controls, and the post-matching standardized mean differences (0.27 and 0.15) remain above the 0.1 threshold. Residual imbalance means residual confounding. The with-replacement, IPW, and entropy-balancing estimates below, which are not constrained to use each control at most once, all land much closer to the truth. For matched pairs, the paired-difference standard error is the natural first check.

7.7. Matching with replacement

With replacement, the same control can be used several times. This can improve balance when good controls are scarce, but the effective sample size is smaller.

```
function nn_match_replace(ps_treated, ps_control)
    [argmin(abs.(ps_control .- p)) for p in ps_treated]
end

matched_repl = nn_match_replace(df.ps[idx_T], df.ps[idx_C])
@printf("Average reuse: %.2f (1.0 = no replacement)\n",
        length(matched_repl) / length(unique(matched_repl)))

att_repl = mean(df.Y[idx_T]) - mean(df.Y[idx_C[matched_repl]])
@printf("Matched-with-replacement ATT: %.3f\n", att_repl)
```

```
Average reuse: 1.76 (1.0 = no replacement)
Matched-with-replacement ATT: 0.988
```

7.8. Matching vs weighting

IPW is very close to matching in spirit. Instead of dropping or pairing observations, we reweight observations by the inverse probability of receiving their observed treatment.

```
# These are the ATE (Horvitz-Thompson/Hajek) weights: treated by 1/ps and
# controls by 1/(1-ps). They target the ATE, not the ATT. With a homogeneous
# +1 effect here, ATE = ATT numerically, but the estimand is the ATE.
w = ifelse.(df.D .== 1, 1 ./ df.ps, 1 ./ (1 .- df.ps))
w_trim = min.(w, quantile(w, 0.99))

ate_ipw = sum(df.Y[df.D .== 1] .* w_trim[df.D .== 1]) / sum(w_trim[df.D .== 1]) -
          sum(df.Y[df.D .== 0] .* w_trim[df.D .== 0]) / sum(w_trim[df.D .== 0])
@printf("IPW ATE:                %.3f\n", ate_ipw)
@printf("Matched ATT (NN):       %.3f\n", att)
@printf("Matched ATT (replace): %.3f\n", att_repl)
@printf("True ATE:               1.000\n")
```

```
IPW ATE:                1.064
Matched ATT (NN):       1.287
Matched ATT (replace): 0.988
True ATE:               1.000
```

When IPW and matching disagree sharply, it is usually a warning about the propensity-score model or lack of overlap.

7.9. Entropy balancing

Entropy balancing chooses weights so that the weighted control group has the same covariate means as the treated group. It does not require a propensity-score model. R's `WeightIt` package implements this and related methods. In Julia, the basic optimization is short enough to write directly.

The entropy-balancing problem for the ATT: find weights w_i on the control units that solve

$$\min_{w_i \geq 0} \sum_i w_i \log w_i \quad \text{subject to} \quad \sum_i w_i x_{ik} = \bar{x}_k^{\text{treated}} \;\; \forall k, \quad \sum_i w_i = 1. \tag{7.1}$$

The dual problem has one parameter per balance constraint:

$$\min_{\lambda} \;\; \log\left(\sum_i \exp(-x_i'\lambda)\right) + \lambda' \bar{x}^{\text{treated}}. \tag{7.2}$$

This is a small convex problem, so Newton's method works well here.

```
"""
    entropy_balance(X_control, X_treated; tol=1e-8, maxiter=50)
Compute entropy-balancing weights on the control units so that the
weighted control mean of every covariate equals the treated mean.
Returns a length-`size(X_control, 1)` vector of normalised weights that
sum to 1.
"""
function entropy_balance(X_control::AbstractMatrix, X_treated::AbstractMatrix;
                         tol::Float64 = 1e-8, maxiter::Int = 50)
    target = vec(mean(X_treated, dims=1))   # length p target means
    Xc = X_control                          # n_c × p
    p  = size(Xc, 2)
       = zeros(p)

    for _ in 1:maxiter
        z = -Xc *
        m = maximum(z)
        w = exp.(z .- m)
        w ./= sum(w)                                    # normalised weights
        mean_w = vec(w' * Xc)                           # weighted control mean
        grad   = target .- mean_w                       # negative gradient
        if maximum(abs.(grad)) < tol
            break
        end
        # Hessian: Σ w_i (x_i - mean_w)(x_i - mean_w)'  (with tiny ridge for stability)
        Xc_centered = Xc .- reshape(mean_w, 1, :)
        H = Matrix((Xc_centered .* w)' * Xc_centered) + 1e-6 .* Matrix{Float64}(I, p, p)
        # Newton step on the convex dual L( )=log Σexp(-x )+ ·target,
        # whose gradient is `grad`=target-mean_w and Hessian is H:   ←   - H ¹ grad
        step = H \ grad
          .-= step
    end

    z = -Xc *
    m = maximum(z)
    w = exp.(z .- m)
    w ./ sum(w)
end

# Apply to the simulated example from earlier sections
Xc_mat = Matrix(df[df.D .== 0, [:X1, :X2]])
Xt_mat = Matrix(df[df.D .== 1, [:X1, :X2]])

w_ebal = entropy_balance(Xc_mat, Xt_mat)
@printf("Sum of weights: %.6f  (should be 1.0)\n", sum(w_ebal))
@printf("Max weight: %.4f, min weight: %.6f\n",
        maximum(w_ebal), minimum(w_ebal))
```

```
# Verify the mean-balance constraints
control_means_unweighted = mean(Xc_mat, dims=1)
control_means_balanced   = w_ebal' * Xc_mat
treated_means            = mean(Xt_mat, dims=1)

@printf("\n%-15s %12s %12s %12s\n",
        "Covariate", "Treated", "Control (raw)", "Control (ebal)")
for (j, name) in enumerate([:X1, :X2])
    @printf("%-15s %12.4f %12.4f %12.4f\n",
            String(name),
            treated_means[j], control_means_unweighted[j],
            control_means_balanced[j])
end
```

```
Sum of weights: 1.000000  (should be 1.0)
Max weight: 0.0095, min weight: 0.000079

Covariate            Treated Control (raw) Control (ebal)
X1                    0.3117      -0.2801       0.3117
X2                    0.2395      -0.1541       0.2395
```

The weighted control means equal the treated means because the optimization imposes that constraint. If overlap is poor, the weights can become very uneven.

The ATT estimate uses these weights on the control outcomes:

```
Yt = df.Y[df.D .== 1]
Yc = df.Y[df.D .== 0]

# ATT = mean(Y_treated) - weighted mean(Y_control)
att_ebal = mean(Yt) - dot(w_ebal, Yc)
@printf("Entropy-balanced ATT: %.3f  (true ATE = 1.000)\n", att_ebal)
```

```
Entropy-balanced ATT: 1.014  (true ATE = 1.000)
```

Entropy balancing is useful when mean balance is the main diagnostic. If we need balance over whole covariate distributions, energy balancing in R's `WeightIt` is a better tool. There is no Julia equivalent yet.

7.10. Matching, weighting, and direct estimation

The estimation chapter estimated a treatment effect by regression adjustment, IPW and AIPW. This chapter has estimated one by matching, by IPW and by entropy balancing. Those are not two

families of estimators. They are one estimand and several ways of building it, and it is worth saying how they line up.

The ATT is

$$\tau_{ATT} = E[Y \mid D = 1] - E_{X|D=1}\big[\, E[Y \mid D = 0, X] \,\big]. \tag{7.3}$$

The first term is observed. The second is not: it is the mean outcome the treated would have had untreated, averaged over the covariate distribution of the treated. Every method here forms that second term, and every one of them ends up as a weighted average of control outcomes, $\sum_{i:D_i=0} w_i Y_i$. What differs is where the weights come from.

Route	What is modelled	Weight on control i
Regression adjustment	the outcome, $\mu_0(x) = E[Y \mid D = 0, X = x]$	implicit in the fitted values used to predict at treated x
IPW	the propensity score $\pi(x)$	the odds $\pi(x_i)/(1 - \pi(x_i))$
Matching	neither; a distance rule	0 if unmatched, 1 if matched
Entropy balancing	the weights themselves	whatever satisfies the balance constraints
AIPW	both	the odds, applied to the residual $Y - \hat{\mu}_0(X)$

Weighting is IPW. That is the second row, and the odds are what the IPW section above computed. Entropy balancing does not change the estimator, only how the weights are found: IPW fits π by maximum likelihood and transforms it, while entropy balancing skips the likelihood and solves for weights satisfying the balance constraints directly. Its solution has the form $w_i \propto \exp(x_i'\lambda)$, which is the odds of a logistic propensity score — so entropy balancing is IPW for a logit model whose coefficients are set by exact mean balance rather than by likelihood.

Matching is weighting too, with weights chosen by a distance rule rather than by a model. One-to-one matching without replacement gives each control a weight of zero or one; matching with replacement or full matching gives a control a weight equal to the number of times, or the share in which, it is used. The zero weights are where the trade-off lives: matching discards observations, while weighting keeps everyone and pays with extreme weights when overlap is poor. Discarding controls leaves an ATT target unchanged; discarding treated units, as calipers and trimming do, changes it. Matching that drops treated units gives stable estimates for a changed estimand; weighting keeps the estimand and has higher variance.

AIPW uses both columns: start from regression adjustment, then add the odds-weighted average of the control residuals $Y - \hat{\mu}_0(X)$. It is consistent if either model is right.

Five routes, one data set. Each row is built by hand from the weight representation so the arithmetic is visible.

```
trt = df.D .== 1

# Outcome model fitted on controls only, then predicted everywhere
mu0_fit = lm(@formula(Y ~ X1 + X2), df[.!trt, :])
```

```
mu0 = predict(mu0_fit, df)

# Odds weights from the logistic propensity score fitted above
odds   = df.ps ./ (1 .- df.ps)
wt_ipw = ifelse.(trt, 1.0, odds)

# Weighted difference in means: the common shape of rows 2 to 4
hajek(w) = sum(df.Y[trt] .* w[trt]) / sum(w[trt]) -
           sum(df.Y[.!trt] .* w[.!trt]) / sum(w[.!trt])

# Matching as 0/1 weights on the units the matcher kept
w_nn = zeros(nrow(df))
w_nn[matched_idx_T] .= 1.0
w_nn[matched_idx_C] .= 1.0

# Entropy-balancing weights, treated at 1
w_eb = zeros(nrow(df))
w_eb[trt] .= 1.0
w_eb[.!trt] .= w_ebal

att_ra   = mean(df.Y[trt] .- mu0[trt])
att_ipw  = hajek(wt_ipw)
att_eb   = hajek(w_eb)
att_nn   = hajek(w_nn)
att_aipw = att_ra - sum((df.Y[.!trt] .- mu0[.!trt]) .* odds[.!trt]) / sum(odds[.!trt])

DataFrame(
    route = ["Regression adjustment (outcome model)",
             "IPW (propensity model)",
             "Entropy balancing (balance constraints)",
             "1:1 matching, no replacement (distance rule)",
             "AIPW (both models)"],
    ATT = round.([att_ra, att_ipw, att_eb, att_nn, att_aipw], digits = 3))
```

	route	ATT
	String	Float64
1	Regression adjustment (outcome model)	1.009
2	IPW (propensity model)	1.014
3	Entropy balancing (balance constraints)	1.014
4	1:1 matching, no replacement (distance rule)	1.287
5	AIPW (both models)	1.014

Two of these confirm the algebra directly. Expressing the matched comparison as a weighted mean with zero-one weights reproduces the 1.287 printed earlier, which is what "matching is a weighting estimator" means concretely. And entropy balancing returns 1.014, the same figure the entropy-balancing section printed, since that section computed the same weighted difference.

The spread is the part worth reading. Four of the five routes land between 1.009 and 1.014 against a

true effect of 1.000. They agree because this DGP gives them every reason to: the treatment effect is constant at 1, so there is no heterogeneity for differing weights to average differently, and the propensity score runs from 0.060 to 0.927, so no unit carries an extreme weight. Under those two conditions the routes are estimating the same number and differ only by noise. The R companion chapter runs the same five routes on the Lalonde data, where effects are heterogeneous and overlap is poor, and there they spread from $1,214 to $1,855.

The exception here is instructive. One-to-one matching without replacement gives 1.287, nearly 30% high. It leaves 364 of the 1,182 controls unused, but unused controls do not by themselves bias an ATT: the treated units, and so the target, are unchanged. The bias comes from match quality. Without replacement, once the best controls are taken, later treated units get worse matches and residual imbalance remains. Matching with replacement, which reuses good controls, gave 0.988 earlier in the chapter. That gap is the cost of the poorer matches, not of matching as an idea.

These are point estimates, and standard errors are not comparable across the rows: matching is not a smooth functional of the data, so the bootstrap is invalid for it, while IPW and AIPW have influence-function variances and AIPW attains the semiparametric efficiency bound that fixed-M matching does not.

7.11. When to use which method

Method	When to use	Limitation
NN matching (no replacement)	Simple, intuitive; ATT estimand	Drops unmatched units
NN matching (with replacement)	Few treated, many controls	Wasteful of controls
IPW	Good propensity model	Sensitive to extreme weights
Entropy balancing	Want exact mean balance; small covariate set	Balances means only, not distributions
Doubly robust (AIPW)	Combine matching/IPW with regression	More complex

For applied work in Julia, IPW or AIPW is usually more practical than 1:1 matching. Julia has good regression tools, but not yet a full matching toolkit. For a matched analysis with many diagnostics, I would still use R's `MatchIt` and `cobalt`.

7.12. Summary

- Matching is for selection on observables. It does not fix unobserved confounding.
- The main diagnostic is balance, not the treatment-effect coefficient.
- Propensity-score nearest-neighbor matching is easy to implement in Julia, but full matching and CEM are better handled in R.
- IPW and entropy balancing are often more practical Julia workflows.

8. Nonparametric Causal Methods

```
using DataFrames
using Distributions
using Random
using Statistics
using MLJ
using EvoTrees
using MLJLinearModels
using DecisionTree
using CausalEstimate
import CausalEstimate: estimate, confint, pvalue
using Printf
```

> **Related reading**: For an R-based TMLE simulation that compares super-learner libraries and stress-tests model misspecification, see the companion blog chapter on TMLE in *Topics on Econometrics and Causal Inference*.

8.1. Why Do We Need Nonparametric Methods?

- Parametric models are often mis-specified
- Nonparametric models are more flexible; less assumptions
- What kind of nonparametrics are good?

Considerations:

- Plug-in (g-computation) estimators are easy.
- The problem is not the decomposition into $E[Y(0)]$ and $E[Y(1)]$ itself: with correctly specified models and regularity, a plug-in estimator can be efficient. The problem appears once the nuisance models are flexible and regularised. Then the error in the nuisance estimate enters the plug-in target *linearly*, and the plug-in has no term that cancels it. An influence-function estimator adds exactly such a term, and the error that survives is a product of two nuisance errors rather than one. Section 8.2.1 does the algebra.
- So they can be consistent, but the convergence rate inherits the (often slow) nuisance rate, which is especially bad in high dimensions.
- Ideally we remove that first-order sensitivity using an orthogonal influence function, and get $\sqrt{n}$ efficiency.
- Influence-function-based (one-step/AIPW/TMLE) estimators are shown to be efficient and orthogonal to first-order nuisance error.

8.2. Influence Function Based Estimators

Estimators based on efficient influence function are often $\sqrt{n}$ consistent, meaning they converge to the truth as fast as $\sqrt{n}$ rate. This means the bias not only goes to 0 with n goes to infinity, but also as fast as $1/\sqrt{n}$ goes to 0. Why does this matter? Because we have limited sample size, when we are based on asymptotics, the faster the bias goes to 0 the smaller sample size we need.

If we have an estimator such that

$$\sqrt{n}(\hat{\psi} - \psi) = \sqrt{n}P_n(\phi) + o_P(1) \to N(0, var(\phi)) \tag{8.1}$$

Then this estimator is $\sqrt{n}$ consistent and asymptotically normal; and ϕ is the influence function. The efficient influence function is the one with variance at the lower bound (similar to parametric case for Cramer-Rao lower bound).

Unfortunately there is no universal formula for efficient influence function for every problem. We need to derive it for each problem. For example, ATE has an IF based estimator, but ATT needs a different formula; and LATE has a different formula too, etc. Deriving them is hard!

For the treated potential-outcome mean, $\psi_1 = E\{E(Y \mid X, A = 1)\}$, the efficient influence function is

$$\phi_1(Z; P) = \frac{A}{\pi(X)}\{Y - \mu_1(X)\} + \mu_1(X) - \psi_1 \tag{8.2}$$

where $\pi(X) = P(A = 1|X)$ is the propensity score model and $\mu_1(X) = E(Y|X, A = 1)$ is the treated outcome model. The ATE influence function is the difference between the treated and untreated components, $\phi_1 - \phi_0$.

For example, an IF-based bias-corrected estimator for the treated-arm mean

$$\hat{\psi}_1 = P_n\left[\frac{A}{\hat{\pi}(X)}\{Y - \hat{\mu}_1(X)\} + \hat{\mu}_1(X)\right] \tag{8.3}$$

where P_n means sample mean. This is the familiar AIPW estimator. It is known to be doubly robust.

Since we know the influence function, we can calculate its variance. Under asymptotic linearity, $\hat{\psi} - \psi \approx \frac{1}{n}\sum_i \phi(Z_i)$, so the variance of the estimator is approximately $\mathrm{Var}(\phi)/n$; the influence function and the estimator do not have the same variance. We estimate the nuisance parameters, form $\hat{\psi}$, and use the sample variance of the estimated influence values divided by n. This needs more than cross-fitting: the nuisance errors must shrink fast enough that their product is $o_p(n^{-1/2})$, and overlap must hold. Cross-fitting handles the separate empirical-process term, so that flexible learners need no complexity restrictions.

In this estimator, both $\hat{\mu}$ and $\hat{\pi}$ can be done using any estimator, such as machine learning. One caveat: with adaptive learners (random forests, boosting), the nuisance fits should be *cross-fit* — predicted for each observation from a model trained without it — otherwise own-observation overfitting can bias the estimate. The manual AIPW below reuses full-sample fits to keep the code short; the `CausalEstimate.jl` calls use `crossfit=5`.

8.2.1. Why the Plug-In Is Off

The bullets above claimed that the plug-in inherits the nuisance error at first order and that the augmentation term removes it. That is algebra, not a simulation finding, so it is worth doing before we run anything.

Take $\psi_1 = E[\mu_1(X)]$ with $\mu_1(x) = E(Y \mid X = x, A = 1)$. The plug-in estimator is $\hat{\psi}_1^{\text{pi}} = P_n[\hat{\mu}_1(X)]$. Split its error into a sampling part and a nuisance part:

$$\hat{\psi}_1^{\text{pi}} - \psi_1 = \underbrace{(P_n - P)[\mu_1]}_{O_P(n^{-1/2})} + \underbrace{P[\hat{\mu}_1 - \mu_1]}_{\text{nuisance error, linear}} + \underbrace{(P_n - P)[\hat{\mu}_1 - \mu_1]}_{\text{smaller still}}. \tag{8.4}$$

The middle term is $E[\hat{\mu}_1(X) - \mu_1(X)]$, and it is linear in the nuisance error. If $\hat{\mu}_1$ converges at rate $n^{-\alpha}$, that term is of order $n^{-\alpha}$, and a regularised learner in even a moderate number of dimensions gives $\alpha < 1/2$. The nuisance error then dominates the sampling error, the estimator is not $\sqrt{n}$ consistent, and nothing in the plug-in cancels it.

Now do the same for the AIPW estimator of Equation 8.3. Evaluate it under the population distribution, holding the estimated nuisances fixed. Because $E[A\{Y - \hat{\mu}_1(X)\} \mid X] = \pi(X)\{\mu_1(X) - \hat{\mu}_1(X)\}$,

$$P\Big[\frac{A}{\hat{\pi}(X)}\{Y - \hat{\mu}_1(X)\} + \hat{\mu}_1(X)\Big] = \psi_1 + E\Big[\frac{\pi(X) - \hat{\pi}(X)}{\hat{\pi}(X)}\{\mu_1(X) - \hat{\mu}_1(X)\}\Big]. \tag{8.5}$$

The bias is a *product* of the two nuisance errors, and that is the entire difference between the two estimators. The term linear in $\hat{\mu}_1 - \mu_1$ has cancelled: the $\hat{\mu}_1$ that the outcome model carries in is subtracted back out by the $\hat{\mu}_1$ sitting inside the augmentation, and what survives is multiplied by the propensity-score error. By Cauchy-Schwarz the bias is bounded by $\|\pi - \hat{\pi}\| \, \|\mu_1 - \hat{\mu}_1\|$ up to the overlap constant, so rates of $o(n^{-1/4})$ on each nuisance suffice to make it $o(n^{-1/2})$ — slower than $\sqrt{n}$ on each piece, fast enough for $\sqrt{n}$ on the target.

Two familiar facts fall out of Equation 8.5 directly. Double robustness is immediate: if either factor is zero the whole bias is zero, which is why one correct model is enough. And the $n^{-1/4}$ requirement is the reason cross-fitting matters, since reusing the same observations to fit and to evaluate $\hat{\mu}_1$ makes the product term behave worse than the rate calculation assumes.

8.2.2. Simulation Setup

```
Random.seed!(1)
n = 1000
w1 = rand(Binomial(1, 0.5), n)
w2 = rand(Binomial(1, 0.5), n)
w3 = round.(rand(Uniform(0, 4), n), digits=3)
w4 = round.(rand(Uniform(0, 5), n), digits=3)

logistic(x) = 1.0 ./ (1.0 .+ exp.(-x))   # inverse logit (sigmoid), not the logit
```

```
prob_A = logistic.(-0.4 .+ 0.2.*w2 .+ 0.15.*w3 .+ 0.2.*w4 .+ 0.15.*w2.*w4)
A = rand.(Binomial.(1, prob_A))

prob_Y1 = logistic.(-1.0 .+ 1.0 .- 0.1.*w1 .+ 0.3.*w2 .+ 0.25.*w3 .+ 0.2.*w4 .+ 0.15.*w2.*w4)
prob_Y0 = logistic.(-1.0 .+ 0.0 .- 0.1.*w1 .+ 0.3.*w2 .+ 0.25.*w3 .+ 0.2.*w4 .+ 0.15.*w2.*w4)

Y_1 = rand.(Binomial.(1, prob_Y1))
Y_0 = rand.(Binomial.(1, prob_Y0))
Y = Y_1 .* A .+ Y_0 .* (1 .- A)

W = DataFrame(w1=w1, w2=w2, w3=w3, w4=w4)
df = DataFrame(w1=w1, w2=w2, w3=w3, w4=w4, A=A, Y=Y, Y_1=Y_1, Y_0=Y_0)

True_Psi = mean(df.Y_1 .- df.Y_0)
@printf "True ATE = %.4g\n" True_Psi
```

```
True ATE = 0.184
```

The line above reports `mean(Y.1 - Y.0)` for this particular draw. That is a legitimate quantity — the *sample* average treatment effect — but it is not a fixed benchmark, and it is easy to mistake for one. The potential outcomes are binary, so at $n = 1000$ this realized average has a sampling standard deviation of about 0.020: two runs of the same DGP under different seeds can differ by 0.04 without anything being wrong. (The R and Julia editions of this chapter print 0.21 and 0.184 for exactly this reason.) Estimator standard errors in this chapter are around 0.03, so a noisy target of that size would swamp the comparison we are trying to make.

The *population* ATE is a definite number, and since the DGP is fully specified we can integrate it out rather than simulate it:

```
# w1, w2 ~ Bernoulli(0.5); w3 ~ U(0,4); w4 ~ U(0,5). Average the true individual
# effect over that covariate distribution on a trapezoid grid.
function population_ate()
    g3 = range(0, 4; length = 1601)
    g4 = range(0, 5; length = 1601)
    trap(g) = (w = fill(step(g), length(g)); w[1] /= 2; w[end] /= 2; w)
    w3w, w4w = trap(g3), trap(g4)
    total = 0.0
    for w1 in (0, 1), w2 in (0, 1)
        acc = 0.0
        for (j, u4) in enumerate(g4), (i, u3) in enumerate(g3)
            lin0 = -1 - 0.1w1 + 0.3w2 + 0.25u3 + 0.2u4 + 0.15w2 * u4
            acc += w3w[i] * w4w[j] * (logistic(lin0 + 1) - logistic(lin0))
        end
        total += acc / (4 * 5)
    end
    total / 4
end
```

```
pop_ate = population_ate()
@printf "Realized-sample ATE (this draw): %.4f\n" True_Psi
@printf "Exact population ATE:           %.4f\n" pop_ate
```

```
Realized-sample ATE (this draw): 0.1840
Exact population ATE:           0.2028
```

Compare the estimators below against 0.2028, not against the realized draw.

```
# Load MLJ models to form our equivalent of SuperLearner
LogisticClassifier = @load LogisticClassifier pkg=MLJLinearModels verbosity=0
EvoTreeClassifier = @load EvoTreeClassifier pkg=EvoTrees verbosity=0
RandomForestClassifier = @load RandomForestClassifier pkg=DecisionTree verbosity=0

# Define a Stack (SuperLearner)
stack = Stack(
    metalearner = LogisticClassifier(),
    resampling = CV(nfolds=5),
    evo = EvoTreeClassifier(max_depth=4, nrounds=100),
    rf = RandomForestClassifier(n_trees=100, max_depth=5),
    glm = LogisticClassifier()
)
```

```
ProbabilisticStack(
  metalearner = LogisticClassifier(
        lambda = 2.220446049250313e-16,
        gamma = 0.0,
        penalty = :l2,
        fit_intercept = true,
        penalize_intercept = false,
        scale_penalty_with_samples = true,
        solver = nothing),
  resampling = CV(
        nfolds = 5,
        shuffle = false,
        rng = TaskLocalRNG()),
  measures = nothing,
  cache = true,
  acceleration = CPU1{Nothing}(nothing),
  evo = EvoTreeClassifier(
        loss = :mlogloss,
        metric = :mlogloss,
        nrounds = 100,
        bagging_size = 1,
        early_stopping_rounds = 9223372036854775807,
```

```
        L2 = 1.0,
        lambda = 0.0,
        gamma = 0.0,
        eta = 0.1,
        max_depth = 4,
        min_weight = 1.0,
        rowsample = 1.0,
        colsample = 1.0,
        nbins = 64,
        tree_type = :binary,
        seed = 123,
        device = :cpu),
  rf = RandomForestClassifier(
        max_depth = 5,
        min_samples_leaf = 1,
        min_samples_split = 2,
        min_purity_increase = 0.0,
        n_subfeatures = -1,
        n_trees = 100,
        sampling_fraction = 0.7,
        feature_importance = :impurity,
        rng = TaskLocalRNG()),
  glm = LogisticClassifier(
        lambda = 2.220446049250313e-16,
        gamma = 0.0,
        penalty = :l2,
        fit_intercept = true,
        penalize_intercept = false,
        scale_penalty_with_samples = true,
        solver = nothing))
```

8.2.3. Plug-in Estimator from Outcome Regression

```
Random.seed!(1)
X_mu = select(df, :w1, :w2, :w3, :w4, :A)
mach_mu = machine(stack, X_mu, categorical(df.Y))
MLJ.fit!(mach_mu, verbosity=0)

# predict the PO Y^1
X_mu1 = copy(X_mu); X_mu1.A .= 1
X_mu0 = copy(X_mu); X_mu0.A .= 0

# Predict probability of Y=1 (assuming 1 is the positive class)
mu1hat = pdf.(MLJ.predict(mach_mu, X_mu1), 1)
mu0hat = pdf.(MLJ.predict(mach_mu, X_mu0), 1)
```

```
plug_in_ate = mean(mu1hat .- mu0hat)
@printf "ATE (plug-in) = %.4g\n" plug_in_ate
```

```
ATE (plug-in) = 0.1759
```

8.2.4. AIPW ATE

```
Random.seed!(123)
X_pi = select(df, :w1, :w2, :w3, :w4)
mach_pi = machine(stack, X_pi, categorical(df.A))
MLJ.fit!(mach_pi, verbosity=0)

pi1hat = pdf.(MLJ.predict(mach_pi, X_pi), 1)

# Bound propensity scores to avoid extreme weights
pi1hat = max.(min.(pi1hat, 0.999), 0.001)
pi0hat = 1.0 .- pi1hat

# Current prediction on observed data
mu1hat_obs = mu1hat
mu0hat_obs = mu0hat

IF_1 = ((df.A ./ pi1hat) .* (df.Y .- mu1hat_obs) .+ mu1hat_obs)
IF_0 = (((1 .- df.A) ./ pi0hat) .* (df.Y .- mu0hat_obs) .+ mu0hat_obs)
IF = IF_1 .- IF_0

aipw_1 = mean(IF_1); aipw_0 = mean(IF_0)
aipw_manual = aipw_1 - aipw_0

ci_lb = mean(IF) - quantile(Normal(), 0.975) * std(IF) / sqrt(length(IF))
ci_ub = mean(IF) + quantile(Normal(), 0.975) * std(IF) / sqrt(length(IF))
res_manual_aipw = [aipw_manual, ci_lb, ci_ub]
@printf "ATE (AIPW)    = %.4g\n" aipw_manual
@printf "95%% CI        = [%.4g, %.4g]\n" ci_lb ci_ub
```

```
ATE (AIPW)    = 0.1918
95% CI        = [0.1311, 0.2525]
```

8.2.5. AIPW ATE Using CausalEstimate.jl

```
using CausalEstimate
```

```
r(x) = round(x, sigdigits=4)

psi_ate = ATE(outcome=:Y, treatment=:A, confounders=[:w1, :w2, :w3, :w4])
aipw_result = estimate(psi_ate, AIPW(crossfit=5), df)

(
    ATE    = r(estimate(aipw_result)),
    CI     = r.(confint(aipw_result)),
    pvalue = r(pvalue(aipw_result)),
)
```

```
(ATE = 0.2072, CI = (0.1206, 0.2939), pvalue = 2.776e-6)
```

8.2.6. Regression-imputation ATT (manual)

Note that the manual estimator below is evaluated on the **treated subset** `df1`, where `(1 .- df1.A)` is identically zero. The augmentation term `((1-A)/ )(Y- )` therefore vanishes and the estimator collapses to the regression-imputation ATT, `mean(Y_treated - ̂ (X_treated))`. That is consistent if the control outcome model is correct, but it is **not** the doubly-robust / augmented ATT, and the influence-function SE understates uncertainty because it ignores estimation of and . The proper ATT efficient influence function (Hahn 1998) is `(1/p)[A(Y- - ) - (1-A)· /(1- )·(Y- )]`, with the ATT, evaluated over the **full** sample; without the `-` it is the score, whose mean is . The `CausalEstimate.jl` call in the next subsection computes the cross-fit, doubly-robust version.

```
Random.seed!(1)
df2 = copy(df)
df2.pi0hat = pi0hat
df2.pi1hat = pi1hat

df0 = filter(row -> row.A == 0, df2)
df1 = filter(row -> row.A == 1, df2)

# Fit outcome model only on controls
X_mu0 = select(df0, :w1, :w2, :w3, :w4)
mach_mu0 = machine(stack, X_mu0, categorical(df0.Y))
MLJ.fit!(mach_mu0, verbosity=0)

# Predict counterfactual for treated
X_mu1 = select(df1, :w1, :w2, :w3, :w4)
mu0hat_treated = pdf.(MLJ.predict(mach_mu0, X_mu1), 1)

IF_0_att = (((1 .- df1.A) ./ df1.pi0hat) .* (df1.Y .- mu0hat_treated) .+ mu0hat_treated)
IF_att = df1.Y .- IF_0_att

aipw_manual_att = mean(IF_att)
ci_lb_att = mean(IF_att) - quantile(Normal(), 0.975) * std(IF_att) / sqrt(length(IF_att))
```

```
ci_ub_att = mean(IF_att) + quantile(Normal(), 0.975) * std(IF_att) / sqrt(length(IF_att))
res_manual_aipw_att = [aipw_manual_att, ci_lb_att, ci_ub_att]
res_manual_aipw_att
```

```
3-element Vector{Float64}:
 0.20145553037952954
 0.1700985774823039
 0.2328124832767552
```

8.2.7. AIPW ATT Using CausalEstimate.jl

```
psi_att = ATT(outcome=:Y, treatment=:A, confounders=[:w1, :w2, :w3, :w4])
att_result = estimate(psi_att, AIPW(crossfit=5), df)

(
    ATT    = r(estimate(att_result)),
    CI     = r.(confint(att_result)),
    pvalue = r(pvalue(att_result)),
)
```

```
(ATT = 0.189, CI = (0.06404, 0.3139), pvalue = 0.00303)
```

8.3. TMLE

TMLE is a variant of IF based estimator. It is also a doubly robust estimator. See Mark van der Laan and his coauthors for the recent development of TMLE.

We should expect AIPW and TMLE to give similar answers.

```
Random.seed!(42)
tmle_result = estimate(psi_ate, TMLE(crossfit=5), df)

(
    TMLE   = r(estimate(tmle_result)),
    CI     = r.(confint(tmle_result)),
    pvalue = r(pvalue(tmle_result)),
)
```

```
(TMLE = 0.1775, CI = (0.08638, 0.2687), pvalue = 0.0001349)
```

```
@printf "Population ATE (exact):    %.4f\n" pop_ate
@printf "Realized-sample ATE:       %.4f\n" True_Psi
```

```
Population ATE (exact):    0.2028
Realized-sample ATE:       0.1840
```

8.4. DoubleML

DoubleML (Double/Debiased Machine Learning) was proposed by Chernozhukov et al. (2018). An R package `DoubleML` implements the framework; as of this writing there is no native Julia package, but the algorithm is straightforward to implement directly. The double machine learning framework consists of three key ingredients: Neyman orthogonality, high-quality machine learning estimation, and sample splitting.

They consider a partially linear model:

$$y_i = \theta d_i + g_0(x_i) + \eta_i \tag{8.6}$$

$$d_i = m_0(x_i) + v_i \tag{8.7}$$

This model is quite general, except it does not allow interaction of d and x; therefore no heterogeneous treatment effect across x. But "DoubleML" implements more than partially linear model, it actually allows for heterogeneous treatment effects, in models such as interactive regression model.

The basic idea of doubleML is to use any machine learning algorithm to estimate outcome equation ($l_0(X) = E(Y|X)$) and treatment equation ($m_0(X) = E(D|X)$). Then get the residuals, namely $\hat{W} = Y - \hat{l}_0(X)$ and $\hat{V} = D - \hat{m}_0(X)$.

Then regress $\hat{W}$ on $\hat{V}$. Based on FWL theorem, you get $\hat{\theta}$.

An important component here is to specify a Neyman-orthogonal score function. The *partialling-out* form, which is what the residual-on-residual regression below implements, is

$$\psi(W;\theta,\eta) = (Y - l(X) - \theta\,(D - m(X)))\,(D - m(X)), \tag{8.8}$$

with $l(X) = E[Y \mid X]$ and $m(X) = E[D \mid X]$. The estimator $\hat{\theta}$ solves the equation that the sample mean of this score function is 0.

The estimator's variance is the sandwich $J^{-1}\,E[\psi^2]\,J^{-1}$ with Jacobian $J = E[\partial\psi/\partial\theta] = -E[(D - m(X))^2]$ — this is what the code computes by dividing the score variance by $\left(\frac{1}{n}\sum \hat{V}_i^2\right)^2$.

```
using MLJ
using Random

# We implement DoubleML PLR (Partially Linear Regression) manually with cross-fitting
Random.seed!(123)
nsplits = 5
n_samples = nrow(df)
s = shuffle(1:n_samples)
folds = [s[1 + floor(Int, (k-1)*n_samples/nsplits) : floor(Int, k*n_samples/nsplits)] for k in

# Base learner (Random Forest for regression)
RandomForestRegressor = @load RandomForestRegressor pkg=DecisionTree verbosity=0
```

```
learner = RandomForestRegressor(n_trees=500, max_depth=5)

W_res = zeros(n_samples)
V_res = zeros(n_samples)

for fold in 1:nsplits
    test_idx = folds[fold]
    train_idx = setdiff(1:n_samples, test_idx)

    # Fit outcome model l_0(X) = E(Y | X)
    mach_l = machine(learner, W[train_idx, :], df.Y[train_idx])
    MLJ.fit!(mach_l, verbosity=0)
    y_pred = MLJ.predict(mach_l, W[test_idx, :])
    W_res[test_idx] = df.Y[test_idx] .- y_pred

    # Fit treatment model m_0(X) = E(D | X)
    mach_m = machine(learner, W[train_idx, :], df.A[train_idx])
    MLJ.fit!(mach_m, verbosity=0)
    a_pred = MLJ.predict(mach_m, W[test_idx, :])
    V_res[test_idx] = df.A[test_idx] .- a_pred
end
```

```
# FWL theorem: regress residual W_res on residual V_res
theta_hat = sum(W_res .* V_res) / sum(V_res .^ 2)

# Variance of score function (Neyman-orthogonal score)
psi = (W_res .- theta_hat .* V_res) .* V_res
var_theta = var(psi) / (mean(V_res .^ 2)^2)
se_theta = sqrt(var_theta / n_samples)

ci_lb = theta_hat - 1.96 * se_theta
ci_ub = theta_hat + 1.96 * se_theta
@printf "DML PLR Estimate = %.4g (SE = %.4g)\n" theta_hat se_theta
@printf "95%% CI           = [%.4g, %.4g]\n" ci_lb ci_ub
```

```
DML PLR Estimate = 0.192 (SE = 0.03206)
95% CI           = [0.1292, 0.2549]
```

8.5. Summary

Flexible machine-learning nuisance models avoid having to guess a functional form, but a plug-in estimator built on them carries their regularisation bias into the average. Estimators based on the efficient influence function remove that bias to first order. AIPW, TMLE and DoubleML all do this. What remains is a product of the outcome-model error and the propensity-score error,

so each nuisance needs to converge only faster than $n^{-1/4}$. Cross-fitting keeps the remaining empirical-process term small.

In the simulation the population ATE is 0.2028. The plug-in gives 0.1759, with no standard error. Cross-fitted AIPW from `CausalEstimate.jl` gives 0.2072, TMLE 0.1775 and DoubleML 0.192 (0.032). The last targets a variance-weighted average of conditional effects, which equals the ATE only when the effect is constant.

9. Heterogeneous Treatment Effects with Machine Learning

```
using DataFrames
using Distributions
using Random
using Statistics
using LinearAlgebra
using Printf
using MLJ
using MLJDecisionTreeInterface
using GLM
using CairoMakie
CairoMakie.activate!(type = "png")
```

The estimands chapter defined the conditional average treatment effect

$$\mathrm{CATE}(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x] \tag{9.1}$$

as the effect of treatment for units with covariates $X = x$. In simulation we can compute CATE from known potential outcomes. In real data we observe only one potential outcome per unit, so CATE has to be estimated from (X, D, Y).

Here I focus on meta-learners: recipes that turn a regression method into a CATE estimator. I implement S-, T-, X-, R-, and DR-learners using `MLJ` and `DecisionTree.jl` random forests.

Julia does not currently have an equivalent of R's `grf::causal_forest` with honest sample splitting and CATE confidence intervals. For that workflow, use the R companion chapter.

9.1. The data-generating process

I use the same CATE function as the estimands chapter, $\tau(x) = 1 + 2x_1$ (the rest of the DGP differs: five covariates and confounding through the observed X_2).

```
Random.seed!(42)
n = 5000
p = 5
```

```
X    = rand(n, p)
# Treatment depends on the OBSERVED covariate X2 (a backdoor confounder we
# adjust for), so unconfoundedness given X holds and the estimators below are
# consistent for the true CATE/ATE.
ps   = @. 1 / (1 + exp(-(-0.3 + 1.5 * X[:, 2])))
D    = Float64.(rand(n) .< ps)
tau  = @. 1 + 2 * X[:, 1]
Y0   = @. 0.5 * X[:, 2] + randn()
Y1   = Y0 .+ tau
Y    = ifelse.(D .== 1, Y1, Y0)

@printf("n = %d, true ATE = %.3f\n", n, mean(tau))
@printf("True CATE at X1 = 0.2: %.2f\n", 1 + 2 * 0.2)
@printf("True CATE at X1 = 0.8: %.2f\n", 1 + 2 * 0.8)
```

```
n = 5000, true ATE = 2.009
True CATE at X1 = 0.2: 1.40
True CATE at X1 = 0.8: 2.60
```

Treatment is related to the observed covariate X_2, which also affects the outcome — a backdoor confounder. Because X_2 is observed and in the conditioning set, unconfoundedness and overlap identify the CATE and the ATE. Whether a fitted learner gets close also depends on the learner, its tuning and the sample size.

9.2. A wrapper for fitting random forests

To keep the code short, define a helper that fits a random forest and returns predictions.

```
const RFR = @load RandomForestRegressor pkg=DecisionTree verbosity=0

"""
    rf_fit_predict(Xtrain, ytrain, Xpredict; n_trees=500, weights=nothing)
Fit a random forest on (Xtrain, ytrain) and return predictions at Xpredict.
DecisionTree's `RandomForestRegressor` does not accept per-sample weights, so
when `weights` are supplied we approximate a weighted fit by resampling the
training rows with probability proportional to the weights (weighted bootstrap).
"""
function rf_fit_predict(Xtrain, ytrain, Xpredict; n_trees::Int=500,
                        weights=nothing)
    if weights !== nothing
        w = Float64.(weights)
        m = size(Xtrain, 1)
        cdf = cumsum(w) ./ sum(w)            # weighted-bootstrap CDF
        idx = [searchsortedfirst(cdf, rand()) for _ in 1:m]
        Xtrain = Xtrain[idx, :]
```

```
        ytrain = ytrain[idx]
    end
    learner = RFR(n_trees=n_trees, max_depth=-1)
    Xtrain_t = MLJ.table(Xtrain)
    Xpred_t  = MLJ.table(Xpredict)
    mach = machine(learner, Xtrain_t, Float64.(ytrain))
    fit!(mach, verbosity=0)
    return MLJ.predict(mach, Xpred_t)
end

"""
    rf_oof(Xtrain, ytrain, Xpredict, folds; n_trees=500, weights=nothing, rows=...)

Out-of-fold version of `rf_fit_predict`. For each fold `k`, fit on the rows *not*
in `k` and predict only the fold-`k` rows of `Xpredict`. Restrict the training
rows further with `rows` (used below to fit an arm-specific model while still
predicting for everybody).

Every CATE estimate in the meta-learner comparison below goes through this
function rather than `rf_fit_predict`, and the reason matters. A random forest asked to predict
same rows it was trained on partly reproduces its own training targets, so an
in-sample CATE is contaminated by the noise in whatever pseudo-outcome it was fit
to. That inflates apparent accuracy for learners whose target is close to `Y`
and destroys it for learners whose target is a high-variance pseudo-outcome --
which is exactly the DR- and R-learners. It also makes any downstream *ranking*
on the estimate invalid; see the GATES section for what that does.
"""
function rf_oof(Xtrain, ytrain, Xpredict, folds; n_trees::Int=500,
                weights=nothing, rows=trues(length(folds)))
    out = zeros(size(Xpredict, 1))
    for k in unique(folds)
        train = (folds .!= k) .& rows
        test  = folds .== k
        w = weights === nothing ? nothing : Float64.(weights)[train]
        out[test] = rf_fit_predict(Xtrain[train, :], ytrain[train],
                                   Xpredict[test, :]; n_trees=n_trees, weights=w)
    end
    return out
end

# One fold assignment, shared by every learner below so that the comparison
# across learners is like-for-like.
Random.seed!(7)
folds = rand(1:5, n)
nothing
```

9.3. S-learner ("single")

Fit one regression of Y on (D, X), then predict the difference between $D = 1$ and $D = 0$ for each x:

```
X_with_D = hcat(D, X)
X1_test  = hcat(ones(n), X)
X0_test  = hcat(zeros(n), X)

mu1_S = rf_oof(X_with_D, Y, X1_test, folds)
mu0_S = rf_oof(X_with_D, Y, X0_test, folds)
tau_S = mu1_S .- mu0_S

@printf("S-learner CATE correlation with truth: %.3f\n", cor(tau_S, tau))
```

```
S-learner CATE correlation with truth: 0.884
```

The S-learner is simple. Its weakness is that the model may treat D as a minor predictor and shrink treatment effects toward zero.

9.4. T-learner ("two")

Fit two separate regressions, one on treated units and one on controls:

```
idx_T = D .== 1
idx_C = D .== 0

mu1_T = rf_oof(X, Y, X, folds; rows = idx_T)   # treated-arm model, predicted for all
mu0_T = rf_oof(X, Y, X, folds; rows = idx_C)   # control-arm model, predicted for all
tau_T = mu1_T .- mu0_T

@printf("T-learner CATE correlation with truth: %.3f\n", cor(tau_T, tau))
```

```
T-learner CATE correlation with truth: 0.882
```

The T-learner gives treatment and control separate outcome models. It can work well, but it can extrapolate badly when treated and control covariate distributions do not overlap.

9.5. X-learner (Künzel et al. 2019)

The X-learner starts from the T-learner. It imputes missing potential outcomes, creates pseudo-treatment effects, smooths them over X, and then combines the two arms with propensity-score weights.

```
# Step 1: pseudo-outcomes, stored full-length so the arm models can be fit
# out-of-fold with `rows` (entries outside each arm are never used for training)
D1_pseudo = zeros(n); D1_pseudo[idx_T] = Y[idx_T] .- mu0_T[idx_T]   # observed - imputed Y(0)
D0_pseudo = zeros(n); D0_pseudo[idx_C] = mu1_T[idx_C] .- Y[idx_C]   # imputed Y(1) - observed

# Step 2: regress pseudo-outcomes on X in each arm
tau_X1 = rf_oof(X, D1_pseudo, X, folds; rows = idx_T)
tau_X0 = rf_oof(X, D0_pseudo, X, folds; rows = idx_C)

# Step 3: weight by propensity score
e_hat = clamp.(rf_oof(X, D, X, folds), 0.02, 0.98)

tau_X = e_hat .* tau_X0 .+ (1 .- e_hat) .* tau_X1
@printf("X-learner CATE correlation with truth: %.3f\n", cor(tau_X, tau))
```

```
X-learner CATE correlation with truth: 0.928
```

The X-learner is useful when one treatment arm is much smaller than the other.

9.6. R-learner (Nie and Wager 2021)

The R-learner partials out the main effects of X from both Y and D:

$$\tilde{Y}_i = \frac{Y_i - \hat{m}(X_i)}{D_i - \hat{e}(X_i)}, \qquad \text{weight}_i = (D_i - \hat{e}(X_i))^2, \tag{9.2}$$

where $\hat{m}(x) = \mathbb{E}[Y \mid X = x]$ and $\hat{e}(x) = \mathbb{E}[D \mid X = x]$ are cross-fitted nuisance estimates.

```
# Nuisances cross-fitted on the shared folds defined in the helper chunk
m_hat    = rf_oof(X, Y, X, folds)
e_hat_cv = clamp.(rf_oof(X, D, X, folds), 0.02, 0.98)

pseudo_R   = (Y .- m_hat) ./ (D .- e_hat_cv)
weights_R  = (D .- e_hat_cv) .^ 2

# The R-learner is a *weighted* regression of pseudo_R on X with weights
# (D - e_hat)^2. DecisionTree's RF does not accept per-sample weights, so we
# pass them through the weighted-bootstrap path of `rf_fit_predict` (reached here
# via `rf_oof`, which forwards `weights`). This downweights
```

```
# observations with D close to e_hat, whose pseudo_R blows up (division by a
# near-zero denominator) and would otherwise dominate the split criterion.
tau_R = rf_oof(X, pseudo_R, X, folds; weights = weights_R)
@printf("R-learner CATE correlation with truth: %.3f\n", cor(tau_R, tau))
```

```
R-learner CATE correlation with truth: 0.714
```

The R-learner is often stable when overlap is reasonable because nuisance model errors have only second-order effects on the target.

9.7. DR-learner (Kennedy 2020)

The DR-learner uses an AIPW-style pseudo-outcome and then regresses it on X:

$$\tilde{Y}_i^{DR} = \hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + \frac{D_i(Y_i - \hat{\mu}_1(X_i))}{\hat{e}(X_i)} - \frac{(1 - D_i)(Y_i - \hat{\mu}_0(X_i))}{1 - \hat{e}(X_i)}. \tag{9.3}$$

```
mu1_cf = rf_oof(X, Y, X, folds; rows = D .== 1)
mu0_cf = rf_oof(X, Y, X, folds; rows = D .== 0)
e_cf   = clamp.(rf_oof(X, D, X, folds), 0.02, 0.98)

pseudo_DR = @. (mu1_cf - mu0_cf) +
               D * (Y - mu1_cf) / e_cf -
               (1 - D) * (Y - mu0_cf) / (1 - e_cf)

tau_DR = rf_oof(X, pseudo_DR, X, folds)
@printf("DR-learner CATE correlation with truth: %.3f\n", cor(tau_DR, tau))
```

```
DR-learner CATE correlation with truth: 0.769
```

9.8. Comparing all five estimators

```
fig = Figure(size = (1000, 700))
labs = ["S-learner", "T-learner", "X-learner", "R-learner", "DR-learner"]
preds = [tau_S, tau_T, tau_X, tau_R, tau_DR]

for (i, (lab, pred)) in enumerate(zip(labs, preds))
    row, col = divrem(i - 1, 3) .+ (1, 1)
    ax = Axis(fig[row, col],
              xlabel = "X1", ylabel = "Estimated CATE",
              title = lab)
```

```
    scatter!(ax, X[:, 1], pred, color = (:steelblue, 0.2), markersize = 4)
    lines!(ax, 0:0.01:1, x -> 1 + 2x, color = :firebrick, linestyle = :dash,
           linewidth = 2)
end
fig
```

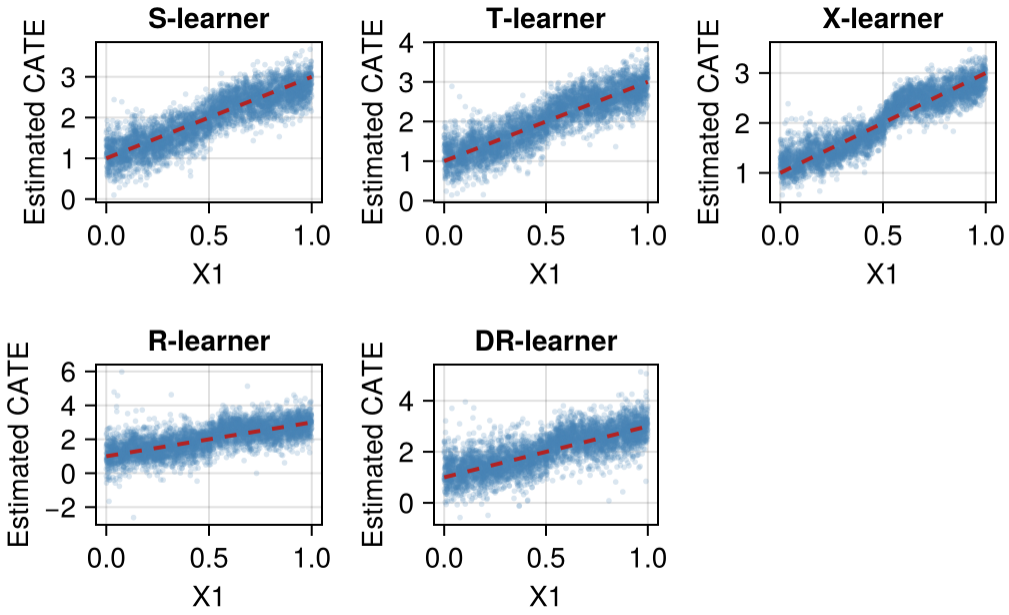


Figure 9.1.: Estimated CATE vs true $\tau(x) = 1 + 2x$ for each meta-learner. Red dashed line = ground truth.

9.9. Best Linear Projection (BLP)

Even if CATE is nonlinear, we often want a regression-style summary: which covariates are associated with larger effects? The best linear projection is the OLS regression of pseudo-outcomes on covariates:

$$(\beta_0^*, \beta^*) = \arg\min_{\beta_0,\beta} \mathbb{E}\left[(\tau(X) - \beta_0 - X'\beta)^2\right], \qquad \mathrm{BLP}(X) = \beta_0^* + X'\beta^*. \tag{9.4}$$

The arg min returns the coefficient vector (β_0^*, β^*); the best linear projection itself is the fitted function $\beta_0^* + X'\beta^*$.

A doubly-robust BLP uses the DR-learner pseudo-outcomes:

```
blp_df = DataFrame(hcat(pseudo_DR, X), [:tau_pseudo, :X1, :X2, :X3, :X4, :X5])
blp_fit = lm(@formula(tau_pseudo ~ X1 + X2 + X3 + X4 + X5), blp_df)
```

```
# `display`, not `println`. StatsBase defines only
# show(io, ::MIME"text/plain", ::CoefTable), so println() falls through to the
# generic struct show and prints the raw constructor -- `CoefTable(Any[[...]])`
# -- instead of the formatted table.
display(coeftable(blp_fit))
```

	Coef.	Std. Error	t	Pr(>	t	)
(Intercept)	1.03165	0.135462	7.62	<1e-13	0.76609	1.29722
X1	2.05583	0.116642	17.63	<1e-66	1.82716	2.28449
X2	-0.0285376	0.117208	-0.24	0.8076	-0.258318	0.201242
X3	-0.0205871	0.118377	-0.17	0.8619	-0.252657	0.211483
X4	-0.0204671	0.118625	-0.17	0.8630	-0.253025	0.212091
X5	0.0117126	0.118624	0.10	0.9214	-0.220842	0.244268

Here the coefficient on X_1 should be close to 2, and the coefficients on the other variables should be close to 0.

9.10. GATES: sorted group ATEs

GATES (group average treatment effects) is a simple way to report heterogeneity (Chernozhukov et al. 2018). Sort observations by predicted CATE, split them into bins, and estimate the ATE in each bin. (The same paper's CLAN — classification analysis — is the natural companion: compare average *covariates* between the most- and least-affected bins; here that would show high X_1 in the top quintile.)

```
nq = 5

# The ranking score MUST be out-of-fold, which `tau_DR` now is (it comes from
# `rf_oof`). This is not a technicality. When this chapter ranked on an in-sample
# forest prediction -- fit on (X, pseudo_DR) and predicted back at the same X --
# the bins sorted on pseudo_DR's own noise and then averaged that same noise, so
# the top bin collected positive noise and the bottom bin negative noise. The
# table ran from -1.22 to 5.28, entirely outside the true CATE range of [1, 3],
# with tight confidence intervals around impossible values.
tau_rank = tau_DR   # already out-of-fold, from rf_oof above

quintile_edges = quantile(tau_rank, range(0, 1, length = nq + 1))
quintiles      = searchsortedfirst.(Ref(quintile_edges), tau_rank) .- 1
quintiles      = clamp.(quintiles, 1, nq)

# AIPW-style ATE within each quintile using the DR pseudo-outcomes
function quintile_ate(tau_pseudo, idx)
    n_q = sum(idx)
    est = mean(tau_pseudo[idx])
```

```
    sd  = std(tau_pseudo[idx]) / sqrt(n_q)
    return (est = est, se = sd)
end

clan_df = DataFrame(quintile = 1:nq,
                    ATE = [quintile_ate(pseudo_DR, quintiles .== q).est for q in 1:nq],
                    SE  = [quintile_ate(pseudo_DR, quintiles .== q).se  for q in 1:nq],
                    # This is a simulation, so report the true within-bin average of
                    # tau alongside the estimate. It makes the table self-checking:
                    # every entry must lie in [1, 3] here, and Truth is the column to
                    # compare ATE against.
                    Truth = [mean(tau[quintiles .== q]) for q in 1:nq])
clan_df.lo = clan_df.ATE .- 1.96 .* clan_df.SE
clan_df.hi = clan_df.ATE .+ 1.96 .* clan_df.SE

@printf("%-9s %8s %7s %8s %8s %8s\n", "Quintile", "ATE", "SE", "95% LB", "95% UB", "Truth")
for row in eachrow(clan_df)
    @printf("%-9d %8.3f %7.3f %8.3f %8.3f %8.3f\n",
            row.quintile, row.ATE, row.SE, row.lo, row.hi, row.Truth)
end
```

```
Quintile       ATE      SE   95% LB   95% UB    Truth
1            1.350   0.075    1.204    1.497    1.393
2            1.602   0.081    1.443    1.760    1.620
3            2.220   0.081    2.061    2.379    2.041
4            2.417   0.074    2.273    2.561    2.388
5            2.612   0.075    2.464    2.759    2.604
```

Quintile 5 has a larger ATE than quintile 1, and because the ranking score is out-of-fold the magnitudes are trustworthy too: compare the `ATE` column against `Truth`, the actual within-bin average of $\tau(x) = 1 + 2X_1$. Both halves of the procedure — forming the groups and estimating the group means — now use predictions from models that did not see the observation being scored, since `pseudo_DR` is built from cross-fitted nuisances and `tau_DR` comes from `rf_oof`.

Do not expect the intervals to have exact nominal coverage even so, and the table shows why it is useful to print `Truth`: the reported standard errors treat the bin membership as fixed, when in fact the *boundaries* were estimated from the same data, and they take no account of the uncertainty in `tau_DR` itself. One quintile here has a truth just outside its interval, which is about what that omission would predict. The estimates are close enough to be read as a heterogeneity profile; the intervals are indicative rather than exact. For inference you would want the groups formed on a genuinely held-out split, or `grf`'s `rank_average_treatment_effect`.

This is worth insisting on because the failure is silent. Ranking on an in-sample score leaves the group means unbiased *on average* — they still average to the ATE — while making the spread meaningless, so nothing in the output looks wrong except the numbers themselves. A useful habit is to sanity-check group ATEs against the range the estimand can possibly take: here every quintile mean must lie in $[1, 3]$, and a table with a negative bottom bin is refuted before you read its standard errors.

9.11. Variable importance — which covariate drives heterogeneity?

A simple diagnostic is to regress the DR pseudo-outcome on each covariate separately and compare R^2. It answers a different question from the BLP, and the two are easy to run together and confuse.

The BLP is a *joint* projection with inference attached. Its target, Equation 9.4, is defined without reference to any fitted object: it is the best linear approximation to $\tau(x)$ using all covariates at once, it comes in the units of the outcome, it has a sign, and `coeftable` gives it a standard error we can test against zero. The importance measure below is *marginal* and descriptive: one univariate R^2 per covariate, with no units, no sign, and no standard error.

The difference that matters in practice is joint versus marginal. A covariate that is merely correlated with the true effect modifier picks up a high univariate R^2, because on its own it does predict the pseudo-outcome, while its BLP coefficient is near zero once the real modifier is in the regression. The projection splits shared variation between correlated covariates; the marginal measure gives each of them full credit for it. Here the five covariates are drawn independently, so the two agree.

Both share one blind spot worth naming, since it is not obvious: this importance measure is itself a *linear* fit, so a covariate that modifies the effect non-monotonically — large effects at both ends of its range, small in the middle — scores near zero on both the BLP and the R^2. A tree-based importance of the kind `grf` reports would flag it, because trees can split on it. Neither diagnostic here can.

```
function single_var_r2(target, x)
    df_one = DataFrame(t = target, x = x)
    fit = lm(@formula(t ~ x), df_one)
    1 - sum(abs2.(residuals(fit))) / sum(abs2.(target .- mean(target)))
end

vi = [single_var_r2(pseudo_DR, X[:, j]) for j in 1:p]
vi_df = DataFrame(variable = ["X$j" for j in 1:p], R2 = vi)
sort!(vi_df, :R2, rev = true)
println(vi_df)
```

```
5×2 DataFrame
 Row   variable  R2
       String    Float64

   1   X1        0.0585715
   2   X4        2.32397e-5
   3   X5        1.20668e-5
   4   X2        2.02436e-6
   5   X3        5.36056e-7
```

X_1 should be the most important variable in this simulation.

9.12. Policy learning: who should we treat?

A CATE estimate is not yet a policy. If treatment has a cost, the policy question is who should be treated. Define a treatment cost c in outcome units:

$$\pi^*(x) = \mathbb{1}\{\tau(x) > c\}. \tag{9.5}$$

For interpretability, we can restrict the policy to a single threshold rule.

```
# cost = 2 makes the problem non-degenerate: tau(x) = 1 + 2 x1 is in [1, 3],
# so the optimal rule is "treat iff x1 > 0.5" (about half the population).
# A cost below 1 would make treat-everyone optimal and there would be
# nothing for a policy to learn.
cost = 2.0

# Welfare of a rule = average gain over treating nobody, evaluated with the
# DR scores: mean over units of treat(x) * (pseudo_DR - cost)
welfare(treat) = mean(treat .* (pseudo_DR .- cost))

welfare_all  = welfare(trues(n))            # treat everyone
treat_est    = tau_DR .> cost               # rule from the ESTIMATED CATE
treat_true   = tau .> cost                  # oracle rule (simulation only)

# Simple threshold-rule family on X1
threshes = 0:0.05:1
welfares = [welfare(X[:, 1] .> t) for t in threshes]
best_t   = threshes[argmax(welfares)]

@printf("Treatment rates: estimated rule %.2f, oracle rule %.2f\n",
        mean(treat_est), mean(treat_true))
@printf("Welfare (treat everyone):                    %.3f\n", welfare_all)
@printf("Welfare (oracle rule  (x) > %.0f):           %.3f\n", cost, welfare(treat_true))
@printf("Welfare (best X1-threshold rule, X1 > %.2f): %.3f\n",
        best_t, maximum(welfares))
@printf("Welfare (τ̂-rule, SAME scores, biased):      %.3f\n", welfare(treat_est))
@printf("Welfare (τ̂-rule, evaluated on true  ):      %.3f\n",
        mean(treat_est .* (tau .- cost)))
```

```
Treatment rates: estimated rule 0.53, oracle rule 0.51
Welfare (treat everyone):                    0.040
Welfare (oracle rule  (x) > 2):           0.300
Welfare (best X1-threshold rule, X1 > 0.45): 0.301
Welfare (τ̂-rule, SAME scores, biased):      0.261
Welfare (τ̂-rule, evaluated on true  ):      0.215
```

The threshold rule gives up flexibility, but it is easy to explain — and it recovers the oracle threshold of 0.5 almost exactly.

One number above deserves a warning. The $\hat{\tau}$-rule "evaluated" with the same DR scores that selected it appears to beat even the oracle rule — which is impossible in expectation. Selecting units whose noisy score is high and then averaging those same scores is optimistic by construction. In a simulation we can evaluate the rule against the true $\tau(x)$ instead (the last line): the honest value is positive — better than treating everyone — but well below the oracle, because the noisy CATE estimates misclassify many units near the threshold. Both lessons matter: never evaluate a rule on the scores that chose it (in real data, estimate the rule and evaluate its welfare on separate folds), and do not expect an estimated rule to attain oracle welfare. Here the simple X_1-threshold rule, which searches a small one-dimensional family, gets essentially the oracle value — restricting the policy class is a form of regularisation.

For more on policy trees with IPW and AIPW losses (using R's `policytree` package), see the companion blog chapter on `policytree`. For a cross-software comparison of CATE estimators (including Stata 19's new `cate` command), see the Stata CATE blog chapter.

9.13. Causal forests with panel data

With panel data, unit effects can be correlated with treatment and covariates. A cross-sectional meta-learner can then be biased. The fixed effect adjustment is to demean by unit before applying the learner.

```
Random.seed!(2024)
n_firms = 200
n_t     = 5
N       = n_firms * n_t

firm_id  = repeat(1:n_firms, inner = n_t)
unit_fe  = randn(n_firms) .* 1.5
V1_firm  = randn(n_firms) .* 1.0
V1_panel = V1_firm[firm_id] .+ 0.3 .* randn(N)
# The unit effect drives BOTH treatment and the outcome (classic
# fixed-effect confounding); unit_fe is unobserved to the learner.
W_panel  = Float64.(rand(N) .<
           @. 1 / (1 + exp(-(0.3 * V1_firm[firm_id] + 0.5 * unit_fe[firm_id]))))
tau_panel = @. 0.5 + 1.0 * V1_panel
Y_panel  = unit_fe[firm_id] .+ V1_panel .+ tau_panel .* W_panel .+ randn(N)

df_panel = DataFrame(firm = firm_id, V1 = V1_panel, W = W_panel, Y = Y_panel)
# Y(1) - Y(0) = tau_panel exactly, so the true ATE is mean(tau_panel).
true_panel_ate = mean(tau_panel)
@printf("True panel ATE: %.3f\n", true_panel_ate)
```

```
True panel ATE: 0.655
```

A naive T-learner ignores the firm fixed effects. Because `unit_fe` raises both the treatment probability and the outcome, and is not in the learner's covariates, the naive estimate is badly biased upward:

```
X_panel_naive = reshape(df_panel.V1, N, 1)
idx_T_p = df_panel.W .== 1
idx_C_p = df_panel.W .== 0

# Note these panel fits are in-sample, unlike the meta-learners above. Splitting
# out-of-fold here would have to split by *firm* rather than by row, since two
# observations of the same firm are not independent. The comparison this section
# makes -- naive against within-transformed -- is driven by fixed-effect
# confounding, which is far larger than any in-sample optimism, so the point
# survives; do not read these two numbers as clean CATE accuracy figures.
mu1_naive = rf_fit_predict(X_panel_naive[idx_T_p, :], df_panel.Y[idx_T_p],
                           X_panel_naive)
mu0_naive = rf_fit_predict(X_panel_naive[idx_C_p, :], df_panel.Y[idx_C_p],
                           X_panel_naive)
tau_naive = mu1_naive .- mu0_naive

@printf("Naive panel T-learner ATE: %.3f  (true = %.3f)\n",
        mean(tau_naive), true_panel_ate)
```

```
Naive panel T-learner ATE: 2.014  (true = 0.655)
```

The within transformation removes firm-level variation and leaves the within-firm variation:

```
# Within transformation: subtract firm means
df_dm = combine(groupby(df_panel, :firm),
                :Y  => (y -> y .- mean(y)) => :Y_dm,
                :W  => (w -> w .- mean(w)) => :W_dm,
                :V1 => (v -> v .- mean(v)) => :V1_dm)

X_dm = reshape(df_dm.V1_dm, N, 1)
idx_T_dm = df_dm.W_dm .> 0
idx_C_dm = df_dm.W_dm .<= 0

mu1_dm = rf_fit_predict(X_dm[idx_T_dm, :], df_dm.Y_dm[idx_T_dm], X_dm)
mu0_dm = rf_fit_predict(X_dm[idx_C_dm, :], df_dm.Y_dm[idx_C_dm], X_dm)
tau_dm = mu1_dm .- mu0_dm

# The two demeaned groups differ in W_dm by less than a full 0 -> 1 switch
# (treated-above-average vs below-average within firm), so the raw contrast
# is attenuated; rescale by the W_dm gap, as in a Wald estimator.
w_gap = mean(df_dm.W_dm[idx_T_dm]) - mean(df_dm.W_dm[idx_C_dm])
@printf("Within-transform T-learner ATE: %.3f  (true = %.3f)\n",
        mean(tau_dm) / w_gap, true_panel_ate)
```

```
Within-transform T-learner ATE: 0.689  (true = 0.655)
```

The within transformation removes the fixed effects, and the rescaled contrast lands near the original ATE in this DGP, although, as the caveat below explains, its target need not equal it. The cost is variance, because identification now comes from within-firm variation.

A caveat on what this estimator targets. Splitting on $W_{dm} > 0$ vs $W_{dm} \leq 0$ and dividing the T-learner contrast by the average W_{dm} gap is *not* a general fixed-effect CATE estimator. It is a heuristic within-firm contrast whose target depends on the distribution of demeaned treatment values and on the grouping rule, and the rescaling is a Wald-style approximation rather than an identified within estimand. It works here because the treatment effect is linear in V_1 and the DGP is benign. Properly identified heterogeneous panel effects require a proper orthogonal score or a dedicated panel causal-forest / DML construction; treat this section as intuition, not a turnkey method.

So how much does ignoring the panel cost? On this DGP the naive T-learner returns 2.014 against a true panel ATE of 0.655 — about three times the truth — while the within transform gives 0.689. The magnitude is specific to this design, since it scales with how strongly the unit effect drives both treatment and outcome, but the direction is not: a unit effect that raises both leaves the naive estimate biased upward. Read those two numbers as the size of the confounding being removed, not as clean CATE accuracy figures, for the estimand reasons given just above.

For R's `grf::causal_forest`, pass `clusters = firm_id` so the honest sample split keeps each firm's observations together. Julia does not currently have an equivalent; see the companion blog chapter on causal forests in panel data and the R companion to this chapter for the GRF-based workflow.

9.14. Summary

- Meta-learners turn regression tools into CATE estimators.
- In Julia they can be built with `MLJ` random forests, but there is no full `grf` equivalent yet.
- BLP and GATES summarize heterogeneity in regression-style output.
- Policy learning turns CATE estimates into treatment rules.
- In applied work, compare at least two CATE estimators and report simple heterogeneity summaries, not only a CATE scatterplot.

10. Continuous and Multivalued Treatments

```
using DataFrames
using Distributions
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

So far most examples use a binary treatment. Many treatments are not binary: dose, dollars, hours, or a treatment with several levels. In those cases "the treatment effect" is not one number. We usually want a dose-response curve,

$$\mu(d) = \mathbb{E}[Y(d)], \quad d \in \mathcal{D}. \tag{10.1}$$

Here I implement two approaches directly in Julia: the generalized propensity score (Hirano-Imbens 2004) and a doubly-robust extension. For modified treatment policies and time-varying treatments, R's `lmtp` package is still the better practical tool; see the R companion chapter and the LMTP blog chapter.

10.1. Setup

```
Random.seed!(42)
n  = 2000
X1 = randn(n)
X2 = randn(n)
# Continuous treatment, depends on X1, X2
D  = @. 2.0 + 0.5 * X1 + 0.5 * X2 + randn()
# Dose-response: concave in D, with diminishing returns. Not "hump-shaped" over
# the observed range: the vertex is at d = 2/0.3 = 6.67, beyond the largest dose
# drawn below, so the slope 2 - 0.3d stays positive throughout the support and no
# downturn is identified from these data.
true_mu(d) = 1.0 + 2.0 * d - 0.15 * d^2
Y  = @. true_mu(D) + 0.3 * X1 - 0.3 * X2 + 0.5 * randn()
```

```
df = DataFrame(Y = Y, D = D, X1 = X1, X2 = X2)
@printf("Treatment range: [%.2f, %.2f]\n", minimum(D), maximum(D))
@printf("True  (d=2) = %.2f,  (d=4) = %.2f,  (d=6) = %.2f\n",
        true_mu(2), true_mu(4), true_mu(6))
```

```
Treatment range: [-2.01, 5.97]
True  (d=2) = 4.40,  (d=4) = 6.60,  (d=6) = 7.60
```

10.2. Naive regression vs adjusted regression

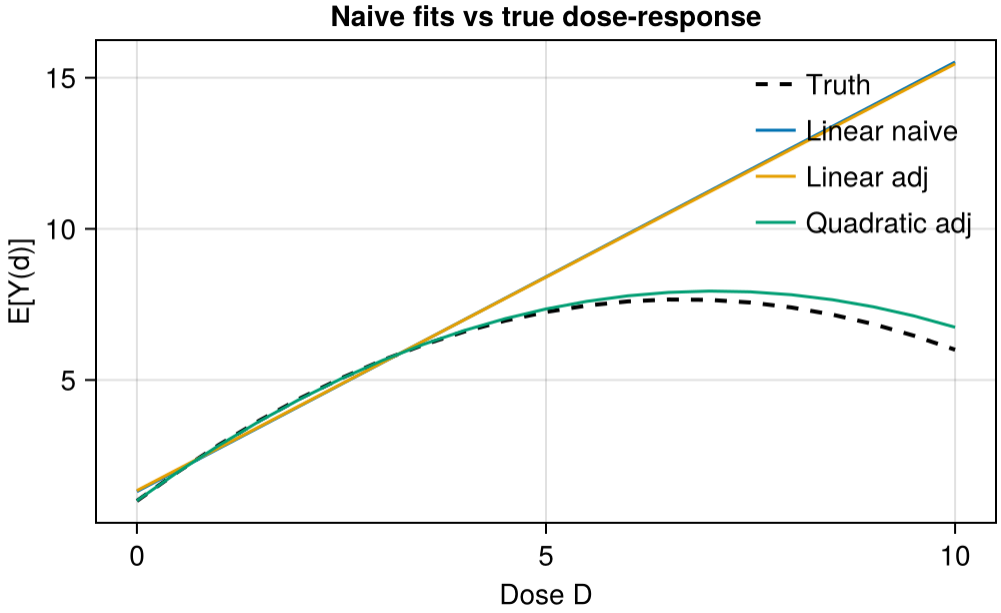
A linear fit on D alone has two problems: it ignores confounding by X, and it imposes a straight line on a nonlinear dose-response curve.

```
df.D2 = df.D .^ 2
linear_naive = lm(@formula(Y ~ D),                    df)
linear_adj   = lm(@formula(Y ~ D + X1 + X2),          df)
poly_adj     = lm(@formula(Y ~ D + D2 + X1 + X2),     df)

ds = 0.0:0.5:10.0

predict_mu(fit) = Float64.(predict(fit, DataFrame(D = collect(ds),
                                                  D2 = collect(ds) .^ 2,
                                                  X1 = 0, X2 = 0)))
fits = (
    linear_naive = predict_mu(linear_naive),
    linear_adj   = predict_mu(linear_adj),
    poly_adj     = predict_mu(poly_adj),
)
truth = true_mu.(ds)

fig = Figure(size = (700, 400))
ax = Axis(fig[1, 1], xlabel = "Dose D", ylabel = "E[Y(d)]",
          title = "Naive fits vs true dose-response")
lines!(ax, ds, truth, color = :black, linestyle = :dash,
       linewidth = 2, label = "Truth")
lines!(ax, ds, fits.linear_naive, label = "Linear naive")
lines!(ax, ds, fits.linear_adj,   label = "Linear adj")
lines!(ax, ds, fits.poly_adj,     label = "Quadratic adj")
axislegend(ax, position = :rt, framevisible = false)
fig
```



The quadratic adjusted model is better in this simulation because the true curve is quadratic. In real applications, the functional form is usually not known.

10.3. Generalized Propensity Score (Hirano-Imbens 2004)

The **generalized propensity score** is the conditional density of the treatment given covariates:

$$r(d, x) = f_{D|X}(d \mid x). \tag{10.2}$$

Two results from Hirano and Imbens (2004) are what make this object useful, and both are worth stating before we compute anything.

The first is the assumption they need, which is weaker than it looks. *Weak unconfoundedness* requires $Y(d) \perp D \mid X$ separately for each dose d. It does not require the whole path $\{Y(d) : d\}$ to be jointly independent of D given X, which would be a much stronger demand at a continuous treatment.

The second is the balancing property. Conditional on the scalar $r(d, X)$, the covariates carry no further information about whether a unit received dose d:

$$f_D(d \mid r(d, X)) = f_D(d \mid r(d, X),\, X). \tag{10.3}$$

So a one-dimensional score does the job of the full covariate vector, exactly as in the binary case. Put the two together and the dose-response function is

$$\mu(d) = E[Y(d)] = E\Big[\, E(Y \mid D = d,\; r(d, X))\Big], \tag{10.4}$$

where the outer expectation runs over the distribution of $r(d, X)$ in the whole sample. Read the argument of r carefully, because it is where this estimator is usually got wrong: at each counterfactual dose d we must recompute every unit's score *at that* d, not reuse the score at the dose the unit happened to receive. The `estimate_dose` function below does exactly that, which is why the GPS is recomputed inside it rather than taken from `df.gps`.

One more point of theory explains why the GPS enters the outcome model as a regressor rather than as a weight. The GPS is a conditional *density*, not a probability, so it carries the units of $1/D$; weights of the form $1/\hat{r}$ are therefore not scale-free and change if the dose is rescaled. Dividing by the marginal density of D would restore scale invariance, but Hirano and Imbens sidestep the issue altogether by putting the score in the outcome model.

In practice we start with a normal model for $D \mid X$, $D \mid X \sim \mathcal{N}(\beta_0 + \beta' X, \sigma^2)$. Fitting D on X_1 and X_2 by OLS gives $\hat{\beta}$ and $\hat{\sigma}$, and the score at any dose d for unit i is the normal density evaluated there,

$$\hat{r}(d, x_i) = \frac{1}{\hat{\sigma}\sqrt{2\pi}} \exp\left\{ -\frac{(d - x_i'\hat{\beta})^2}{2\hat{\sigma}^2} \right\}. \tag{10.5}$$

That is the whole calculation: one linear regression, then one density evaluation per unit per dose.

```
gps_model = lm(@formula(D ~ X1 + X2), df)
d_pred    = predict(gps_model)
sigma_e   = sqrt(sum(abs2.(residuals(gps_model))) / dof_residual(gps_model))

df.gps = @. pdf(Normal(d_pred, sigma_e), df.D)
@printf("GPS summary: min=%.4f, mean=%.4f, max=%.4f\n",
        minimum(df.gps), mean(df.gps), maximum(df.gps))
```

```
GPS summary: min=0.0009, mean=0.2806, max=0.3962
```

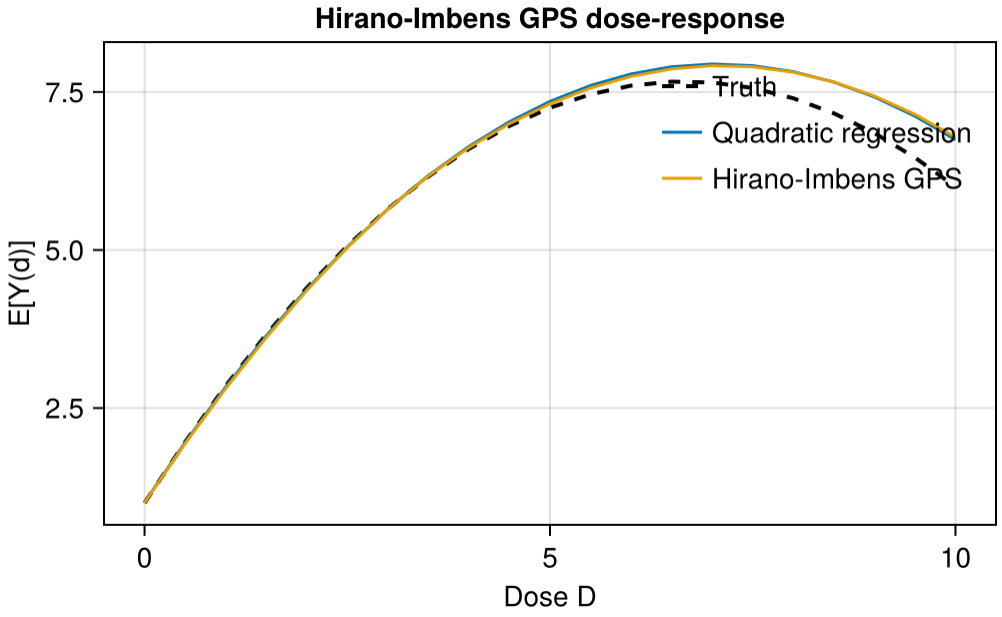
The Hirano-Imbens estimator has two steps. First regress Y on flexible functions of D and the estimated GPS. Then, for each dose d, average the predicted outcome over the sample.

```
# Stage 2: regress Y on flexible function of D and the GPS
df.gps2 = df.gps .^ 2
hi_fit = lm(@formula(Y ~ D + D2 + gps + gps2 + D & gps), df)

# Dose-response at each d: average over the empirical GPS distribution at that d
function estimate_dose(d)
    gps_at_d = @. pdf(Normal(d_pred, sigma_e), d)
    nd = DataFrame(D = fill(d, n), D2 = fill(d^2, n),
                   gps = gps_at_d, gps2 = gps_at_d .^ 2)
    mean(predict(hi_fit, nd))
end

hi_dose = [estimate_dose(d) for d in ds]
```

```
fig2 = Figure(size = (700, 400))
ax2 = Axis(fig2[1, 1], xlabel = "Dose D", ylabel = "E[Y(d)]",
           title = "Hirano-Imbens GPS dose-response")
lines!(ax2, ds, truth, color = :black, linestyle = :dash,
       linewidth = 2, label = "Truth")
lines!(ax2, ds, fits.poly_adj, label = "Quadratic regression")
lines!(ax2, ds, hi_dose, label = "Hirano-Imbens GPS")
axislegend(ax2, position = :rt, framevisible = false)
fig2
```



In this example the GPS estimator tracks the nonlinear curve much better than the naive linear regression.

10.4. Doubly-robust dose-response estimation

A kernel-localized AIPW estimator — in the spirit of Kennedy et al. (2017), who develop a more elaborate pseudo-outcome version with formal guarantees — is the continuous-treatment analogue of AIPW. It combines an outcome model $\hat{\mu}(d, x)$ with the GPS $\hat{r}(d \mid x)$:

$$\hat{\mu}_{DR}(d) = \frac{1}{n}\sum_i \hat{\mu}(d, X_i) + \sum_i \tilde{w}_i(d)\,(Y_i - \hat{\mu}(D_i, X_i)), \qquad \tilde{w}_i(d) = \frac{w_i(d)}{\sum_j w_j(d)}, \quad w_i(d) = \frac{K_h(D_i - d)}{\hat{r}(D_i \mid X_i)} \tag{10.6}$$

where K_h is a kernel around dose d. The code self-normalizes the kernel weights (a Hájek/stabilized form, $\sum_i \tilde{w}_i = 1$) rather than dividing by n, which stabilizes the correction term in finite samples.

Two features of this estimator are worth understanding. The first is why it is doubly robust, and the algebra is the same as in the binary case worked through in Section 8.2.1: the first term is a plug-in built from the outcome model, the second subtracts that model's own residual reweighted by the score, so a first-order error in $\hat{\mu}$ enters twice with opposite signs and cancels, and what survives from the nuisances involves the *product* of the error in $\hat{\mu}$ and the error in $\hat{r}$. If either nuisance is right, that product is zero. At a fixed bandwidth a second term remains, the smoothing bias of the kernel. With a correct generalized propensity score, a zero outcome model and true curve $\mu(a) = a^2$, the correction averages μ over a normal window of standard deviation h around d, so its expected value is $E[(d + hZ)^2] = d^2 + h^2$ for standard normal Z, which is 4.25 rather than 4 at $d = 2$ and $h = 0.5$. The double robustness holds in the limit as h shrinks.

The second is why a bandwidth appears here when none appeared for the ATE. The ATE is a single number and is estimable at the $\sqrt{n}$ rate. But $\mu(d)$ at a *point* d is a regression function, and no $\sqrt{n}$ estimator of it exists without smoothness assumptions, because the data hold only finitely many observations near any given dose. The kernel is how we borrow from neighbouring doses, and h sets the exchange rate: too wide pulls in doses where the truth is different, too narrow leaves too few observations to average. Under the usual second-order smoothness the best trade is $h \propto n^{-1/5}$ and the rate is $n^{-2/5}$, slower than $\sqrt{n}$ by construction. Bandwidth sensitivity is therefore a cost of the method, not a tuning detail.

Kennedy et al. (2017) handle the localisation more carefully: instead of smoothing the correction term, they construct a pseudo-outcome whose conditional mean given $D = d$ is exactly $\mu(d)$, then run a single nonparametric regression of that pseudo-outcome on D. All the smoothing sits in one place, which is what lets them state formal guarantees. The version here keeps the doubly-robust structure and is easier to read, with weaker theory behind it.

```
# Outcome model: ^(D, X)
mu_fit = lm(@formula(Y ~ D + D2 + X1 + X2 + D & X1 + D & X2), df)

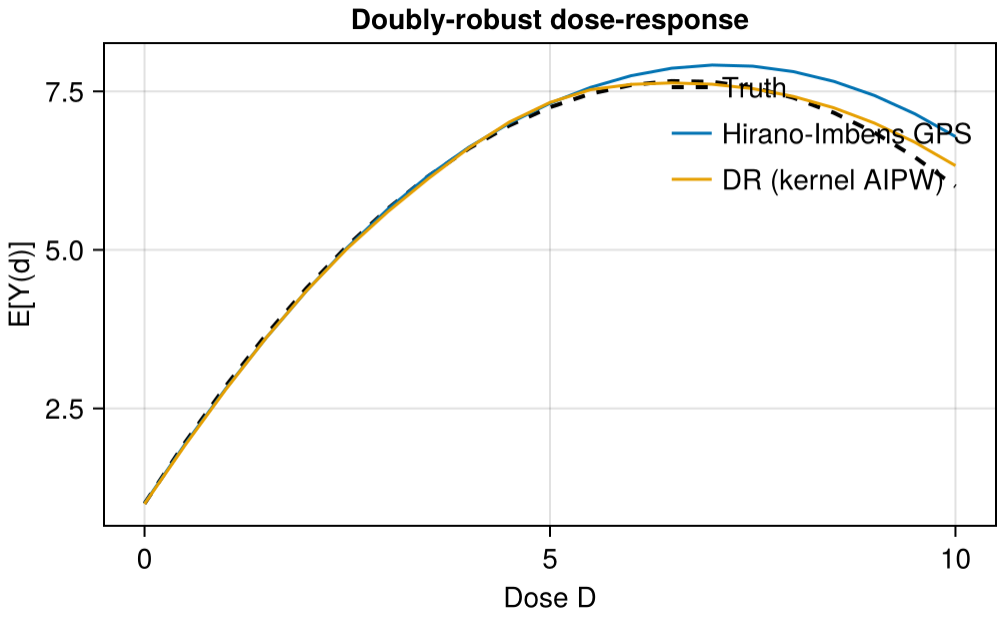
# Predict ^(D_i, X_i) for the observed treatments
mu_at_obs = predict(mu_fit)

# DR dose-response at a grid
function dr_estimate(d; h = 0.5)
    nd = DataFrame(D = fill(d, n), D2 = fill(d^2, n),
                   X1 = df.X1, X2 = df.X2)
    mu_at_d = predict(mu_fit, nd)
    kernel  = @. pdf(Normal(0, h), df.D - d)
    weight  = @. kernel / df.gps
    weight ./= mean(weight)
    correction = @. weight * (df.Y - mu_at_obs)
    mean(mu_at_d) + mean(correction)
end

dr_dose = [dr_estimate(d) for d in ds]

fig3 = Figure(size = (700, 400))
ax3 = Axis(fig3[1, 1], xlabel = "Dose D", ylabel = "E[Y(d)]",
```

```
          title = "Doubly-robust dose-response")
lines!(ax3, ds, truth, color = :black, linestyle = :dash,
       linewidth = 2, label = "Truth")
lines!(ax3, ds, hi_dose, label = "Hirano-Imbens GPS")
lines!(ax3, ds, dr_dose, label = "DR (kernel AIPW)")
axislegend(ax3, position = :rt, framevisible = false)
fig3
```



Read this self-normalized kernel correction as an *illustrative* kernel-AIPW construction, not a turnkey doubly robust theorem. The informal "consistent if either the outcome model or the GPS is correct" statement is the discrete-treatment intuition; for continuous treatments a formal doubly robust dose-response result additionally needs bandwidth, nuisance-convergence-rate, smoothness, and orthogonalization conditions (see Kennedy et al. on continuous-treatment DR estimators). The bandwidth h trades bias against variance.

10.5. Multivalued treatments

For a discrete multivalued treatment $D \in \{0, 1, \dots, K\}$, the GPS is a multinomial probability:

$$\hat{r}(d, x) = \hat{P}(D = d \mid X = x). \tag{10.7}$$

A multinomial regression supplies $\hat{r}$. The dose-response curve becomes the average outcome under each treatment level.

```
# Simulate a 4-level treatment
Random.seed!(99)
n_mv = 2000
X_mv = randn(n_mv, 2)
util  = hcat(zeros(n_mv),
             @.(0.2 + 0.5 * X_mv[:, 1]),
             @.(0.3 + 0.4 * X_mv[:, 2]),
             @.(0.5 + 0.3 * X_mv[:, 1] + 0.3 * X_mv[:, 2]))
# Softmax to get probabilities
exp_util = exp.(util)
prob_mv  = exp_util ./ sum(exp_util, dims = 2)
D_mv  = [rand(Distributions.Categorical(prob_mv[i, :])) - 1 for i in 1:n_mv]
Y_mv  = @. 0.5 * D_mv + 0.3 * X_mv[:, 1] - 0.2 * X_mv[:, 2] + randn()

# Unadjusted group means by treatment level (NOT causal: treatment assignment
# depends on X, and Y also depends on X, so these are confounded comparisons).
mv_df = DataFrame(Y = Y_mv, D = D_mv, X1 = X_mv[:, 1], X2 = X_mv[:, 2])
by_d = combine(groupby(mv_df, :D), :Y => mean => :y_mean, :Y => length => :n)
@printf("%-6s %12s %12s\n", "D", "Unadj Mean(Y)", "N")
for r in eachrow(by_d)
    @printf("%-6d %12.3f %12d\n", r.D, r.y_mean, r.n)
end
```

```
D      Unadj Mean(Y)            N
0            -0.022          406
1             0.622          486
2             0.879          521
3             1.569          587
```

The table above shows **unadjusted** group means, not causal dose-response estimates: because treatment assignment depends on X and Y also depends on X, the level-to-level differences are confounded. To recover causal contrasts one would weight by the multinomial GPS (or model the outcome). In practice, I would report adjusted contrasts between levels, such as $D = 1$ versus $D = 0$ or $D = 2$ versus $D = 1$.

10.6. When to reach for these methods

These methods are useful when:

1. Treatment is continuous or multivalued.
2. The research question is about the shape of the dose-response curve, not just one binary contrast.

For binary treatments, the standard Estimation and Heterogeneous Effects methods are sufficient.

For time-varying continuous treatments and modified-policy interventions, use LMTP in R for now. A Julia version would be useful, but this book does not need to reimplement it.

10.7. Summary

- Continuous and multivalued treatments usually require a dose-response curve, not a single coefficient.
- GPS extends propensity-score logic to continuous treatments by modeling the density of treatment given covariates.
- The kernel-AIPW doubly-robust estimator adds an outcome model to the GPS.
- For modified treatment policies and time-varying continuous treatments, use R's `lmtp` package.

11. Bayesian Causal Inference

```
using DataFrames
using Distributions
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

Bayesian methods return a posterior distribution, not only a point estimate and a standard error. For causal inference this is useful when we want uncertainty about conditional average treatment effects, policy values, or other derived quantities.

The two common models in this area are:

1. **BART for causal inference** (Hill 2011) — Bayesian Additive Regression Trees as a flexible outcome model.
2. **Bayesian Causal Forests** (BCF; Hahn-Murray-Carvalho 2020) — BART extended to separate prognostic and treatment-effect components.

Both have mature R packages (`BART`, `bcf`). Julia does not currently have a comparable BART/BCF workflow. So this chapter shows the basic Bayesian g-computation idea with a conjugate linear model, and points to R for the tree-based versions.

> **Related reading**: For a hands-on Bayesian model with `Numpyro`, see the Using Numpyro chapter in *Topics on Econometrics and Causal Inference*. The R companion gives the BART and BCF version.

11.1. Setup

```
Random.seed!(42)
n = 1500
p = 3

X = randn(n, p)
ps = @. 1 / (1 + exp(-(-0.3 + 0.5 * X[:, 1] + 0.3 * X[:, 2])))
```

```
D  = Float64.(rand(n) .< ps)
tau = @. 1 + 2 * X[:, 1]   # true CATE
Y0  = @. 0.5 * X[:, 2] + 0.3 * X[:, 3] + randn()
Y1  = Y0 .+ tau
Y   = ifelse.(D .== 1, Y1, Y0)

df = DataFrame(Y = Y, D = D, X1 = X[:, 1], X2 = X[:, 2], X3 = X[:, 3])
@printf("True ATE = %.3f\n", mean(tau))
```

```
True ATE = 0.901
```

11.2. Bayesian g-computation

The simplest Bayesian causal inference recipe is Bayesian g-computation:

1. Specify a Bayesian model for $\mathbb{E}[Y \mid D, X]$.
2. Sample from the posterior of the regression parameters.
3. For each posterior draw, compute counterfactual outcomes $\hat{Y}(1), \hat{Y}(0)$.
4. The posterior of the ATE is the distribution of the average difference across posterior draws.

Both potential outcomes in the simulation share the same residual, so every unit's individual effect equals $\tau(X_i)$. That equality is a feature of the simulation. In general the difference of predicted means at x is the conditional average effect $E[Y(1) - Y(0) \mid X = x]$; the data show one potential outcome per person and do not identify the individual effect without a model for how the two potential outcomes are related.

A Bayesian linear regression with the usual improper prior $p(\beta, \sigma^2) \propto 1/\sigma^2$ is enough to show the idea:

```
"""
    bayes_linreg(X, y; n_samples=2000)
Bayesian linear regression under the improper prior p( ,  ²)   1/ ²:  ² is
drawn from its Inverse-Gamma marginal posterior and   |  ² from a normal centred
at the OLS estimates, so the spread is close to the OLS standard errors. Returns `n_samples` p
draws of ( ,  ²).
"""
function bayes_linreg(X::Matrix, y::Vector; n_samples::Int = 2000)
    n, p = size(X)
    XtX_inv = inv(X' * X)
    beta_hat = XtX_inv * X' * y
    sse = sum(abs2.(y .- X * beta_hat))
    #  ² ~ InverseGamma((n-p)/2, sse/2)
     ²_post = [rand(InverseGamma((n - p) / 2, sse / 2)) for _ in 1:n_samples]
    #   |  ² ~ MultivariateNormal(beta_hat,  ² * XtX_inv)
    betas = zeros(n_samples, p)
    for i in 1:n_samples
```

```
        cov_mat = σ²_post[i] .* XtX_inv
        betas[i, :] = rand(MvNormal(beta_hat, Symmetric(cov_mat)))
    end
    return betas, σ²_post
end

# Fit on (D, X1, X2, X3, D*X1, D*X2, D*X3) - allow treatment effect heterogeneity
DesignMat = hcat(ones(n), df.D, df.X1, df.X2, df.X3,
                 df.D .* df.X1, df.D .* df.X2, df.D .* df.X3)
betas, σ²_draws = bayes_linreg(DesignMat, df.Y; n_samples = 2000)

# Counterfactual prediction matrices
X_treat   = hcat(ones(n), ones(n),  df.X1, df.X2, df.X3,
                 df.X1, df.X2, df.X3)           # D = 1, so D*X = X
X_control = hcat(ones(n), zeros(n), df.X1, df.X2, df.X3,
                 zeros(n), zeros(n), zeros(n))  # D = 0

# Posterior ITEs: n_samples x n
ite_post = (X_treat * betas')' .- (X_control * betas')'
# ATE posterior: average over n at each draw
ate_post = vec(mean(ite_post, dims = 2))

@printf("Bayesian g-computation:\n")
@printf("  Posterior mean ATE:  %.3f\n", mean(ate_post))
@printf("  Posterior SD:        %.3f\n", std(ate_post))
@printf("  95%% CrI:             [%.3f, %.3f]\n",
        quantile(ate_post, 0.025), quantile(ate_post, 0.975))
@printf("  True ATE:            %.3f\n", mean(tau))
```

```
Bayesian g-computation:
  Posterior mean ATE:  0.904
  Posterior SD:        0.056
  95% CrI:             [0.798, 1.016]
  True ATE:            0.901
```

With this prior, the credible interval is close to the usual frequentist interval. With informative priors, the posterior reflects both the prior and the data.

11.2.1. Credible intervals for the CATE

```
ite_mean = vec(mean(ite_post, dims = 1))
ite_lo   = [quantile(ite_post[:, i], 0.025) for i in 1:n]
ite_hi   = [quantile(ite_post[:, i], 0.975) for i in 1:n]
```

```
# Sort by X1 for plotting
ord = sortperm(df.X1)

fig = Figure(size = (700, 400))
ax = Axis(fig[1, 1], xlabel = "X1", ylabel = "CATE",
          title = "Posterior CATEs vs truth")
band!(ax, df.X1[ord], ite_lo[ord], ite_hi[ord],
      color = (:steelblue, 0.2), label = "95% CrI")
scatter!(ax, df.X1, ite_mean, color = (:steelblue, 0.4), markersize = 3,
         label = "Posterior mean CATE")
lines!(ax, [-3, 3], 1 .+ 2 .* [-3, 3], color = :firebrick,
       linestyle = :dash, linewidth = 2, label = "True  (x)")
axislegend(ax, position = :rb, framevisible = false)
fig
```

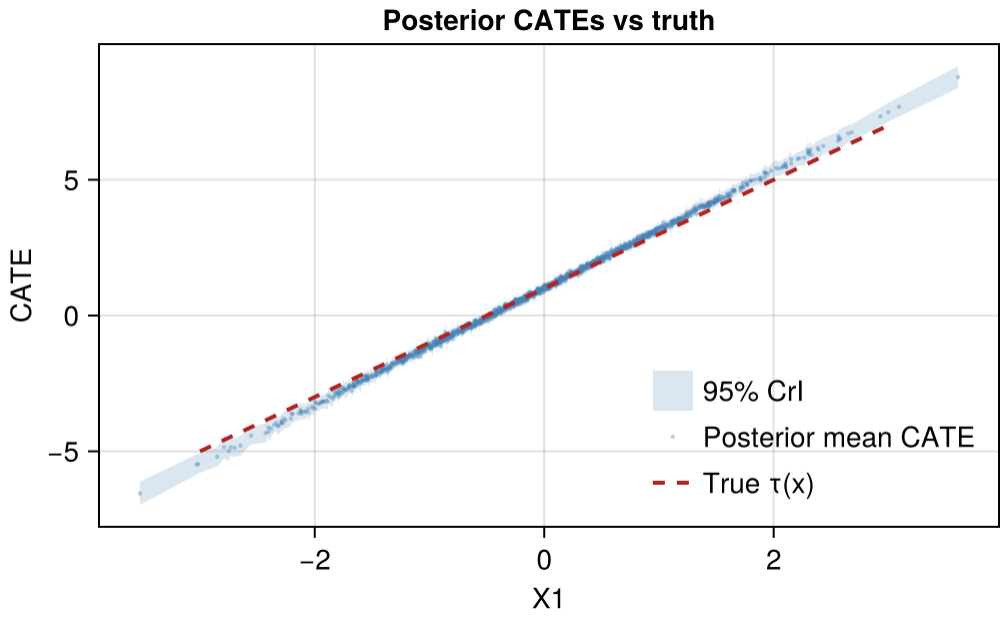


Figure 11.1.: Posterior mean of the conditional average effect at each unit's covariates vs true (x) = 1 + 2 X . Blue band shows 95% pointwise credible interval across posterior draws.

The blue ribbon is the pointwise 95% credible interval for the conditional average effect, given the linear model and the prior. The red dashed line is the true treatment-effect function.

11.3. Why use BART/BCF in R instead?

The linear g-computation above is useful when:

- the outcome model can be approximated with a few interactions;

- the effect modifiers are known in advance.

For flexible Bayesian causal inference, R's `BART` and `bcf` are better tools. They use sums of regression trees and provide:

- flexible outcome models without manually specifying interactions;
- posterior draws of conditional average treatment effects;
- in BCF, separate priors for baseline prediction and treatment-effect heterogeneity.

Compare the rough sketch:

Capability	Julia (this chapter)	R (BART/BCF)
Bayesian point estimate of ATE		
Posterior credible intervals		
Flexible nonparametric outcome model	(limited to linear + interactions)	
Native CATE posterior with regularisation	(linear-model heterogeneity only)	
Separation of prognostic and treatment effects	–	(BCF)

For now, Julia users who need BART/BCF should call out to R. Reimplementing these models in Julia would be a separate package project.

11.4. When to use Bayesian methods

Reason to use Bayesian	Reason to use Frequentist
Prior information matters	Asymptotic theory is well-understood
Need full posterior for downstream decisions	Computational cost matters at scale
Small sample, weak identification	Effects are well-identified
Want posterior intervals for conditional effects	Existing frequentist toolkit suffices

Bayesian methods are most useful when the posterior itself will be used: for treatment recommendations, policy values, or hierarchical models. If the target is just a well-identified ATE in a large sample, the frequentist tools are usually simpler.

11.5. Connections

- The Heterogeneous Treatment Effects chapter covers frequentist meta-learners. BCF is the Bayesian counterpart to causal forests.
- The Sensitivity Analysis chapter's Cinelli-Hazlett bounds are frequentist; the Bayesian analog puts a prior on the unmeasured-confounder strength (McCandless et al. 2007).
- For a hands-on Numpyro example of Bayesian modelling, see Using Numpyro.

11.6. Summary

- Bayesian g-computation computes counterfactual outcomes for each posterior draw.
- A conjugate linear model is easy to implement in Julia and is useful for exposition.
- For BART and BCF, use the R packages for now.
- For a Python/Numpyro Bayesian model, see the Using Numpyro chapter.

12. Distributional Treatment Effects

```
using DataFrames
using Distributions
using Random
using Statistics
using CairoMakie
CairoMakie.activate!(type = "png")
```

The previous chapters focus mostly on the average treatment effect. But many questions are not only about the mean. A policy can raise average earnings because high earners gain a lot, while low earners gain little. A clinical intervention can improve the median outcome but do nothing for the lower tail. To see this, we need distributional effects.

12.1. Beyond the average

The objects are the two marginal distributions $F_{Y(1)}$ and $F_{Y(0)}$: the outcome distribution if everyone were treated and if nobody were treated. From them we can compute quantile differences, density shifts, and inequality measures.

The most common summary is the **quantile treatment effect** (QTE):

$$\mathrm{QTE}(\tau) = F^{-1}_{Y(1)}(\tau) - F^{-1}_{Y(0)}(\tau), \quad \tau \in (0,1). \tag{12.1}$$

$\mathrm{QTE}(\tau)$ reports the difference between the τ-th quantile of the treated and control distributions. It is *not* the same as the average effect *at* a given pre-treatment quantile — QTE does not track individuals; it compares quantile to quantile across two marginal distributions.

Consider a treatment whose effect grows with the baseline outcome:

```
Random.seed!(42)
n = 5000

# Pre-treatment outcome (counterfactual under no treatment)
Y0 = rand(LogNormal(0, 0.6), n)

# Heterogeneous treatment effect: bigger at the top of the Y0 distribution
true_te = 0.2 .* Y0 .+ 0.1
Y1 = Y0 .+ true_te
```

```
ate     = mean(Y1 .- Y0)
qte_25  = quantile(Y1, 0.25) - quantile(Y0, 0.25)
qte_50  = quantile(Y1, 0.50) - quantile(Y0, 0.50)
qte_75  = quantile(Y1, 0.75) - quantile(Y0, 0.75)
qte_90  = quantile(Y1, 0.90) - quantile(Y0, 0.90)

@printf("ATE          = %.3f\n", ate)
@printf("QTE(0.25)    = %.3f\n", qte_25)
@printf("QTE(0.50)    = %.3f\n", qte_50)
@printf("QTE(0.75)    = %.3f\n", qte_75)
@printf("QTE(0.90)    = %.3f\n", qte_90)
```

```
ATE          = 0.339
QTE(0.25)    = 0.232
QTE(0.50)    = 0.296
QTE(0.75)    = 0.397
QTE(0.90)    = 0.535
```

The ATE is one number. The QTEs show that the effect is much larger near the top of the distribution.

```
 _grid = 0.05:0.025:0.95
qte_curve = [quantile(Y1,  ) - quantile(Y0,  ) for   in  _grid]

fig = Figure(size = (820, 360), fontsize = 13)

ax1 = Axis(fig[1, 1], xlabel = "Outcome Y", ylabel = "Density",
           title = "Outcome distributions")
density!(ax1, Y0, color = (:steelblue, 0.4), label = "Y(0)")
density!(ax1, Y1, color = (:firebrick, 0.4), label = "Y(1)")
axislegend(ax1, position = :rt, framevisible = false)

ax2 = Axis(fig[1, 2], xlabel = "Quantile  ", ylabel = "QTE( )",
           title = "Quantile treatment effects")
lines!(ax2, collect( _grid), qte_curve, color = :black, linewidth = 2)
hlines!(ax2, [ate], color = :gray40, linestyle = :dash)
text!(ax2, 0.05, ate + 0.02; text = "ATE", color = :gray40, fontsize = 11)

fig
```

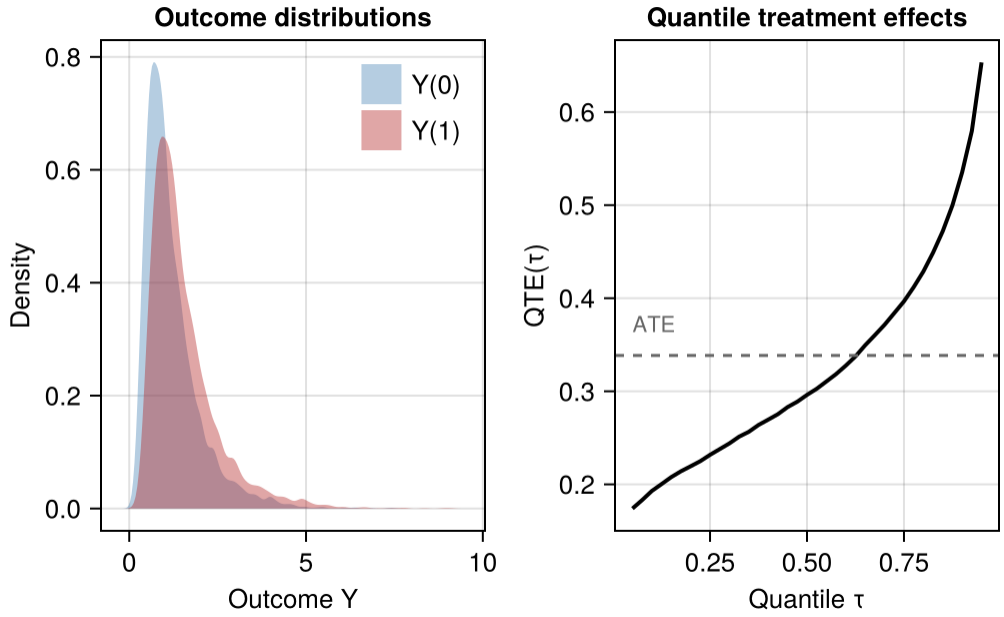


Figure 12.1.: Treated and control marginal distributions. The QTE curve traces the horizontal distance between the two CDFs at each quantile.

The treatment shifts and stretches the distribution. The QTE curve shows that low quantiles move little and high quantiles move more.

12.2. Identifying counterfactual distributions

QTE requires the two counterfactual marginal distributions. Observational data gives only $F_{Y|D=1}$ and $F_{Y|D=0}$. Identification needs the same kind of assumptions as ATE, but applied to the whole distribution.

Common approaches are:

Inverse propensity weighting on the CDF. Estimate the propensity score $\hat{\pi}(x)$ and weight observations to construct counterfactual empirical CDFs:

$$\hat{F}_{Y(d)}(y) = \frac{\sum_{i=1}^{n} w_i^{(d)} \mathbf{1}\{D_i = d\}\mathbf{1}\{Y_i \le y\}}{\sum_{i=1}^{n} w_i^{(d)} \mathbf{1}\{D_i = d\}}, \qquad w_i^{(d)} = \frac{1}{\hat{\pi}(x_i)^d (1-\hat{\pi}(x_i))^{1-d}}. \tag{12.2}$$

We self-normalize (divide by the sum of the weights rather than by n) so that $\hat{F}$ is a proper CDF that reaches 1; the unnormalized Horvitz–Thompson form tends to 1 only in expectation and need not reach 1 in any given sample, which destabilizes the upper quantiles. Then $\widehat{\mathrm{QTE}}(\tau) = \hat{F}^{-1}_{Y(1)}(\tau) - \hat{F}^{-1}_{Y(0)}(\tau)$. This is the Firpo (2007) estimator for QTE under unconfoundedness.

Distribution regression. Model $P(Y \le y \mid X, D)$ as a flexible function of X for each threshold y (Chernozhukov, Fernández-Val, Melly 2013). Integrating out X gives the counterfactual marginal.

Distribution regression handles the entire distribution simultaneously rather than estimating each quantile separately.

Generative / engression approaches. Train a stochastic model that learns $Y \mid X, D$ as a distribution rather than a conditional mean. Sampling from the trained model gives counterfactual draws, from which any distributional summary follows. This is the route taken by `Engression.jl` and by `Endid.jl` for the DiD setting.

12.3. Distributional DiD with Engression

For panel data, distributional DiD extends the DiD idea from means to distributions. Instead of estimating only the counterfactual mean outcome, it estimates a counterfactual outcome distribution.

`Endid.jl` uses Engression for this. It learns a stochastic model for $Y \mid X$ rather than only a conditional mean. Counterfactual draws from that model give QTEs, but of a transformed outcome. Following Lee and Wooldridge (2025), `Endid.jl` first subtracts each unit's pre-treatment mean (or pre-treatment trend) and then works with the post-treatment average of what remains. Its QTEs are quantile effects on that transformed outcome, not on the outcome level. The two can differ even when the mean effects agree. Let B and e be independent standard normals, with $Y_{pre} = B$, $Y_{post}(0) = B + e$ and $Y_{post}(1) = 2B + e$. The level QTE at quantile q is $(\sqrt{5} - \sqrt{2})\,\Phi^{-1}(q)$, while after subtracting B it is $(\sqrt{2} - 1)\,\Phi^{-1}(q)$.

> **Note**
>
> The block below is **not executed** (`eval: false`). `Endid.jl` and its Engression backend are not part of this book's `Project.toml`, so the example will not run in the book's default environment. To run it, add `Endid` (and its dependencies) to a separate project environment first; see the `Endid.jl` repository for installation instructions.

```julia
using Endid

# Panel data: y outcome, id unit, time period, post 1 if post-treatment
# D = 1 marks treated units (optional; otherwise inferred from post)
res = endid(
    df, :y, :id, :time, :post;
    dvar       = :D,
    controls   = [:age, :income],
    nboot      = 100,
    num_epochs = 500,
    hidden_dim = 64,
)

println(res)   # ATT + bootstrap SE
res.qte        # DataFrame of QTE estimates at default quantiles 0.1:0.1:0.9
plot(res)      # QTE curve with confidence band
```

The `endid` function returns an `EndidResult` containing:

- `att` — the average treatment effect on the treated, with bootstrap SE and CI
- `qte` — a `DataFrame` of quantile treatment effects on the treated, one row per quantile in the grid (default `0.1:0.1:0.9`), each with bootstrap standard errors
- `model` — the trained Engression model (for further sampling / introspection)
- `design` — `"common_timing"` or `"staggered"` depending on which interface was used

For staggered adoption (different treatment times across cohorts), use `endid_staggered` with a `gvar` column carrying each unit's first-treatment period.

12.4. When QTE differs from ATT: a panel simulation

Now simulate panel data where the treatment effect depends on unit type:

```
Random.seed!(11)
n_units = 500
n_time  = 4
T0      = 2                # last pre-treatment period
treated_frac = 0.5
n_treated    = Int(round(n_units * treated_frac))

# Unit type (latent ability); drives both Y and the treatment effect size
ability = rand(Normal(0, 1), n_units)

# Build long panel
rows = NamedTuple[]
for i in 1:n_units, t in 1:n_time
    post = t > T0 ? 1 : 0
    treated = i <= n_treated ? 1 : 0
    # Heterogeneous TE: large at the top of the ability distribution
    te = (treated == 1 && post == 1) ? 0.4 * ability[i] + 0.5 : 0.0
    y = ability[i] + 0.3 * t + te + 0.4 * randn()
    push!(rows, (id = i, time = t, y = y, post = post, D = treated))
end
df = DataFrame(rows)
first(df, 5)
```

	id	time	y	post	D
	Int64	Int64	Float64	Int64	Int64
1	1	1	-0.946791	0	1
2	1	2	-0.253741	0	1
3	1	3	0.619691	1	1
4	1	4	1.32825	1	1
5	2	1	-0.603713	0	1

A standard ATT regression reports the mean effect. Here that hides the fact that low-type and high-type units have different effects.

```
# Compute empirical QTE on the simulated data by comparing treated post-period
# outcomes to a benchmark constructed from pre-period treated outcomes shifted
# by the control units' time trend (the standard DiD-style counterfactual,
# applied at each quantile rather than at the mean).

post_treated = df[(df.D .== 1) .& (df.post .== 1), :y]
pre_treated  = df[(df.D .== 1) .& (df.post .== 0), :y]
post_control = df[(df.D .== 0) .& (df.post .== 1), :y]
pre_control  = df[(df.D .== 0) .& (df.post .== 0), :y]

 _grid = 0.1:0.1:0.9
qte_did = [
    (quantile(post_treated,  ) - quantile(pre_treated,  )) -
    (quantile(post_control,  ) - quantile(pre_control,  ))
    for   in  _grid
]

result = DataFrame(quantile = collect( _grid),
                   QTE_DiD  = qte_did)
result
```

	quantile	QTE_DiD
	Float64	Float64
1	0.1	0.0542613
2	0.2	0.330757
3	0.3	0.282768
4	0.4	0.454995
5	0.5	0.441014
6	0.6	0.658351
7	0.7	0.697324
8	0.8	0.833802
9	0.9	1.02507

The curve below is a difference-in-quantiles: it subtracts quantile-by-quantile the control change from the treated change. Reading it as the QTE for the treated, the difference between the marginal quantiles of $Y_{post}(1)$ and $Y_{post}(0)$ among treated units, needs one assumption: that the treated group's untreated quantiles would have changed by the same amount as the control group's,

$$Q_{Y_{post}(0)|D=1}(q) = Q_{Y_{pre}(0)|D=1}(q) + Q_{Y_{post}(0)|D=0}(q) - Q_{Y_{pre}(0)|D=0}(q),$$

with the right side itself a valid quantile function. Parallel trends in means does not deliver this. Changes-in-changes is a different restriction with its own counterfactual construction. No rank condition is needed for the QTE, which is a contrast of marginal quantiles. A rank condition matters only for a different object, the distribution of individual effects. Rank similarity does not identify that distribution. Rank invariance does, but even then the difference of marginal quantiles is the effect for the unit at rank q, and it is a quantile of the individual effects only if it rises with q.

Its *average*, though, is not a separate quantity: because $\int_0^1 F^{-1}(\tau)\,d\tau = E[Y]$, averaging this curve over the whole unit interval returns exactly the ordinary difference-in-means DiD. On the nine-point grid used here the average is 0.531 against a difference-in-means DiD of 0.538, and the gap closes as the grid is refined (0.534 on a 199-point grid, 0.536 on a 999-point grid) — it is discretization, not a difference in estimand. So the honest summary is that the quantile-DiD curve adds nothing to what we already know about the *mean* contrast, which its own average reproduces, while it does add descriptive information about the *shape* of the change: here, larger differences at the top of the outcome distribution.

```
fig = Figure(size = (640, 360), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Quantile τ", ylabel = "Difference-in-quantiles",
           title = "Difference-in-quantiles (descriptive quantile-DiD)")
lines!(ax, result.quantile, result.QTE_DiD, color = :firebrick, linewidth = 2)
scatter!(ax, result.quantile, result.QTE_DiD, color = :firebrick, markersize = 7)
hlines!(ax, [mean(result.QTE_DiD)], color = :gray40, linestyle = :dash)
text!(ax, 0.12, mean(result.QTE_DiD) + 0.02;
      text = "mean = difference-in-means DiD", color = :gray40, fontsize = 11)
fig
```

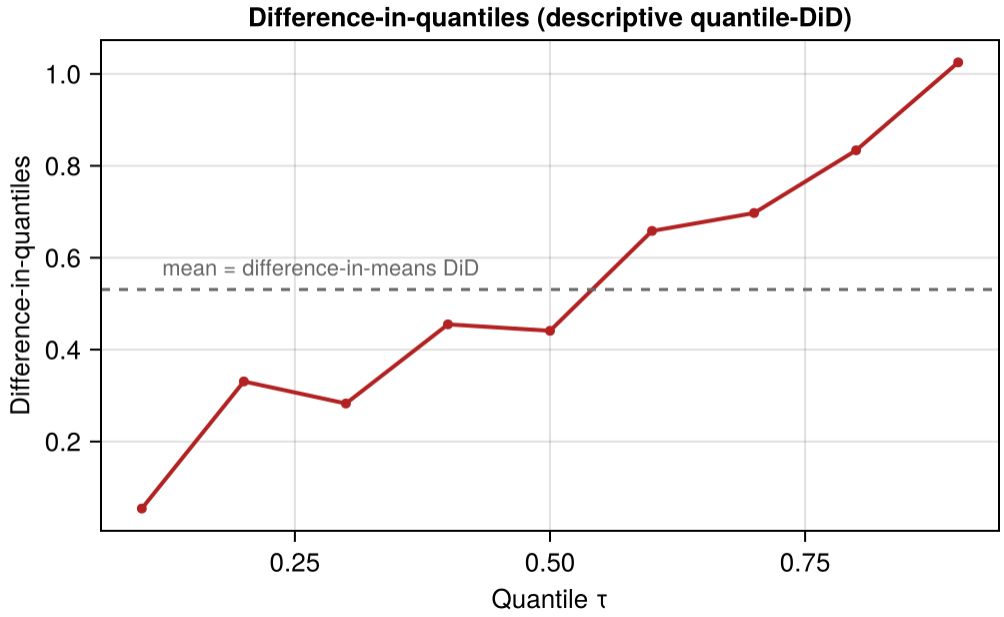


Figure 12.2.: Difference-in-quantiles (a descriptive quantile-DiD), computed at each quantile. It identifies the QTE for the treated only under the additive quantile-change restriction in the text; it is a quantile of individual effects only under rank invariance and a monotone curve. Its average, by contrast, *is* the ordinary difference-in-means DiD – averaging a quantile function returns a mean – so the information the curve adds is its shape, not its level.

The simple quantile-DiD above ignores covariate adjustment and bootstrap inference. `Endid.jl` adds both.

12.5. Beyond QTE: posterior bands as distributional output

TASC also produces more than a point estimate, but for a different object: the counterfactual path of one treated unit. QTE and TASC answer different questions:

- **QTE / Engression DiD**: how does the treatment effect *vary across the marginal distribution of outcomes* when many units are treated?
- **TASC posterior band**: how uncertain is the *single counterfactual path* when one unit is treated?

Use QTE when many treated units may have heterogeneous effects. Use TASC when the problem is a single treated unit with serial dependence.

12.6. Summary

- ATE summarizes the mean. QTE summarizes changes across the outcome distribution.
- QTE identification uses the same assumptions as ATE, but for CDFs.
- `Endid.jl` estimates distributional DiD through Engression.
- Report QTEs when the effect may differ across wages, test scores, health outcomes, sales, or other outcome distributions.

Part III.

Designs

13. Difference-in-Differences

```
using DataFrames
using DataFramesMeta
using Panelest
using Vcov
using StatsModels
import StatsAPI: coeftable
using ReadStatTables
using CSV
using SynthDiD
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using Printf
using RegressionTables
using Random
using Distributions
```

When unconfoundedness fails, we need a different assumption. Parallel trends, which leads to the difference-in-differences estimator, restricts a different counterfactual object. It is not weaker in general: neither assumption implies the other.

> **Related reading**: For a side-by-side comparison of DiD, synthetic DiD, synthetic control, and TASC on a *single* panel — the Proposition 99 California smoking data, where the four estimators do not agree — see the Same data, different estimators chapter in *Topics on Econometrics and Causal Inference*.

13.1. $T = 2$ Panel Data

Suppose we have two periods, $t = (1, 2)$. Some units are treated just prior to period 2. For each individual i, there are four potential outcomes:

$$[Y_{i1}(0), Y_{i1}(1), Y_{i2}(0), Y_{i2}(1)] \tag{13.1}$$

Let D denote the treated group. The ATT in period 2 is

$$\tau_{2,att} = E(Y_2(1) - Y_2(0)|D = 1) \tag{13.2}$$

The first term is observed. The second is the counterfactual.

13.2. Parallel Trends Assumption

$$E[Y_2(0) - Y_1(0)|D = 1] = E[Y_2(0) - Y_1(0)|D = 0] \tag{13.3}$$

or

$$E[\Delta Y(0)|D = 1] = E[\Delta Y(0)|D = 0] \tag{13.4}$$

In words, the change in untreated potential outcomes between periods 1 and 2 is the same for treated and control groups.

13.3. No Anticipation Assumption

$$E[Y_1(1) - Y_1(0)|D = 1] = 0 \tag{13.5}$$

This is to say, in period 1, there is no treatment effect for the treated group.

13.4. DiD is Identified with PT and NA

With Parallel Trends, we notice

$$E[Y_2(0)|D = 1] = E[Y_1(0)|D = 1] + E[Y_2(0) - Y_1(0)|D = 0] \tag{13.6}$$

Then, with No Anticipation,

$$\begin{aligned} E[Y_2(0)|D = 1] &= E[Y_1(1)|D = 1] + E[Y_2(0) - Y_1(0)|D = 0] \\ &= E[Y_1|D = 1] + E[Y_2 - Y_1|D = 0] \end{aligned} \tag{13.7}$$

$$\tau_{2,att} = E[Y_2 - Y_1|D = 1] - E[Y_2 - Y_1|D = 0] \tag{13.8}$$

Parallel trends and no anticipation are the two assumptions we name, but they are not the whole set the derivation uses. It also maintains consistency and a well-defined treatment, so that $Y = DY(1) + (1 - D)Y(0)$ and the treatment has no hidden versions with different effects. It maintains no interference for the stated estimand: one unit's outcome does not move with another unit's treatment. Section 13.13 returns to this and shows what the estimand becomes when it fails. It maintains overlap, so both groups are observed in both periods and, once covariates enter, comparable units exist in each. And it maintains stable composition, so the same population is sampled in both periods. Differential attrition or entry breaks the comparison even when trends are parallel.

13.5. Advantages and Disadvantages of DiD

Advantages:

- No need to assume unconfoundedness. PT and NA take its place, alongside the maintained conditions listed above. Or say we don't need treatment itself to be independent of potential outcomes, but we do need it to be independent to the change in potential outcome at least for untreated state. This allows selection on untreated outcome levels. If the selection bias does not change over time, then DiD can handle it. It is not a relaxation of unconfoundedness, though: groups can have equal untreated post-period levels and unequal untreated changes, and then unconfoundedness holds while parallel trends fails.

Disadvantages:

- PT and NA might be violated.
- PT is not scale free, in the sense that even if outcome can have PT, but then non-linear transformation of outcome won't have PT (for example log of Y).

13.6. Traditional DiD

In practice, DiD in many period setting is usually done with

$$Y_{it} = \alpha_i + \phi_t + W_{it}\beta + \epsilon_{it} \tag{13.9}$$

here $W_{it} = (1[t = 2] \cdot D_i)$, which is the interaction of post-treatment indicator and treatment group indicator.

This is usually called Two Way Fixed Effect (TWFE). There are multiple ways to implement the same model in practice.

TWFE can be done by "Pooled OLS". That is, using OLS on time dummies and firm (individual) dummies. Wooldridge (2021) shows it's equivalent to use treatment dummies, instead of individual dummies. He also shows that this model can be equivalently implemented with fixed effect model and random effect model.

13.7. TWFE with Staggered Treatment Timing

The problem comes in when there is different timing of treatment. People used to still use

$$Y_{it} = \alpha_i + \phi_t + W_{it}\beta + \epsilon_{it} \tag{13.10}$$

where W_{it} now is a dummy when an individual i gets treated at time t.

However, what is β here?

13.8. Goodman-Bacon Decomposition

Goodman-Bacon (2021) showed that β in the TWFE is a weighted average of many different treatment effects, between treated cohorts, and control units, both can be different at different time points. The weights are a function of the size of the subsample, relative size of treatment and control units, and the timing of treatment in the sub sample. The decomposition weights themselves are non-negative and sum to one; the problem is that the units treated earlier are used as controls later. When those "forbidden" 2x2 comparisons are expressed in terms of the underlying cohort-time ATTs, some ATTs receive negative weight (de Chaisemartin and D'Haultfœuille 2020). Therefore there is no meaningful interpretation of β: it does not need to be a convex combination of treatment effects.

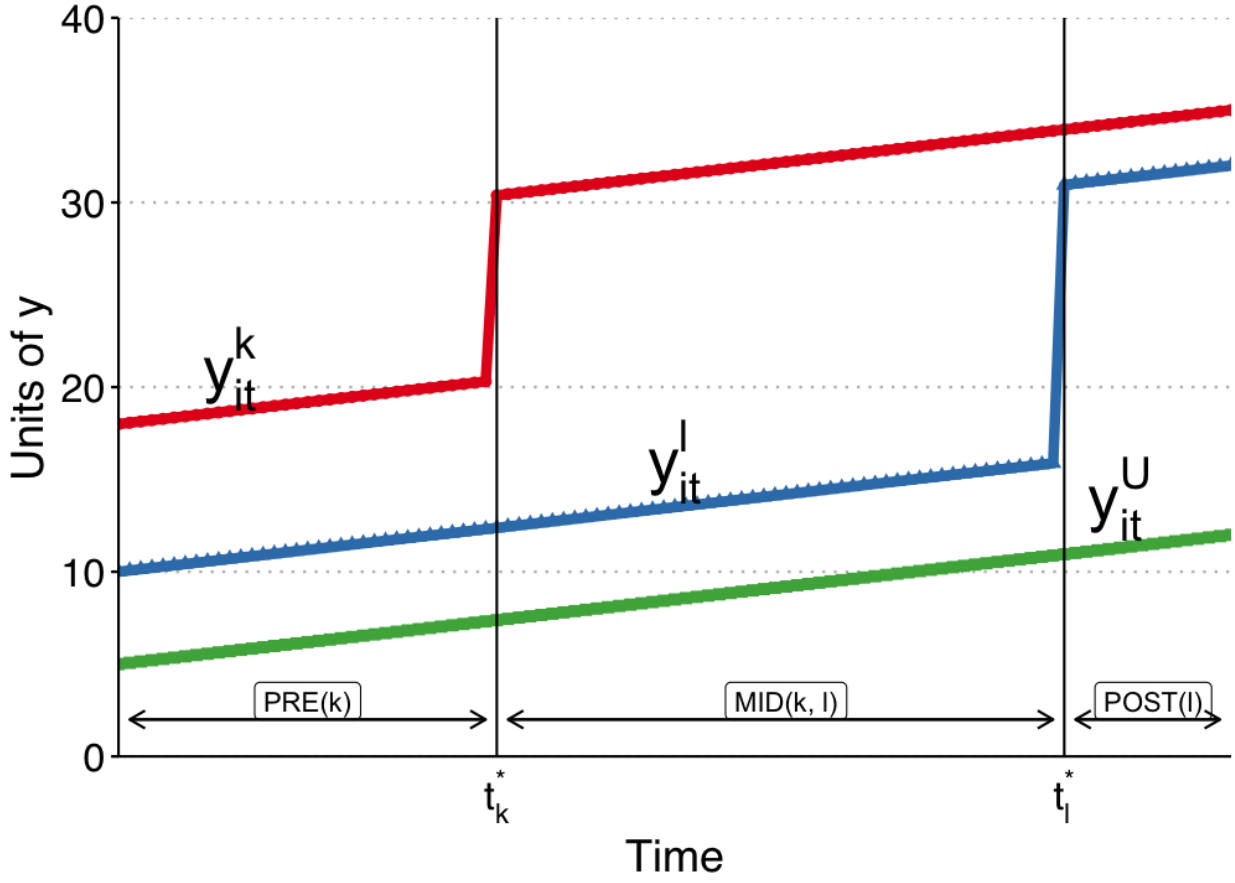


Figure 13.1.: Bacon decomposition

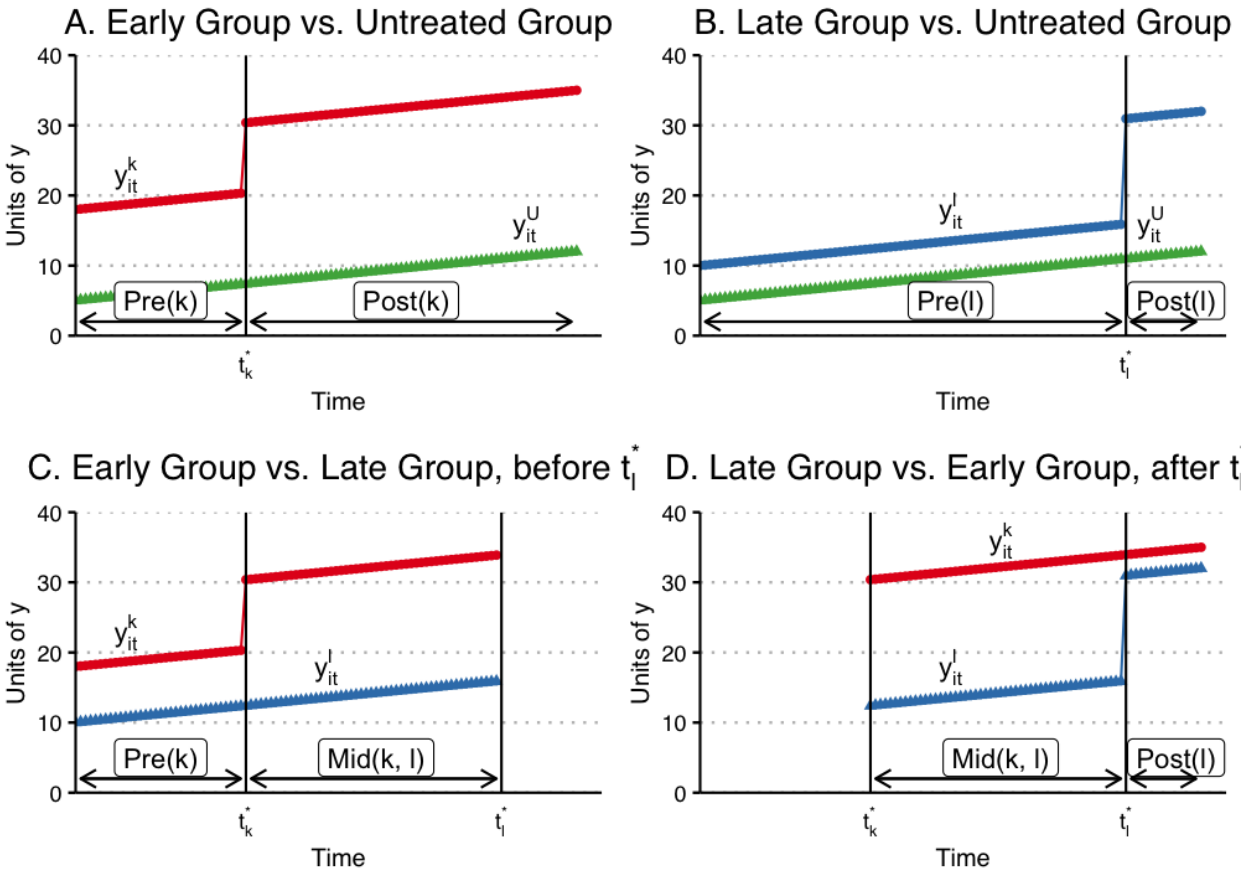


Figure 13.2.: Bacon decomposition

13.9. Wooldridge's ETWFE

There are a lot of ways to deal with staggered DiD situation. Wooldridge (2021) is basically saying: This is not a problem of TWFE, it's a mis-use of TWFE. The reason we get non-sensible result of β is that we know there is heterogeneous treatment effect, in the sense the treatment effect differs across cohort, but we force them to be the same. If we relax it, it can work. As he shows, this works when we specify cohort effects accordingly.

$$y_{it} = \alpha_i + \phi_t + \sum_{g=g_0}^{G} \sum_{t=g}^{T} \lambda_{g,t} \times 1(g,t) + \epsilon_{i,t} \tag{13.11}$$

Here g is a cohort indicator, a cohort is determined by the time of getting treatment. ETWFE is allowing each cohort to have different effect at each different time point after being treated. The baseline group is the never treated group. If there is no never treated group, it can easily be changed to comparing to the last treated group.

When the treatment comes in amounts, such as a dose or a count, the same regression extends to it. The chapter DiD with Continuous Treatment covers that case.

13.10. Example 1: US Teen Employment

We'll use the mpdta dataset on US teen employment from the did package. "Treatment" in this dataset refers to an increase in the minimum wage rate. Our goal is to estimate the effect of this minimum wage treatment (treat) on the log of teen employment (lemp). Notice that the panel ID is at the county level (countyreal), but treatment was staggered across cohorts (first_treat) so that a group of counties were treated at the same time. In addition to these staggered treatment effects, we also observe log population (lpop) as a potential control variable.

```
# mpdta from the did R package (Callaway & Sant'Anna 2021)
mpdta = CSV.read("data/mpdta.csv", DataFrame)
rename!(mpdta, "first.treat" => "first_treat")
first(mpdta, 6)
```

	year	countyreal	lpop	lemp	first_treat	treat
	Int64	Int64	Float64	Float64	Int64	Int64
1	2003	8001	5.89676	8.46147	2007	1
2	2004	8001	5.89676	8.33687	2007	1
3	2005	8001	5.89676	8.34022	2007	1
4	2006	8001	5.89676	8.37816	2007	1
5	2007	8001	5.89676	8.48735	2007	1
6	2003	8019	2.23238	4.99721	2007	1

The three tables below give the panel shape: observations per year, the ever-treated indicator, and the treatment cohorts.

```
display(combine(groupby(mpdta, :year), nrow))
display(combine(groupby(mpdta, :treat), nrow))
display(combine(groupby(mpdta, :first_treat), nrow))
```

	year	nrow
	Int64	Int64
1	2003	500
2	2004	500
3	2005	500
4	2006	500
5	2007	500

	treat	nrow
	Int64	Int64
1	0	1545
2	1	955

	first_treat	nrow
	Int64	Int64
1	0	1545
2	2004	100
3	2006	200
4	2007	655

The panel is balanced: 500 counties observed in each of 2003 through 2007, 2,500 observations. 955 observations belong to ever-treated counties and 1,545 to never-treated ones. The cohorts are 2004 (100 observations, so 20 counties), 2006 (200, or 40 counties) and 2007 (655, or 131 counties), with 309 counties never treated. Treatment timing is staggered, which is exactly the case where plain TWFE misbehaves.

We fit ETWFE with log population as a control, clustering on county.

```
mod = etwfe(mpdta, @formula(lemp ~ lpop);    # outcome ~ controls
            tvar = :year,                    # time variable
            gvar = :first_treat,             # group variable
            vcov = Vcov.cluster(:countyreal));  # clustered by county
```

```
show_regression_html(mod.model)
```

	Estimate	Std. Error	t value	Pr(>|t|)
_D_g2004_t2004	-0.021	0.022	-0.978	0.328
_D_g2004_t2004_x_lpop	0.005	0.018	0.263	0.792
_D_g2004_t2005	-0.082	0.027	-2.991	0.003 **
_D_g2004_t2005_x_lpop	0.025	0.018	1.403	0.161
_D_g2004_t2006	-0.138	0.031	-4.478	7.534e-06 ***
_D_g2004_t2006_x_lpop	0.051	0.021	2.408	0.016 *
_D_g2004_t2007	-0.110	0.032	-3.390	6.997e-04 ***
_D_g2004_t2007_x_lpop	0.011	0.027	0.423	0.672
_D_g2006_t2006	0.003	0.019	0.134	0.893
_D_g2006_t2006_x_lpop	0.039	0.016	2.364	0.018 *
_D_g2006_t2007	-0.045	0.022	-2.051	0.040 *
_D_g2006_t2007_x_lpop	0.038	0.022	1.694	0.090 .
_D_g2007_t2007	-0.046	0.018	-2.557	0.011 *
_D_g2007_t2007_x_lpop	-0.020	0.016	-1.225	0.221
_x_lpop	1.065	0.022	48.821	< 2e-16 ***
_x_lpop_g2004	0.051	0.038	1.350	0.177
_x_lpop_g2006	-0.041	0.047	-0.867	0.386
_x_lpop_g2007	0.056	0.039	1.416	0.157
_x_lpop_t2004	0.011	0.008	1.458	0.145
_x_lpop_t2005	0.021	0.008	2.558	0.011 *
_x_lpop_t2006	0.011	0.011	0.974	0.330
_x_lpop_t2007	0.021	0.012	1.772	0.076 .

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The raw model has one coefficient per cohort-year cell plus the control interactions, 22 in all. That is not something to read directly. `emfx()` aggregates the cells into the ATT.

```
emfx(mod)
```

	type	estimate	std_error	conf_low	conf_high
	String	Float64	Float64	Float64	Float64
1	ATT	-0.050627	0.0124973	-0.0751212	-0.0261329

The ATT is -0.0506 with a standard error of 0.0125 and a 95% interval of $[-0.0751, -0.0261]$. Raising the minimum wage lowered teen employment by about 5% on average across treated county-years.

The same cells can be aggregated by time since treatment instead.

```
mod_es = emfx(mod, type = "event")
mod_es
```

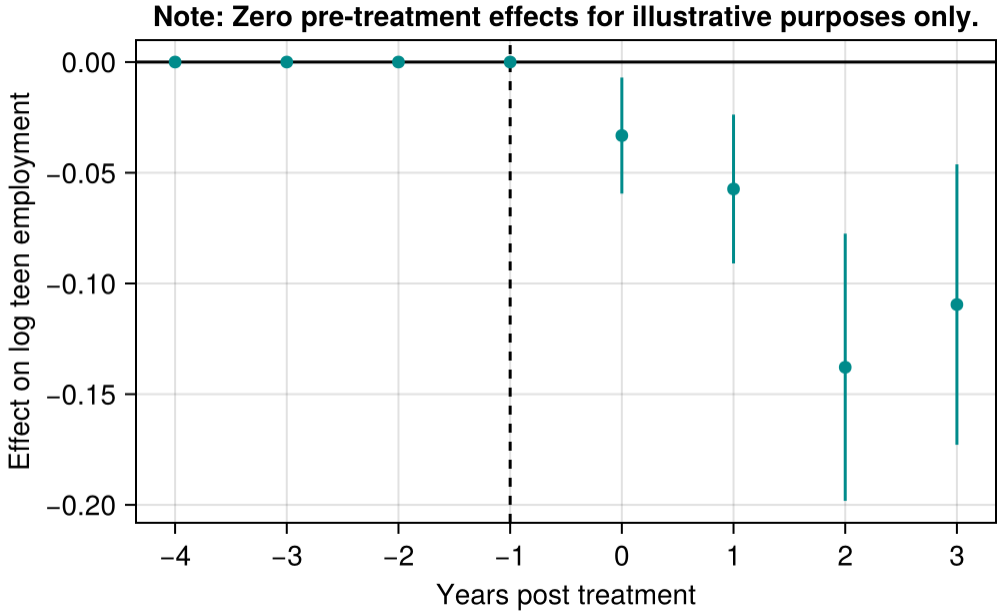
	event	estimate	std_error	conf_low	conf_high
	Int64	Float64	Float64	Float64	Float64
1	0	-0.0332122	0.013366	-0.059409	-0.00701545
2	1	-0.0573456	0.0171496	-0.0909583	-0.023733
3	2	-0.13787	0.0307884	-0.198214	-0.0775264
4	3	-0.109539	0.0323153	-0.172876	-0.0462028

The effect grows with exposure: -0.033 in the treatment year, -0.057 after one year, -0.138 after two, -0.110 after three. All four are significant. The standard errors widen with event time, from 0.013 to 0.032, because the later horizons rest on fewer cohorts: event time 0 uses all three cohorts (191 counties), event time 1 uses the 2004 and 2006 cohorts (60 counties), and event times 2 and 3 rest on the 2004 cohort alone (20 counties).

Adding `post_only = FALSE` keeps the pre-treatment periods in the plot.

```
mod_es2 = emfx(mod, type = "event", post_only = false)

fig = Figure()
ax = Axis(fig[1, 1],
    xlabel = "Years post treatment",
    ylabel = "Effect on log teen employment",
    xticks = -4:3,
    title = "Note: Zero pre-treatment effects for illustrative purposes only.")
hlines!(ax, 0, color = :black)
vlines!(ax, -1, color = :black, linestyle = :dash)
ok = .!isnan.(mod_es2.std_error)
rangebars!(ax, mod_es2.event[ok], mod_es2.conf_low[ok], mod_es2.conf_high[ok], color = :darkcya
scatter!(ax, mod_es2.event, mod_es2.estimate, color = :darkcyan)
fig
```



Read the pre-treatment points with care, and note the caption on the figure. With `etwfe`'s default comparison group the pre-treatment estimates are forced to zero by construction, so their sitting on the line is arithmetic, not evidence for parallel trends. The Poisson example below uses `cgroup = "never"`, which frees them and makes the pre-period a real check.

13.11. Example 2: Staggered DiD

To see whether ETWFE recovers what it should, we switch to simulated data where the answer is recorded in the file.

`data/did_staggered_6.dta` is Wooldridge's simulated staggered panel: 500 units observed annually from 2001 to 2006, 3,000 observations. Units are treated in 2004 (119 units), 2005 (83) or 2006 (36), and 262 are never treated. The outcome is `y`, `x` is a covariate, `d4`, `d5` and `d6` are cohort dummies, and `f04`, `f05`, `f06` are year dummies. What makes the file useful is `te4`, `te5` and `te6`: the true unit-level treatment effect each unit would experience under each cohort's treatment date. The effects are heterogeneous across units and grow with time since treatment, which is precisely the configuration that breaks plain TWFE.

```
# now we do staggered did.
did_data = DataFrame(readstat("data/did_staggered_6.dta"))
did_data.first_treat = [r.d4 == 1 ? 2004 : r.d5 == 1 ? 2005 : r.d6 == 1 ? 2006 : 0
                        for r in eachrow(did_data)]

first(select(did_data, :id, :year, :y, :x, :d4, :d5, :d6, :te4, :te5, :te6, :first_treat), 6)
```

	id	year	y	x	d4	d5	d6	te4	te5	te6	first_treat
	Int16	Int16	Float32	Float32	Int8	Int8	Int8	Float32	Float32	Float32	Int64
1	1	2001	18.3596	0.899984	0	0	0	0.0	0.0	0.0	0
2	1	2002	18.1016	0.899984	0	0	0	0.0	0.0	0.0	0
3	1	2003	18.5956	0.899984	0	0	0	0.0	0.0	0.0	0
4	1	2004	16.5003	0.899984	0	0	0	1.43223	0.0	0.0	0
5	1	2005	19.9573	0.899984	0	0	0	2.57319	2.24723	0.0	0
6	1	2006	19.8659	0.899984	0	0	0	5.06752	4.23316	0.785892	0

```
combine(groupby(did_data, :first_treat), nrow)
```

	first_treat	nrow
	Int64	Int64
1	0	1572
2	2004	714
3	2005	498
4	2006	216

The counts are observations, six per unit: 1,572 never-treated, 714 in the 2004 cohort, 498 in 2005, 216 in 2006.

Because the true effects are in the data, we can compute the target before estimating it. Grouping the true effects by cohort and year gives the true ATT in each cohort-year cell.

```
cohort_cell(d, f, lab) = ifelse.((d .== 1) .& (f .== 1), lab, missing)
did_data.treated_cohort1 = coalesce.(cohort_cell(did_data.d4, did_data.f04, "d4f04"),
                                     cohort_cell(did_data.d4, did_data.f05, "d4f05"),
                                     cohort_cell(did_data.d4, did_data.f06, "d4f06"))
did_data.treated_cohort2 = coalesce.(cohort_cell(did_data.d5, did_data.f05, "d5f05"),
                                     cohort_cell(did_data.d5, did_data.f06, "d5f06"))
did_data.treated_cohort3 = cohort_cell(did_data.d6, did_data.f06, "d6f06")

combine(groupby(dropmissing(did_data, :treated_cohort1), :treated_cohort1), :te4 => mean => :me
```

	treated_cohort1	mean4
	String	Float32
1	d4f04	3.75709
2	d4f05	4.01757
3	d4f06	4.57895

```
display(combine(groupby(dropmissing(did_data, :treated_cohort2), :treated_cohort2), :te5 => mea
display(combine(groupby(dropmissing(did_data, :treated_cohort3), :treated_cohort3), :te6 => mea
```

	treated_cohort2	mean5
	String	Float32
1	d5f05	2.73754
2	d5f06	3.51606

	treated_cohort3	mean6
	String	Float32
1	d6f06	1.84886

So the true cohort-year ATTs are: the 2004 cohort gains 3.76, 4.02 and 4.58 in 2004, 2005 and 2006; the 2005 cohort gains 2.74 and 3.52 in 2005 and 2006; the 2006 cohort gains 1.85 in 2006. Effects grow with time since treatment and shrink across later cohorts — heterogeneity in both directions.

Averaging those cells over the 559 treated observations gives a true overall ATT of **3.677**. That is the number ETWFE has to reproduce.

```
# this replicates Jeff's results with pooled ols or xtreg, with covariate x.
mod = etwfe(did_data, @formula(y ~ x);    # outcome ~ controls
            tvar = :year,                 # time variable
            gvar = :first_treat,          # group variable
            vcov = Vcov.cluster(:id))

show_regression_html(mod.model)
```

	Estimate	Std. Error	t value	Pr(>|t|)
_D_g2004_t2004	3.510	0.303	11.579	< 2e-16 ***
_D_g2004_t2004_x_x	-0.373	1.046	-0.357	0.721
_D_g2004_t2005	4.240	0.329	12.886	< 2e-16 ***
_D_g2004_t2005_x_x	0.532	1.045	0.509	0.611
_D_g2004_t2006	4.210	0.331	12.732	< 2e-16 ***
_D_g2004_t2006_x_x	2.588	1.064	2.431	0.015 *
_D_g2005_t2005	3.008	0.341	8.822	< 2e-16 ***
_D_g2005_t2005_x_x	0.959	0.633	1.515	0.130
_D_g2005_t2006	3.702	0.368	10.057	< 2e-16 ***
_D_g2005_t2006_x_x	0.157	0.862	0.182	0.856
_D_g2006_t2006	2.017	0.571	3.533	4.109e-04 ***
_D_g2006_t2006_x_x	0.724	1.171	0.618	0.537
_x_x	0.241	0.453	0.532	0.595
_x_x_g2004	-1.191	0.934	-1.275	0.202
_x_x_g2005	0.551	0.607	0.908	0.364
_x_x_g2006	0.994	0.828	1.202	0.229
_x_x_t2002	0.396	0.328	1.209	0.227
_x_x_t2003	0.976	0.319	3.054	0.002 **
_x_x_t2004	0.331	0.351	0.942	0.346
_x_x_t2005	0.239	0.398	0.600	0.549
_x_x_t2006	0.360	0.422	0.855	0.393

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
emfx(mod)
```

	type	estimate	std_error	conf_low	conf_high
	String	Float64	Float64	Float64	Float64
1	ATT	3.67208	0.172081	3.33481	4.00936

ETWFE gives 3.67 with a standard error of 0.172 and a 95% interval of $[3.33, 4.01]$, against a true 3.677. It recovers the ATT to three decimals.

The same holds for the disaggregations. By time since treatment:

```
mod_es = emfx(mod, type = "event")
mod_es
```

	event	estimate	std_error	conf_low	conf_high
	Int64	Float64	Float64	Float64	Float64
1	0	3.10909	0.212614	2.69237	3.5258
2	1	4.01879	0.248267	3.5322	4.50539
3	2	4.20954	0.330621	3.56154	4.85755

Estimated 3.11, 4.02 and 4.21 at event times 0, 1 and 2, against true values of 3.11, 3.81 and 4.58. Every interval covers its truth, and the estimates reproduce the growth in the effect with exposure.

And by calendar year:

```
mod_es2 = emfx(mod, type = "calendar")
mod_es2
```

	calendar	estimate	std_error	conf_low	conf_high
	Int64	Float64	Float64	Float64	Float64
1	2004	3.5099	0.303131	2.91578	4.10403
2	2005	3.73388	0.253321	3.23738	4.23038
3	2006	3.70073	0.249441	3.21183	4.18962

Estimated 3.51, 3.73 and 3.70 in 2004, 2005 and 2006, against true values of 3.76, 3.49 and 3.80. Again all three intervals cover.

Note that the calendar profile is nearly flat while the event-time profile rises. There is no contradiction: later calendar years mix an older cohort with large accumulated effects and a newly treated cohort with small ones, and the two movements offset. This is why the aggregation has to be chosen to match the question being asked.

13.12. Nonlinear ETWFE: Count and Binary Outcomes

> **Related reading**: The theory behind nonlinear ETWFE — Wooldridge's conditional parallel-trends assumption on the linear index, the incidental parameters argument for cohort rather than unit intercepts, and the average partial effects that follow — is developed at length in Extended two-way fixed effects in *Topics on Econometrics and Causal Inference*, with count data and Poisson with exposure covering the count case in more detail.

ETWFE is not limited to linear (continuous) outcomes. When the dependent variable is a count or a binary indicator, linear TWFE can give nonsensical results — negative predicted probabilities, additive effects that ignore the bounded or non-negative nature of Y. `etwfe()` supports the Poisson family via the `family` argument, which fits the model with `fepois()`.

The key identifying assumption shifts slightly. Instead of parallel trends on $E[Y_{it}(\infty)]$, we assume parallel trends on the **linear index** — that is, on $\log E[Y_{it}(\infty)]$ for Poisson, or on $\text{logit}\, E[Y_{it}(\infty)]$ for logit. This is Wooldridge's (2023) Conditional Parallel Trends assumption stated on $G^{-1}(E[Y])$. When the true treatment effect is multiplicative (e.g., a policy raises counts by 50% regardless of baseline), this assumption is more natural than additive PT on Y itself.

One implementation detail matters: the nonlinear ETWFE specification uses **cohort fixed effects** (one intercept per treatment cohort) plus year fixed effects — not individual unit fixed effects. `etwfe()` handles this automatically. In the linear case the two are numerically equivalent (Wooldridge 2021), but in nonlinear models unit fixed effects raise the incidental-parameters problem and make average partial effects uncomputable — the unit intercepts cannot be averaged out of the nonlinear link. Wooldridge (2023) instead uses cohort intercepts (a Mundlak device), which is what `etwfe` implements.

13.12.1. Simulated example: staggered count data

We simulate $N = 400$ units over $T = 6$ periods, 2,400 observations. Units are split into four groups of 100 by id: the first is treated from period 3, the second from period 4, the third from period 5, and the fourth never. Each unit draws a fixed effect $\alpha_i \sim N(0, 0.3^2)$. The log mean is $\log \mu_{it} = 1 + 0.2t + \alpha_i + 0.4D_{it}$, and the outcome is $Y_{it} \sim \text{Poisson}(\mu_{it})$.

The treatment effect is multiplicative and homogeneous on the log scale: every treated unit's expected count rises by a factor of $\exp(0.4)$, that is by $\exp(0.4) - 1 = 0.492$, or 49%. Parallel trends holds on the log scale by construction. Here it also holds on the level scale. Cohort membership is independent of the unit effects α_i, so every cohort's untreated mean, $\exp(1 + 0.2t)\, E[e^{\alpha_i}]$, follows the same path. What differs across scales is the effect: a constant proportional effect is a larger absolute effect where the baseline is higher, so level-scale effects change over calendar time. A design where level parallel trends fails would give cohorts different untreated baseline multipliers while keeping common proportional growth.

On the count scale the ATT is not 0.492. It is $E[\mu_{it}(0) \mid \text{treated}] \times (\exp(0.4) - 1)$, and the mean untreated count among treated observations is 7.518, so the true ATT is $7.518 \times 0.4918 = 3.698$ counts.

The table below reports mean counts by cohort and treatment status.

```
Random.seed!(42)
N = 400; Tmax = 6

unit_fe_vals = randn(N) .* 0.3

df_count = DataFrame([(id = i, year = t) for i in 1:N for t in 1:Tmax])
cohort_grp = ((df_count.id .- 1) .÷ 100) .+ 1
df_count.first_treat = [3, 4, 5, 0][cohort_grp]   # 0 = never treated
```

```
df_count.treated = (df_count.first_treat .> 0) .& (df_count.year .>= df_count.first_treat)
df_count.unit_fe = unit_fe_vals[df_count.id]
df_count.log_mu = 1 .+ 0.2 .* df_count.year .+ df_count.unit_fe .+ 0.4 .* df_count.treated
df_count.count = rand.(Poisson.(exp.(df_count.log_mu)))

# true count-scale ATT: mean untreated mean among treated observations times exp(0.4) - 1
mu0 = exp.(df_count.log_mu .- 0.4 .* df_count.treated)
@printf("mean untreated count among treated = %.3f, true ATT = %.3f\n",
        mean(mu0[df_count.treated]), mean(mu0[df_count.treated]) * (exp(0.4) - 1))

combine(groupby(df_count, [:first_treat, :treated]), :count => mean => :mean_count)
```

```
mean untreated count among treated = 7.518, true ATT = 3.698
```

	first_treat	treated	mean_count
	Int64	Bool	Float64
1	0	0	5.75
2	3	0	3.75
3	3	1	10.59
4	4	0	4.31667
5	4	1	11.8167
6	5	0	4.5
7	5	1	12.12

The never-treated group averages 5.75 counts. Treated cohorts average 3.75, 4.32 and 4.50 before treatment and 10.6, 11.8 and 12.1 after. The pre-treatment means look low only because they are drawn from earlier periods, and the time trend adds 0.2 to the log mean each period. This is exactly the comparison that a raw before-after difference would get wrong.

13.12.2. Poisson ETWFE

Pass `family = "poisson"` to `etwfe()`. Everything else — cohort dummies, `emfx()`, event study — works identically to the linear case. We use `cgroup = "never"` so that pre-treatment estimates are not mechanistically forced to zero.

```
mod_pois = etwfe(df_count, @formula(count ~ 1);
                 tvar   = :year,
                 gvar   = :first_treat,
                 family = "poisson",
                 cgroup = "never",
                 vcov   = Vcov.cluster(:id));
```

```
emfx(mod_pois)
```

	type	estimate	std_error	conf_low	conf_high
	String	Float64	Float64	Float64	Float64
1	ATT	3.62343	0.34363	2.94993	4.29693

The estimate is 3.62 counts with a standard error of 0.34 and a 95% interval of $[2.95, 4.30]$, against the true 3.698 derived above. The interval covers it.

`emfx()` returns average marginal effects — the average difference $E[Y(1)] - E[Y(0)]$ in **count units** across treated observations. This is the ATT on the count scale. The underlying model imposes PT on the log scale, so each cohort-time treatment coefficient in the raw model can be exponentiated to get the incidence rate ratio for that cell (here they share a common value because the simulated effect is homogeneous).

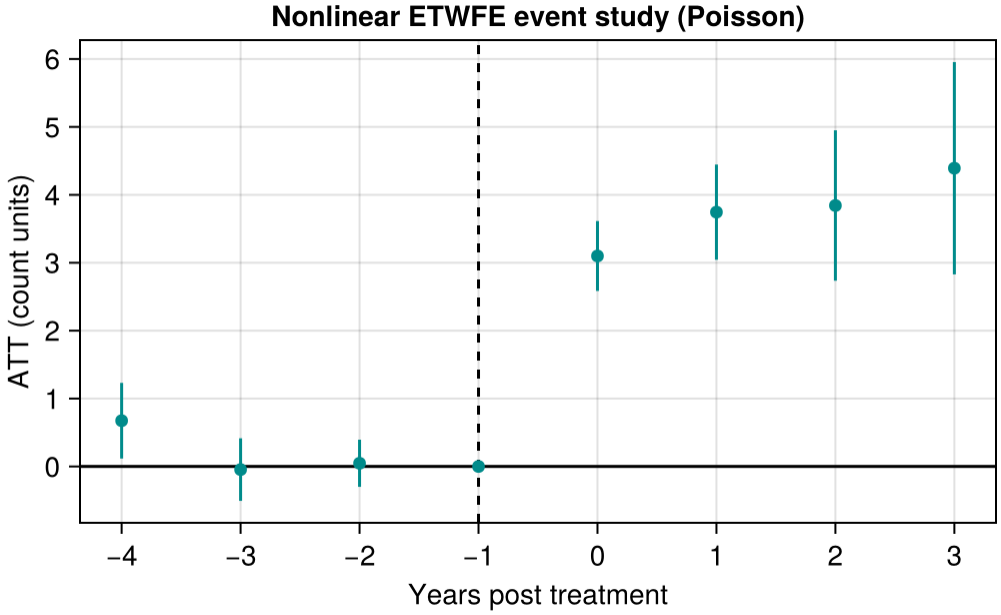
13.12.3. Event study

Because we passed `cgroup = "never"`, the pre-treatment estimates are free parameters rather than zeros by construction, so the pre-period is an actual test here. On the count scale the post-treatment effect should also grow with event time even though the log-scale effect is constant, because the baseline count grows with the time trend: the true values are 3.06, 3.74, 4.18 and 4.55 at event times 0 through 3.

```
mod_pois_es = emfx(mod_pois, type = "event", post_only = false)
display(mod_pois_es)

fig_p = Figure()
ax_p  = Axis(fig_p[1, 1],
    xlabel = "Years post treatment",
    ylabel = "ATT (count units)",
    xticks = -4:3,
    title  = "Nonlinear ETWFE event study (Poisson)")
hlines!(ax_p, 0, color = :black)
vlines!(ax_p, -1, color = :black, linestyle = :dash)
ok = .!isnan.(mod_pois_es.std_error)
rangebars!(ax_p, mod_pois_es.event[ok], mod_pois_es.conf_low[ok], mod_pois_es.conf_high[ok],
           color = :darkcyan)
scatter!(ax_p, mod_pois_es.event, mod_pois_es.estimate, color = :darkcyan)
fig_p
```

	event	estimate	std_error	conf_low	conf_high
	Int64	Float64	Float64	Float64	Float64
1	-4	0.673962	0.284111	0.117115	1.23081
2	-3	-0.045772	0.235216	-0.506785	0.415241
3	-2	0.0470628	0.176909	-0.299671	0.393797
4	-1	0.0	NaN	NaN	NaN
5	0	3.09953	0.262425	2.58519	3.61387
6	1	3.74512	0.357564	3.04431	4.44593
7	2	3.84242	0.565125	2.7348	4.95004
8	3	4.39207	0.797697	2.82861	5.95552



The pre-treatment estimates at event times −3 and −2 are close to zero. The one at −4 is 0.67 with a standard error of 0.28, and its interval excludes zero. It rests on a single cohort-period, the period-5 cohort in period 1. Parallel trends holds by construction in this simulation, so this is one rejection in three pre-period tests at the 5% level, which chance produces often enough. With real data it would call for a closer look. The post-treatment estimates are 3.10, 3.75, 3.84 and 4.39, each with an interval that covers its true value. They rise with event time, which is the count-scale consequence of a constant log-scale effect applied to a growing baseline, not evidence of a growing treatment effect.

13.12.4. Comparison: linear ETWFE on log(count + 1)

A common alternative is to log-transform the count and run linear ETWFE. This is biased when zeros are present (the +1 shift distorts the linear index) and misinterprets the scale of the effect. For comparison:

```
df_count.log_count = log.(df_count.count .+ 1)

mod_lin = etwfe(df_count, @formula(log_count ~ 1);
                tvar = :year,
                gvar = :first_treat,
                vcov = Vcov.cluster(:id))
emfx(mod_lin)
```

	type	estimate	std_error	conf_low	conf_high
	String	Float64	Float64	Float64	Float64
1	ATT	0.399088	0.0323772	0.33563	0.462546

The linear model returns 0.40 with a 95% interval of $[0.336, 0.463]$. That number is neither the count-scale ATT of 3.62 nor the log-scale parameter 0.4, though it happens to sit near the latter because the counts here are large enough that $\log(Y+1) \approx \log Y$. With many zeros the approximation breaks and the estimate drifts.

The linear model estimates the ATT on the $\log(Y+1)$ scale — not the same quantity as the Poisson ATT and harder to interpret. When treatment effects are multiplicative, Poisson ETWFE is the more appropriate model.

13.13. Spatial Interference

Everything in this chapter so far assumes SUTVA: one unit's outcome does not depend on another unit's treatment. That is not always plausible. A treated municipality may affect deforestation in nearby untreated municipalities. A vaccinated household may reduce transmission to nearby households. A labor-market policy in one commuting zone may change flows to its neighbors. With this kind of spatial spillover, the usual DiD estimand no longer has the interpretation we want. Xu (2023) shows that ignoring interference can produce an estimand that is neither a direct effect nor a spillover effect.

Xu (2023, 2026) develops a doubly robust DiD that keeps the identifying logic of conditional parallel trends but adds an exposure mapping $G_{it} = G(i, W_{-it})$ for unit i's neighborhood treatment status. The estimand is the direct ATT at exposure level g,

$$\tau_g(t,c) = \frac{1}{|S_M|} \sum_{i \in S_M} \mathbb{E}[y_{it}(c,g) - y_{it}(\infty,g) \mid z_i, C_i = c, G_{it} = g], \tag{13.12}$$

where C_i is unit i's first treatment period (so $\{C_i > t\}$ is the not-yet-directly-treated comparison group), $S_M = \{C_i = c\} \cup \{C_i > t\}$, and g is a chosen exposure level. The DR plug-in averages three pieces over S_M: an IPW term from the treated cohort, an IPW term from the not-yet-treated, and a regression-imputation term, with three propensity models (η_{tc} for cohort, η_{tcg} and $\eta_{t\infty g}$ for exposure) and two outcome-change regressions. It is consistent if either all three propensities or both outcome models are correctly specified.

The companion package `DidInterference.jl` implements this. The 2x2 base case (Xu 2023):

```
using DidInterference, DataFrames

# Wide-format two-period data with a binary exposure G.
res = did_int_2x2(panel;
    yname      = :Y_post,
    yname_pre  = :Y_pre,
    treat      = :W,
    exposure   = :G,
    g          = 1,
    covariates = [:z1, :z2],
    trim       = 0.01)   # Xu (2026) uses 0.01 in the Brazil application

println(res.estimate, "  ", res.ci)
```

For staggered adoption (Xu 2026), `did_int_staggered()` loops over `(cohort, time)` cells with $t \geq c$, restricts to S_M in each cell, fits the DR estimator, and aggregates across cells using joint-IF stacking (per-cell IFs share the never-treated comparison group, so independence-style SEs underestimate uncertainty noticeably). The R port `didint` has identical estimators; its `vignettes/brazil_amazon.Rmd` vignette replicates Section III of Xu (2026), the *Lista de Municípios Prioritários* policy on Amazon deforestation, using the public Assunção-McMillan-Murphy-Souza-Rodrigues replication archive on Zenodo.

13.14. Persistent Outcomes and Heterogeneous Dynamics

Standard event-study TWFE regressions like

$$Y_{it} = \alpha_i + \gamma_t + \sum_j D_{it}^j \delta_j + X_{it}'\beta + U_{it} \tag{13.13}$$

leave the dynamics of Y unmodelled. Three things get run together here and are worth separating.

The first is serial correlation in U_{it}. That is an inference problem rather than an identification one: with exogenous treatment timing the event-time coefficients remain consistent, but conventional standard errors are wrong, which is why DiD is routinely clustered by unit.

The second is dynamic selection. If units are treated *because* of their recent outcomes, then treatment timing is related to lagged Y and the parallel-trends restriction the specification relies on no longer holds. That is an identification failure, and no amount of clustering repairs it.

The third is what motivates the rest of this section. When the outcome is genuinely persistent — earnings, employment, consumption, anything with habits or adjustment costs — the *structural* process carries its own lag, and Equation 13.13 is a static equation fitted to dynamic data. Which target the event-time coefficients estimate then matters. Under permanent treatment in $Y_{it} = \rho Y_{i,t-1} + \delta D_{it} + U_{it}$, the total effect at horizon k is $\delta \sum_{j=0}^{k} \rho^j$, the immediate effect carried forward through the lag. A well-identified event study estimates that total effect. Its gap from the structural coefficient δ grows with the horizon (Botosaru and Liu, 2025, Figure 1), but that gap is bias only if the target is δ. Persistence in the residual alone does not produce it.

Botosaru and Liu (2025) introduce a **dynamic panel with correlated random coefficients** that handles both persistence and unit-level heterogeneity in dynamic responses:

$$Y_{it} = \rho_Y Y_{i,t-1} + \alpha_i + \sum_j D_{it}^j \delta_{ij} + X_{it}'\beta + U_{it}, \tag{13.14}$$

with a parsimonious AR(1) structure on the event-time effects to keep dimensionality manageable:

$$\delta_{ij} = \rho_\delta \delta_{i,j-1} + \varepsilon_{ij}, \quad j \geq 1. \tag{13.15}$$

The latent $\lambda_i = (\alpha_i, \delta_{i0})$ is a unit-specific correlated random coefficient. A two-step semiparametric estimator: step 1 is **quasi-maximum likelihood** for the common parameters $(\rho_Y, \rho_\delta, \sigma_U^2, \sigma_\varepsilon^2, \beta)$ under a Gaussian working assumption on λ_i (consistent under misspecification of that working

assumption); step 2 is **Tweedie / Gaussian-conjugate empirical Bayes** for the unit-level posterior trajectories $\{\delta_{i,j}\}_{j=0}^{J}$, achieving asymptotic ratio optimality.

A companion paper (Botosaru and Liu 2026) adds a **homogeneous-feedback extension**: covariates X_{it} are allowed to adjust endogenously to past Y and treatment (the canonical example: minimum-wage policy affects wages directly and also indirectly through firms re-optimising input demand). Under the assumption that the covariate adjustment rule does not depend on λ_i given the observable history, the likelihood factors into a structural piece (for $Y \mid X$, history) and a feedback piece (for X given history), each separately identified. The factorisation yields a clean decomposition of dynamic treatment effects into **direct** (through $\delta_{i,j}$) and **indirect** (through covariate feedback) components.

The package `TVHTE.jl` implements both papers:

```
using TVHTE

# Fit on a panel with staggered cohorts and a covariate
fit = tvhte(panel_Y, baseline_Y; t0 = cohort, J = max_event_time, X = panel_X)

# Feedback model + direct/indirect counterfactual decomposition
fb = fit_feedback(panel_Y, baseline_Y, panel_X, baseline_X)
cf = simulate_counterfactual(fit, fb, alt_cohort)
```

An R port with the same API is at `tvhte`; both packages have deployed docs at https://xiangao.github.io/TVHTE.jl/ and https://xiangao.github.io/tvhte/.

13.15. Synthetic Control and Synthetic DiD

The biggest problem with DiD is the PT assumption. Synthetic control and synthetic DiD replace it with weighted comparisons. The chapter Synthetic Control, DiD, and TASC covers both, with the California Proposition 99 example.

13.16. Summary

DiD replaces unconfoundedness with parallel trends and no anticipation. The untreated trend of the treated group is taken from the control group. With staggered timing, a single TWFE coefficient averages over comparisons that can use already-treated units as controls. ETWFE fixes this by giving each cohort its own effect in each period and then aggregating. A count outcome calls for the Poisson version, which assumes parallel trends on the log scale. Spillovers, persistent outcomes and a failure of parallel trends each need their own tools: interference-robust DiD, dynamic panels, and synthetic control or synthetic DiD, which the next chapter takes up.

On the minimum-wage data, ETWFE gives an ATT of -0.0506 (0.0125). On Wooldridge's simulated panel it returns 3.67 against a true 3.677. In the count simulation, Poisson ETWFE gives 3.62 (0.34) counts against a true 3.698, while linear ETWFE on $\log(Y+1)$ returns 0.40, a number on a different scale.

14. DiD with Continuous Treatment

The DiD chapter treats the treatment as on or off. Many treatments come in amounts. A county gets one Walmart or five. A state raises the minimum wage by fifty cents or by three dollars. A hospital receives a small grant or a large one. With a dose we want more than the effect of being treated. We want to know how the effect changes with the dose.

We first look at two periods, where Callaway, Goodman-Bacon, and Sant'Anna (2024), CGBS from here on, show what parallel trends identifies with a dose and what it leaves out. Two assumptions carry everything that follows: parallel trends and strong parallel trends. We then move to staggered adoption, using the heterogeneous-slopes extension of extended two-way fixed effects (ETWFE) in Wooldridge (2026), and replicate his study of Walmart and county retail employment.

14.1. Two periods

We observe units $i = 1, \dots, n$ in two periods, $t = 1, 2$. Nobody is treated in period 1. Before period 2, each unit receives a dose $D_i \geq 0$, and units with $D_i = 0$ stay untreated. For each dose d, unit i has a potential outcome $Y_{it}(d)$, and we observe $Y_{it} = Y_{it}(D_i)$. No anticipation means $Y_{i1}(d) = Y_{i1}(0)$ for every d. Let $\Delta Y_i = Y_{i2} - Y_{i1}$.

There are two natural estimands. Take two doses d and d'. The first estimand, $ATT(d \mid d')$, is the average effect of dose d, relative to no treatment, among the units that actually received dose d':

$$ATT(d \mid d') = E[Y_{i2}(d) - Y_{i2}(0) \mid D_i = d']. \tag{14.1}$$

The leading case is $ATT(d \mid d)$, the effect of each group's own dose. The second estimand, $ATE(d)$, is the effect of dose d averaged over all treated units, whatever dose they received:

$$ATE(d) = E[Y_{i2}(d) - Y_{i2}(0) \mid D_i > 0]. \tag{14.2}$$

Its derivative $ATE'(d)$ is the average causal response at d: the average effect of a small increase in the dose.

14.1.1. Parallel trends

The first assumption is parallel trends. With a dose it says that, without treatment, every dose group would have followed the trend of the untreated:

$$E[Y_{i2}(0) - Y_{i1}(0) \mid D_i = d] = E[Y_{i2}(0) - Y_{i1}(0) \mid D_i = 0] \quad \text{for all } d. \tag{14.3}$$

In words, the change in the untreated potential outcome between periods 1 and 2 is the same at every dose, including zero. The assumption restricts untreated outcomes only. Under Equation 14.3 and no anticipation,

$$E[\Delta Y_i \mid D_i = d] - E[\Delta Y_i \mid D_i = 0] = ATT(d \mid d). \tag{14.4}$$

This is the binary DiD done separately at each dose. Parallel trends identifies the whole curve $ATT(d \mid d)$, each dose group's effect of its own dose. With more periods before treatment we can check it in the usual way: the dose groups should have moved together before anyone was treated.

Parallel trends does not identify comparisons across doses. For the two doses d and d',

$$ATT(d \mid d) - ATT(d' \mid d') = \underbrace{ATT(d \mid d) - ATT(d' \mid d)}_{\text{causal response}} + \underbrace{ATT(d' \mid d) - ATT(d' \mid d')}_{\text{selection on gains}}. \tag{14.5}$$

The first term is causal. It is what moving the dose from d' to d does for the units that chose d. The second term compares two groups at the same dose d'. It is non-zero when the units that chose d would gain differently from dose d' than the units that chose d'. Parallel trends says nothing about this term, and pre-treatment data cannot reveal it, because it involves treated outcomes. So the slope of the curve in Equation 14.4 mixes the causal response with selection on gains.

14.1.2. Strong parallel trends

The second assumption changes the comparison so that the selection term no longer enters. CGBS call it strong parallel trends:

$$E[Y_{i2}(d) - Y_{i1}(0) \mid D_i = d] = E[Y_{i2}(d) - Y_{i1}(0) \mid D_i > 0] \quad \text{for all } d > 0. \tag{14.6}$$

In words, the average path from untreated in period 1 to dose d in period 2 is the same for the units that chose d as for all treated units. Together with Equation 14.3 it gives $ATT(d \mid d) = ATE(d)$ for every d: each dose group's own-dose effect equals the effect of that dose averaged over all treated units. The identified curve is then $ATE(d)$, and a comparison across doses, $ATE(d) - ATE(d')$, sets two doses for the same population, so it is causal. Its slope is the average causal response of the treated population. This does not make the selection term in Equation 14.5 zero group by group. The group that chose d can still respond to dose d' differently from the group that chose d'; strong parallel trends only rules this out on average. For example, with three equally likely dose groups and zero untreated outcomes, let row g of the matrix below give the effects of doses 1, 2 and 3 for the group that chose dose g:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 6 \\ 2 & 4 & 3 \end{pmatrix}.$$

The diagonal and the column means are both $(1, 2, 3)$, so $ATT(d \mid d) = ATE(d)$ at every dose. Yet for the group that chose dose 2, moving from dose 1 to dose 2 is worth 2, while the identified contrast is 1.

Strong parallel trends is a stronger assumption than parallel trends, and of a different kind. It restricts treated potential outcomes, so no pre-treatment data can check it, and passing a pre-trend test says nothing about it. It restricts how units choose their dose. It is plausible when the dose is set for reasons unrelated to the gain, for example by a formula or a lottery. It is hard to defend when units choose their dose knowing what they will gain.

Wooldridge (2026) reaches the same point from a regression. Let B_i be the gain of unit i from one unit of dose, so that the effect is linear in the dose with a slope that varies across units: $Y_{i2}(d) = Y_{i2}(0) + dB_i$. Let $\gamma = E[Y_{i2}(0) - Y_{i1}(0)]$ be the untreated trend, and let $\delta = E(B_i \mid D_i > 0)$ be the average slope among the treated. Under Equation 14.3,

$$E(\Delta Y_i \mid D_i) = \gamma + \delta D_i \tag{14.7}$$

holds when $E(B_i \mid D_i = d) = \delta$ for every $d > 0$. That condition is strong parallel trends for the linear model. Adding powers of D_i to the regression relaxes the linearity.

14.2. Staggered adoption

Now units enter treatment at different times, as in the ETWFE section of the DiD chapter. Periods run $t = 1, \ldots, T$, and the first period in which any unit is treated is q. A cohort $g \in \{q, \ldots, T\}$ is the set of units first treated in period g. Let $D_{ig} = 1$ if unit i is in cohort g, and $D_{ig} = 0$ otherwise. Never-treated units have $D_{ig} = 0$ for every g. The vector $\mathbf{D}_i = (D_{iq}, \ldots, D_{iT})$ collects unit i's cohort dummies. For each period s, the time dummy fs_t equals 1 if $t = s$ and 0 otherwise.

The treatment $\mathbf{W}_{it}$ is a $1 \times J$ row vector, where J is the number of treatment variables. A single dose has $J = 1$. A dose and its square has $J = 2$. A set of mutually exclusive treatment levels has J equal to the number of levels. Every unit has $\mathbf{W}_{it} = \mathbf{0}$ before its entry period, and $\mathbf{W}_i$ stacks unit i's rows over all T periods. We write $\mathbf{w}$ for a possible value of $\mathbf{W}_{it}$. The dose may change after entry. In the Walmart data it is the number of stores open.

Wooldridge (2026) writes potential outcomes as linear in the treatment, with slopes that vary by unit and period. Let $\mathbf{B}_{it}$ be a $J \times 1$ vector of random slopes, whose j-th element is unit i's gain in period t from one unit of treatment variable j. Let γ_t be a time effect, C_i an unobserved unit effect, and U_{it} an idiosyncratic error. The potential outcome at treatment value $\mathbf{w}$ is

$$Y_{it}(\mathbf{w}) = \mathbf{w}\,\mathbf{B}_{it} + \gamma_t + C_i + U_{it}. \tag{14.8}$$

For a single dose, $J = 1$, write the scalar dose as w. Subtracting the untreated potential outcome cancels the time effect, the unit effect and the error:

$$Y_{it}(w) - Y_{it}(0) = w\, B_{it}. \tag{14.9}$$

So B_{it} is the difference in potential outcomes per unit of dose. With several treatment variables, the j-th element of $\mathbf{B}_{it}$ is the change in the potential outcome from one more unit of treatment variable j, holding the others fixed. The additive $\gamma_t + C_i$ is parallel trends. Nothing restricts $\mathbf{B}_{it}$. The estimand is its mean for each cohort and period:

$$\delta_{gs} = E(\mathbf{B}_{is} \mid D_{ig} = 1), \qquad s \geq g. \tag{14.10}$$

In words, δ_{gs} is the average effect per unit of each treatment variable for cohort g in period s. Because the potential outcome is linear in the dose, it is also the effect of one more unit at any dose: going from 0 to 1 has the same effect as going from 1 to 2. Adding powers of the dose, or dummies for dose levels, to $\mathbf{W}_{it}$ relaxes this. With a binary treatment, δ_{gs} is the cohort-period ATT of ETWFE.

Wooldridge assumes no anticipation and strict exogeneity. Strict exogeneity says the error U_{it} has mean zero given the whole treatment path $\mathbf{W}_i$, the unit effect C_i and the slopes $\mathbf{B}_{it}$. He needs one more assumption, which has no counterpart in the binary case. He calls it limited selection:

$$E(\mathbf{B}_{it} \mid \mathbf{W}_i) = E(\mathbf{B}_{it} \mid \mathbf{D}_i). \tag{14.11}$$

Once we know when a unit entered, its dose path carries no information about its average gain. With a binary treatment the entry date fixes the whole treatment path, so Equation 14.11 holds automatically. With two periods and a scalar dose it is the strong parallel trends of Equation 14.6 in its linear form. Parallel trends enters through the additive $\gamma_t + C_i$ in Equation 14.8. So the staggered estimates rest on the same two assumptions as the two-period ones.

Let $A_i = E(C_i \mid \mathbf{W}_i)$ be the mean of the unit effect given the treatment path. Under these assumptions,

$$E(Y_{it} \mid \mathbf{W}_i) = \sum_{g=q}^{T} \sum_{s=g}^{T} (D_{ig}\, fs_t\, \mathbf{W}_{it})\, \delta_{gs} + \gamma_t + A_i. \tag{14.12}$$

A_i is a unit effect. So a two-way fixed effects (TWFE) regression of Y_{it} on the interactions $D_{ig}\, fs_t\, \mathbf{W}_{it}$, with unit and time effects, estimates every δ_{gs}. This is ETWFE with the treatment indicator replaced by the dose. Standard errors are clustered by unit.

There are many cohort-period slopes, so we aggregate them. Let e be the exposure time, the number of periods since entry, and let N_g be the number of units in cohort g. Let $\hat{\delta}_{gs}$ be the TWFE estimate of δ_{gs}. The estimated effect at exposure e, written $\hat{\tau}_e$, averages over the cohorts observed that long, with weights proportional to cohort size:

$$\hat{\tau}_e = \sum_{g=q}^{T-e} \frac{N_g}{N_q + \cdots + N_{T-e}}\, \hat{\delta}_{g,g+e}. \tag{14.13}$$

The overall effect weights every cohort-period slope by N_g. In a balanced panel it is the average slope over all treated unit-periods.

Pre-trends can be tested in two ways. A lead is a dummy $D_{ig}\, fs_t$ for a period s before cohort g enters. The first test fits TWFE on the untreated observations only, the unit-periods with $\mathbf{W}_{it} = \mathbf{0}$, with leads for $s \leq g - 2$, so that the period just before entry is the base. It then tests whether the leads are zero. The second test adds the same leads to Equation 14.12. With a binary treatment the two give identical estimates. With a dose they differ. Both tests check parallel trends. Neither checks limited selection. When the leads show a trend, Wooldridge adds cohort-specific linear trends to Equation 14.12. For each cohort g, the regressor $D_{ig} \cdot t$ is a linear time trend for that cohort only. The assumption then becomes that each cohort's gap from the never-treated units would have kept growing along the same line after entry.

14.3. Wooldridge's regression and the CGBS estimator

The two approaches target the same objects. Under parallel trends and strong parallel trends, each identifies the average effect of the dose for every cohort and period. They differ in how they estimate it and in what they can handle.

The first difference is the comparison group. Wooldridge runs one TWFE regression in levels on all periods, Equation 14.12. Every untreated observation serves as a control: the never-treated units, the not-yet-treated units, and the pre-entry periods of every cohort. In the binary case this regression is the imputation estimator of Borusyak, Jaravel, and Spiess (2024). Callaway, Goodman-Bacon, and Sant'Anna (2024) build on Callaway and Sant'Anna (2021) instead. For each cohort and period they compare the cohort with the never-treated or the not-yet-treated units, using the long difference from the period just before entry, and fit the dose response in that comparison. The regression uses more data, and it is more efficient when the errors are homoskedastic and serially uncorrelated. In exchange it needs parallel trends in every pre-entry period. The differencing approach needs it only from the base period on.

The second difference is the shape of the dose response. In Wooldridge's regression, the shape comes from the functions of the dose placed in $\mathbf{W}_{it}$: the dose itself, its powers, or dummies for dose levels. The estimand is the average slope δ_{gs}. CGBS fit a B-spline in the dose within each cohort and period, or a nonparametric curve whose complexity is chosen from the data. They report the curve $ATT(d \mid d)$ and its derivative. The two also treat a dose near zero differently. With two periods, Wooldridge's regression has no separate intercept for the treated units, so its fitted effect goes to zero as the dose goes to zero. The CGBS fit gives the treated units their own intercept, so the effect can jump at any positive dose. The two coincide once a dummy for a positive dose is added to $\mathbf{W}_{it}$.

The third difference is scope. The table compares Wooldridge's regression with the R package `contdid`, version 0.1.1, which implements CGBS.

	Wooldridge (2026)	`contdid` 0.1.1
Dose changes after entry	Yes	No
Covariates	Yes, in the trends and in the slopes	No
Cohort-specific trends	Yes	No
Unbalanced panel	Yes	No
Multi-level or vector treatment	Yes, through $\mathbf{W}_{it}$	No

	Wooldridge (2026)	`contdid` 0.1.1
Exit from treatment	Yes	No
Standard errors	Analytic, clustered by unit	Multiplier bootstrap
Reported effects	Average slopes by cohort and period, aggregated by exposure	$ATT(d \mid d)$ and its derivative, aggregated by dose or by exposure

The first row decides the Walmart application. The number of stores changes after entry, so the CGBS estimator, as implemented, does not apply. The other rows show what the single regression adds: covariates, trends, exit and missing years enter as extra terms. In the other direction, CGBS trace out the whole dose-response curve within each cohort and period. Wooldridge reports average slopes and needs powers of the dose to show curvature.

14.4. Walmart and county retail employment

Wooldridge (2026) applies the method to the county panel of Brown and Butts (2025). It covers 1,288 counties from 1977 to 1999. No county had a Walmart before 1986. Stores then opened in 14 cohorts, from 1986 to 1999, and 395 counties never got one. The treatment is a single dose, so $J = 1$ and we write it without bold: W_{it} is the number of Walmarts open in county i in year t. The outcome is log retail employment.

```
# extract of walmart_lw.dta from Wooldridge's replication files (data/gen_walmart_lw.R)
wm = CSV.read("data/walmart_lw.csv", DataFrame)
display(sort(combine(groupby(wm[wm.year .== 1999, :], :cohort), nrow => :N), :cohort))
counts = sort(combine(groupby(wm[wm.n_open .> 0, :], :n_open), nrow => :N), :n_open)
counts.share = round.(counts.N ./ sum(counts.N), digits = 3)
counts
```

	cohort	N
	Int64	Int64
1	0	395
2	1986	69
3	1987	74
4	1988	60
5	1989	77
6	1990	118
7	1991	113
8	1992	88
9	1993	98
10	1994	48
11	1995	53
12	1996	25
13	1997	26
14	1998	23
15	1999	21

	n_open	N	share
	Int64	Int64	Float64
1	1	6278	0.797
2	2	1042	0.132
3	3	300	0.038
4	4	128	0.016
5	5	54	0.007
6	6	17	0.002
7	7	20	0.003
8	8	11	0.001
9	9	7	0.001
10	10	5	0.001
11	11	12	0.002
12	12	5	0.001
13	13	1	0.0

Most treated counties have a single store. One store accounts for 80% of treated county-years, two for 13%, and the largest count is 13. Within a cohort and year, the slope is fit from the change relative to untreated counties and from the differences in store counts. With so many single-store counties, it mostly measures the effect of the first store.

A single-coefficient TWFE regression is the natural starting point.

```
coeftable(feols(wm, @formula(log_retail_emp ~ n_open + fe(fips) + fe(year)),
                vcov = Vcov.cluster(:fips)))
```

	Estimate	Std. Error	z value	Pr(>
n_open	0.034412	0.00565117	6.09	<1e-08

One more Walmart raises retail employment by 3.4%, with a standard error of 0.57 percentage points. In the binary case this regression makes forbidden comparisons, which use already-treated units as controls, when effects vary by cohort and time. The same holds with a dose. So we build the regressors of Equation 14.12, one for each cohort and year from entry on: W_{it} times the cohort dummy times the year dummy.

```
cohorts = 1986:1999
cells = [(g, s) for g in cohorts for s in cohorts if s >= g]
wcols = [Symbol("w_$(g)_$(s)") for (g, s) in cells]
for ((g, s), c) in zip(cells, wcols)
    wm[!, c] = (wm.cohort .== g) .* (wm.year .== s) .* wm.n_open
end
trcols = [Symbol("tr_$g") for g in cohorts]
for (g, c) in zip(cohorts, trcols)
    wm[!, c] = (wm.cohort .== g) .* wm.year
end

fit_het = feols(wm, term(:log_retail_emp) ~ sum(term.(wcols)) + fe(:fips) + fe(:year),
```

```
                vcov = Vcov.cluster(:fips))
length(coef(fit_het))
```

```
105
```

That gives 105 slopes. We aggregate them with Equation 14.13. Each aggregate is a linear combination of the slopes with fixed weights, so its standard error comes from the clustered covariance matrix. This matches Stata's `margins` in Wooldridge's code.

```
Ng = Dict(g => sum((wm.year .== 1999) .& (wm.cohort .== g)) for g in cohorts)

function aggregate_slopes(fit)
    pos = Dict(n => i for (i, n) in enumerate(coefnames(fit)))
    b = coef(fit); V = vcov(fit)
    lincomb(a) = (dot(a, b), sqrt(a' * V * a))
    out = map(0:13) do e
        gs = [g for g in cohorts if g + e <= 1999]
        a = zeros(length(b))
        for g in gs
            a[pos["w_$(g)_$(g + e)"]] = Ng[g] / sum(Ng[h] for h in gs)
        end
        lincomb(a)
    end
    a = zeros(length(b))
    for (g, s) in cells
        a[pos["w_$(g)_$(s)"]] = Ng[g]
    end
    push!(out, lincomb(a ./ sum(a)))
end
agg_het = aggregate_slopes(fit_het);
```

Before reading the effects, we check pre-trends. Following the first test above, we fit TWFE on the untreated county-years with a lead for each cohort and each year at least two years before entry. We then average the leads by years until entry, with the cohort weights of Equation 14.13.

```
un = wm[wm.n_open .== 0, [:fips, :year, :cohort, :log_retail_emp]]
leads = [(g, s) for g in cohorts for s in 1977:1999 if s <= g - 2]
lcols = [Symbol("l_$(g)_$(s)") for (g, s) in leads]
for ((g, s), c) in zip(leads, lcols)
    un[!, c] = Float64.((un.cohort .== g) .& (un.year .== s))
end
fit_pre = feols(un, term(:log_retail_emp) ~ sum(term.(lcols)) + fe(:fips) + fe(:year),
                vcov = Vcov.cluster(:fips))
pos = Dict(n => i for (i, n) in enumerate(coefnames(fit_pre)))
b = coef(fit_pre); V = vcov(fit_pre)
pre = DataFrame(map(2:22) do k
```

```
    gs = [g for g in cohorts if g - k >= 1977]
    a = zeros(length(b))
    for g in gs
        a[pos["1_$(g)_$(g - k)"]] = Ng[g] / sum(Ng[h] for h in gs)
    end
    (time = -k, estimate = dot(a, b), se = sqrt(a' * V * a))
end)

fig = Figure(size = (700, 360))
ax = Axis(fig[1, 1], xlabel = "Years until first Walmart", ylabel = "Log retail employment")
hlines!(ax, 0, color = :black)
rangebars!(ax, pre.time, pre.estimate .- 1.96 .* pre.se, pre.estimate .+ 1.96 .* pre.se,
           color = :darkcyan)
scatter!(ax, pre.time, pre.estimate, color = :darkcyan)
fig
```

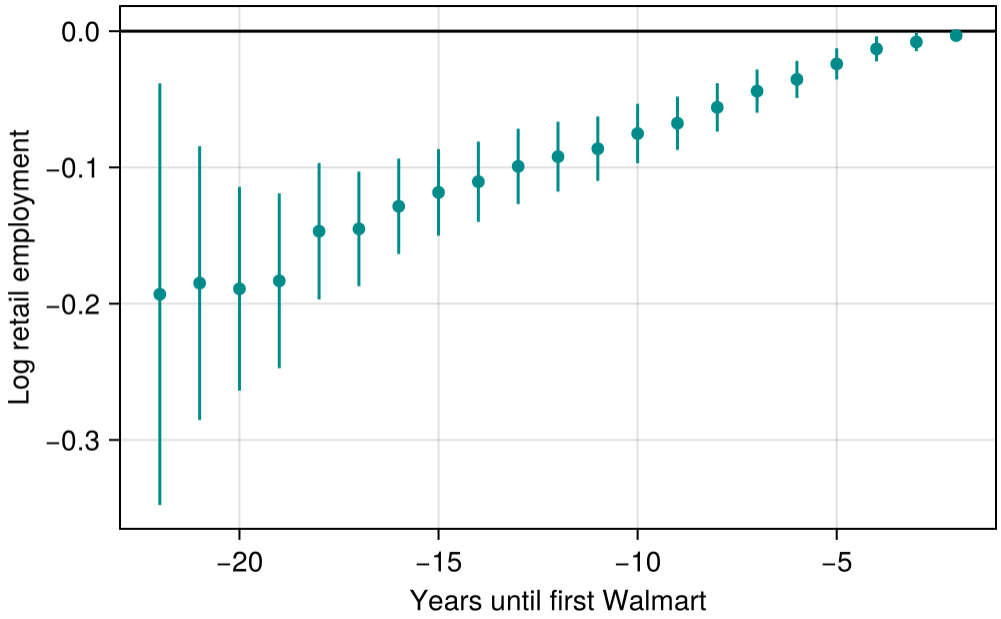


Figure 14.1.: Weighted pre-treatment estimates from the untreated observations, with 95% intervals. The year before entry is the base.

The leads fall almost linearly as we go back in time. Twenty years before entry, counties that later got a Walmart had retail employment about 19 log points lower, relative to the year before entry, than the never-treated counties. Retail employment was growing faster in these counties long before any store opened. Parallel trends fails, so Wooldridge also reports estimates with the cohort-specific linear trends.

```
fit_trend = feols(wm, term(:log_retail_emp) ~ sum(term.(vcat(wcols, trcols))) +
                      fe(:fips) + fe(:year),
                  vcov = Vcov.cluster(:fips))
agg_trend = aggregate_slopes(fit_trend)

fmt(x) = @sprintf("%.4f (%.4f)", x[1], x[2])
DataFrame(Exposure = vcat(string.(0:13), "overall"),
          var"No trends" = fmt.(agg_het), var"Cohort trends" = fmt.(agg_trend))
```

	Exposure	No trends	Cohort trends
	String	String	String
1	0	0.0291 (0.0041)	0.0049 (0.0033)
2	1	0.0439 (0.0042)	0.0196 (0.0034)
3	2	0.0357 (0.0044)	0.0113 (0.0035)
4	3	0.0301 (0.0046)	0.0070 (0.0036)
5	4	0.0286 (0.0048)	0.0060 (0.0039)
6	5	0.0316 (0.0051)	0.0073 (0.0041)
7	6	0.0344 (0.0055)	0.0106 (0.0046)
8	7	0.0412 (0.0063)	0.0200 (0.0054)
9	8	0.0459 (0.0072)	0.0236 (0.0064)
10	9	0.0480 (0.0082)	0.0274 (0.0083)
11	10	0.0624 (0.0104)	0.0373 (0.0111)
12	11	0.0671 (0.0121)	0.0422 (0.0146)
13	12	0.0706 (0.0160)	0.0587 (0.0200)
14	13	0.0895 (0.0210)	0.0925 (0.0302)
15	overall	0.0386 (0.0051)	0.0156 (0.0046)

The table reproduces Table 7.1 of Wooldridge (2026), estimates and standard errors, to all four decimals.

Without trends, one more store raises retail employment by about 3% to 4% in the first years. The effect climbs to 9% by year 13. The overall effect is 0.0386 (0.0051). It is close to the single-coefficient TWFE estimate of 0.0344, and a little more precise.

The cohort trends change the estimates a lot. The effect in the entry year is 0.0049 (0.0033), indistinguishable from zero. From the entry year to year 6, the effects are between a sixth and a half of their values without trends. They then rise steadily, and by year 13 the two columns agree at about 0.09. The overall effect falls to 0.0156 (0.0046), less than half the estimate without trends. The trends also make the estimates more precise up to about year 8. The long exposures rest on the earliest cohorts only, so their standard errors grow. Year 13 comes from the 1986 cohort alone.

For comparison, we add the same 14 cohort trends to the single-coefficient regression. One slope on the store count is then shared by every cohort and year.

```
s1 = feols(wm, term(:log_retail_emp) ~ term(:n_open) + sum(term.(trcols)) +
               fe(:fips) + fe(:year),
           vcov = Vcov.cluster(:fips))
k = findfirst(==("n_open"), coefnames(s1))
(estimate = coef(s1)[k], se = sqrt(vcov(s1)[k, k]))
```

```
(estimate = 0.009987276364962341, se = 0.004501561610753822)
```

With the same trends, the single-coefficient regression gives 0.0100 (0.0045). The flexible model costs almost nothing in precision.

The table averages over cohorts. To see the path of a single cohort, we take the 1990 cohort, the largest, with 118 counties and ten years after entry. Under Equation 14.8, with $J = 1$, the effect of w stores on cohort g in year s is $w\,\delta_{gs}$. The figure plots the estimated effect of one store, $\hat{\delta}_{1990,s}$, for each year after entry, with and without cohort trends. The effect of two stores is twice as large in every year, because the model is linear in the dose.

```
g0 = 1990
nm = ["w_$(g0)_$(s)" for s in g0:1999]
fig = Figure(size = (700, 360))
ax = Axis(fig[1, 1], xlabel = "Years since the first Walmart opened",
          ylabel = "Effect of one store", xticks = 0:9)
hlines!(ax, 0, color = :black)
for (fit, label, shift, col) in [(fit_het, "No trends", -0.12, :darkcyan),
                                 (fit_trend, "Cohort trends", 0.12, :darkorange)]
    pos = Dict(n => i for (i, n) in enumerate(coefnames(fit)))
    idx = [pos[n] for n in nm]
    est = coef(fit)[idx]; se = sqrt.(diag(vcov(fit))[idx])
    x = (0:9) .+ shift
    rangebars!(ax, x, est .- 1.96 .* se, est .+ 1.96 .* se, color = col)
    scatter!(ax, x, est, color = col, label = label)
end
Legend(fig[2, 1], ax, orientation = :horizontal, framevisible = false)
fig
```

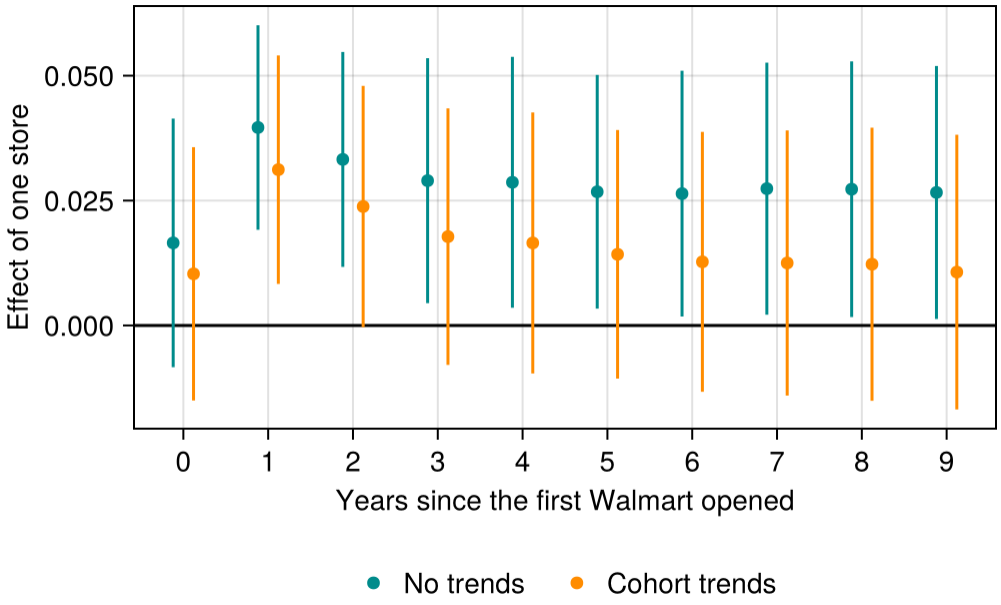


Figure 14.2.: Effect of one Walmart on log retail employment in the 1990 cohort, by year since entry, with 95% intervals.

Without trends, one store raises retail employment in the 1990 cohort by 1.7% in the entry year and by 4.0% a year later, and the effect then settles near 2.7%. From year 1 on, the estimates are about twice their standard errors or more. With cohort trends the path is lower, and the gap grows from 0.6 percentage points in the entry year to 1.6 by year 9. Only the year-1 estimate, 3.1%, is more than twice its standard error. The growing gap is what we would see if the cohort trend, which is also fit on post-entry years, absorbs part of an effect that grows with exposure. The plot holds the dose at one store. The cohort's average number of stores rises from 1.08 in 1990 to 1.75 in 1999, so the effect it actually experienced grows faster than the plotted per-store effect.

14.5. Summary

Wooldridge's method is ETWFE with the treatment indicator replaced by the dose. The regression in Equation 14.12 interacts cohort, year and dose. Each interaction has its own slope δ_{gs}, the average effect of one more unit of dose for cohort g in year s, and the slopes are averaged by exposure time. Covariates, cohort trends, exit and unbalanced panels enter as extra terms in the same regression.

Two assumptions identify the slopes. Parallel trends lets us compare each cohort's change with the change of the untreated units, and the pre-trend tests check it. Strong parallel trends, which Wooldridge's limited selection extends to staggered adoption, says the dose a unit ends up with tells us nothing about its average gain. It makes comparisons across doses causal, and no test in the data checks it. In the Walmart data parallel trends fails, so we add cohort trends, and the overall effect of one more store falls from 3.9% to 1.6%.

15. Synthetic Control, DiD, and TASC

```
using DataFrames
using SynthDiD
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using Printf

using TASC
```

Here I put three related panel-data designs together: classical synthetic control, synthetic DiD, and Time-Aware Synthetic Control (TASC). All compare a treated unit to a donor pool over time. They differ in what structure they put on the untreated counterfactual.

15.1. Synthetic Control

The biggest problem with DiD is the parallel trends assumption. There is not really a test for it, just like the unconfoundedness assumption. It restricts a different counterfactual object: it allows treated and control units to differ in untreated levels, but requires their untreated mean changes to be equal. Neither assumption implies the other, and parallel trends is still hard to justify in many applications.

Synthetic control starts from a different idea. Instead of giving every control unit the same weight, it constructs a synthetic control unit from the donor pool:

$$\hat{Y}_{1t}(0) = \sum_{i=2}^{N} \hat{\omega}_i Y_{it}, \tag{15.1}$$

where the weights are chosen to make the synthetic unit close to the treated unit before treatment. The usual Abadie-style version restricts the donor weights to be nonnegative and to sum to one. The identifying hope is not that the average control follows the treated unit, but that some weighted average of controls does.

Classical synthetic control is time-agnostic in an important sense: the pre-treatment periods enter the weight-fitting problem as a set of matching moments. If we permute the order of the pre-treatment periods, the fitted donor weights are unchanged. This is often a strength because it avoids imposing a time-series model, but it can leave predictive information unused when there is a stable trend.

15.2. Synthetic DiD

Arkhangelsky et al. (2021) combine the idea of DiD and synthetic control. Standard DiD assigns equal weight to all control units and all pre-treatment time periods. Synthetic control assigns data-driven weights to donor units. Synthetic DiD uses both unit weights and time weights.

The three estimators below are written as a *unifying weighted-regression sketch*: the same two-way fixed-effect regression, with different weights switched on. This is a useful way to see how DiD, synthetic control, and synthetic DiD relate, but it is **not** the estimators' actual optimization problems. In particular, the synthetic-control weights $\widehat{\omega}_i^{sc}$ are not free regression weights — in Abadie's formulation they are chosen by a separate optimization that matches pre-treatment outcomes (and predictors) of the treated unit, subject to $\omega_i \geq 0,\ \sum_i \omega_i = 1$. Likewise the SDID unit and time weights $\widehat{\omega}_i^{sdid}, \widehat{\lambda}_t^{sdid}$ come from their own penalized pre-period optimization problems. Treat the displays below as schematic; see the package documentation (or Arkhangelsky et al. 2021) for the exact weight-selection problems.

The DiD objective can be written as

$$(\hat{\tau}^{did}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau,\mu,\alpha,\beta}{\arg\min} \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2. \tag{15.2}$$

Synthetic control can be viewed as a weighted version that puts the donor weights into the comparison:

$$(\hat{\tau}^{sc}, \hat{\mu}, \hat{\beta}) = \underset{\tau,\mu,\beta}{\arg\min} \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \mu - \beta_t - W_{it}\tau)^2 \widehat{\omega}_i^{sc}. \tag{15.3}$$

Synthetic DiD adds time weights:

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau,\mu,\alpha,\beta}{\arg\min} \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \widehat{\omega}_i^{sdid}\widehat{\lambda}_t^{sdid}. \tag{15.4}$$

The unit weights try to make the donor pool look like the treated unit. The time weights put more emphasis on pre-treatment periods that are more informative about the post-treatment period.

15.3. Time-Aware Synthetic Control

TASC was proposed by Rho et al. (2026). It starts from the same panel-data structure, but it models the donor and treated outcomes through a latent time-series system:

$$x_t = Ax_{t-1} + q_t, \qquad q_t \sim N(0, Q), \tag{15.5}$$

$$y_t = Hx_t + r_t, \qquad r_t \sim N(0, R). \tag{15.6}$$

Here y_t is the vector of outcomes across units at time t, and x_t is a low-dimensional latent state. The matrix H maps the latent state into unit outcomes, so HX is a low-rank signal. The matrix A is the new piece relative to classical synthetic control: it says that the latent time factors evolve according to a stable trend.

The counterfactual step is simple. TASC learns the state-space model from the pre-treatment panel. After treatment, the treated unit's observed outcome is treated as missing. The filter and smoother continue to use the donor units to infer the latent state, and the treated row of H maps that state into the untreated counterfactual:

$$\hat{Y}_{1t}(0) = h_1^\top \hat{x}_t, \qquad t > T_0. \tag{15.7}$$

The estimated treatment effect is then

$$\hat{\tau}_t = Y_{1t} - \hat{Y}_{1t}(0). \tag{15.8}$$

Relative to classical synthetic control, TASC uses the order of time. Relative to synthetic DiD, it puts the time dependence in a generative latent-state model rather than in a set of pre-period weights. This can help when trends are stable and outcomes are noisy, but it is also a stronger modeling assumption.

15.4. Example: California Proposition 99

California's Proposition 99 raised cigarette taxes and funded anti-smoking programs. The usual synthetic-control exercise compares California's cigarette sales after 1988 to a donor pool of other states. The outcome is per-capita cigarette sales in packs.

```
california = california_prop99()
setup = panel_matrices(california, :State, :Year, :PacksPerCapita, :treated)

tau_sdid = synthdid_estimate(setup.Y, setup.N0, setup.T0)
tau_sc   = sc_estimate(setup.Y, setup.N0, setup.T0)
tau_did  = did_estimate(setup.Y, setup.N0, setup.T0)
nothing
```

`SynthDiD.jl` orders the donor states first and treated states last. `TASC.jl` expects the treated unit to be first, so we explicitly reorder the matrix before fitting the state-space model. The paper's Proposition 99 example uses a low latent dimension; here we set `d = 2`.

```
Y_tasc = vcat(setup.Y[(setup.N0 + 1):end, :], setup.Y[1:setup.N0, :])

tasc_model = fit_tasc(
    Y_tasc;
    d = 2,
    T0 = setup.T0,
```

```
    n_em = 200,
    tol = 1e-3,
)

tasc_pred = predict_counterfactual(tasc_model, Y_tasc)
tasc_att = mean(tasc_pred.effect[(setup.T0 + 1):end])
nothing
```

```
estimates = DataFrame(
    Method = ["Diff-in-Diff", "Synthetic Control", "Synthetic Diff-in-Diff", "TASC"],
    Estimate = [tau_did.estimate, tau_sc.estimate, tau_sdid.estimate, tasc_att],
)

DataFrames.transform(estimates, :Estimate => ByRow(x -> round(x, digits = 2)) => :EstimateRound
```

	Method	Estimate	EstimateRounded
	String	Float64	Float64
1	Diff-in-Diff	-27.3491	-27.35
2	Synthetic Control	-19.6197	-19.62
3	Synthetic Diff-in-Diff	-15.6038	-15.6
4	TASC	-17.1127	-17.11

The estimates are all negative: California's observed cigarette sales fell below the estimated no-Proposition-99 counterfactual. The methods differ because they impose different restrictions on the untreated path. DiD compares average changes. Synthetic control chooses donor weights. Synthetic DiD adds time weights. TASC uses a low-rank state-space model and updates the latent state with post-treatment donor observations.

```
years = Int.(setup.times)
treated = vec(setup.Y[(setup.N0 + 1), :])
sc_counterfactual = vec(tau_sc.weights.omega' * setup.Y[1:setup.N0, :])
# The SDiD estimate is mean(post gap) - sum(  .* pre gap), where gap = treated -  'Y_donors.
# Shifting  'Y_donors by the  -weighted pre-period gap gives the path whose average
# post-period gap equals the reported estimate.
sdid_raw = vec(tau_sdid.weights.omega' * setup.Y[1:setup.N0, :])
sdid_intercept = sum(tau_sdid.weights.lambda .* (treated[1:setup.T0] .- sdid_raw[1:setup.T0]))
sdid_counterfactual = sdid_raw .+ sdid_intercept
@printf("SDiD estimate %.3f; mean post gap to plotted path %.3f\n", tau_sdid.estimate,
        mean(treated[(setup.T0 + 1):end] .- sdid_counterfactual[(setup.T0 + 1):end]))
tasc_counterfactual = tasc_pred.target
tasc_se = sqrt.(max.(tasc_pred.variance, 0.0))
tasc_lower = tasc_counterfactual .- 1.96 .* tasc_se
tasc_upper = tasc_counterfactual .+ 1.96 .* tasc_se

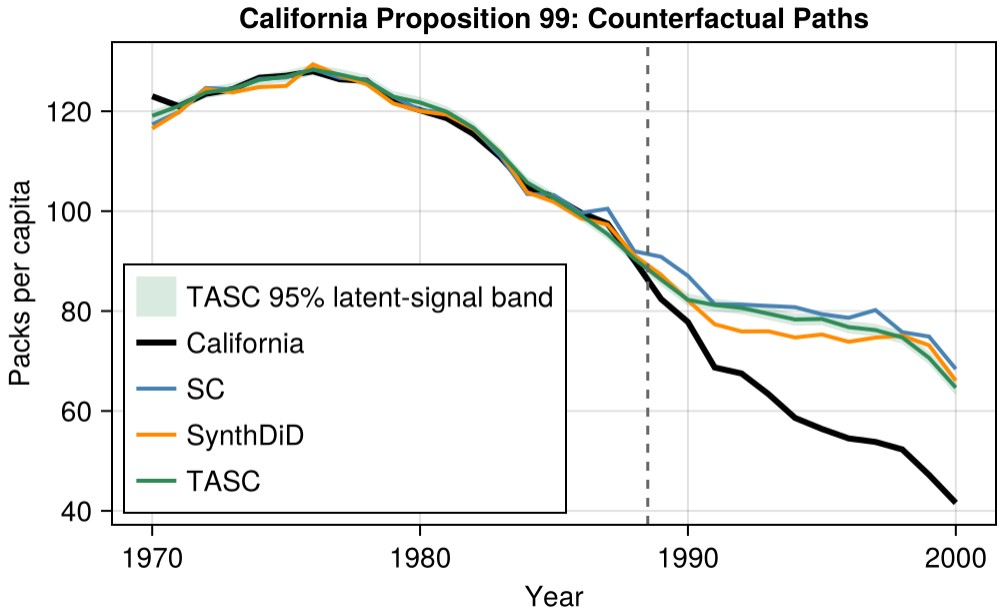
fig = Figure(size = (760, 430))
ax = Axis(
    fig[1, 1],
```

```
    xlabel = "Year",
    ylabel = "Packs per capita",
    title = "California Proposition 99: Counterfactual Paths",
)

band!(ax, years, tasc_lower, tasc_upper, color = (:seagreen, 0.18), label = "TASC 95% latent-si
lines!(ax, years, treated, color = :black, linewidth = 3, label = "California")
lines!(ax, years, sc_counterfactual, color = :steelblue, linewidth = 2, label = "SC")
lines!(ax, years, sdid_counterfactual, color = :darkorange, linewidth = 2, label = "SynthDiD")
lines!(ax, years, tasc_counterfactual, color = :seagreen, linewidth = 2, label = "TASC")
vlines!(ax, years[setup.T0] + 0.5, color = :gray40, linestyle = :dash)

axislegend(ax, position = :lb)
fig
```

```
SDiD estimate -15.604; mean post gap to plotted path -15.604
```



The figure emphasizes where the counterfactual enters. After 1988, the black line is observed California. The colored lines are estimates of California's untreated path. The treatment-effect path is the vertical gap between the black line and each counterfactual. The synthetic DiD line is the weighted donor average shifted by the time-weighted pre-1989 gap, so its average post-1988 gap equals the synthetic DiD estimate.

15.5. Summary

All three designs compare a treated unit with a donor pool over time. They differ in the structure they put on the untreated path. Synthetic control replaces parallel trends with a weighted average of donors chosen to match the treated unit before treatment. Synthetic DiD adds time weights that favour the pre-treatment periods most like the post-treatment ones. TASC models outcomes through a low-rank latent state that evolves over time, so it uses the order of time that synthetic control ignores. That is a stronger modelling assumption, and it helps when trends are stable and outcomes are noisy.

For Proposition 99 all four estimates are negative: -27.35 for DiD, -19.62 for synthetic control, -15.60 for synthetic DiD and -17.11 for TASC. Each says California's cigarette sales fell below its estimated counterfactual. The spread across methods reflects their different restrictions on the untreated path.

16. Randomization Inference for Synthetic Control

```
using DataFrames
using SynthDiD
using Statistics
using Random
using CairoMakie
CairoMakie.activate!(type = "png")

using TASC
```

Synthetic control is often used when there is one treated unit: California, West Germany, the Basque Country. Usual large-sample standard errors are not very helpful because there is no large sample of treated units. Two approaches are common:

- **Randomization inference**: use donor units as placebos.
- **Model-based inference**: put a model on the counterfactual path and compute its uncertainty under that model.

Here I use both on the Proposition 99 panel.

16.1. The placebo test

The placebo idea is simple. Take each control unit in turn, pretend it was treated, and re-run synthetic control. This produces a placebo gap for each donor. The collection of placebo gaps is a *reference* distribution: it shows how large a gap this estimator returns for a unit we are prepared to treat as exchangeable with California. It becomes an empirical null distribution only if that exchangeability can be defended. In an observational setting that is an assumption, not something the data supply.

If California's gap is large relative to the placebo gaps, that is evidence against the no-effect null.

```
california = california_prop99()
setup = panel_matrices(california, :State, :Year, :PacksPerCapita, :treated)
Y    = setup.Y
N0   = setup.N0           # 38 donor states
T0   = setup.T0           # 1989 onward is post-treatment
N1   = size(Y, 1) - N0    # 1 treated state (California)
```

```
years = Int.(setup.times)

# California's observed effect (SC estimate)
tau_sc = sc_estimate(Y, N0, T0)
ca_obs = vec(Y[N0 + 1, :])
ca_cfact = vec(tau_sc.weights.omega' * Y[1:N0, :])
ca_effect = mean(ca_obs[(T0 + 1):end]) - mean(ca_cfact[(T0 + 1):end])

@printf("California ATT (SC): %.2f packs per capita\n", ca_effect)
```

```
California ATT (SC): -19.62 packs per capita
```

Now run the placebo: each control state takes a turn as the "treated" unit. California stays out of every donor pool, so each placebo uses the other 37 states while California uses all 38. This follows Abadie, Diamond, and Hainmueller (2010). An exact reassignment argument would instead apply the same procedure under every allowed assignment, giving each hypothetical treated state the other 38 as donors.

```
function run_placebo(Y, N0, T0, treated_idx)
    # Swap treated_idx into the last row (treated slot); remaining donors fill 1..N0-1
    donor_idx = setdiff(1:N0, treated_idx)
    Y_perm    = vcat(Y[donor_idx, :], Y[treated_idx:treated_idx, :])
    fit       = sc_estimate(Y_perm, length(donor_idx), T0)
    treated   = vec(Y_perm[end, :])
    cfact     = vec(fit.weights.omega' * Y_perm[1:length(donor_idx), :])
    return treated, cfact
end

placebo_effects = Float64[]
placebo_paths   = Matrix{Float64}(undef, N0, size(Y, 2))
placebo_cfacts  = Matrix{Float64}(undef, N0, size(Y, 2))

for i in 1:N0
    treated_i, cfact_i = run_placebo(Y, N0, T0, i)
    placebo_paths[i, :]  = treated_i .- cfact_i
    placebo_cfacts[i, :] = cfact_i
    push!(placebo_effects, mean(treated_i[(T0 + 1):end] .- cfact_i[(T0 + 1):end]))
end

# California's treatment effect path
ca_gap = ca_obs .- ca_cfact

@printf("California ATT:        %.2f\n", ca_effect)
@printf("Placebo distribution mean: %.2f\n", mean(placebo_effects))
@printf("Placebo distribution sd:   %.2f\n", std(placebo_effects))
```

```
California ATT:        -19.62
Placebo distribution mean: 0.32
Placebo distribution sd:   10.76
```

16.1.1. The two-sided p-value

A Fisher-style exact p-value is the fraction of placebo effects at least as extreme (in absolute value) as the observed effect:

```
# Fisher exact convention: count California itself in numerator and denominator,
# so the p-value floor is 1/(N0+1) = 1/39 rather than 0.
p_value = (1 + sum(abs.(placebo_effects) .>= abs(ca_effect))) / (length(placebo_effects) + 1)
@printf("Two-sided p-value: %.3f\n", p_value)
```

```
Two-sided p-value: 0.077
```

With 38 donor states, the smallest possible p-value is $1/39 \approx 0.026$ (California plus 38 placebos). A p-value at this floor means the observed effect is more extreme than every placebo.

Call it an *exact* randomization p-value only when an assignment mechanism earns the word. Exactness needs the treated label to be exchangeable across the 39 units under the null, and it needs the estimator reapplied symmetrically to each of them. Proposition 99 was not randomly assigned to California. Nothing in the design delivers that exchangeability, California differs from most donors in size, and its assignment probability was not 1/39. Abadie (2021) makes the point directly: the procedure reduces to classical randomization inference when treatment is randomly assigned, which is an unusual situation in aggregate observational applications. Absent that argument, read the number as an in-space placebo diagnostic, a measure of how unusual California looks among the donors. Ferman and Pinto (2017) show separately that placebo tests can be distorted in size when pre-treatment fit varies across units and the weights are estimated.

16.1.2. The placebo plot

The usual plot overlays the placebo gaps in grey and California's gap in black. If the black line is outside the grey cloud, California looks unusual.

```
fig = Figure(size = (740, 380), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Year", ylabel = "Gap: observed − synthetic",
           title = "Placebo gap trajectories")
for i in 1:N0
    lines!(ax, years, placebo_paths[i, :],
           color = (:grey, 0.35), linewidth = 1)
end
lines!(ax, years, ca_gap, color = :black, linewidth = 2.5, label = "California")
hlines!(ax, [0.0]; color = :gray40, linestyle = :dot, linewidth = 1)
vlines!(ax, [years[T0] + 0.5]; color = :gray40, linestyle = :dash, linewidth = 1)
axislegend(ax, position = :lb, framevisible = false)
fig
```

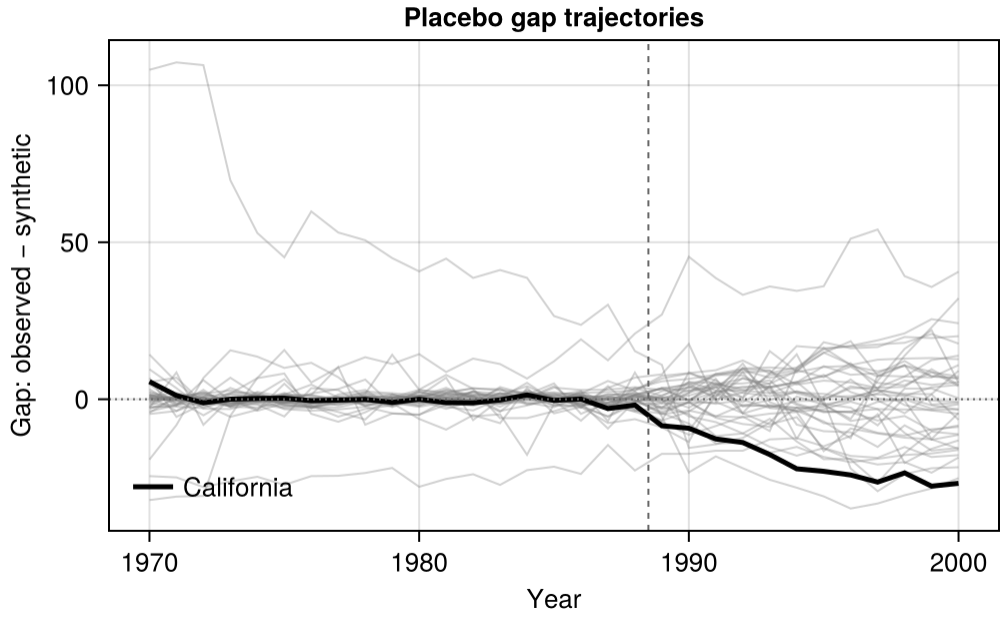


Figure 16.1.: California's treatment-effect gap (black) vs placebo gaps from each donor state (grey). The vertical dashed line marks 1989.

16.2. The MSPE ratio test

Some donor states fit California's pre-period trajectory better than others. A placebo state whose synthetic control fits its own pre-period poorly will produce a large *pre-period* gap by construction, inflating its placebo effect even under the sharp null. Abadie, Diamond, and Hainmueller (2015) propose normalising by pre-treatment fit:

$$\text{MSPE ratio}(\ell) = \frac{\frac{1}{T-T_0}\sum_{t>T_0}(Y_{\ell t} - \hat{Y}_{\ell t})^2}{\frac{1}{T_0}\sum_{t\le T_0}(Y_{\ell t} - \hat{Y}_{\ell t})^2}. \tag{16.1}$$

A large ratio means post-treatment gap is large *relative to* pre-treatment fit. The MSPE ratio test compares California's ratio to the distribution of placebo ratios.

```
function mspe_ratio(treated, cfact, T0)
    pre  = mean((treated[1:T0] .- cfact[1:T0]).^2)
    post = mean((treated[(T0 + 1):end] .- cfact[(T0 + 1):end]).^2)
    return post / max(pre, eps())
end

ca_ratio = mspe_ratio(ca_obs, ca_cfact, T0)

placebo_ratios = Float64[]
for i in 1:N0
```

```
    treated_i, cfact_i = run_placebo(Y, N0, T0, i)
    push!(placebo_ratios, mspe_ratio(treated_i, cfact_i, T0))
end

p_ratio = (1 + sum(placebo_ratios .>= ca_ratio)) / (length(placebo_ratios) + 1)
@printf("California MSPE ratio: %.2f\n", ca_ratio)
@printf("Placebo ratio 75th pct: %.2f\n", quantile(placebo_ratios, 0.75))
@printf("Placebo ratio max:     %.2f\n", maximum(placebo_ratios))
@printf("One-sided p-value (ratio): %.3f\n", p_ratio)
```

```
California MSPE ratio: 154.94
Placebo ratio 75th pct: 35.00
Placebo ratio max:     536.00
One-sided p-value (ratio): 0.077
```

The MSPE ratio discounts placebo states that were poorly fit before treatment.

```
fig = Figure(size = (640, 360), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Post/Pre MSPE ratio",
           ylabel = "Count", title = "MSPE ratio distribution")
hist!(ax, placebo_ratios, bins = 20, color = (:steelblue, 0.6))
vlines!(ax, [ca_ratio]; color = :firebrick, linewidth = 2, linestyle = :dash,
        label = "California")
axislegend(ax, position = :rt, framevisible = false)
fig
```

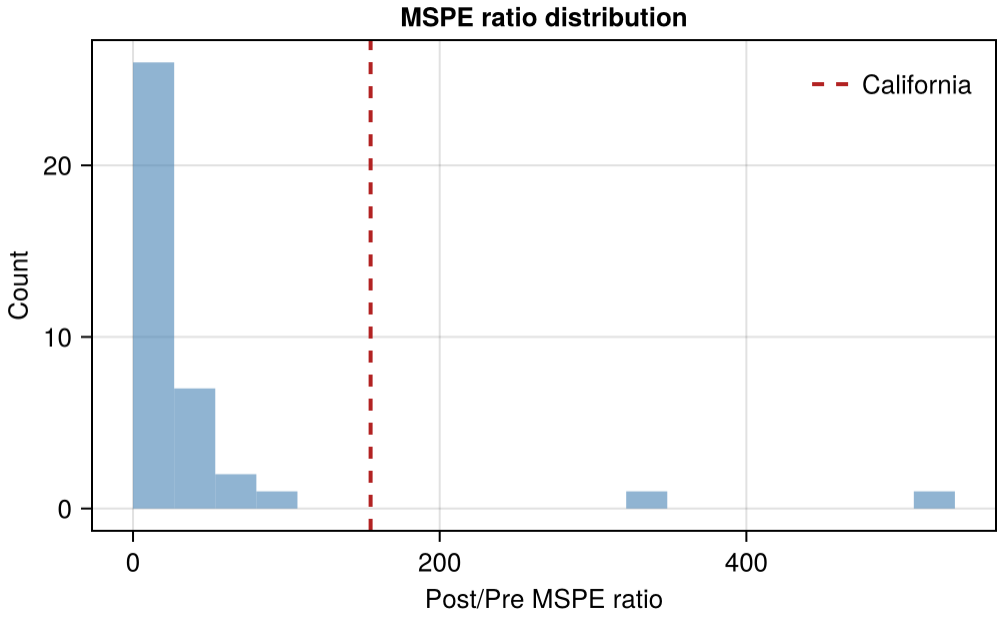


Figure 16.2.: Distribution of post/pre MSPE ratios across donor states (histogram). California's ratio is the red dashed line.

16.3. Time placebo

Another check moves the treatment date backward into the pre-period. If the method finds a large placebo effect before Proposition 99, the post-treatment effect is less credible.

```
# Pretend treatment started in 1980 instead of 1989
T0_placebo = findfirst(==(1980), years) - 1
Y_pre      = Y[:, 1:T0]   # cut off real treatment period
fit_pre    = sc_estimate(Y_pre, N0, T0_placebo)

ca_pre     = vec(Y_pre[N0 + 1, :])
ca_pre_cf  = vec(fit_pre.weights.omega' * Y_pre[1:N0, :])
placebo_effect_time = mean(ca_pre[(T0_placebo + 1):end] .- ca_pre_cf[(T0_placebo + 1):end])

@printf("Time-placebo effect (treatment moved to 1980): %.2f\n", placebo_effect_time)
@printf("Real effect (treatment in 1989):               %.2f\n", ca_effect)
```

```
Time-placebo effect (treatment moved to 1980): -3.31
Real effect (treatment in 1989):               -19.62
```

Here the time-placebo effect is much smaller than the real effect.

16.4. TASC: a model-based band

TASC models the panel with a linear Gaussian state-space process. Its band comes from the Kalman smoother with the parameters fixed at their EM estimates: at each year it is the variance $h' P_t h$ of the latent untreated signal, where h is California's loading vector and P_t the smoothed state covariance. So it is a pointwise interval for the latent signal, conditional on the fitted parameters. It leaves out the observation noise, the uncertainty in the parameters, and donor selection. A predictive interval for California's untreated outcome would be wider, and an interval for the average effect would also need the covariance across years.

```
Y_tasc = vcat(Y[(N0 + 1):end, :], Y[1:N0, :])
tasc_model = fit_tasc(Y_tasc; d = 2, T0 = T0, n_em = 200, tol = 1e-3)
tasc_pred  = predict_counterfactual(tasc_model, Y_tasc)

tasc_cfact = vec(tasc_pred.target)
tasc_se    = sqrt.(max.(vec(tasc_pred.variance), 0.0))
tasc_lower = tasc_cfact .- 1.96 .* tasc_se
tasc_upper = tasc_cfact .+ 1.96 .* tasc_se

# At each post-treatment year, is California outside the band?
years_post = years[(T0 + 1):end]
outside    = (ca_obs[(T0 + 1):end] .< tasc_lower[(T0 + 1):end]) .|
             (ca_obs[(T0 + 1):end] .> tasc_upper[(T0 + 1):end])
@printf("Years California is outside the 95%% band: %d/%d\n",
        sum(outside), length(outside))
```

```
Years California is outside the 95% band: 12/12
```

The count is not a predictive check. The band covers the latent signal only, so even with no treatment effect, observed outcomes would fall outside it more often than one year in twenty.

```
# Convert placebo gaps to counterfactual paths for visualisation
fig = Figure(size = (820, 420), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Year", ylabel = "Packs per capita",
           title = "Placebo cloud vs TASC latent-signal band")

# Placebo counterfactuals (treated outcome minus gap, mapped to California's level)
for i in 1:N0
    cf = placebo_cfacts[i, :] .+ (ca_obs[T0] - placebo_cfacts[i, T0])
    lines!(ax, years, cf, color = (:grey, 0.20), linewidth = 1)
end

band!(ax, years, tasc_lower, tasc_upper;
      color = (:seagreen, 0.25), label = "TASC 95% interval")
lines!(ax, years, ca_obs;    color = :black,     linewidth = 3, label = "California")
lines!(ax, years, ca_cfact;  color = :steelblue, linewidth = 2, label = "SC counterfactual")
```

```
lines!(ax, years, tasc_cfact; color = :seagreen,  linewidth = 2, label = "TASC counterfactual")
vlines!(ax, [years[T0] + 0.5]; color = :gray40, linestyle = :dash, linewidth = 1)
axislegend(ax, position = :lb, framevisible = false)
fig
```

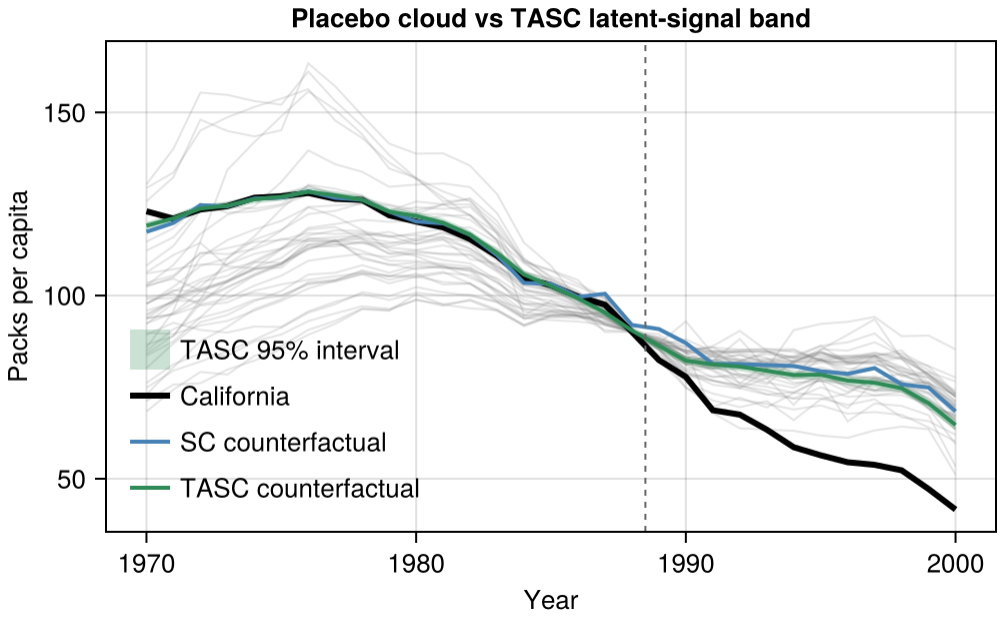


Figure 16.3.: California's observed path (black) with the TASC latent-signal band (green), classical SC counterfactual (blue), and placebo gap envelope (grey).

16.5. How the two inference strategies compare

	Randomization inference	TASC band
What is uncertain?	Which unit was treated	Latent factor path
Null hypothesis	Sharp null: no effect anywhere	None — direct uncertainty
Validity requires	An assignment or exchangeability argument for the treated label, plus symmetric reapplication of the estimator	SSM correctly specified
Output	Permutation p-value	Pointwise latent-signal band
Smallest p possible	$1/(N_0+1)$ (here ~0.026)	N/A — continuous
Sensitivity to donor pool	High (small N0 → coarse p)	Not priced: the band is conditional on the chosen donors and model

The two strategies answer different questions. Randomization inference asks whether California is unusual relative to donor-state placebos. TASC asks how uncertain the model is about California's counterfactual path. I would report both when using TASC.

16.6. Summary

- With one treated unit, randomization inference is more useful than asymptotic standard errors.
- Unit placebos give a reference distribution. It is design-based randomization inference only under an explicit assignment or exchangeability assumption; otherwise it is a placebo diagnostic.
- MSPE ratios account for pre-treatment fit.
- Time placebos check whether the method finds effects before treatment.
- TASC bands give model-based uncertainty for the latent untreated signal, conditional on the donor pool, the state-space model and its EM parameter estimates. They leave out observation noise, parameter uncertainty, donor selection and misspecification.

17. IV & Regression Discontinuity

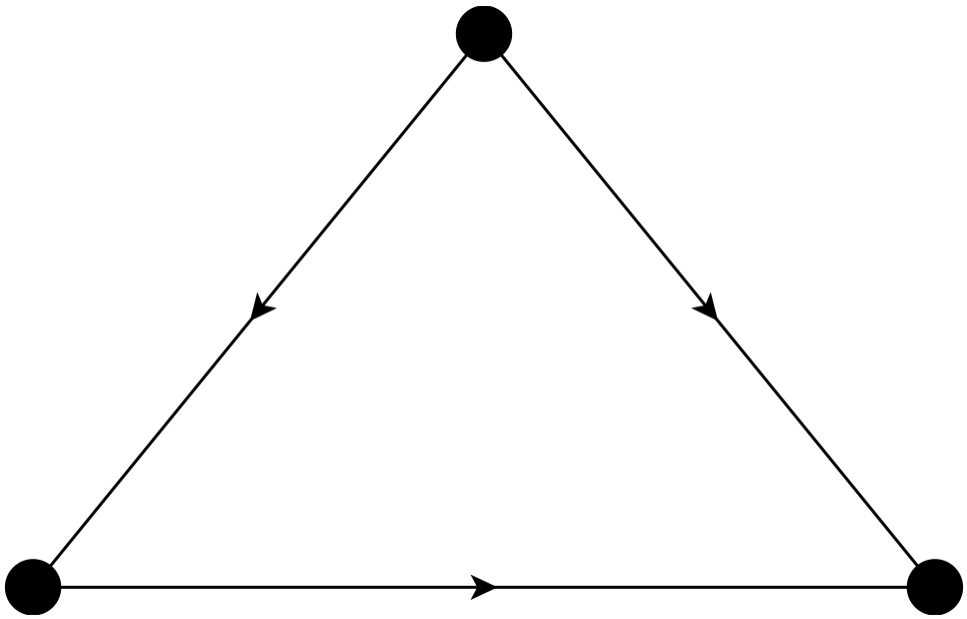
```
using CairoMakie
CairoMakie.activate!(type = "png")
using GraphMakie, Graphs
using Distributions, Random, LinearAlgebra
using RDRobust
using Panelest
using Vcov
using DataFrames
using StatsModels, DataFramesMeta
import StatsAPI: coeftable
using Printf
using CSV
using Statistics
using RegressionTables
```

With an unobserved confounder, unconfoundedness fails. Even a randomized experiment can face this problem through noncompliance: subjects are randomly assigned but some do not comply, so the actually-treated group is self-selected and that selection may be correlated with the outcome.

17.1. Instrumental Variables

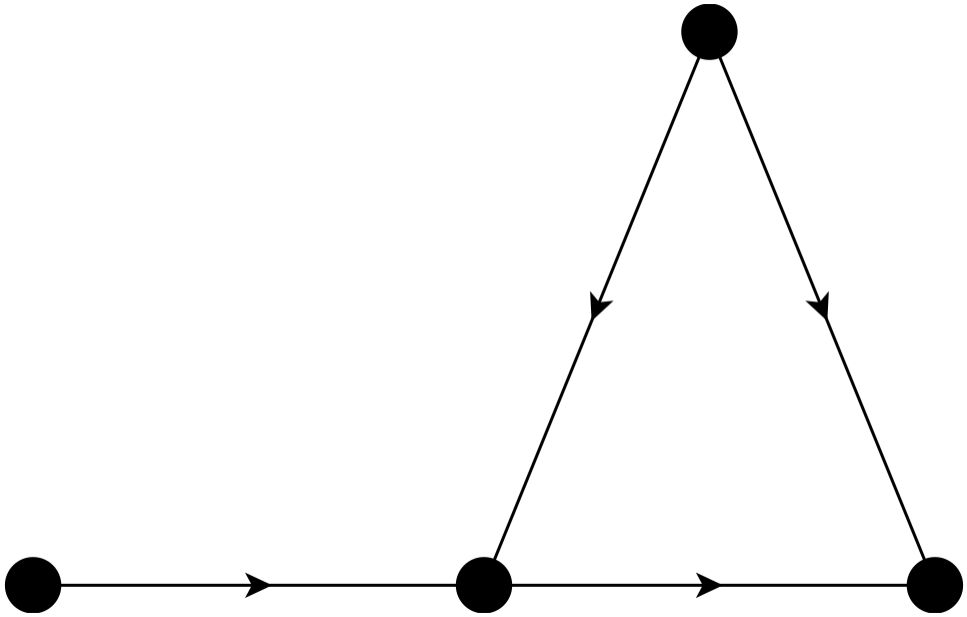
17.1.1. Uncontrolled Confounder

The graph below is the problem: c is an unobserved common cause of the treatment w and the outcome y. There is no adjustment set, because c is not measured.



17.1.2. Why Does IV Work?

Adding z gives a variable that moves w and reaches y only through w. The arrow $z \to w$ is the relevance condition, and the *absence* of any arrow from z to y or to c is the exclusion restriction and the exogeneity condition. The missing arrows carry the assumptions.



17.1.3. IV Under Potential Outcomes

- W has two potential outcomes W(1) and W(0), as a function of Z.
- Y has two potential outcomes Y(1) and Y(0), as a function of W.

Intention to Treat (ITT) is the average treatment effect of the treatment assignment Z. It is the difference between the average potential outcome under treatment and the average potential outcome under control. It is the average treatment effect of the treatment assignment Z, regardless of whether the subject complies with the assignment.

There are four possible groups of subjects in the case of binary treatment and binary instrument. The groups are defined by the *potential* treatment status $(W(0), W(1))$ — how the subject would take treatment under each value of the instrument — not by the observed (Z, W):

- $W(0) = 0, W(1) = 0$: never-taker
- $W(0) = 0, W(1) = 1$: complier
- $W(0) = 1, W(1) = 0$: defier
- $W(0) = 1, W(1) = 1$: always-taker

Table 17.1.: compliance groups

	W(1)=0	W(1)=1
W(0)=0	N	C
W(0)=1	D	A

These four groups (strata) have different causal mechanisms. The compliers and defiers are the groups whose *treatment status* responds to the assignment Z — compliers in the same direction as the assignment, defiers in the opposite direction. The always-takers and never-takers take the same treatment regardless of Z.

Unfortunately we cannot identify the compliance group by looking at the data.

Z	W	G
0	0	C, N
0	1	A, D
1	0	N, D
1	1	C, A

17.1.4. Assumptions

- Exclusion restriction: $Y(w, 1) = Y(w, 0)$
- Exogeneity of instrument: $Z \perp [W(0), W(1), Y(0), Y(1)]$
- Monotonicity (no defiers): $W(0) \leq W(1)$

17.1.5. LATE Identification

Since we excluded defiers, and the other two groups (always-takers and never-takers) are not affected by the treatment assignment Z. The only effect we can identify is the treatment effect on the compliers. This is called the Local Average Treatment Effect (LATE).

Below we write potential outcomes indexed by the instrument, $Y(Z = z)$, rather than by treatment status, $Y(d)$. The exclusion restriction is exactly what licenses this: since $Y(w, 1) = Y(w, 0)$, Y depends on Z only through W, so $Y(Z = z) = Y(W(z), z) = Y(W(z))$ in the usual treatment-indexed notation. For a complier $(W(1) = 1, W(0) = 0)$, this means $Y(Z = 1) = Y(1)$ and $Y(Z = 0) = Y(0)$, so $E[Y(Z = 1) - Y(Z = 0) \mid G = C]$ below is exactly $E[Y(1) - Y(0) \mid G = C]$, the LATE in treatment-indexed notation.

$$
\begin{aligned}
&E[Y(Z=1) - Y(Z=0) | G = C] \\
&= \frac{E[Y(Z=1) - Y(Z=0)]}{P(W(1)=1, W(0)=0)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{1 - P(W=0|Z=1) - P(W=1|Z=0)} \text{ (monotonicity rules out D; the A and N shares subtract)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{P(W=1|Z=1) - P(W=1|Z=0)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{E[W|Z=1] - E[W|Z=0]}
\end{aligned}
\tag{17.1}
$$

17.1.6. Parametric Models

With a binary instrument this is exactly the ratio of two OLS coefficients — no linearity assumption is needed, since a regression on a binary regressor just computes the two group means. This is sometimes called the Wald estimator.

$$
\begin{aligned}
\tau_{LATE} &= \frac{E[Y|Z=1] - E[Y|Z=0]}{E[W|Z=1] - E[W|Z=0]} \\
&= \frac{Cov(Y, Z)}{Cov(W, Z)}
\end{aligned}
\tag{17.2}
$$

Why do covariates help? It is important to separate two assumptions here. Conditioning on covariates X can make the *as-if-random* (conditional independence) assumption more credible: the instrument may be unconfounded only within levels of X. But the *exclusion restriction* – that the instrument has no direct effect on the outcome – is a substantive structural assumption that covariates do not generally repair. Adding X removes a direct $Z \to Y$ path only if that path runs entirely through X, or if exclusion is conditional by design. So covariates help with conditional independence, not with exclusion per se.

17.1.7. Example

We simulate $n = 10{,}000$ observations. Three variables (w, m, z) are drawn jointly normal with mean zero and unit variances, so their covariances are also their correlations: $\text{corr}(w, m) = 0.36$, $\text{corr}(w, z) = 0.64$, and $\text{corr}(m, z) = 0$. A fourth variable $u \sim N(0, 1)$ is independent of everything. The outcome is $y = w + m + u$.

So m is the omitted confounder, correlated with the treatment w but not with the instrument z, and the true coefficient on w is 1. Because all variances are 1, we can predict the omitted-variable bias exactly: regressing y on w alone gives $1 + \text{Cov}(m, w) = 1.36$.

We fit three models: OLS of y on w, OLS of y on w and m, and 2SLS using z as the instrument for w.

```
## DGP: data.y = data.w + data.m + data.u
Random.seed!(66)
nobs = 10000
## here we have three variables w, m, z.
## m is the omitted variable; w and m are correlated; z is the instrument,
## which is correlated with w, but not m. u is independent of everything else.
## with unit variances, the covariances below ARE the correlations
crwm = 0.36; crmz = 0.0; crwz = 0.64
covarMat = [1.0  crwm crwz;
            crwm 1.0  crmz;
            crwz crmz 1.0]
mat  = rand(MvNormal(zeros(3), covarMat), nobs)'
data = DataFrame(w = mat[:, 1], m = mat[:, 2], z = mat[:, 3])
data.u = rand(Normal(0, 1), nobs)
# dgp
data.y = data.w .+ data.m .+ data.u
lm_biased  = feols(data, @formula(y ~ w))
lm_full    = feols(data, @formula(y ~ w + m))
tsls_model = feiv(data, @formula(y ~ 1); endo = :w, inst = :z);
```

```
# lm is biased
display(coeftable(lm_biased))
# lm_full is good
display(coeftable(lm_full))
# tsls is good.
display(coeftable(tsls_model))
```

	Estimate	Std. Error	z value	Pr(>
(Intercept)	-0.0116415	0.0136663	-0.85	0.3943
w	1.35705	0.0136527	99.40	<1e-99

	Estimate	Std. Error	z value	Pr(>
(Intercept)	-0.00914669	0.00997142	-0.92	0.3590
w	0.997584	0.0106743	93.46	<1e-99
m	0.998357	0.0106525	93.72	<1e-99

	Estimate	Std. Error	z value	Pr(>
w	0.999871	0.0219208	45.61	<1e-99
(Intercept)	-0.00577212	0.0141291	-0.41	0.6829

OLS on w alone gives 1.3571, matching the predicted 1.36 to two decimals. OLS controlling for m gives 0.9976 on w and 0.9984 on m, both close to the true 1, which is what we would do if the confounder were observed. 2SLS gives 0.9999 with a standard error of 0.0219, recovering the truth without ever using m.

Note the cost of instrumenting: the 2SLS standard error is 0.0219 against 0.0107 for the OLS that controls for m, twice as large. The instrument uses only the part of w that z explains, and here that is 41% of its variance.

17.1.8. Control Function Approach

Two-stage least squares can be implemented equivalently via the **control function (CF) approach**. Understanding why makes the extension to nonlinear models (next chapter) transparent.

Algebraic motivation. Because `w` is endogenous, decompose the structural error:

$$\varepsilon = \rho v + \eta, \qquad v \equiv w - E[w \mid z], \qquad E(z\eta) = 0,\ E(zv) = 0. \tag{17.3}$$

If we *knew* v, controlling for it directly would eliminate the confounding channel. We don't know v, but the first-stage residual $\hat{v} = w - \hat{w}$ is a consistent estimate. Substituting into the structural equation:

$$y = \alpha + \beta w + \rho\,\hat{v} + \text{error}. \tag{17.4}$$

OLS on this **augmented** second stage gives a consistent $\hat{\beta}$ — identical to 2SLS by the Frisch–Waugh–Lovell (FWL) theorem. The coefficient $\hat{\rho}$ on $\hat{v}$ doubles as a **Durbin–Wu–Hausman (DWH) endogeneity test**: under H_0 (w is exogenous), $\rho = 0$.

```
import StatsBase: coef, stderror, coefnames
pick(m, v, name) = v[findfirst(==(name), coefnames(m))]

# Manual 2SLS, explicitly
stage1      = feols(data, @formula(w ~ z))
data.w_hat  = data.w .- stage1.residuals
tsls_manual = feols(data, @formula(y ~ w_hat))
```

```
# Control function: first-stage residual v_hat in the augmented regression
data.v_hat = stage1.residuals
lm_cf      = feols(data, @formula(y ~ w + v_hat))

# Compare ^_w from manual 2SLS and CF - should be identical by FWL
@printf("Manual 2SLS   beta_w = %.8f\n", pick(tsls_manual, coef(tsls_manual), "w_hat"))
@printf("Control funct beta_w = %.8f\n", pick(lm_cf, coef(lm_cf), "w"))

# DWH endogeneity test: t-stat on v_hat
rho, rho_se = pick(lm_cf, coef(lm_cf), "v_hat"), pick(lm_cf, stderror(lm_cf), "v_hat")
@printf("DWH test on v_hat: coef = %.4f  se = %.4f  t = %.3f  p = %.4g\n",
        rho, rho_se, rho / rho_se, 2 * ccdf(Normal(), abs(rho / rho_se)))
```

```
Manual 2SLS   beta_w = 0.99987086
Control funct beta_w = 0.99987086
DWH test on v_hat: coef = 0.6100  se = 0.0270  t = 22.563  p = 9.987e-113
```

Both routes give $\hat{\beta}_w = 0.99987086$, agreeing to all eight printed digits, which confirms FWL. The coefficient on $\hat{v}$ is 0.6100 with a standard error of 0.0270, so $t = 22.6$ and $H_0 : \rho = 0$ is rejected: `w` is endogenous. That coefficient also has a value we can derive. With $\varepsilon = m + u$ and $v = w - 0.64z$, we have $\text{Cov}(\varepsilon, v) = \text{Cov}(m, w) = 0.36$ and $\text{Var}(v) = 1 - 0.64^2 = 0.5904$, so $\rho = 0.36/0.5904 = 0.610$. The estimate matches it to three decimals. One caveat: the OLS standard errors from the augmented regression are not valid for $\hat{\beta}$ ($\hat{v}$ is a generated regressor; the DWH test is fine because under H_0 the first-stage estimation error does not matter). Use 2SLS standard errors, or bootstrap the two stages jointly as we do in the Poisson IV chapter.

Why FWL fails in nonlinear models. FWL is a property of linear projections; once we replace OLS with Poisson or Probit, the estimator is no longer a linear projection and the partialling-out logic does not apply. In a nonlinear second stage, $\hat{v}$ must enter *inside* the link function (e.g. $\exp(\mathbf{x}\beta + \rho\hat{v})$), which requires a structural assumption beyond instrument exogeneity. This is the subject of the next chapter.

17.2. When 2SLS with Covariates Is *Actually* LATE

The LATE identification above is clean because we have no covariates. The Wald estimator gives us LATE, full stop. But in practice we usually add covariates to defend conditional exogeneity:

$$Y = \beta W + \gamma^\top X + U, \qquad W = \pi Z + \delta^\top X + V. \tag{17.5}$$

Does 2SLS on this still give us a LATE? Angrist and Pischke (2009) say it gives a weighted average of covariate-specific LATEs. Blandhol et al. (2025) show that this is generally not true.

17.2.1. Why the Linear Specification Hides an Assumption

By Frisch-Waugh-Lovell, the IV estimand is

$$\beta_{iv} = \frac{E[Y\tilde{Z}]}{E[W\tilde{Z}]}, \qquad \tilde{Z} = Z - L[Z \mid X], \tag{17.6}$$

where $L[Z \mid X]$ is the **linear** projection of Z on X. So $\tilde{Z}$ is the part of Z that a linear regression on X cannot explain.

What if the true $E[Z \mid X]$ is not linear in X? Then $L[Z \mid X] \neq E[Z \mid X]$, and the residual instrument $\tilde{Z}$ still picks up some of the conditional mean. The IV estimand then mixes treatment effects across compliance groups. Blandhol et al. (2025) prove that β_{iv} is a non-negatively weighted average of conditional LATEs only if the rich covariates condition holds:

$$L[Z \mid X] = E[Z \mid X]. \tag{17.7}$$

This is a parametric assumption about $E[Z \mid X]$. It is implicit in the linear-in-X specification, and it is rarely defended. When it fails, β_{iv} picks up treatment effects for always-takers as well as compliers, and some always-taker terms enter with negative weight.

"2SLS with covariates estimates LATE" is true only if X enters saturated (one dummy per cell), or the true $E[Z \mid X]$ happens to be linear in X.

17.2.2. A Simulation

We build a DGP where $E[Z \mid X]$ is strongly nonlinear in X, so the linear-in-X specification has to fail, and then compare four estimators against the true LATE.

We draw $n = 8000$ observations with a single covariate $X \sim U(-1, 1)$. The instrument is $Z \sim \text{Bernoulli}(p_Z(X))$ with $p_Z(X) = \Lambda(0.3 + X + 2\sin(2.5\pi X))$ — the sine term is what a linear projection cannot reproduce. Potential treatments come from one uniform draw U compared against two thresholds, $T_0 = \mathbb{1}\{U < \Lambda(-1.6 + X)\}$ and $T_1 = \mathbb{1}\{U < \min(\Lambda(-0.4 + X) + 0.2,\, 0.95)\}$, so $T_1 \geq T_0$ and monotonicity holds by construction. Observed treatment is $W = ZT_1 + (1 - Z)T_0$. The effect is heterogeneous in X, $\tau(X) = 0.5 + 1.5X$, and the potential outcomes are $Y(0) = 1 + 2X + 0.8X^2 + \varepsilon$ with $\varepsilon \sim N(0, 1)$ and $Y(1) = Y(0) + \tau(X)$.

Two features matter. X enters both the instrument propensity and the treatment thresholds, so Z is exogenous only conditional on X. And τ depends on X, so which units the instrument moves determines the answer.

```
using MLJ
using DecisionTree
using Random
using Statistics
using LinearAlgebra

Random.seed!(13)
```

```
n = 8000
plogis(z) = 1 / (1 + exp(-z))

# Single covariate on [-1, 1]
X = 2 .* rand(n) .- 1

# True conditional mean of Z is wildly nonlinear in X.
# A linear projection L[Z|X] cannot reproduce this.
pZ_true = plogis.(0.3 .+ 1.0 .* X .+ 2.0 .* sin.(2.5 .*   .* X))
Z       = Int.(rand(n) .< pZ_true)

# Potential treatment states with monotonicity (T1 >= T0).
# X enters the propensity, so X confounds W <-> Y.
U  = rand(n)
p0 = plogis.(-1.6 .+ 1.0 .* X)
p1 = min.(plogis.(-0.4 .+ 1.0 .* X) .+ 0.2, 0.95)
T0 = Int.(U .< p0)
T1 = Int.(U .< p1)
W  = ifelse.(Z .== 1, T1, T0)

G  = [t1 == 1 && t0 == 1 ? :AT :
      t1 == 0 && t0 == 0 ? :NT : :CP
      for (t1, t0) in zip(T1, T0)]

# Heterogeneous treatment effect - varies with X
  = 0.5 .+ 1.5 .* X
Y0 = 1.0 .+ 2.0 .* X .+ 0.8 .* X .^ 2 .+ randn(n)
Y1 = Y0 .+
Y  = ifelse.(W .== 1, Y1, Y0)

true_LATE = mean( [G .== :CP])
@printf("Compliance shares  AT=%.3f  CP=%.3f  NT=%.3f\n",
        mean(G .== :AT), mean(G .== :CP), mean(G .== :NT))
@printf("True unconditional LATE = %.3f\n", true_LATE)
```

```
Compliance shares  AT=0.188  CP=0.425  NT=0.388
True unconditional LATE = 0.617
```

The compliance shares are 18.8% always-takers, 42.5% compliers and 38.8% never-takers, and the true LATE, the mean of τ over compliers, is 0.617.

We now compare four estimators against it: the unconditional Wald ratio, 2SLS with X entered linearly, 2SLS with a degree-7 polynomial in X, and DDML PLIV with random-forest nuisance learners. Julia has no counterpart of R's `DoubleML` package, so we write the PLIV cross-fitting out.

```
# Helper: TSLS via the matrix expression (Xmat is the exogenous matrix incl.
# constant). Returns the coefficient on W and its conventional standard error.
function tsls_matrix(Y, W, Z, Xmat)
    Wmat = hcat(W, Xmat)
    Zmat = hcat(Z, Xmat)
    What = Zmat * ((Zmat' * Zmat) \ (Zmat' * Wmat))
         = (What' * Wmat) \ (What' * Y)
    e    = Y .- Wmat *
    2    = sum(abs2, e) / (length(Y) - size(Wmat, 2))
    return  [1], sqrt( 2 * inv(What' * What)[1, 1])
end

# (a) Wald - biased: Z is not unconditionally exogenous because X confounds.
wald = (mean(Y[Z .== 1]) - mean(Y[Z .== 0])) /
       (mean(W[Z .== 1]) - mean(W[Z .== 0]))

# (b) TSLS with X entered linearly - the textbook spec.
b_lin, se_lin = tsls_matrix(Float64.(Y), Float64.(W), Float64.(Z), hcat(ones(n), X))

# (c) TSLS with a rich basis (polynomial up to degree 7) - closer to saturation.
b_poly, se_poly = tsls_matrix(Float64.(Y), Float64.(W), Float64.(Z),
                              hcat([X .^ k for k in 0:7]...))

# (d) DDML PLIV with random-forest nuisance learners and 5-fold cross-fitting.
RandomForestRegressor = @load RandomForestRegressor pkg=DecisionTree verbosity=0
learner = RandomForestRegressor(n_trees = 500, max_depth = 5)

nsplits = 5
s       = shuffle(1:n)
folds   = [s[1 + floor(Int, (k-1)*n/nsplits) : floor(Int, k*n/nsplits)]
           for k in 1:nsplits]

mY = zeros(n); mW = zeros(n); mZ = zeros(n)
Xtab = DataFrame(X = X)
for fold_idx in 1:nsplits
    test  = folds[fold_idx]
    train = setdiff(1:n, test)
    for (out, dest) in ((Y, mY), (W, mW), (Z, mZ))
        mach = machine(learner, Xtab[train, :], Float64.(out[train]))
        MLJ.fit!(mach, verbosity = 0)
        dest[test] = MLJ.predict(mach, Xtab[test, :])
    end
end

rY = Y .- mY; rW = W .- mW; rZ = Z .- mZ
b_pliv  = sum(rZ .* rY) / sum(rZ .* rW)
        = (rY .- b_pliv .* rW) .* rZ              # PLIV score
```

```
se_pliv = sqrt(mean(  .^ 2) / mean(rW .* rZ)^2 / n)

results = DataFrame(
    Estimator = ["True LATE",
                 "Wald (no covariates)",
                 "TSLS, X linear",
                 "TSLS, poly(X, 7)",
                 "DDML PLIV (cross-fit, random forest)"],
    Estimate  = round.([true_LATE, wald, b_lin, b_poly, b_pliv], digits = 3),
    SE        = [missing, missing, round(se_lin, digits = 3), round(se_poly, digits = 3),
                 round(se_pliv, digits = 3)]
)
```

	Estimator	Estimate	SE
	String	Float64	Float64?
1	True LATE	0.617	*missing*
2	Wald (no covariates)	1.91	*missing*
3	TSLS, X linear	0.552	0.059
4	TSLS, poly(X, 7)	0.585	0.067
5	DDML PLIV (cross-fit, random forest)	0.582	0.07

The linear-in-X 2SLS does not recover the true LATE — and neither do the more flexible versions. Two things are going on, and the standard errors in the table help separate them. First, the estimands differ: for this DGP the linear-projection 2SLS converges to 0.55, and the polynomial 2SLS and DDML PLIV converge to approximately $\beta_{rich} = 0.58$ (computable by integrating the weight formula below), all below the LATE of 0.617 — a real but modest gap that persists at any sample size. Second, sampling noise: the SEs are 0.06 to 0.07, and in this draw each estimate lands within 0.01 of its own estimand, at 0.552, 0.585 and 0.582, so here the whole gap to the LATE is the difference in estimands. The Wald estimator is far off for a different reason entirely, at 1.910 against a truth of 0.617: Z is not unconditionally exogenous, and Wald has no way to use X.

What are the polynomial 2SLS and DDML PLIV estimating, if not LATE? PLIV targets β_{rich}, a weighted average of conditional LATEs with weights proportional to $\text{Var}(Z \mid X) \cdot P(\text{complier} \mid X)$; the polynomial 2SLS targets nearly the same quantity (a degree-7 polynomial cannot fully reproduce this $E[Z \mid X]$, so its estimand is a misspecified-projection variant sitting just below β_{rich}). These are non-negatively weighted, so weakly causal. But they are not the unconditional LATE we usually want.

17.2.3. Diagnostic: RESET on Z ~ X

Rich covariates says $E[Z \mid X]$ is linear in X. Ramsey's RESET test (Ramsey 1969) checks one implication of that: whether a few higher-order terms add to a regression of Z on X. It can detect some departures from linearity. It cannot establish linearity, and it says nothing about exclusion, instrument independence or monotonicity.

```julia
using GLM
# RESET with type = "regressor", power = 2:3: F-test of X^2 and X^3 added to Z ~ X
df_reset       = DataFrame(Z = Float64.(Z), X = X, X2 = X .^ 2, X3 = X .^ 3)
m_restricted   = lm(@formula(Z ~ X), df_reset)
m_unrestricted = lm(@formula(Z ~ X + X2 + X3), df_reset)
rss_r, rss_u   = sum(abs2, residuals(m_restricted)), sum(abs2, residuals(m_unrestricted))
q, k_u         = 2, 4
F_stat         = ((rss_r - rss_u) / q) / (rss_u / (n - k_u))
@printf("RESET = %.2f, df1 = %d, df2 = %d, p-value = %.3g\n",
        F_stat, q, n - k_u, ccdf(FDist(q, n - k_u), F_stat))
```

```
RESET = 131.57, df1 = 2, df2 = 7996, p-value = 6.03e-57
```

It rejects strongly here, RESET = 131.6 on 2 and 7,996 degrees of freedom with $p < 10^{-56}$, which is the right answer given that we put a sine wave in $E[Z \mid X]$. The same test on real data is cheap. A rejection is a clear warning against reading 2SLS as the LATE; a non-rejection is weaker evidence, because the test looks only at the terms we included.

17.2.4. What to Do Instead

Blandhol et al. (2025) give four steps for empirical work.

1. **Reconsider the covariates.** If Z is unconditionally exogenous, drop them. A kitchen-sink set of controls makes rich covariates *less* likely.
2. **Run RESET on `Z ~ X`.** A rejection rules out reading linear IV as the LATE. A non-rejection is consistent with rich covariates but does not prove it, and the other IV assumptions still need their own argument.
3. **If RESET rejects, report DDML PLIV alongside 2SLS.** That is what the cross-fitting chunk above does. DDML PLIV targets β_{rich}, a non-negatively weighted average of conditional LATEs.
4. **For a binary instrument, also estimate the unconditional LATE.** With instrument propensity score weighting (Słoczyński 2024),

$$\hat{\beta}_{late} = \frac{\sum_i Y_i[Z_i/\hat{p}(X_i) - (1-Z_i)/(1-\hat{p}(X_i))]}{\sum_i W_i[Z_i/\hat{p}(X_i) - (1-Z_i)/(1-\hat{p}(X_i))]}, \tag{17.8}$$

   where $\hat{p}(X) = P(Z = 1 \mid X)$ is estimated nonparametrically. Stata has `kappalate` (Słoczyński, Uysal and Wooldridge), which implements this and related weighting estimators of the LATE. In Julia it is straightforward to build once $\hat{p}(X)$ is in hand.

17.2.5. A Note on the FRDD Example

The fuzzy-RDD example at the end of this chapter is 2SLS with race, state of birth, and quarter-of-birth fixed effects entered as covariates. That section traces the gap between the 2SLS and the local `rdrobust` estimates to the first stage, which is close to zero near the cutoff. Rich covariates is a separate question about the 2SLS estimate itself, and the RESET test on the above-cutoff instrument against those covariates is the right first thing to run.

17.2.6. Bottom Line

"2SLS with covariates estimates LATE" needs the rich covariates condition. That condition is a parametric assumption on $E[Z \mid X]$ that researchers rarely defend, and a simple RESET test often rejects it. The honest alternatives are DDML PLIV for β_{rich} and IPSW (or DDML) for the unconditional LATE. Even when rich covariates holds, β_{rich} can be quite different from the unconditional LATE we usually care about.

17.3. Regression Discontinuity Design

RDD (regression discontinuity design) is a quasi-experimental design that is used to estimate causal effects of interventions when assignment to the intervention is determined by whether a subject's value on an observed covariate exceeds a threshold. The idea is that the assignment is as good as random, so we can estimate the causal effect of the intervention by comparing the outcomes of subjects who are just above and just below the threshold.

17.3.1. Sharp RDD

We have a continuous variable X, called the running variable, which determines the binary treatment W. The treatment is assigned according to a threshold c. The outcome variable Y is a function of X and W.

$$W = \mathbb{1}(X > c) \tag{17.9}$$

Lee (2008) studies the effect of incumbency advantage in elections. His identification strategy is based on the discontinuity generated by the rule that the party with a majority vote share wins. The forcing variable X_i is the difference in vote share between the Democratic and Republican parties in one election, with the threshold $c = 0$. The outcome variable Y_i is vote share at the second election.

`rdplot` bins the running variable and returns bin means and a polynomial fit on each side of the cutoff, which we plot. A visible jump at zero is the design working.

```
senate = CSV.read("data/rdrobust_senate.csv", DataFrame)

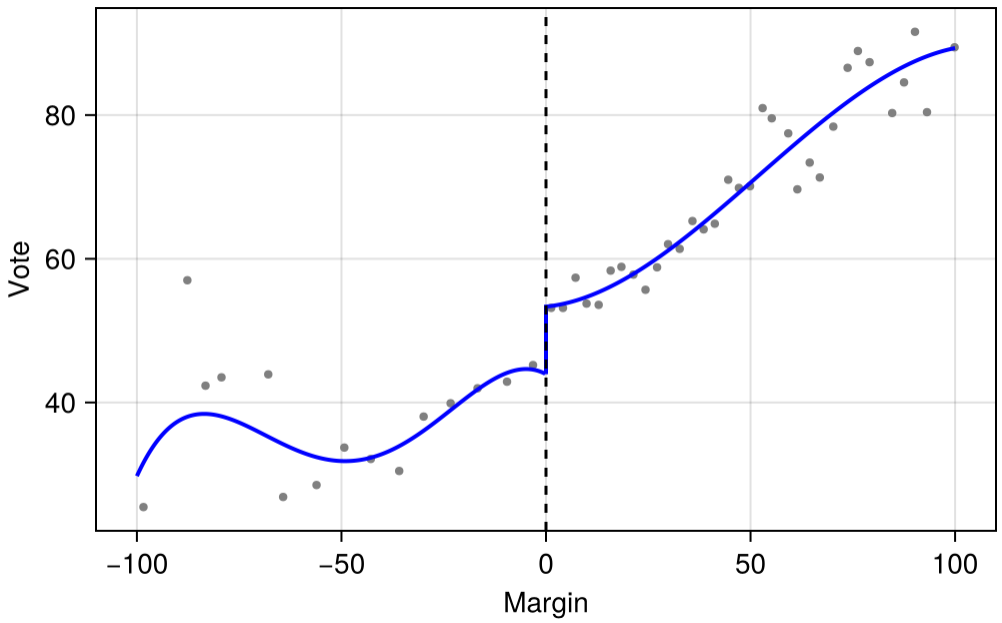
# Calculate RD plot data
plot_data = rdplot(senate.vote, senate.margin, c=0.0)

fig = Figure()
ax = Axis(fig[1, 1], xlabel="Margin", ylabel="Vote")
scatter!(ax, plot_data.vars_bins.rdplot_mean_x, plot_data.vars_bins.rdplot_mean_y,
         color=:gray, markersize=6)

# Plot polynomial fit for left side
poly_l = filter(r -> r.rdplot_x < 0.0, plot_data.vars_poly)
lines!(ax, poly_l.rdplot_x, poly_l.rdplot_y, color=:blue, linewidth=2)

# Plot polynomial fit for right side
poly_r = filter(r -> r.rdplot_x >= 0.0, plot_data.vars_poly)
lines!(ax, poly_r.rdplot_x, poly_r.rdplot_y, color=:blue, linewidth=2)

vlines!(ax, [0.0], color=:black, linestyle=:dash)
fig
```



The `rdrobust_RDsenate` data are 1,297 US Senate races. The running variable `margin` is the Democratic margin of victory in an election and `vote` is the Democratic vote share in the next election for the same seat. The bin means step up discontinuously at a margin of zero.

17.3.2. Identification of SRDD

$$\tau_{RD} = \lim_{x \to c^+} E[Y|X = x] - \lim_{x \to c^-} E[Y|X = x] \tag{17.10}$$

In words, the treatment effect is the difference in the outcome from the right side and from the left side. This is the same as the difference in the outcome at the threshold, as if the treatment is assigned randomly.

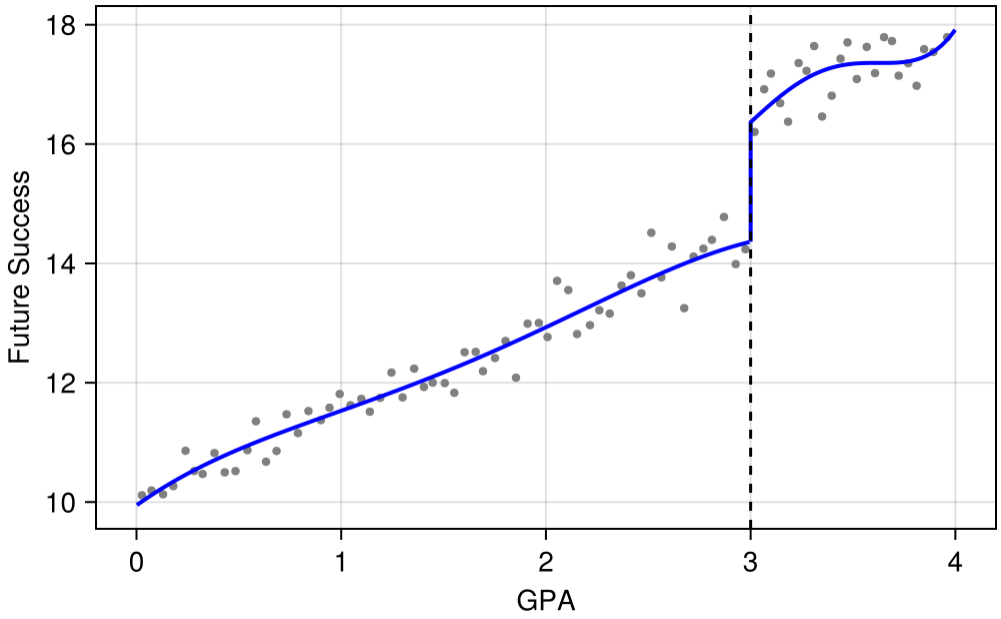
17.3.3. Estimation of SRDD

Before the real data, a simulation where we know the answer. We draw $n = 1000$ students with GPA $\sim U(0, 4)$ and outcome $Y = 10 + 1.5\,\text{GPA} + 2 \cdot \mathbb{1}\{\text{GPA} > 3\} + \varepsilon$ with $\varepsilon \sim N(0, 1)$. The cutoff is 3 and the true jump is 2. The running variable also has a direct effect on the outcome, with slope 1.5, which is what the local regression on each side has to absorb.

```
Random.seed!(2024)
GPA = rand(Uniform(0, 4), 1000)
# Treatment is W = 1(GPA > 3), so the +2 jump must sit on the right
# (above-cutoff) side to match the right-minus-left estimand above.
future_success = 10 .+ 1.5 .* GPA .+ 2 .* (GPA .> 3) .+ rand(Normal(0, 1), 1000)

plot_data_gpa = rdplot(future_success, GPA, c = 3.0)

fig2 = Figure()
ax2 = Axis(fig2[1, 1], xlabel = "GPA", ylabel = "Future Success")
scatter!(ax2, plot_data_gpa.vars_bins.rdplot_mean_x, plot_data_gpa.vars_bins.rdplot_mean_y,
         color = :gray, markersize = 6)
poly_gpa_l = filter(r -> r.rdplot_x < 3.0, plot_data_gpa.vars_poly)
lines!(ax2, poly_gpa_l.rdplot_x, poly_gpa_l.rdplot_y, color = :blue, linewidth = 2)
poly_gpa_r = filter(r -> r.rdplot_x >= 3.0, plot_data_gpa.vars_poly)
lines!(ax2, poly_gpa_r.rdplot_x, poly_gpa_r.rdplot_y, color = :blue, linewidth = 2)
vlines!(ax2, [3.0], color = :black, linestyle = :dash)
fig2
```




```
# a compact summary, like R's summary() of an rdrobust fit
function rd_summary(r)
    @printf("Bandwidth h = %.3f, b = %.3f; N = %d left, %d right; effective N = %d left, %d rig
            r.bws.h_left[1], r.bws.b_left[1], r.N[1], r.N[2], r.N_h[1], r.N_h[2])
    DataFrame(Method = r.coef.Method, Coef = round.(r.coef.Coeff, digits = 4),
              SE = round.(r.se.StdErr, digits = 4), p = round.(r.pv.PValue, sigdigits = 3),
              CI_low = round.(r.ci.CILower, digits = 3), CI_high = round.(r.ci.CIUpper, digits
end

# estimate the sharp RDD model
rdd_gpa = rdrobust(future_success, GPA, c = 3.0)
rd_summary(rdd_gpa)
```

```
Bandwidth h = 0.373, b = 0.536; N = 736 left, 264 right; effective N = 94 left, 98 right
```

	Method	Coef	SE	p	CI_low	CI_high
	String	Float64	Float64	Float64	Float64	Float64
1	Conventional	2.0776	0.3272	2.17e-10	1.436	2.719
2	Bias-Corrected	2.0456	0.3272	4.08e-10	1.404	2.687
3	Robust	2.0456	0.3925	1.88e-7	1.276	2.815

The estimated jump is 2.078 with a robust 95% interval of $[1.276, 2.815]$, covering the true 2. Note how little data the estimate uses: the MSE-optimal bandwidth is 0.373, so of the 1,000 observations only 94 on the left and 98 on the right are effective. A local estimator pays for its weak assumptions with a small effective sample, which is why the interval is 1.5 wide for a jump of 2.

17.3.4. Nonparametric Estimation

Now the same estimator on the Senate data, with the cutoff at a margin of zero.

```
rdd_house = rdrobust(senate.vote, senate.margin, c = 0.0)
rd_summary(rdd_house)
```

```
Bandwidth h = 17.754, b = 28.028; N = 595 left, 702 right; effective N = 360 left, 323 right
```

	Method	Coef	SE	p	CI_low	CI_high
	String	Float64	Float64	Float64	Float64	Float64
1	Conventional	7.4141	1.4587	3.72e-7	4.555	10.273
2	Bias-Corrected	7.5065	1.4587	2.66e-7	4.647	10.366
3	Robust	7.5065	1.7413	1.63e-5	4.094	10.919

The incumbency effect is 7.414 percentage points with a robust 95% interval of $[4.094, 10.919]$. Barely winning a Senate seat raises the party's vote share in the next election for that seat by about 7 points. The bandwidth is 17.75 margin points, leaving 360 observations on the left and 323 on the right out of 1,297.

17.3.5. Fuzzy RDD

In fuzzy RDD, the treatment is assigned according to a threshold c, but there exist non-compliance. This is similar to IV case.

$$\tau_{FRD} = \frac{\lim_{x \to c^+} E[Y|X = x] - \lim_{x \to c^-} E[Y|X = x]}{\lim_{x \to c^+} E[W|X = x] - \lim_{x \to c^-} E[W|X = x]} \tag{17.11}$$

For example, if food stamp eligibility is given to all households below a certain income, but not all households receive the food stamps. In other words, income does not solely determine the assignment. Cutoff point increases the probability of treatment but doesn't completely determine treatment.

FRDD is IV.

17.3.6. FRDD Example

Fetter (2013)'s main question of interest is how much of the increase in the home ownership rate in the midcentury US was due to mortgage subsidies given out by the government.

We're using the running variable quarter of birth (qob), which has been centered on the quarter of birth you'd need to be to be eligible for a mortgage subsidy for fighting in the Korean War (qob_minus_kw). This determines whether you were a veteran of either the Korean War or World War II (vet_wwko).

```
vet = CSV.read("data/mortgages.csv", DataFrame)   # causaldata::mortgages
# Create an "above-cutoff" variable as the instrument
vet.above = Float64.(vet.qob_minus_kw .> 0)
# Impose a bandwidth of 12 quarters on either side
vet = vet[abs.(vet.qob_minus_kw) .< 12, :]
# The running variable qob_minus_kw is exogenous, so it goes with the controls.
# The endogenous regressors are veteran status and its interaction with the
# running variable, instrumented by being above the cutoff and that interaction.
vet.vet_inter   = vet.qob_minus_kw .* vet.vet_wwko
vet.above_inter = vet.qob_minus_kw .* vet.above
m = feiv(vet, @formula(home_ownership ~ nonwhite + qob_minus_kw + fe(bpl) + fe(qob));
         endo = [:vet_wwko, :vet_inter], inst = [:above, :above_inter],
         vcov_type = Vcov.robust())   # heteroskedasticity-robust SEs
```

```
    Demean Variables: [=====>                          ]  1/7    Demean Variables: [==========
```

```
Panelest IV/2SLS Model
Number of obs: 56901
Endogenous:    vet_wwko, vet_inter
Instruments:   above, above_inter


                 Estimate  Std. Error  z value  Pr(>|z|)

vet_wwko       0.170172    0.0459329      3.70    0.0002
vet_inter     -0.00285263  0.00263744    -1.08    0.2794
nonwhite      -0.190434    0.00689193   -27.63    <1e-99
qob_minus_kw  -0.00715077  0.00177347    -4.03    <1e-04


--- Diagnostics ---
First-stage F (vet_wwko): 315.7
First-stage F (vet_inter): 1861.18
Wu-Hausman:   F(2, 56840) = 6.7536  [p = 0.0012]
```

The 2SLS estimate is 0.1702 with a heteroskedasticity-robust standard error of 0.0459: veteran status raises home ownership by 17 percentage points. The sample is 56,901 people within 12 quarters of the cutoff, with birthplace and quarter-of-birth fixed effects. The first-stage F statistics are 316 and 1,861, so the instruments are strong, and the Wu-Hausman test rejects exogeneity of veteran status at $p = 0.001$. The interaction of veteran status with the running variable is not significant, so there is little evidence that the slope differs across the cutoff.

Now the same design estimated locally with `rdrobust`. We pass race and state of birth as covariates, but not quarter of birth. Quarter of birth is a function of the running variable, and the local window holds only three quarters on each side of the cutoff. There the quarter-of-birth dummies duplicate the local linear fits, and the jump is not identified with them in. `rdrobust` still returns

a number, but the number is arbitrary: with the dummies included, R's `rdrobust` gives 0.096 or 0.838, depending only on whether the covariate matrix carries an intercept column. `RDRobust.jl` gives a third value and warns that the estimate is not identified.

```
# race and state of birth; quarter of birth is left out (see text)
states    = sort(unique(vet.bpl))
conmatrix = hcat(Float64.(vet.nonwhite), [Float64.(vet.bpl .== s) for s in states[2:end]]...)
m_rdd = rdrobust(vet.home_ownership,
                 vet.qob_minus_kw,
                 fuzzy = vet.vet_wwko,
                 c = 0.0,
                 covs = conmatrix)
rd_summary(m_rdd)
```

```
Bandwidth h = 2.612, b = 5.371; N = 28776 left, 28125 right; effective N = 6911 left, 6756 rig
```

	Method	Coef	SE	p	CI_low	CI_high
	String	Float64	Float64	Float64	Float64	Float64
1	Conventional	2.3684	4.9681	0.634	-7.369	12.106
2	Bias-Corrected	6.5115	4.9681	0.19	-3.226	16.249
3	Robust	6.5115	5.7613	0.258	-4.78	17.803

`rdrobust` returns a conventional estimate of 2.37 with a standard error of 4.97, and a bias-corrected estimate of 6.51 with a robust standard error of 5.76. The robust 95% interval runs from −4.78 to 17.80. For an effect on a probability, that says nothing.

The reason is the first stage. A fuzzy RD divides the jump in the outcome by the jump in treatment at the cutoff. Here we estimate the second jump directly, with the same covariates and bandwidth.

```
# first stage: the jump in veteran status at the cutoff
fs = rdrobust(Float64.(vet.vet_wwko), vet.qob_minus_kw, c = 0.0,
              covs = conmatrix, h = m_rdd.bws.h_left[1])
display(rd_summary(fs))
# veteran share by quarter near the cutoff
combine(groupby(filter(:qob_minus_kw => q -> abs(q) < 4, vet), :qob_minus_kw, sort = true),
        :vet_wwko => mean => :vet_share)
```

```
Bandwidth h = 2.612, b = 2.612; N = 28776 left, 28125 right; effective N = 6911 left, 6756 rig
```

	Method	Coef	SE	p	CI_low	CI_high
	String	Float64	Float64	Float64	Float64	Float64
1	Conventional	-0.0102	0.0205	0.618	-0.05	0.03
2	Bias-Corrected	-0.0025	0.0205	0.904	-0.043	0.038
3	Robust	-0.0025	0.0332	0.941	-0.067	0.063

	qob_minus_kw	vet_share
	Float64	Float64
1	-3.5	0.561224
2	-2.5	0.558109
3	-1.5	0.524569
4	-0.5	0.480088
5	0.5	0.417344
6	1.5	0.352348
7	2.5	0.2943
8	3.5	0.276429

Within 2.6 quarters of the cutoff, veteran status does not jump: the estimate is −0.010 with a standard error of 0.021. The veteran share falls from 0.56 two and a half quarters before the cutoff to 0.29 two and a half quarters after, but it falls steadily across those quarters, with no break at zero. The fuzzy estimate divides a noisy jump in home ownership by a jump in treatment that is indistinguishable from zero.

2SLS finds a strong first stage because it fits one line on each side over 12 quarters and reads the jump off where the two lines meet. The steep decline near the cutoff then registers as a large jump. Its 0.17 rests on that linear specification. The local estimator drops the assumption, and the data near the cutoff then have nothing to say. The two estimates do not contradict each other. The local one is uninformative here.

17.4. Summary

An instrument recovers an effect when the confounder is unobserved. It must move the treatment (relevance), reach the outcome only through the treatment (exclusion) and be as good as random. With heterogeneous effects and no defiers, the Wald ratio identifies the LATE, the effect on compliers. Adding covariates linearly keeps that interpretation only under the rich-covariates condition, that the instrument's conditional mean is linear in the covariates. A RESET test on the instrument checks it. Regression discontinuity replaces the instrument with a cutoff in a running variable and assumes continuity at the cutoff. Fuzzy RDD is IV at the cutoff.

In the simulation, OLS gives 1.3571, and 2SLS gives 0.9999 against a true 1. With a nonlinear instrument propensity, the true LATE is 0.617 and the linear 2SLS converges to 0.55. The Senate incumbency effect is 7.4 percentage points. In the Korean War example, 2SLS gives 0.1702 (0.0459). The local `rdrobust` estimate, 2.37 (4.97), is uninformative, because veteran status does not jump at the cutoff within the local window. The 2SLS first stage comes from linear fits over 12 quarters.

18. Shift-Share Instrumental Variables

```
using DataFrames
using LinearAlgebra
using Statistics
using Random
using CairoMakie
CairoMakie.activate!(type = "png")
using Panelest
using StatsModels
using CSV

using ShiftShareIV
```

Shift-share, or Bartik, instruments answer one specific problem, so it is clearest to write the regression down first.

We want the effect of a local economic shock on a local outcome. For region ℓ,

$$Y_\ell \;=\; \beta X_\ell + Z_\ell'\gamma + u_\ell, \tag{18.1}$$

where X_ℓ is the shock the region actually experienced — local import competition per worker, in the China-shock literature — and Y_ℓ is the outcome, say the change in the local manufacturing employment share. The parameter we want is β.

The endogenous regressor is X_ℓ. Regions do not receive their shocks at random. A region whose import competition rises sharply may be a region whose industries were already losing ground, and whatever caused that decline sits in u_ℓ and moves Y_ℓ on its own. Then $\mathrm{Cov}(X_\ell, u_\ell) \neq 0$, and OLS mixes the effect of the shock with the local conditions that produced it. We cannot sign that bias in advance; it depends on whether those local conditions push the outcome up or down.

The shift-share instrument replaces the realised local shock with a predicted one, assembled from two pieces that are meant to have nothing to do with u_ℓ: each region's *baseline* industry shares, fixed before the outcome period, and *national* or foreign industry-level shocks, common across regions. A region with a large baseline share in an industry is mechanically more exposed to a shock hitting that industry, and that exposure is predicted rather than realised.

Building the instrument is the easy part. The difficult part is saying which of the two pieces carries the exogeneity assumption — the shares, the shocks, or both — because the two answers lead to different identification arguments, different diagnostics, and different standard errors. That is what the rest of the chapter is about.

18.1. The construction

For each location ℓ and industry k, let $s_{\ell k}$ be the share of local employment in industry k at baseline, and let g_k be a national growth rate (or trade shock) for that industry. The shift-share instrument is

$$B_\ell \;=\; \sum_{k=1}^{K} s_{\ell k}\, g_k. \tag{18.2}$$

This is "shift-share" because it combines a local share with an industry-level shift. Bartik (1991) used local industry mix to predict local employment growth. Autor, Dorn, and Hanson (2013) used local industry shares interacted with industry-level Chinese import growth in other high-income countries.

The first-stage regression is then

$$\Delta L_\ell \;=\; \pi_0 + \pi_1 B_\ell + X_\ell'\gamma + \epsilon_\ell, \tag{18.3}$$

with B_ℓ instrumenting for the observed local shock in a 2SLS for the outcome of interest.

18.2. Two identification views

There are two ways to justify a shift-share instrument. They put the exogeneity assumption on different objects.

18.2.1. Share view (GPSS 2020)

Treat the shares $s_{\ell k}$ as the source of identifying variation, with the shocks g_k acting as weights. The instrument is valid if shares are exogenous to the outcome (conditional on controls). GPSS prove that the shift-share IV is numerically equivalent to a GMM combination of K just-identified IVs, one per industry share:

$$\hat{\beta}^{SS} \;=\; \sum_{k=1}^{K} \hat{\alpha}_k\, \hat{\beta}_k, \tag{18.4}$$

where $\hat{\beta}_k$ is the just-identified IV using share $s_{\cdot k}$ alone and $\hat{\alpha}_k$ is the **Rotemberg weight**. This gives an important diagnostic: which industry shares are driving the estimate?

18.2.2. Shock view (BHJ 2022)

Treat the shocks g_k as the source of identifying variation. Shares are exposure weights. Under this view, shocks should be uncorrelated with the second-stage unobservables, conditional on industry-level controls. BHJ show how to rewrite the regression at the shock level.

The two views are not mutually exclusive. In an application, we should be clear which one is being defended and report diagnostics for both.

18.3. Inference

Standard OLS or cluster-robust standard errors on the regional regression **underestimate uncertainty** because the same industry shocks appear in many regions, inducing cross-region correlation that region-level clustering does not capture. Two corrections are now standard:

- **Cluster on shocks (BHJ)**: equivalent to running the regression at the industry-shock level. Implemented via the `bhj_collapse` function.
- **AKM SE (Adão, Kolesár, Morales 2019)**: explicit formula accounting for shock-level correlation; requires custom implementation or a dedicated package.

18.4. Simulation: build intuition

Use a small DGP with regions, industries, and a known causal effect.

```
# The CSVs below are 500 locations x 20 industries; beta_true is the DGP coefficient.
beta_true  = 0.5

df     = CSV.read("data/shift_share_sim.csv", DataFrame)
shares = Matrix(CSV.read("data/shift_share_shares.csv", DataFrame))
shocks = CSV.read("data/shift_share_shocks.csv", DataFrame).shock
X = df.X; Y = df.Y; u = df.u
first(df[:, [:region, :X, :Y, :u]], 6)
```

	region	X	Y	u
	Int64	Float64	Float64	Float64
1	1	-0.518194	0.11729	-0.308258
2	2	0.212703	-1.06765	-0.538806
3	3	0.652136	0.892118	1.37164
4	4	0.285676	0.861261	1.09974
5	5	-0.334493	-0.329605	-0.111544
6	6	0.37613	-1.72747	-1.75462

A naive OLS suffers from the confounder `u`:

```
ols  = feols(df, @formula(Y ~ X))
ivss = feiv(df, @formula(Y ~ 1), endo = :X, inst = :B)

@printf("%-25s %8s %8s\n", "Estimator", "Coef", "SE")
@printf("%-25s %8.3f %8.3f\n", "OLS (biased)",
        coef(ols)[end], stderror(ols)[end])
@printf("%-25s %8.3f %8.3f\n", "Shift-share IV",
        coef(ivss)[1], stderror(ivss)[1])
@printf("True beta: %.3f\n", beta_true)
```

```
Estimator                      Coef       SE
OLS (biased)                  1.451    0.075
Shift-share IV                0.531    0.131
True beta: 0.500
```

OLS is biased upward by the confounder; the shift-share IV recovers a value close to the true effect.

18.4.1. Rotemberg weights

The GPSS decomposition says the shift-share IV is a weighted average of K just-identified IVs (one per industry share). Compute the weights:

```
rw = rotemberg_weights(shares, shocks, X, Y)

@printf("%-10s %8s %8s %8s\n", "Industry", "Shock", " _k", " _k")
for row in eachrow(rw)
    @printf("%-10d %8.3f %8.3f %8.3f\n",
            row.industry, row.shock, row.alpha, row.beta_k)
end
@printf("\nGPSS identity: Σ  _k  _k = %.4f\n", sum(rw.alpha_beta))
@printf("Shift-share IV estimate: %.4f\n", coef(ivss)[1])
```

```
Industry       Shock       _k       _k
1              1.250    0.054    0.604
2             -0.959    0.048    0.244
3             -0.212    0.005    2.576
4              0.745    0.002  -10.225
5              2.663    0.387    0.469
6             -1.068    0.082    1.025
7              0.100   -0.001   -1.627
8             -0.718    0.025   -0.689
9              1.181    0.058    0.853
10            -0.197    0.004    1.188
11             0.741    0.023   -0.112
```

```
12             0.051   -0.001   -0.055
13            -0.442    0.014    0.611
14             0.270    0.003   -0.013
15             0.631    0.003   -3.240
16             2.271    0.204    0.746
17            -0.212    0.001    0.174
18            -0.115    0.000  -39.532
19             0.020   -0.000    1.234
20            -1.183    0.089    0.554

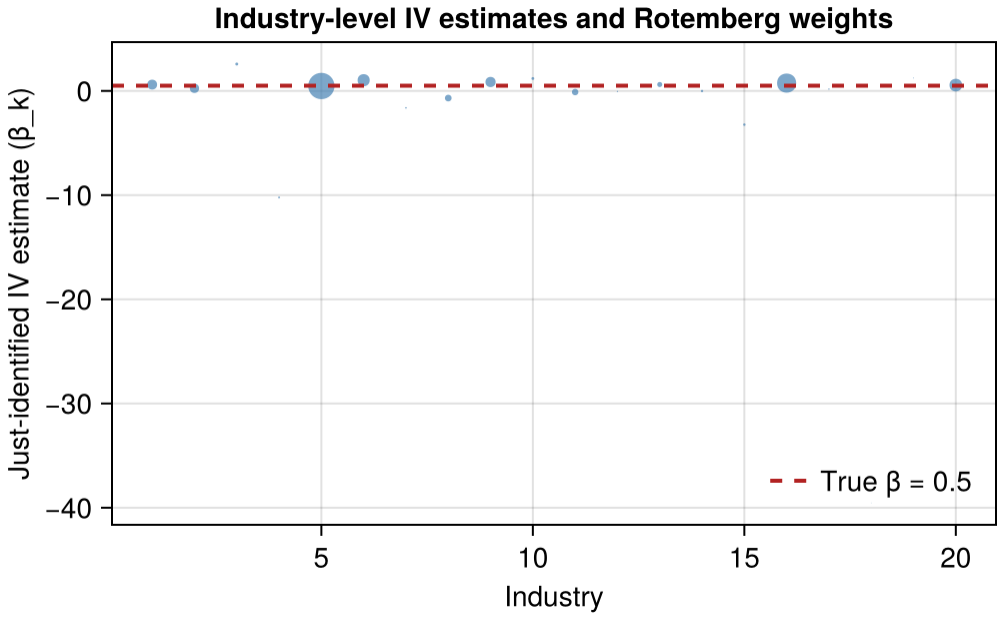
GPSS identity: Σ _k  _k = 0.5309
Shift-share IV estimate: 0.5309
```

Two diagnostics matter:

1. **Weight concentration**: if one or two industries carry most of the weight, the identifying variation is essentially coming from those industries' shares. Their exogeneity is the assumption you are really leaning on.
2. **Heterogeneous** $\hat{\beta}_k$: in this benign simulation, the industry-specific IVs are all near the true effect (with sampling noise). In real data, if a few industries give wildly different estimates, the "average" shift-share IV is averaging over heterogeneous treatment effects you should report explicitly.

```
fig = Figure(size = (640, 380))
ax  = Axis(fig[1, 1],
           xlabel = "Industry",
           ylabel = "Just-identified IV estimate ( _k)",
           title  = "Industry-level IV estimates and Rotemberg weights")

# Bubble plot: position = beta_k, size = |alpha_k|
scatter!(ax, rw.industry, rw.beta_k,
         markersize = 30 .* sqrt.(abs.(rw.alpha)),
         color = (:steelblue, 0.7))
hlines!(ax, [beta_true], color = :firebrick, linestyle = :dash,
        linewidth = 2, label = "True   = $(beta_true)")
axislegend(ax, position = :rb, framevisible = false)
fig
```



18.4.2. What goes wrong when shares are endogenous

Now break the share-exogeneity assumption: make industry 1's share correlated with the second-stage error. Under the share-exogeneity view of GPSS that alone makes the instrument invalid. Under the shock view of BHJ, endogenous shares are allowed, but only because many independent shocks average the share-confounder correlation away. With twenty industries and one realized set of shocks, industry 1's shock does not average out, so the shock argument fails here too, for a different reason.

```
v = CSV.read("data/shift_share_bad_v.csv", DataFrame).v
shares_bad = copy(shares)
shares_bad[:, 1] .= max.(0.01, shares[:, 1] .+ 0.15 .* v)
shares_bad = shares_bad ./ sum(shares_bad, dims=2)   # renormalise

bad_noise = CSV.read("data/shift_share_bad_noise.csv", DataFrame)
B_bad = bartik_iv(shares_bad, shocks)
X_bad = B_bad .+ 0.3 .* u .+ bad_noise.noise_x
Y_bad = beta_true .* X_bad .+ u .+ 0.6 .* v .+ bad_noise.noise_y

df_bad  = DataFrame(X = X_bad, Y = Y_bad, B = B_bad)
iv_bad  = feiv(df_bad, @formula(Y ~ 1), endo = :X, inst = :B)

@printf("True beta: %.3f\n", beta_true)
@printf("Shift-share IV with bad share 1: %.3f\n", coef(iv_bad)[1])

# Would the Rotemberg weights have flagged industry 1?
rw_bad = rotemberg_weights(shares_bad, shocks, X_bad, Y_bad)
```

```
ord    = sortperm(abs.(rw_bad.alpha), rev = true)
rank1  = findfirst(==(1), rw_bad.industry[ord])
@printf("Rotemberg weight of industry 1: %.3f (rank %d of %d by |·|)\n",
        rw_bad.alpha[rw_bad.industry .== 1][1], rank1, nrow(rw_bad))

# Leave-one-industry-out: rebuild the instrument without industry 1
B_loo  = bartik_iv(shares_bad[:, 2:end], shocks[2:end])
df_loo = DataFrame(X = X_bad, Y = Y_bad, B = B_loo)
iv_loo = feiv(df_loo, @formula(Y ~ 1), endo = :X, inst = :B)
@printf("Leave-industry-1-out IV: %.3f\n", coef(iv_loo)[1])
```

```
True beta: 0.500
Shift-share IV with bad share 1: 0.798
Rotemberg weight of industry 1: 0.110 (rank 3 of 20 by |·|)
Leave-industry-1-out IV: 0.480
```

The estimate is biased, and the bias comes from one industry's share being related to the error. Note what the diagnostics do and do not catch here. The endogenous industry's Rotemberg weight is unremarkable — it ranks third of twenty — because the weights measure *influence* (whose exogeneity the estimate leans on most), not *endogeneity*. An industry with a modest weight can still move the estimate substantially when its share is correlated with the error. What points to the problem is the **leave-one-industry-out** check: dropping industry 1 from the instrument restores an estimate close to the truth. The isolation is not clean. Renormalizing the rows put the confounder into every share, so the remaining nineteen shares still carry it; the return toward the truth says that contamination is small in this draw.

18.5. BHJ shock-level inference

Collapse the location-level data to the shock level and run a weighted IV at the shock (industry) level. This reframes the identifying assumption: shocks must be uncorrelated with the shock-level aggregated outcome residual.

```
collapsed = bhj_collapse(shares, shocks, Y, X)

# BHJ shock-level 2SLS: regress Y_agg ~ X_agg | shock, weighted by weight
df_collapsed = collapsed
iv_bhj = feiv(df_collapsed, @formula(Y_agg ~ 1),
              endo = :X_agg, inst = :shock, weights = :weight)

@printf("Location-level shift-share IV: %.3f\n", coef(ivss)[1])
@printf("BHJ shock-level IV:             %.3f\n", coef(iv_bhj)[1])
@printf("(Identical in the no-controls case)\n")
```

```
Location-level shift-share IV: 0.531
```

```
BHJ shock-level IV:             0.531
(Identical in the no-controls case)
```

The BHJ shock-level regression gives exactly the location-level IV estimate when there are no controls. Its value is interpretive: validity is now a statement about shocks (20 observations, one per industry) rather than locations (500 observations), and testing shock-level exogeneity is more transparent.

18.6. Empirical: the China shock (Autor, Dorn, Hanson 2013)

The standard shift-share application is ADH (2013). Commuting zones are exposed differently to Chinese import competition because their baseline industry mixes differ. The instrument is

$$\text{IV}_\ell = \sum_k \frac{L_{\ell k,\,1990}}{L_{\ell,\,1990}} \frac{\Delta M_k^{\text{other}}}{L_{k,\,1990}}, \tag{18.5}$$

where the shocks $\Delta M_k^{\text{other}}$ are growth in Chinese imports into other high-income countries. This leave-one-out construction removes US-specific demand from the shock.

The snippet below is schematic — the full ADH specification additionally needs period effects, the ADH control set, and population weights (a validated replication against the GPSS benchmark lives in the `ShiftShareIV.jl` repository).

```julia
# David Dorn distributes the replication data at ddorn.net
# Files needed:
#   workfile_china.csv  (commuting zone data, 1990-2007)
#   industry_shares.csv (czone × industry shares)

using CSV
cz     = DataFrame(CSV.File("data/workfile_china.csv"))
shares = Matrix(select(cz, r"share_"))   # L × K share matrix
shocks = cz.import_shock_other            # K-vector of "other-countries" shocks

B   = bartik_iv(shares, shocks)               # Bartik instrument
df_adh = hcat(cz, DataFrame(B = B))

# First-stage + second-stage
iv_adh = feiv(df_adh, @formula(d_sh_empl_mfg ~ 1),
              endo = :d_tradeusch_pw, inst = :B,
              vcov_type = Vcov.cluster(:statefip))

# Rotemberg decomposition
rw_adh = rotemberg_weights(shares, shocks, df_adh.d_tradeusch_pw,
                           df_adh.d_sh_empl_mfg)
# Top 5 industries by |alpha|:
sort(rw_adh, :alpha, rev=true)[1:5, :]
```

```
# BHJ shock-level regression
collapsed_adh = bhj_collapse(shares, shocks, df_adh.d_sh_empl_mfg,
                             df_adh.d_tradeusch_pw)
iv_bhj_adh = feiv(collapsed_adh, @formula(Y_agg ~ 1),
                  endo = :X_agg, inst = :shock, weights = :weight)
```

For a new shift-share paper, I would expect:

1. **Rotemberg weights** (via `rotemberg_weights`): identify which industries drive the estimate; flag weight concentration and heterogeneous $\widehat{\beta}_k$.
2. **BHJ shock-level regression** (via `bhj_collapse`): reframe validity as a statement about 20 industry shocks rather than 500 commuting zones.
3. **AKM standard errors** (Adão, Kolesár, Morales 2019): account for shock-level correlation in inference. Requires custom implementation; see the accompanying R chapter for an R-based version.

18.7. Summary

- Shift-share IV combines local shares and industry shocks.
- The GPSS view puts exogeneity on shares; the BHJ view puts it on shocks.
- Rotemberg weights show which industry shares drive the 2SLS estimate.
- Region-level clustering is usually too optimistic. Use shock-level inference and AKM standard errors when possible.
- `ShiftShareIV.jl` provides `bartik_iv`, `rotemberg_weights`, and `bhj_collapse`; `Panelest.jl` provides `feols` and `feiv`.

19. Recentered Formula Instruments

Consider a randomized experiment in two strata of 100 units each. In stratum A each unit is treated with probability 0.8, and in stratum B with probability 0.2. Untreated units have outcome 10 in A and 0 in B, and treatment raises the outcome by 1 for everyone. In a typical draw 80 units in A and 20 in B are treated. The treated then have mean $(80 \times 11 + 20 \times 1)/100 = 9$, and the controls $(20 \times 10 + 80 \times 0)/100 = 2$. The difference in means is 7, although every effect is 1. Assignment is random, but A is treated more often and has higher outcomes anyway, and the comparison mixes the two.

The usual fix is to compare treated and control units within each stratum, or to control for the stratum. There is another way to see it. Let D_i be unit i's treatment and p_i its assignment probability, 0.8 or 0.2. The difference $D_i - p_i$ averages zero within each stratum, so it is uncorrelated with anything fixed within a stratum, including the baseline outcome. In this draw, using $D_i - p_i$ as the instrument for D_i gives exactly 1, and so does controlling for p_i.

Many treatments are computed from a formula that combines a random shock with non-random exposure. Medicaid eligibility is a state's rules applied to a person's income. Import exposure is national industry shocks applied to a region's industry mix. The Bartik instrument of the previous chapter is one such formula. These treatments have the structure of the stratified experiment: units with more exposure expect more treatment in every draw, as stratum A did. Here the stratum is the exposure itself, such as income or a region's industry shares. It is continuous, so no two units share one and we cannot compare within strata. The expected value of the formula reduces it to one number, as a propensity score does, and plays the role of the assignment probability. This chapter covers the general version of Borusyak and Hull (Borusyak and Hull 2023), their result on which formula is best (Borusyak and Hull 2026), and what both mean for shift-share.

19.1. Formula instruments

Unit i has outcome y_i and treatment x_i, with a constant effect β:

$$y_i = \beta x_i + \varepsilon_i. \tag{19.1}$$

The treatment is $x_i = h_i(g, w, u)$, where h_i is a known function, g collects the random shocks, w collects predetermined variables such as income or industry shares, and u collects anything else. The identifying assumption puts all the randomness on the shocks (Borusyak and Hull 2026, Assumption 1):

$$E[\varepsilon_i \mid g, w] = E[\varepsilon_i \mid w]. \tag{19.2}$$

The error may depend on w in any way. Poorer people may have worse outcomes, and regions with more manufacturing may have been declining anyway. We need exogeneity of the shocks, not of the exposure.

A formula instrument is $z_i = f_i(g, w)$ for a fixed function f_i. Its expected value over the shock draws that could have occurred, holding w fixed, is the expected instrument $\mu_i = E[z_i \mid w]$. The average is over draws for the same unit, not over units. In the stratified experiment μ_i is the assignment probability p_i. The recentered instrument $\tilde{z}_i = z_i - \mu_i$ therefore averages zero over the draws for every unit, $E[\tilde{z}_i \mid w] = 0$. It is valid, because by iterated expectations and Equation 19.2,

$$E[\tilde{z}_i \varepsilon_i] = E[\tilde{z}_i \, E[\varepsilon_i \mid g, w]] = E[\tilde{z}_i \, E[\varepsilon_i \mid w]] = E[E[\tilde{z}_i \mid w] \, E[\varepsilon_i \mid w]] = 0.$$

Without recentering the same steps give $E[z_i \varepsilon_i] = E[\mu_i E[\varepsilon_i \mid w]]$, which is not zero when the expected instrument is correlated with the predictable part of the error. We can also keep z_i as the instrument and control for μ_i. In the population the regression of z_i on μ_i has slope one and intercept zero, so its residual is $z_i - \mu_i$ and the two approaches use the same variation. In a sample the fitted slope is not exactly one, and the estimates differ.

The work is in computing μ_i, and it takes three steps in every application below. First, state the shocks and the design: how they could have been drawn. Second, apply the formula to each possible draw, holding the unit's w fixed. Third, average over the draws, weighting each by its probability. When the design has many draws, we simulate them, or permute the shocks if they are exchangeable, and take the mean. When the formula is linear in the shocks, the second and third steps reduce to applying the formula to the expected shocks. When it is not, they do not, and we have to average.

When the instrument is the treatment itself, as eligibility in the simulation and spillover exposure in the deworming example are, μ_i is also the expected treatment given w. If that treatment is binary, as eligibility is, μ_i is a design-based propensity score: the probability of treatment given w, computed from the design. When the instrument differs from the treatment, as Bartik differs from US import exposure in the China-shock application, μ_i is the expected instrument, and it is not the expected treatment. In each case μ_i differs from the propensity score of an observational study in three ways. It is computed from the formula and the design, like the known assignment probability of a stratified experiment, and not estimated from a model of treatment on covariates. It is used by subtracting it, as the expected treatment is in the partially linear model, and not as a weight. And the assumption is about the shocks. We assume nothing about how w affects the outcome, so $z_i - \mu_i$ is uncorrelated with every fixed characteristic, measured or not.

Recentering and instrumenting fix two different problems. Recentering removes the bias from fixed exposure: some units expect a larger treatment in every draw. When the treatment is the formula itself, $x_i = h_i(g, w)$, as eligibility in the simulation and spillover exposure in the deworming example are, this is the only problem, and OLS controlling for μ_i is enough. When the treatment also depends on u, it can be endogenous through u even given μ_i. In ADH, US imports also respond to US demand. We then need an instrument built from the shocks alone, such as Bartik with imports into other countries, and that instrument is itself recentered, or its expected value controlled for. ADH need both fixes, and the simulation and deworming need only the first. The expected value itself is never the instrument. It is the part predictable from exposure, the part that can be correlated with the error.

Any recentered formula is valid, so we can pick the most precise. Borusyak and Hull (2026, Theorem 1) show it starts from the best prediction of the treatment, recentered:

$$\tilde{z} = E[x \mid g, w] - E[x \mid w]. \tag{19.3}$$

Among recentered formulas it has the strongest first stage (Borusyak and Hull 2026, Lemma 1). The full optimal instrument adds two refinements, partly controlling for the predictable part of the error and GLS weighting, which need a model of the error. The authors drop them. Their algorithm predicts the treatment from its formula, recenters the prediction by redrawing the shocks, and uses the result as the instrument. When x_i depends only on (g, w), the prediction is x_i itself.

19.2. Shift-share as a formula instrument

The Bartik instrument $B_\ell = \sum_k s_{\ell k} g_k$ of Equation 18.2 is the linear formula, shares times shocks. It differs from Equation 19.3 in two ways.

The first is recentering. The three steps give the expected instrument. The shocks are the industry shocks g_k, and suppose they are exchangeable within period t: any reshuffling of them across industries is equally likely. Each industry's expected shock is then the period mean $\bar{g}_t$. The formula is linear, so its average is the shares times $\bar{g}_t$. Let $S_\ell = \sum_k s_{\ell k}$ be region ℓ's sum of shares. Then

$$\mu_\ell = S_\ell \, \bar{g}_t. \tag{19.4}$$

If the shares sum to one, $\mu_\ell = \bar{g}_t$ is common to all regions and period effects absorb it, so Bartik is already recentered. If they are incomplete, as with manufacturing shares, heavily manufacturing regions get a large instrument in every draw, and under the shock view Bartik is biased unless we control for S_ℓ interacted with period (Borusyak, Hull, and Jaravel 2022).

The second is the formula. Recentering and choosing a formula are separate decisions. Take two regions with half their employment in electronics, and an import shock to electronics. Region A's plants compete with Chinese imports. Region B's make military components and do not. Bartik gives both half the shock, but only A's import competition rises, so the instrument varies where the treatment does not. The best predictor gives A half the shock and B nothing, then recenters each by its own expected value. Once recentered, both instruments are valid, and the second has the stronger first stage. When the treatment really is shares times shocks, the best predictor is Bartik itself and the two differ only by recentering. The formula matters when the treatment has more structure than Bartik uses: eligibility rules, export exposure, transport networks, nonlinear indices.

Table 19.1.: Shift-share and recentered formula instruments.

	Shift-share (Bartik)	Recentered formula instrument
Formula	Shares times shocks	Best prediction of the treatment
Expected instrument	Usually not subtracted	Subtracted, or controlled for
Shock design	Implicit	Written down to compute μ_ℓ
Inputs	Shares and shocks	Treatment formula and shock design

The formula instrument needs more inputs: the treatment's formula and an explicit shock design. With heterogeneous effects, different instruments weight units differently, so a gain in precision is a gain for a different weighted average (Borusyak and Hull 2026, Proposition D1).

19.2.1. Why recenter

Bartik needs one of two assumptions. If the shares are as good as random given the controls (Goldsmith-Pinkham, Sorkin, and Swift 2020), Bartik is valid as it stands and recentering is not needed. If only the shocks are random (Borusyak, Hull, and Jaravel 2022), the shares may be correlated with the error. Writing each shock as the mean shock plus a deviation splits the instrument in two:

$$B_\ell = S_\ell \, \bar{g}_t + \sum_k s_{\ell k} \, (g_k - \bar{g}_t). \tag{19.5}$$

The first part is the expected instrument of Equation 19.4. It depends only on how much exposure the region has, so with incomplete shares a region with more exposure gets a larger value in every draw, and this part is correlated with the error whenever exposure is. The second part depends on whether the region's industries received larger or smaller shocks than average. If the shocks are independent of the shares and the errors, the deviations average to zero whatever the shares are, so this part is uncorrelated with the error. Recentering keeps only the second part. When the shares sum to one, the first part is $\bar{g}_t$ for every region and the period effects absorb it.

This gives the formula instrument four advantages. Its validity rests only on the shocks, such as a policy choice or a foreign supply shock, which are often easier to defend than the shares. The design has to be written down, so the assumption is explicit and can be varied, as the two ADH designs below show. The instrument can use exposure to gain power, because the best predictor tracks the treatment more closely than a shock-only index. And it is not tied to a linear formula or to independent units: it covers eligibility rules, networks and nonlinear indices, with shocks shared by many units.

19.3. Simulation

The aim is to compare a shock-only instrument with the recentered treatment when we know the true effect. There are 50 states and 10,000 people. Person i lives in state $s(i)$ and has income inc_i, in multiples of the poverty line, drawn from a log-normal distribution with median 1.4. The shock is the states' policy: each state k draws an eligibility threshold T_k from $\{0.5, 1, 1.38, 2\}$, each value with probability 1/4, independently across states. Eligibility and the outcome are

$$x_i = 1[\text{inc}_i < T_{s(i)}], \qquad y_i = 0.3\, x_i + \varepsilon_i, \qquad \varepsilon_i = -0.5 \log \text{inc}_i + \alpha_{s(i)} + \nu_i, \tag{19.6}$$

where $\alpha_k \sim N(0, 0.3^2)$ is a state effect and $\nu_i \sim N(0,1)$. The true effect is 0.3. In the notation of Equation 19.2, the shocks g are the 50 thresholds and the predetermined variables w are the incomes and states. Poorer people are more often eligible and have higher errors, so eligibility is correlated with the error through income and OLS is biased upward. The thresholds are drawn independently of incomes and errors, so Equation 19.2 holds.

Expected eligibility follows the three steps. The shocks are the thresholds, and the design gives each of the four values probability 1/4. The formula is $x_i = 1[\text{inc}_i < T_{s(i)}]$. Applying it to each threshold, holding income fixed, and averaging gives μ_i, the probability that person i is eligible. It is the share of the four thresholds above their income: 1 below 0.5, 0.75 between 0.5 and 1, 0.5 between 1 and 1.38, 0.25 between 1.38 and 2, and 0 above 2. It depends on income only, because every state draws from the same four values. Given income, the recentered treatment $x_i - \mu_i$ moves only with the draw, which is why it is exogenous. A person with income 1.2 has $\mu_i = 0.5$. The instrument is 0.5 if their state drew 1.38 or 2 and -0.5 if it drew 0.5 or 1, with equal chance whatever their error. People below 0.5 or above 2 always have zero and carry no identifying variation. The formula is not linear, so μ_i is not eligibility at the mean threshold, 1.22. That would make the person with income 1.2 eligible for sure, while their μ_i is 0.5.

We compare four estimators, all with standard errors clustered by state.

1. OLS of y_i on x_i. This is the same as instrumenting x_i with itself, the best predictor without recentering. It is biased.
2. The simulated instrument of Currie and Gruber (1996). They apply each state's rules to one fixed national sample and use the share of that sample who would be eligible as the state's instrument. The same population is run through every state's rules, so the instrument measures the generosity of the rules and not the composition of the state's residents. Here the national sample is all 10,000 people, and everyone in a state gets the same value. It is valid.
3. The recentered treatment, instrumenting x_i with $x_i - \mu_i$. It varies within states, among people whose income lies between the lowest and highest thresholds. It is valid.
4. OLS of y_i on x_i and μ_i. Instrumenting x_i with $x_i - \mu_i$ while controlling for μ_i reduces to this regression, because $x_i - \mu_i$ and μ_i add up to x_i, so the instrument reproduces the treatment (Borusyak and Hull 2026, footnote 8). It uses the same variation as 3, but the estimates differ in a finite sample. It is valid.

The Monte Carlo is design-based. We generate the people once: their incomes, states and errors are the same in every replication. In each of 1,000 replications we draw new thresholds for the 50 states, recompute eligibility and outcomes, and fit the four estimators. The only randomness across replications is the policy draw, which is what the identifying assumption treats as random, so the spread of the estimates across replications measures each estimator's precision under that design. The people and the 1,000 × 50 thresholds are stored as CSVs so the Julia companion computes the same numbers.

```
people = CSV.read("data/recenter_people.csv", DataFrame)
draws  = Matrix(CSV.read("data/recenter_policy_draws.csv", DataFrame)[:, 2:end])
pool = [0.5, 1.0, 1.38, 2.0]
beta_true = 0.3
mu    = [mean(v .< pool) for v in people.income]   # expected eligibility
gener = [mean(people.income .< t) for t in pool]    # state generosity

function coef_x(f)
    i = findfirst(==("x"), coefnames(f))
    return coef(f)[i], stderror(f)[i]
end

const CL = Vcov.cluster(:state)
```

```
function one_draw(r)
    thr = draws[r, people.state]
    d = copy(people)
    d.x     = Float64.(d.income .< thr)
    d.y     = beta_true .* d.x .+ d.eps
    d.z_sim = gener[[findfirst(==(t), pool) for t in thr]]
    d.z_rc  = d.x .- mu
    d.mu    = mu
    fits = (feols(d, @formula(y ~ x), vcov = CL),
            feiv(d, @formula(y ~ 1), endo = :x, inst = :z_sim, vcov_type = CL),
            feiv(d, @formula(y ~ 1), endo = :x, inst = :z_rc, vcov_type = CL),
            feols(d, @formula(y ~ x + mu), vcov = CL))
    r2 = [NaN, cor(d.x, d.z_sim)^2, cor(d.x, d.z_rc)^2, NaN]
    return [collect(coef_x(f)) for f in fits], r2
end

R = size(draws, 1)
est = zeros(R, 4); se = zeros(R, 4); r2 = zeros(R, 4)
Threads.@threads for r in 1:R                # one slot per draw, race-free
    cs, r2[r, :] = one_draw(r)
    for j in 1:4; est[r, j], se[r, j] = cs[j]; end
end

DataFrame(estimator = ["OLS", "Simulated instrument", "Recentered treatment",
                       "OLS controlling for mu"],
          mean     = round.(vec(mean(est, dims = 1)), digits = 3),
          sd       = round.(vec(std(est, dims = 1)), digits = 3),
          coverage = round.(vec(mean(abs.(est .- beta_true) .<= 1.96 .* se, dims = 1)),
                            digits = 3),
          first_stage_R2 = round.(vec(mean(r2, dims = 1)), digits = 3))
```

	estimator	mean	sd	coverage	first_stage_R2
	String	Float64	Float64	Float64	Float64
1	OLS	0.637	0.049	0.0	NaN
2	Simulated instrument	0.302	0.167	0.937	0.263
3	Recentered treatment	0.3	0.086	0.927	0.589
4	OLS controlling for mu	0.3	0.08	0.935	NaN

The table gives each estimator's mean and standard deviation over the 1,000 replications, the share of clustered 95% intervals that contain 0.3, and the average first-stage R^2. OLS averages 0.637 against a true 0.3, and its intervals never contain 0.3. The three other estimators are centred on 0.3, with coverage of 92.7% to 93.7%, slightly below 95% with 50 clusters. They differ in precision. The simulated instrument has a standard deviation of 0.167 and a first-stage R^2 of 0.26. It gives everyone in a state the same value, so it cannot separate people whose eligibility depends on the threshold from people who are eligible, or ineligible, under every threshold. The recentered treatment has a standard deviation of 0.086, half as large, and an R^2 of 0.59, because it uses that variation within states. Its controlled form has 0.080.

In real data, Borusyak and Hull (2026) instrument Medicaid eligibility in the 2014 ACA expansion with actual minus expected eligibility, treating expansion as random within the governor's party. The first-stage partial R^2 rises from 0.01 to 0.80 over the simulated instrument and the standard error on crowd-out falls by 44%. Most of the gain comes from the 18% of the sample whose eligibility the expansion could change; outside it, the recentered instrument is zero, and in the full sample without an exposure control it is less precise than the simulated one (Borusyak and Hull 2026, Table A4). Their 2012–2013 placebo on private insurance is −0.011, half the main estimate of −0.023 (Table A2).

19.4. The China shock

In ADH the shocks are at the industry-period level: the growth of Chinese imports into eight other high-income countries, per US worker, in 397 manufacturing industries over 1990–2000 and 2000–2007. ADH argue they are exogenous because the growth came from China's side, through reform, rising productivity and WTO accession, and because imports into other countries do not respond to US demand. The argument is weaker for demand or technology shocks shared by all rich countries, and the identifying variation is concentrated in a few industries such as computers, toys and audio equipment (Goldsmith-Pinkham, Sorkin, and Swift 2020).

All four regressions below use the same instrument and differ only in what they control for. The treatment, US import exposure per worker, is shares times US import growth, and US imports also reflect US demand. Replacing them with the other-country shocks gives the prediction, which is the Bartik instrument itself, so the formula-instrument approach does not change the instrument. It changes the control. The shares, lagged employment shares in the 722 commuting zones, are incomplete, so the expected instrument μ_ℓ varies across zones, and we control for it. The shocks are the 397 industry shocks in each period and the formula is shares times shocks, so μ_ℓ depends only on which reshufflings of the shocks the design treats as equally likely.

- Design 1: the 397 shocks are exchangeable within each period. Every industry then has the same expected shock, the period mean $\bar{g}_t$, which is 4.8 in the first period and 17.3 in the second, and Equation 19.4 gives $\mu_\ell = S_\ell \bar{g}_t$. A zone with 20 percent of its workers in manufacturing has $\mu_\ell = 0.2 \times 4.8 = 0.96$ in the first period and $0.2 \times 17.3 = 3.46$ in the second. Only a zone's manufacturing intensity is treated as predictable.
- Design 2: the shocks are exchangeable only within each of the 20 two-digit SIC sectors in each period. Each industry's expected shock is its sector's mean $\bar{g}_{mt}$, and $\mu_\ell = \sum_m S_{\ell m} \bar{g}_{mt}$, where $S_{\ell m}$ is the zone's share in sector m. A zone's sector mix is also treated as predictable: a zone heavy in electronics is expected to receive a large instrument.

ADH control for the start-of-period manufacturing share with one coefficient for both periods, which approximates design 1's μ_ℓ. The second row drops that control, so nothing recenters the instrument, and it is valid only if the shares are exogenous. All fits use ADH's other controls, census-division effects, population weights and standard errors clustered by state, with the GPSS replication files. The clustered standard errors reproduce ADH; we return to inference under the shock design below.

```
adh = CSV.read("data/adh_master.csv", DataFrame)
tr  = CSV.read("data/adh_shares_triplets.csv", DataFrame)
shk = CSV.read("data/adh_shocks.csv", DataFrame)

function by_row(v)                          # sum over industries, by zone
    out = zeros(nrow(adh))
    for (r, x) in zip(tr.row, v); out[r] += x; end
    return out
end
sector  = string.(shk.sic87dd .÷ 100, " ", shk.year)
g_bar_m = Dict(s => mean(shk.g[sector .== s]) for s in unique(sector))

adh.B    = by_row(tr.share .* shk.g[tr.col])
adh.S    = by_row(tr.share)
adh.S_t2 = adh.S .* adh.t2
adh.t2n  = Float64.(adh.t2)
adh.mu2  = by_row(tr.share .* [g_bar_m[sector[c]] for c in tr.col])

controls = term(:l_sh_popedu_c) + term(:l_sh_popfborn) + term(:l_sh_empl_f) +
           term(:l_sh_routine33) + term(:l_task_outsource) + term(:t2n) +
           fe(:division)
function iv(extra)
    rhs = isempty(extra) ? controls : sum(term.(extra)) + controls
    feiv(adh, term(:d_sh_empl_mfg) ~ rhs, endo = :shock, inst = :B,
         weights = :weights, vcov_type = Vcov.cluster(:statefip))
end
fits = ["Bartik, ADH controls"               => iv([:l_shind_manuf_cbp]),
        "Bartik, no manufacturing control"   => iv(Symbol[]),
        "Bartik, design 1 mu (S x period)"   => iv([:S, :S_t2]),
        "Bartik, design 2 mu (sector means)" => iv([:mu2])]
DataFrame(specification = first.(fits),
          estimate = [round(coef(f)[1], digits = 3) for f in last.(fits)],
          se       = [round(stderror(f)[1], digits = 3) for f in last.(fits)])
```

	specification	estimate	se
	String	Float64	Float64
1	Bartik, ADH controls	-0.596	0.1
2	Bartik, no manufacturing control	-0.712	0.083
3	Bartik, design 1 mu (S x period)	-0.267	0.087
4	Bartik, design 2 mu (sector means)	-0.144	0.117

The first row reproduces ADH's −0.596: an increase in import exposure of $1,000 per worker over a decade lowers the manufacturing employment share by about 0.6 percentage points. Without the manufacturing-share control the estimate is −0.712.

Each of the last two rows removes one source of variation. ADH's estimate partly compares zones with more and less manufacturing across the two periods: because the mean shock more than triples, a manufacturing-heavy zone gets a much larger instrument in the second period whichever

industries China hit. Controlling for design 1's μ_ℓ removes that comparison, and the estimate falls to −0.267 (0.087). ADH's control is close to S_ℓ, with a correlation of 0.92, but one coefficient cannot absorb a term whose slope more than triples between periods. Design 2 also removes the comparison across sectors, such as electronics and toys against food and paper, and the estimate falls to −0.144 (0.117).

Whether the removed comparisons are confounded is the substantive question. If manufacturing-heavy zones declined faster in the 2000s for other reasons, such as automation, the design 1 control is needed and ADH overstate the effect. But the average shock is itself part of the China shock, since Chinese imports rose across nearly all of manufacturing. Designs 1 and 2 treat that average as predictable and use only the variation in which industries China hit harder than average, so they cannot measure an effect that works through manufacturing as a whole. ADH's −0.596 does use it, and it is valid if the shares are exogenous given the controls. The data cannot tell us which assumption holds. Every estimate is negative, and recentering shows how much of the size depends on the assumption.

19.4.1. Inference under the shock design

The standard errors above are clustered by state. They reproduce ADH, but they describe a different source of randomness from the one designs 1 and 2 rely on. Under those designs the random part is which industries received which shocks, and every zone's instrument is built from the same 397 shocks in each period. Inference that follows the design reshuffles the shocks, as the simulation redrew the thresholds, and asks how unusual the observed data are among the reshuffles (Borusyak and Hull 2023).

Let y_ℓ be the change in zone ℓ's manufacturing employment share and x_ℓ its US import exposure. For a hypothesised effect b, form $e_\ell(b) = y_\ell - bx_\ell$. If $b = \beta$, $e_\ell(b)$ is the contribution of the controls plus the error, and under the design it is the same whichever reshuffle of the shocks occurred. Let $\theta(b)$ be the coefficient on the Bartik instrument B_ℓ in a weighted regression of $e_\ell(b)$ on B_ℓ and the controls, including the design's μ_ℓ. We recompute B_ℓ and $\theta(b)$ under 1,999 reshuffles permitted by the design, holding $e_\ell(b)$ fixed. The p-value for b is the share of reshuffles, counting the observed one, whose $|\theta(b)|$ is at least the observed value. The 95% interval collects the values of b whose p-value is above 0.05. At the IV estimate θ is zero, so the interval contains the estimate, and the test does not need a strong first stage. It does need the reshuffles to be equally likely given the errors, which is a little more than Equation 19.2. The reshuffles are stored in the data folder, so both books use the same ones.

```
using LinearAlgebra
perms = [Matrix{Int}(CSV.read("data/adh_perms_design$d.csv", DataFrame)) for d in 1:2]
divs = sort(unique(adh.division))
W0 = hcat(ones(nrow(adh)), adh.l_sh_popedu_c, adh.l_sh_popfborn, adh.l_sh_empl_f,
          adh.l_sh_routine33, adh.l_task_outsource, adh.t2n,
          [Float64.(adh.division .== v) for v in divs[2:end]]...)
W = [hcat(W0, adh.S, adh.S_t2),                # design 1: mu = S x period mean
     hcat(W0, adh.mu2)]                         # design 2: mu = sector means
b_grid = (-1500:1000) ./ 1000                   # hypothesised effects b
sw = sqrt.(adh.weights)
```

```
function ri_test(W, perm)
    Q = Matrix(qr(sw .* W).Q)
    res(v) = (u = sw .* v; u .- Q * (Q' * u))   # weighted residual on controls and mu
    yt = res(adh.d_sh_empl_mfg); xt = res(adh.shock)
    # theta(b) = sum(Bt .* (yt .- b .* xt)) / sum(Bt.^2) is linear in b, so keep three sums
    sums(B) = (bt = res(B); [dot(bt, yt), dot(bt, xt), dot(bt, bt)])
    obs = sums(adh.B)
    P = size(perm, 2)
    sims = zeros(P, 3)
    Threads.@threads for p in 1:P                # one reshuffling per column
        sims[p, :] = sums(by_row(tr.share .* shk.g[perm[tr.col, p]]))
    end
    theta(m, b) = abs.(m[:, 1] .- b .* m[:, 2]) ./ m[:, 3]
    p_val = [(1 + sum(theta(sims, b) .>= theta(obs', b))) / (P + 1) for b in b_grid]
    ok = b_grid[p_val .> 0.05]
    return minimum(ok), maximum(ok), p_val[findfirst(==(0), b_grid)]
end
ri = [ri_test(W[d], perms[d]) for d in 1:2]

est = [coef(last(fits[k]))[1] for k in 3:4]
se_ = [stderror(last(fits[k]))[1] for k in 3:4]
DataFrame(design   = ["Design 1 (S x period)", "Design 2 (sector means)"],
          estimate = round.(est, digits = 3),
          state_lo = round.(est .- 1.96 .* se_, digits = 3),
          state_hi = round.(est .+ 1.96 .* se_, digits = 3),
          ri_lo = [round(r[1], digits = 3) for r in ri],
          ri_hi = [round(r[2], digits = 3) for r in ri],
          ri_p0 = [round(r[3], digits = 3) for r in ri])
```

	design	estimate	state_lo	state_hi	ri_lo	ri_hi	ri_p0
	String	Float64	Float64	Float64	Float64	Float64	Float64
1	Design 1 (S x period)	-0.267	-0.438	-0.096	-0.526	-0.016	0.036
2	Design 2 (sector means)	-0.144	-0.373	0.085	-0.418	0.107	0.226

The intervals that reshuffle the shocks are wider than the clustered ones. Under design 1 the interval runs from −0.526 to −0.016, against −0.438 to −0.096 with state clustering, and the p-value for no effect is 0.036. Under design 2 it runs from −0.418 to 0.107, and the p-value is 0.226. With the design's own inference, design 1 still finds a negative effect at the 5% level, but its interval nearly reaches zero, and design 2 cannot rule out no effect.

19.4.2. A two-zone example

To see what the manufacturing-share control does, take two zones, P and Q, and three manufacturing industries, 1, 2 and 3, in one period. Zone P has 10 percent of its workers in each industry, so its manufacturing share is $S_P = 0.3$. Zone Q has 10 percent in industry 3 only, so $S_Q = 0.1$. Suppose we observe China shocks $g_1 = 30$, $g_2 = 0$ and $g_3 = 0$, in thousands of dollars of imports per worker. Under design 1 any reshuffling of these shocks across the industries is equally likely, so

the shock of 30 could have gone to any of the three industries, each with probability 1/3. To keep the example short, suppose US import exposure equals the Bartik measure B_ℓ, and let the change in the manufacturing employment share, in percentage points, be

$$y_\ell = -0.6B_\ell - 10S_\ell.$$

Each \$1,000 of exposure per worker lowers the share by 0.6 points, and manufacturing-heavy zones decline by a further 10 points per unit of manufacturing share for other reasons, such as automation. P has the larger share, so it declines 2 points more than Q in every draw.

Each row of the table is one draw. A zone's exposure is its shares times the shocks, and its outcome follows from the equation above. The last row averages each zone's exposure over the three draws, which is its expected exposure.

Draw	g_1	g_2	g_3	B_P	B_Q	y_P	y_Q
1	30	0	0	3	0	−4.8	−1.0
2	0	30	0	3	0	−4.8	−1.0
3	0	0	30	3	3	−4.8	−2.8
Average				$\mu_P = 3$	$\mu_Q = 1$		

P's exposure is $0.1 \times 30 = 3$ in every draw, because its three shares are equal. Q's is 3 only when industry 3 is shocked. The averages match Equation 19.4 with the expected shock $\bar{g} = 10$: $\mu_P = 0.3 \times 10 = 3$ and $\mu_Q = 0.1 \times 10 = 1$.

Comparing the two zones within a draw is biased. In draw 1, P has 3 more units of exposure than Q and an outcome 3.8 points lower:

$$\frac{-4.8 - (-1.0)}{3 - 0} = -1.27 = -0.6 - \frac{2}{3}.$$

Of the 3.8 points, 1.8 come from imports and 2 from P's larger manufacturing share. P has more exposure in most draws, so the comparison counts the 2 points as an effect of imports, and the effect looks larger than it is.

Comparing each zone with its own expected exposure is not biased. P's exposure equals its average in every draw, so $B_P - \mu_P$ is always zero and P carries no information. Q's exposure is 0 in two draws and 3 in one, and when it rises by 3, Q's outcome falls from −1.0 to −2.8:

$$\frac{-2.8 - (-1.0)}{3 - 0} = -0.6.$$

Q's manufacturing share is the same in every draw, so it cancels. This is what the instrument $B_\ell - \mu_\ell$ does. It measures how much more or less exposure a zone got than it expected, which averages zero for every zone, so it is unrelated to any fixed characteristic of the zone. Pooling the three draws of both zones, the raw slope of y_ℓ on B_ℓ is −1.1, and using $B_\ell - \mu_\ell$ as the instrument gives exactly −0.6. Since $\mu_\ell = 10S_\ell$, controlling for S_ℓ does the same.

In real data we see one draw but many zones. The instrument then compares zones that got more exposure than they expected with zones that got less. Whether a zone was lucky does not depend on its manufacturing share, so the lucky and unlucky zones are alike in their shares, as treated and control units are in an experiment.

With two periods the mean shock changes, and $\mu_\ell = S_\ell \bar{g}_t$ has a different slope on S_ℓ in each period. With $\bar{g}_t$ equal to 4.8 and 17.3, a single coefficient on S_ℓ cannot match both, and part of the decline gap stays in the estimate. This is the difference between ADH's control and design 1.

19.5. Spillovers: deworming

Miguel and Kremer (2004) study a deworming programme in 75 primary schools in Busia, Kenya. The schools were listed alphabetically within strata and assigned in turn to three groups of 25, which started treatment in different years. Worms spread between children, so treating one school can lower infection at schools nearby. To measure this, they count the pupils at treated schools within 3 km of each school, and they control for the total number of pupils within 3 km.

Let D_j be 1 if school j is treated, n_j its number of pupils, p_j the probability that it is treated, and $j \sim i$ mean that j is another school within 3 km of school i. The spillover exposure of school i and its expected value are

$$x_i = \sum_{j \sim i} n_j D_j, \qquad \mu_i = \sum_{j \sim i} n_j p_j. \tag{19.7}$$

The three steps give μ_i. The shocks g are the assignments, and the predetermined variables w are the locations and pupil counts. The formula is linear in the assignments, so averaging it over the draws replaces each D_j by its probability p_j. If exposure were instead 1 for any treated neighbour, the formula would not be linear, and μ_i would be the share of draws in which school i has at least one treated neighbour, found by redrawing the assignment. Assignment is random, but a school in a dense area has more treated neighbours in every draw. If density also affects the outcome, through crowding or sanitation, the exposure is correlated with the error. In the first year one third of the schools were treated, so $p_j = 1/3$ for every school and μ_i is one third of the total pupils within 3 km. Miguel and Kremer's total-pupils control is therefore the recentering. The exposure is the treatment itself, as eligibility was in the simulation, so the estimator is OLS controlling for μ_i, estimator 4 above. If the treatment probability differed across schools, for example by stratum, the control would have to be μ_i and not the total pupils.

A second simulation checks this in a district like Busia. We place 75 schools in a 30 km by 30 km district, half around a town and half spread over the rest, and give each about 400 pupils. In each draw exactly 25 schools are treated, so $p_j = 1/3$. Let N_i be the total pupils at other schools within 3 km and $\eta_i \sim N(0, 0.05^2)$ a school error. The outcome, school participation, is

$$y_i = 0.05 D_i + 0.03 x_i - 0.02 N_i + \eta_i, \tag{19.8}$$

with x_i and N_i in thousands of pupils. Treatment raises participation by 5 points at the school and by 3 points per thousand treated pupils nearby. Schools in dense areas have lower participation for other reasons. As before, the schools, their pupils and errors are fixed, and the 1,000 replications

redraw which 25 schools are treated. We regress y_i on D_i and on (a) x_i, (b) $x_i - \mu_i$, or (c) x_i and N_i, with heteroskedasticity-robust standard errors. The table reports the mean and standard deviation of each estimate over the replications, the coverage of the robust 95% intervals for 0.03, and the mean robust standard error. Only the assignment changes across replications, so the standard deviation is the estimate's sampling variation under the design, and a robust standard error is accurate when it matches it.

```
schools = CSV.read("data/deworm_schools.csv", DataFrame)
treat   = Matrix(CSV.read("data/deworm_draws.csv", DataFrame)[:, 2:end])
ns = nrow(schools)
near = [i != j && hypot(schools.east[i] - schools.east[j],
                        schools.north[i] - schools.north[j]) < 3
        for i in 1:ns, j in 1:ns]                # within 3 km
N  = near * schools.pupils ./ 1000              # total pupils nearby, thousands
mu = N ./ 3                                     # expected treated pupils nearby

function one_assign(r)
    D = Float64.(treat[r, :])
    x = near * (schools.pupils .* D) ./ 1000
    d = DataFrame(D = D, x = x, N = N, x_rc = x .- mu,
                  y = 0.05 .* D .+ 0.03 .* x .- 0.02 .* N .+ schools.eta)
    fits = (feols(d, @formula(y ~ D + x), vcov = Vcov.robust()),
            feols(d, @formula(y ~ D + x_rc), vcov = Vcov.robust()),
            feols(d, @formula(y ~ D + x + N), vcov = Vcov.robust()))
    return [(coef(f)[3], stderror(f)[3]) for f in fits]
end

R = size(treat, 1)
dw_est = zeros(R, 3); dw_se = zeros(R, 3)
Threads.@threads for r in 1:R                # one slot per draw, race-free
    for (j, (b, s)) in enumerate(one_assign(r))
        dw_est[r, j], dw_se[r, j] = b, s
    end
end

DataFrame(estimator = ["(a) Exposure, no control", "(b) Recentered exposure",
                       "(c) Exposure, control for nearby pupils"],
          mean     = round.(vec(mean(dw_est, dims = 1)), digits = 3),
          sd       = round.(vec(std(dw_est, dims = 1)), digits = 3),
          coverage = round.(vec(mean(abs.(dw_est .- 0.03) .<= 1.96 .* dw_se, dims = 1)),
                            digits = 3),
          mean_se  = round.(vec(mean(dw_se, dims = 1)), digits = 3))
```

	estimator	mean	sd	coverage	mean_se
	String	Float64	Float64	Float64	Float64
1	(a) Exposure, no control	-0.024	0.012	0.0	0.005
2	(b) Recentered exposure	0.03	0.038	0.43	0.016
3	(c) Exposure, control for nearby pupils	0.03	0.02	0.92	0.017

Without the control, the spillover estimate averages −0.024 against a true 0.03, and its intervals never contain 0.03. Dense schools have more treated neighbours and lower participation, and the regression assigns that gap to the spillover. Both recentered versions are centred on 0.03. Regressing on $x_i - \mu_i$ alone leaves $-0.02N_i$ in the error, so its standard deviation is 0.038, about twice the 0.020 of (c). Neighbouring schools share that error and the same treated schools, so the robust standard error of (b) averages 0.016 against a standard deviation of 0.038, and its intervals contain 0.03 only 43% of the time. With the control for nearby pupils the remaining error is independent across schools, the robust standard error averages 0.017 against 0.020, and coverage is 92%. That match is a property of this simulation, where the school errors are independent. The design does not guarantee it, because every school's exposure comes from the same 25 treated schools. In a single data set the inference that follows the design redraws the assignment, as we reshuffled the shocks for ADH. It has to hypothesise both the direct effect and the spillover, because a school's outcome under another assignment depends on both.

19.5.1. A two-school example

Take two untreated schools, A and B, and three other schools, 1, 2 and 3, with 400 pupils each. Exactly one of the three is treated, each with probability 1/3, as in the full design, where exactly 25 of the 75 schools are treated. School A is in town and has all three within 3 km, so $N_A = 1.2$ thousand pupils nearby. School B is rural and has only school 3 nearby, so $N_B = 0.4$. With both schools untreated and η_i set to zero, Equation 19.8 gives

$$y_i = 0.03x_i - 0.02N_i.$$

Density lowers A's participation 1.6 points below B's in every draw, because A has 0.8 thousand more pupils nearby.

Each row of the table is one draw. A school's exposure is the pupils at its treated neighbours, in thousands, and its outcome follows from the equation above. The last row averages each school's exposure over the three draws, which is its expected exposure.

Treated school	x_A	x_B	y_A	y_B
1	0.4	0	−0.012	−0.008
2	0.4	0	−0.012	−0.008
3	0.4	0.4	−0.012	0.004
Average	$\mu_A = 0.4$	$\mu_B = 0.133$		

Each expected exposure is the pupils nearby times the treatment probability 1/3. The town school expects more treated neighbours because it has more neighbours.

Comparing the two schools within a draw is biased. In draw 1, A has 0.4 more exposure than B and participation 0.004 lower:

$$\frac{-0.012 - (-0.008)}{0.4 - 0} = -0.01 = 0.03 - 0.04.$$

The spillover raises A by 0.012 relative to B, and density lowers it by 0.016. A has more exposure in most draws, so the comparison counts the density gap as an effect of exposure. This is omitted-variable bias. Density affects participation, and density is correlated with exposure, because exposure is built from nearby pupils. If either link were absent, there would be no bias. Here the bias is large enough to flip the sign.

Comparing each school with its own expected exposure is not biased. A's exposure equals its average in every draw, so $x_A - \mu_A$ is always zero and A carries no information. B's exposure is 0 in two draws and 0.4 in one, and when it rises, B's participation rises from -0.008 to 0.004:

$$\frac{0.004 - (-0.008)}{0.4 - 0} = 0.03.$$

B's density is the same in every draw, so it cancels. Pooling the three draws of both schools, the raw slope of y_i on x_i is 0, and using $x_i - \mu_i$ as the instrument gives exactly 0.03.

The argument uses nothing specific to density. Any fixed characteristic of a school, such as local wealth or sanitation, cancels in the same way, because $x_i - \mu_i$ comes only from the draw. We do not need to know which fixed variables affect the outcome, or to measure them. We need only μ_i, which comes from the design.

With 75 schools and one draw, the instrument compares schools that got more treated neighbours than they expected with schools that got fewer. Their densities are alike on average but not exactly, which is the extra noise of estimator (b). Controlling for N_i, which here is the same as controlling for μ_i, removes the density term exactly. The comparison is then between schools with the same number of pupils nearby, which is why (c) is tighter.

19.6. Summary

The calculation in each setting is the same: hold exposure fixed, redraw the shocks under an explicit design, and average the formula. The result is the expected instrument μ, which depends on the shocks, the fixed exposure and the design:

Table 19.4.: Expected instrument in each application. In the stratified experiment, the Medicaid simulation and deworming, the instrument is the treatment, so μ is also the expected treatment.

Case	Shocks	Fixed exposure	Design	Expected instrument μ
Stratified experiment	Who is treated	Stratum	Probability 0.8 or 0.2 by stratum	The assignment probability
Medicaid simulation	State thresholds	Income	Each threshold with probability 1/4	Share of thresholds above income

Case	Shocks	Fixed exposure	Design	Expected instrument μ
Bartik, design 1	Industry shocks	Industry shares	Reshuffling within period	$S_\ell \bar{g}_t$
Bartik, design 2	Industry shocks	Industry shares	Reshuffling within sector and period	$\sum_m S_{\ell m} \bar{g}_{mt}$
De-worming	Which schools are treated	Locations and pupils	25 of 75 schools at random	Pupils nearby times 1/3

Shift-share is one formula instrument, with the formula and the design fixed by convention. Any formula of shocks and exposure is valid once its expected value over the shock design is subtracted, and the most efficient formula is the recentered best prediction of the treatment. Applied to Bartik, this has two consequences. When the shares do not sum to one, the regression must control for the sum of shares times the period's mean shock. When the treatment depends on the shocks through more than shares times shocks, a prediction built from the treatment's own formula is a stronger instrument than Bartik. In the ADH data the first consequence alone moves the estimate from -0.60 to -0.27, and inference that reshuffles the shocks gives it a 95% interval of -0.53 to -0.02, wider than the state-clustered one. The same logic covers spillovers. A school's expected exposure to treated neighbours is the treatment probability times its nearby pupils, which is the control Miguel and Kremer used.

For the shift-share construction, the share and shock views, and inference, see the previous chapter, Shift-Share Instrumental Variables.

20. IV in Poisson with Fixed Effects

Poisson regression specifies the conditional mean as an exponential function:

$$E(y \mid \mathbf{x}) = \exp(\mathbf{x}\beta)$$

As a quasi-maximum likelihood estimator (QMLE), Poisson regression requires only that the conditional mean is correctly specified; the true count distribution does not need to follow a Poisson distribution, and standard errors are easily made robust to arbitrary overdispersion or heteroskedasticity.

However, when an explanatory variable is **endogenous**—correlated with unobserved omitted factors—standard linear two-stage least squares (2SLS) intuition fails. Because the conditional mean is nonlinear, unobserved confounders enter multiplicatively inside the exponential function.

This chapter covers:

1. Why linear 2SLS intuition fails in nonlinear exponential models.
2. The control function approach for continuous vs. binary endogenous variables.
3. Terza's exact correction (Terza 1998) and its profile-likelihood implementation.
4. Generalized residuals as a first-order local approximation, logit response residuals, and the Mundlak device for panel fixed effects (Lin and Wooldridge 2019).
5. Why applied econometrics generally recommends the generalized residual approach over the exact Terza correction.
6. Multiplicative-moment GMM (Mullahy 1997) and quasi-differencing for panel fixed effects.
7. Why the exogeneity test should not be used to choose between the naive and control-function models (Guggenberger 2010a).
8. Step-by-step implementation demos in Julia (`Panelest.jl`) and Stata (`ppmlhdfe`).

20.1. The model and setup

Let y_1 denote the count outcome, y_2 the endogenous explanatory variable, and $\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2)$ the vector of exogenous variables ($\mathbf{z}_1$ included in the structural equation, $\mathbf{z}_2$ excluded instruments).

The structural conditional mean is:

$$E(y_1 \mid \mathbf{z}_1, y_2, c_1) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + c_1) \tag{20.1}$$

where c_1 is an unobserved omitted variable correlated with y_2. Our primary estimand of interest is α_1, the structural semi-elasticity of y_1 with respect to y_2.

20.1.1. Subscripts index equations

Following the simultaneous-equations convention of Wooldridge (2010) (ch. 18), **subscripts index equations rather than individual variables**: - **Equation 1 (Structural equation):** outcome y_1, parameters (δ_1, α_1), unobserved error c_1. - **Equation 2 (Reduced-form equation):** outcome y_2, coefficients π_2, reduced-form error v_2.

Role	Linear IV Notation	Nonlinear Notation Here	Description
Outcome (the count)	Y	y_1	Dependent variable in structural equation
Endogenous regressor	W or D	y_2	Explanatory variable correlated with c_1
Excluded instrument	Z	$\mathbf{z}_2$	Exogenous instrument excluded from Eq. 1
Included exogenous	X	$\mathbf{z}_1$	Exogenous covariates included in Eq. 1
Reduced-form error	V	v_2	Error from projecting y_2 on $\mathbf{z}$
Structural unobservable	U	c_1	Confounder correlated with y_2
Endogeneity parameter	ρ	ρ_1	Projection coefficient: $c_1 = \rho_1 v_2 + e_1$

Two departures from Wooldridge's notation are maintained for clarity: 1. Wooldridge writes the coefficient on y_2 as γ_1; we write α_1 to align with standard treatment effect notation. 2. Where Wooldridge writes $\mathbf{x}_1\beta_1$, we write $\mathbf{z}_1\delta_1 + \alpha_1 y_2$.

20.1.2. Fixed effects in the exponential model

Everything below absorbs fixed effects without comment, and it is worth saying once why that is allowed. Fixed-effects Poisson has no incidental parameters problem (Wooldridge 1999; Lin and Wooldridge 2019). With $N \to \infty$ and T fixed, the unit effects enter multiplicatively, factor out of the score, and are conditioned out cleanly. The estimator of β is consistent even though the number of unit parameters grows with the sample. Probit and logit have no such property, which is why the first stage of Section 20.3.4 needs the Mundlak device while the second stage does not.

The software follows from this. In Stata, `xtpoisson, fe` fits the conditional model, but the community standard is `ppmlhdfe` (Correia, Guimarães, and Zylkin 2020): it absorbs several dimensions of fixed effects at once, it is fast, it detects separated and singleton observations, and it recovers predicted values with the absorbed effects put back. In Julia, `Panelest.jl` provides `fepois()` for the same job, with `feols()` for the linear first stage.

Stata's `ivpoisson` handles endogenous regressors, but it cannot absorb high-dimensional fixed effects. That is the practical reason the panel literature routes endogeneity through a control function rather than through a packaged IV command: the control function leaves the second stage as an ordinary FE Poisson regression, which `ppmlhdfe` and `fepois` already do well.

20.2. Control functions and GMM

20.2.1. A linear second stage: the control function *is* 2SLS

To see where the nonlinear complication begins, consider first the linear model:

$$y_1 = \mathbf{z}_1\delta_1 + \alpha_1 y_2 + \varepsilon_1, \qquad E(\mathbf{z}'\varepsilon_1) = \mathbf{0}$$

Two-stage least squares (2SLS) replaces y_2 with its linear projection $\hat{y}_2 = \mathbf{z}\hat{\pi}_2$:

$$y_1 = \mathbf{z}_1\delta_1 + \alpha_1\hat{y}_2 + (\varepsilon_1 + \alpha_1\hat{v}_2)$$

where $\hat{v}_2 = y_2 - \hat{y}_2$ is the OLS residual.

Alternatively, the **control function (CF)** approach decomposes the structural error ε_1 via a linear projection on the first-stage residual v_2:

$$\varepsilon_1 = \rho_1 v_2 + \eta_1$$

where $\rho_1 = \frac{\mathrm{Cov}(\varepsilon_1, v_2)}{\mathrm{Var}(v_2)}$. By the properties of linear projection, $E(v_2\eta_1) = 0$ and $E(\mathbf{z}'\eta_1) = E(\mathbf{z}'\varepsilon_1) - \rho_1 E(\mathbf{z}'v_2) = \mathbf{0}$.

Plugging this into the structural equation yields the augmented regression:

$$y_1 = \mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1\hat{v}_2 + \eta_1$$

By the **Frisch–Waugh–Lovell (FWL) theorem**, OLS estimation of this augmented regression produces an estimate $\hat{\alpha}_1$ that is **numerically identical to 2SLS**. No additional distributional assumptions are required beyond the standard orthogonality condition $E(\mathbf{z}'\varepsilon_1) = \mathbf{0}$.

20.2.2. A nonlinear second stage: the control function needs more than uncorrelatedness

When the conditional mean is exponential, this automatic numerical equivalence breaks down.

Substituting the error decomposition $c_1 = \rho_1 v_2 + e_1$ into Equation 20.1 yields:

$$\begin{aligned} E(y_1 \mid \mathbf{z}, y_2, v_2) &= E\left[\exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 v_2 + e_1) \mid \mathbf{z}, y_2, v_2\right] \\ &= \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 v_2) \cdot E\left[\exp(e_1) \mid \mathbf{z}, y_2, v_2\right] \end{aligned} \tag{20.2}$$

For the augmented index $\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 v_2$ to correctly specify the conditional mean in a Poisson second stage, the remaining expectation factor must be **strictly constant**:

$$E\left[\exp(e_1) \mid \mathbf{z}, y_2, v_2\right] = \text{constant}$$

This is why Wooldridge (2010) (sec. 18.5) normalizes $E[\exp(e_1)] = 1$ rather than $E(e_1) = 0$.

This condition restricts the **conditional moment-generating function** of e_1 at a single point. Every moment of e_1 enters it, so it is far stronger than the zero-covariance condition that suffices in the linear model. It is not, however, an independence condition. Independence of e_1 from $(\mathbf{z}, v_2)$, together with $v_2 \perp \mathbf{z}$, is a **sufficient** assumption that delivers it, and it is the assumption we maintain below because it is simple to state and is what the standard derivation uses. Any weaker restriction that holds $E[\exp(e_1) \mid \mathbf{z}, y_2, v_2]$ constant would serve equally well.

Property	Linear Model (OLS / 2SLS)	Nonlinear Model (Poisson CF)
Second stage	$y_1 = \mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 \hat{v}_2$	$y_1 \sim \text{Poisson}(\exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 \hat{v}_2))$
CF vs. 2SLS/GMM	**Identical** (by FWL theorem)	**Different** (FWL does not hold)
Required assumption	Uncorrelatedness: $E(\mathbf{z}'\varepsilon_1) = \mathbf{0}$	Constant conditional exponential moment; **sufficient:** $v_2 \perp \mathbf{z}$ and $e_1 \perp (\mathbf{z}, v_2)$
Second-stage standard errors	Valid under $H_0 : \rho_1 = 0$	**Invalid** (must bootstrap generated regressor)
Endogeneity test	t-statistic on $\hat{v}_2 = 0$ (DWH test)	t-statistic on $\hat{v}_2 = 0$ (Variable addition test)

20.3. Estimators for endogenous regressors

20.3.1. Continuous y_2: the linear reduced form holds

When y_2 is continuous (e.g., hours worked, prices, expenditures), a linear reduced form with additive errors is natural:

$$y_2 = \mathbf{z}\pi_2 + v_2, \qquad c_1 = \rho_1 v_2 + e_1 \tag{20.3}$$

with $v_2 \perp \mathbf{z}$ and $e_1 \perp (\mathbf{z}, v_2)$. These are two separate restrictions and it is worth keeping them apart. Joint normality of (c_1, v_2) delivers the second: e_1 is the projection residual, and for jointly normal variables uncorrelated implies independent. It says nothing about the first. $v_2 \perp \mathbf{z}$ is a restriction on the reduced form — the conditional distribution of v_2 must not vary with $\mathbf{z}$ — and a linear equation with a homoskedastic normal additive error satisfies it. Both hold here; neither is implied by the other. The structural conditional mean becomes:

$$E(y_1 \mid \mathbf{z}_1, y_2, v_2) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 v_2) \tag{20.4}$$

Estimation Procedure: 1. **Step 1 (First Stage):** Estimate the linear regression $y_2 = \mathbf{z}\pi_2 + v_2$ by OLS (or with fixed effects absorbed) and compute residuals $\hat{v}_2 = y_2 - \mathbf{z}\hat{\pi}_2$. 2. **Step 2 (Second Stage):** Estimate Poisson QMLE with y_2 and $\hat{v}_2$ entering simultaneously:

$$y_1 \sim \text{Poisson}(\exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 \hat{v}_2))$$

3. **Inference:** Standard errors must be bootstrapped (e.g., clustered by panel unit) to account for the first-stage estimation uncertainty of $\hat{v}_2$.

This is Lin and Wooldridge (2019)'s Procedure 3. In a panel, step 1 absorbs the unit effects (`reghdfe` in Stata, `feols` in Julia) and step 2 absorbs them again (`ppmlhdfe`, `fepois`).

20.3.1.1. The first stage may absorb or average: both give the same answer

There are two ways to handle unit effects in a linear first stage, and it does not matter which we pick. Regressing y_{it2} on $\mathbf{z}_{it}$ and unit dummies gives the fixed-effects residual $\hat{v}_{it2}^{FE}$. Regressing y_{it2} on $\mathbf{z}_{it}$ and the unit means $\bar{\mathbf{z}}_i$ — the Mundlak device — gives the Mundlak residual $\hat{v}_{it2}^{M}$. Substituting either one into the second-stage FE Poisson gives **numerically identical** $\hat{\alpha}_1$ and $\hat{\rho}_1$ (Lin and Wooldridge 2019, app. A.1–A.2).

The reason is short. Mundlak's result is that the coefficient on $\mathbf{z}_{it}$ in the second regression equals the within estimator $\hat{\pi}_2^{FE}$, so the two residuals share a slope and differ only in what is subtracted off unit by unit:

$$\hat{v}_{it2}^{FE} = \hat{v}_{it2}^{M} + g_i$$

for some unit-specific constant g_i. Put that into the second-stage index:

$$\mathbf{z}_{1,it}\delta_1 + \alpha_1 y_{it2} + \rho_1 \hat{v}_{it2}^{FE} + c_i = \mathbf{z}_{1,it}\delta_1 + \alpha_1 y_{it2} + \rho_1 \hat{v}_{it2}^{M} + (c_i + \rho_1 g_i)$$

The swap moves $\rho_1 g_i$ into the unit effect, and the unit effect is a free parameter that FE Poisson absorbs without restriction. The model is reparameterized, not changed. Nothing reaches $\hat{\alpha}_1$ or $\hat{\rho}_1$.

This holds only when the first stage averages or absorbs on the same dimension the second stage absorbs. It is also specific to the linear first stage: for a binary y_2 the first stage is nonlinear, the residual is not shifted by a constant, and the choice between dummies and Mundlak means becomes a real one (Section 20.3.4).

20.3.2. Binary y_2: why the linear reduced form fails

When $y_2 \in \{0, 1\}$ is binary, the linear reduced-form residual $v_2 = y_2 - \mathbf{z}\pi_2$ takes exactly two values conditional on $\mathbf{z}$:

$$v_2 \mid \mathbf{z} = \begin{cases} 1 - \mathbf{z}\pi_2 & \text{with probability } p(\mathbf{z}) \\ -\mathbf{z}\pi_2 & \text{with probability } 1 - p(\mathbf{z}) \end{cases} \tag{20.5}$$

The support $\{-\mathbf{z}\pi_2, 1 - \mathbf{z}\pi_2\}$, the probabilities, and the conditional variance $\text{Var}(v_2 \mid \mathbf{z}) = p(\mathbf{z})[1 - p(\mathbf{z})]$ are all **direct functions of z**.

As Wooldridge (2010, 746) emphasizes: > "The assumption that e_1 in (18.39) is independent of v_2 and **z rules out most cases where y_2 has some discreteness, such as with binary, count, or corner responses.** (In particular, the independence assumption likewise fails unless v_2 is independent of **z**, and it would be very unusual for a discrete variable to be expressible as a linear equation with additive error independent of **z**.)"

A distribution whose support moves with **z** cannot be independent of **z** (unless $\pi_2 = \mathbf{0}$, meaning there is no first stage). Therefore, $v_2 \perp \mathbf{z}$ fails mathematically for any discrete endogenous variable.

20.3.2.1. Where the independence assumption belongs

The issue is not that the observed residual is discrete. The generalized residual of Section 20.3.4 is also two-valued given **z**.

The critical distinction is **which object independence is asserted about**: - The linear CF asserts independence of the **observed discrete residual** v_2, which is impossible. - Terza's threshold model asserts independence of the **continuous latent probit error** $v_2 \mid \mathbf{z} \sim N(0,1)$, which is the standard assumption of probit models.

20.3.2.2. What survives: the exogeneity test

The linear-residual construction remains **fully valid as a Hausman-type exogeneity test** (Wooldridge 2010, 744). Under the null hypothesis of exogeneity ($H_0 : \rho_1 = 0$), c_1 does not enter the model, so none of the joint independence assumptions are needed. A standard t-test on $\widehat{v}_2 = 0$ in the Poisson second stage provides an asymptotically valid test for endogeneity.

20.3.3. Terza's (1998) exact correction

To resolve the discreteness problem, Terza (1998) and Wooldridge (2010) (sec. 18.5) replace the linear probability model with a **latent threshold model**:

$$y_2 = \mathbf{1}[\mathbf{z}\pi_2 + v_2 \geq 0], \qquad v_2 \mid \mathbf{z} \sim N(0,1) \tag{20.6}$$

Let (c_1, v_2) be jointly normal, mean zero, and independent of **z**, with $c_1 = \rho_1 v_2 + e_1$, where $e_1 \mid \mathbf{z}, v_2 \sim N(0, \tau_1^2 - \rho_1^2)$.

Conditioning on the latent error v_2 yields:

$$E(y_1 \mid \mathbf{z}, v_2) = \exp((\tau_1^2 - \rho_1^2)/2 + \mathbf{z}_1\delta_1 + \alpha_1 y_2)\exp(\rho_1 v_2)$$

Averaging over the truncated normal distribution of v_2 conditional on the observed binary outcome $y_2 \in \{0,1\}$ gives the exact conditional mean:

$$E(y_1 \mid \mathbf{z}, y_2) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2) \cdot h(y_2, \mathbf{z}\pi_2, \rho_1) \tag{20.7}$$

where:

$$h(y_2, \mathbf{z}\pi_2, \rho_1) = \exp(\rho_1^2/2)\left[y_2 \frac{\Phi(\rho_1 + \mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)} + (1 - y_2)\frac{1 - \Phi(\rho_1 + \mathbf{z}\pi_2)}{1 - \Phi(\mathbf{z}\pi_2)}\right] \tag{20.8}$$

Here $\Phi(\cdot)$ denotes the standard normal CDF, $\rho_1 = \mathrm{Cov}(c_1, v_2)$ is a projection coefficient on the scale of the probit index, and $\exp(\rho_1^2/2)$ is the lognormal correction factor.

20.3.3.1. Estimation by a one-dimensional profile likelihood with an offset

Notice that for any fixed value of ρ_1, $\log h(y_2, \mathbf{z}\hat{\pi}_2, \rho_1)$ is a known data column. The conditional mean can be written as:

$$E(y_1 \mid \mathbf{z}, y_2, \alpha_i) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \alpha_i + \log h(\rho_1)) \tag{20.9}$$

This is a standard Poisson regression with an **offset**.

Because $h(\cdot)$ enters multiplicatively, individual fixed effects factor out in closed form:

$$\exp(\hat{\alpha}_i) = \frac{\sum_t y_{1,it}}{\sum_t \exp(\mathbf{z}_{1,it}\delta_1 + \alpha_1 y_{2,it})h_{it}(\rho_1)}$$

Fixed effects are absorbed without an incidental parameters problem. The full model is estimated by profiling out (δ_1, α_1) and optimizing over the single scalar $\rho_1 \in [-2, 2]$ using 1D optimization.

20.3.3.2. Terza's estimators and FIML

Terza (1998) proposes three estimators: 1. **Full-Information Maximum Likelihood (FIML):** Evaluates the joint likelihood of y_1 and y_2 by integrating out v_2 via Gauss–Hermite quadrature. 2. **Two-Stage Method of Moments / QMLE:** The two-step estimator above (probit followed by QMLE on Equation 20.7). 3. **Nonlinear Weighted Least Squares:** For the fully parametric specification.

In panel data, FIML has two major drawbacks: - It requires the full Poisson density inside the quadrature integral, losing QMLE robustness to overdispersion. - It cannot absorb fixed effects; entering unit dummies destroys likelihood separability, leading to severe incidental parameters bias and convergence failures.

By contrast, the two-step profile QMLE concentrates out fixed effects cleanly.

20.3.4. The generalized-residual CF and the Mundlak device

A first-order Taylor expansion of $h(y_2, \mathbf{z}\pi_2, \rho_1)$ around $\rho_1 = 0$ yields:

$$\frac{\partial}{\partial \rho_1} \frac{\Phi(\rho_1 + \mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)}\bigg|_{\rho_1=0} = \frac{\phi(\mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)} \equiv \lambda(\mathbf{z}\pi_2) \tag{20.10}$$

$$\frac{\partial}{\partial \rho_1} \frac{1 - \Phi(\rho_1 + \mathbf{z}\pi_2)}{1 - \Phi(\mathbf{z}\pi_2)}\bigg|_{\rho_1=0} = -\frac{\phi(\mathbf{z}\pi_2)}{1 - \Phi(\mathbf{z}\pi_2)} = -\lambda(-\mathbf{z}\pi_2)$$

Since $h(y_2, \mathbf{z}\pi_2, 0) = 1$ and $\frac{d}{d\rho_1} \exp(\rho_1^2/2)\big|_{\rho_1=0} = 0$, we have:

$$h(y_2, \mathbf{z}\pi_2, \rho_1) \approx 1 + \rho_1 \hat{r}_2 \tag{20.11}$$

where $\hat{r}_2$ is the **generalized residual** (Gourieroux et al. 1987):

$$\hat{r}_2 = y_2 \frac{\phi(\mathbf{z}\hat{\pi}_2)}{\Phi(\mathbf{z}\hat{\pi}_2)} - (1 - y_2) \frac{\phi(\mathbf{z}\hat{\pi}_2)}{1 - \Phi(\mathbf{z}\hat{\pi}_2)}$$

Using $1 + u \approx \exp(u)$ for small $u = \rho_1 \hat{r}_2$, the conditional mean becomes:

$$E(y_1 \mid \mathbf{z}, y_2) \approx \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 \hat{r}_2)$$

The generalized-residual CF is a **first-order local approximation to Terza's exact correction with error** $O(\rho_1^2)$.

20.3.4.1. Probit and logit generalized residuals

For a **logit** first stage ($\Lambda(\eta) = \frac{1}{1+e^{-\eta}}$), the generalized residual collapses algebraically:

$$\hat{r}_2 = y_2 \frac{\Lambda'(\eta)}{\Lambda(\eta)} - (1 - y_2) \frac{\Lambda'(\eta)}{1 - \Lambda(\eta)} = y_2(1 - \hat{p}_2) - (1 - y_2)\hat{p}_2 = y_2 - \hat{p}_2 \tag{20.12}$$

The logit generalized residual is simply the **raw response residual** $(y_2 - \hat{p}_2)$.

20.3.4.2. Panel fixed effects: the Mundlak device

In short panels (T small, $N \to \infty$), estimating individual unit dummies in a nonlinear first stage (probit or logit) suffers from the **incidental parameters problem**, producing heavily biased first-stage estimates and distorted generalized residuals.

As Lin and Wooldridge (2019) show, the standard solution is the **Mundlak device** (Correlated Random Effects): 1. Add the unit-specific time averages of all time-varying exogenous covariates and instruments:

$$\mathbf{w}_{it} = (1, \mathbf{z}_{it}, \bar{\mathbf{z}}_i)$$

2. Estimate a pooled probit or logit model on $\mathbf{w}_{it}$ and compute the generalized residuals $\hat{r}_{2,it}$. 3. Include $\hat{r}_{2,it}$ in a fixed-effects Poisson second stage. The unit dummies are absorbed there, and FE Poisson is consistent with T fixed, so the incidental parameters problem does not reappear.

20.3.5. Multiplicative-moment GMM

Instead of a control function, Mullahy (1997) proposes multiplicative moment conditions:

$$E\left[\mathbf{z}'\left(y_1 \exp(-\mathbf{z}_1\delta_1 - \alpha_1 y_2) - 1\right)\right] = \mathbf{0} \tag{20.13}$$

Under $E[\exp(c_1) \mid \mathbf{z}] = 1$, this condition holds without requiring the first-stage model for y_2 to be correctly specified.

20.3.5.1. Panel fixed effects by quasi-differencing

In panel data with unobserved individual heterogeneity η_i, $E[y_{it} \mid \mathbf{z}_i, \eta_i] = \eta_i \exp(\mathbf{x}_{it}\beta)$. Individual effects cannot be removed by additive demeaning; they must be eliminated via **multiplicative quasi-differencing** (Chamberlain 1992; Wooldridge 1999; Windmeijer 2000):

$$E\left[\mathbf{z}_{it}'\left(y_{it}\frac{\exp(\mathbf{x}_{is}\beta)}{\exp(\mathbf{x}_{it}\beta)} - y_{is}\right)\right] = \mathbf{0} \tag{20.14}$$

This requires repeated time periods and strict exogeneity of instruments across periods.

20.4. Why recommend generalized residuals over exact Terza

Given that Terza's correction (Terza 1998) is theoretically exact under bivariate normality, why do applied econometricians and textbooks (Wooldridge 2010, 2014; Lin and Wooldridge 2019) frequently recommend the **Generalized-Residual Control Function**?

There are five key reasons:

20.4.1. A built-in Hausman exogeneity test in one step

In the generalized residual model:

$$\log E(y_1 \mid \mathbf{z}, y_2) = \mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho_1 \hat{r}_2$$

The coefficient on $\hat{r}_2$ estimates ρ_1, and its t-statistic is a **direct, asymptotically valid variable-addition test for exogeneity** ($H_0 : \rho_1 = 0$). - Under H_0, none of the joint normality or reduced-form assumptions are required (Wooldridge 2010, 744). - Terza's exact method does not yield a simple linear t-statistic directly beside $\hat{\alpha}_1$.

20.4.2. Standard software

- **Generalized Residual:** Fits directly in standard software without custom numerical optimization.
- **Terza's Exact 2-Step:** Requires writing a custom 1D profile likelihood wrapper to search over ρ_1 with an offset.

20.4.3. Distributional flexibility beyond the probit link

- **Terza's $h(\cdot)$ is strictly probit-specific:** The formula in Equation 20.8 is derived exclusively from truncated moments of a bivariate normal distribution, and no closed-form analogue for logit appears in Terza (1998), Wooldridge (2010) or Imbens and Wooldridge (2007).
- **Generalized residuals apply to any GLM link:** For logit, the generalized residual collapses to the simple **raw response residual** $\hat{r}_2 = y_2 - \hat{p}_2$, requiring zero Mills-ratio machinery.

20.4.4. Robustness to non-normality

- Terza's correction is **exact only under strict joint bivariate normality**.
- If the true unobservables (c_1, v_2) are non-normal (e.g. skewed or heavy-tailed), $h(\cdot)$ has the wrong functional form. The profile then picks the ρ_1 that best fits a curve the data do not follow, and that misspecification passes straight into $\hat{\alpha}_1$. This is misspecification bias, not overfitting — there is only one extra parameter, so there is nothing to overfit with.
- The generalized residual commits to less. It controls for $E[v_2 \mid y_2, \mathbf{z}]$ and stops at first order, so a wrong h cannot drag it. Whether this actually wins under non-normality is an open question; Lin and Wooldridge (2019) leave "investigations into the quality of CF approximations in discrete cases to future research."

20.4.5. The approximation error shrinks with the endogeneity

The error is $O(\rho_1^2)$, so it vanishes quadratically as $\rho_1 \to 0$, and the second stage reports a coefficient on $\hat{r}_2$ that estimates ρ_1 to that same first order. The error is also asymmetric, but the asymmetry is **specific to the treatment state** and runs in opposite directions for the two states. For a treated observation h scales the mean by $\Phi(\mathbf{z}\pi_2 + \rho_1)/\Phi(\mathbf{z}\pi_2)$, bounded by $1/\Phi(\mathbf{z}\pi_2)$, which grows without bound in the lower tail of the index when $\rho_1 > 0$. For an untreated observation the factor is $[1 - \Phi(\mathbf{z}\pi_2 + \rho_1)]/[1 - \Phi(\mathbf{z}\pi_2)]$, which grows without bound in the upper tail when $\rho_1 < 0$. The two cases are mirror images, so neither sign of ρ_1 is the safe one. What governs the error is $|\rho_1|$ together with the treatment state and how far the probit index reaches into the relevant tail. It is design-specific, not a property of the sign of ρ_1.

20.4.6. Practical strategy

Situation	Recommended Estimator	Rationale
Routine empirical analysis & testing	**Generalized-Residual CF**	Fast, standard 1-line command, built-in t-test for $\rho_1 = 0$, works with Logit ($y_2 - \hat{p}_2$), handles panel FE via Mundlak.
$\lvert\hat{\rho}_1\rvert$ **small**	**Generalized-Residual CF alone**	The $O(\rho_1^2)$ error is smallest exactly here.
$\lvert\hat{\rho}_1\rvert$ **large, either sign, or an index reaching far into either tail**	**Add the Terza Exact Profiled 2-Step**	This is where the Taylor remainder can bite, and it can do so for either sign. Fit the exact model alongside and report both; if they agree the approximation was safe, and if they do not, the exact one is right provided the probit is.

20.5. The four estimators compared

Estimator	First Stage	Second Stage Adjustment	Consistency (Binary y_2)	Panel Fixed Effects	Standard Errors
Linear CF	Linear OLS	Add $\hat{v}_2$ inside $\exp(\cdot)$	Inconsistent	Yes (Poisson with FE)	Cluster Bootstrap
Generalized CF	Probit / Logit (with Mundlak)	Add $\hat{r}_2$ inside $\exp(\cdot)$	Approx. ($O(\rho_1^2)$ error)	Yes (Mundlak in 1st stage)	Cluster Bootstrap
Terza Exact 2-Step	Probit	Profile ρ_1 with offset $\log h(\rho_1)$	**Yes** (under normality)	Yes (Poisson with offset)	Cluster Bootstrap
Multiplicative GMM	Any ($\hat{p}_2$)	Instrument y_2 in moments	**Yes** (moment-based; needs valid multiplicative moments)	Needs quasi-differencing	Sandwich vcov

20.6. Practical guidance: an observable diagnostic workflow

In empirical practice, we do not need to guess the confounding parameter ρ_1—**the second-stage Poisson table reports a coefficient on the generalized residual $\hat{r}_2$ that estimates it.** Read

that number as the control-function coefficient. It is a first-order variable-addition coefficient, and it coincides with the ρ_1 inside Terza's exact correction only to that order and only with a probit first stage. With a logit first stage the residual is on a different scale, so the two are not directly comparable without an explicit mapping.

A standard applied workflow proceeds in four steps:

1. **Estimate the Generalized-Residual CF as the baseline model:**
   - For continuous y_2: Add the OLS residual $\hat{v}_2$.
   - For binary y_2: Add the generalized residual $\hat{r}_2$ (or raw response residual $y_2 - \hat{p}_2$ if using logit).
   - In short panels (T small): Use the Mundlak device ($\mathbf{w}_{it} = (1, \mathbf{z}_{it}, \bar{\mathbf{z}}_i)$) in the first stage to avoid incidental parameters bias.
2. **Check the t-statistic on the residual (Exogeneity Test):**
   The t-statistic on $\hat{r}_2$ (testing $\hat{\rho}_1 = 0$) is an asymptotically valid Hausman-type exogeneity test. Read it as a description of how much endogeneity the data can detect. Do not read it as a licence to drop the residual — see the pretest warning below.
3. **Read the magnitude of $\hat{\rho}_1$ (Approximation Diagnostic):** judge magnitude from the coefficient and its confidence interval, not from the t-statistic.
   - **If $|\hat{\rho}_1|$ is small:** the $O(\rho_1^2)$ error is at its smallest and the Generalized-Residual CF stands on its own.
   - **If $|\hat{\rho}_1|$ is large, whatever its sign:** fit Terza's exact 2-step alongside it and report both. Which tail is dangerous depends on the treatment state. Positive ρ_1 inflates treated observations whose index sits in the lower tail; negative ρ_1 inflates untreated observations whose index sits in the upper tail. Do not read a threshold off $\hat{\rho}_1$ alone — the comparison of the two fits is the diagnostic, not a rule of thumb.
4. **Inference:** the second-stage standard errors ignore first-stage estimation error, so they need a correction whenever $\rho_1 \neq 0$. In a panel, resampling whole units is the natural implementation, provided there are enough units for the bootstrap to behave; with few clusters it needs a small-cluster correction of its own. In a cross-section, resample observations or use an analytic two-step correction. Under $H_0 : \rho_1 = 0$ the uncorrected t-statistic on $\hat{r}_2$ is already valid, which is why the exogeneity test in step 2 needs no bootstrap.

20.6.1. Do not use the exogeneity test to choose the model

The tempting next move is to let step 2 pick the specification: if the t-statistic on $\hat{r}_2$ is insignificant, drop the residual and report the plain FE Poisson; if it is significant, keep the control function. This is a pretest estimator. The objection is not that its point estimates are uniformly worse than those of either model it chooses between. It is that the inference reported afterwards is no longer valid at its nominal level.

Guggenberger (2010a) works this out for the linear case. A Hausman pretest followed by OLS-or-2SLS, with the second-stage t-test reported at its nominal level, has **asymptotic size equal to one**: there are points in the parameter space where the procedure rejects a true null with probability approaching 1. The mechanism is a sequence of local alternatives. Let the endogeneity drift toward

zero at rate $n^{-1/2}$. The pretest then has low power and often fails to reject, so the procedure falls back on the estimator that ignores endogeneity — but at that drift rate the bias of that estimator is the same order as its standard error, so the reported confidence interval is centred in the wrong place by about its own width. Taking the supremum of rejection probability over the parameter space picks out exactly these points. Guggenberger (2010b) obtains the same conclusion for the panel Hausman test between fixed and random effects.

These results are proved for linear models, and we are not aware of an analogous theorem for the Poisson control function. The mechanism does not depend on linearity: it needs only a pretest with low power against local endogeneity and a fallback estimator whose bias does not vanish faster than its standard error, and both are present here. We treat it as a warning that carries over rather than a result that has been established in this setting.

The practical rule is to **report the naive FE Poisson and the control-function FE Poisson side by side** and let the reader see both. The t-statistic on $\hat{r}_2$ still belongs in the table as a test of $\rho_1 = 0$. For the magnitude that step 3 needs, read $\hat{\rho}_1$ itself and its confidence interval. A t-statistic combines the coefficient with its precision and measures neither on its own. What it must not do is silently delete one of the two columns.

20.7. Implementation in Julia and Stata

Below are concise, copy-pasteable implementation templates in Julia (`Panelest.jl`) and Stata (`ppmlhdfe`).

20.7.1. Julia (`Panelest.jl`)

```
# 1. Simulate a short panel dataset (50 units, 5 periods)
Random.seed!(123)
n_i, n_t = 50, 5; n = n_i * n_t
id   = repeat(1:n_i, inner = n_t)
time = repeat(1:n_t, outer = n_i)
z    = randn(n)                  # Excluded instrument
x    = randn(n)                  # Exogenous covariate
c_i  = randn(n_i)[id]
e2   = randn(n)

# Endogenous binary treatment and Poisson outcome
y2     = Int.(0.8 .* z .+ 0.5 .* x .+ c_i .+ e2 .>= 0)
lambda = exp.(0.5 .+ 0.8 .* y2 .+ 0.4 .* x .+ c_i .+ 0.2 .* e2)
y1     = rand.(Poisson.(lambda))
df     = DataFrame(y1 = y1, y2 = y2, z = z, x = x, id = id, time = time)

# 2. Mundlak device: unit means
```

```
transform!(groupby(df, :id), :z => mean => :mean_z, :x => mean => :mean_x)

# 3. Step 1: pooled probit first stage and generalized residual
fs  = feprobit(df, @formula(y2 ~ z + x + mean_z + mean_x))
eta = fs.eta
nd  = Normal()
df.r2 = @. ifelse(df.y2 == 1, pdf(nd, eta) / cdf(nd, eta), -pdf(nd, eta) / (1 - cdf(nd, eta)))

# (With a logit first stage, r2 is simply y2 - fitted probability.)

# 4. Step 2: second-stage Poisson with fixed effects
m_naive = fepois(df, @formula(y1 ~ y2 + x + fe(id) + fe(time)))
m_cf    = fepois(df, @formula(y1 ~ y2 + r2 + x + fe(id) + fe(time)))

show_regression_html(m_naive; title = "Naive FE Poisson")
show_regression_html(m_cf; title = "Generalized residual CF")

b(m, v) = coef(m)[findfirst(==(v), coefnames(m))]
s(m, v) = stderror(m)[findfirst(==(v), coefnames(m))]
@printf "true alpha1 = %.2f, true rho1 = %.2f\n" 0.8 0.2
@printf "naive          alpha1 = %.4f (SE %.4f)\n" b(m_naive, "y2") s(m_naive, "y2")
@printf "generalized CF alpha1 = %.4f (SE %.4f), rho1 = %.4f\n" b(m_cf, "y2") s(m_cf, "y2") b(
```

	Estimate	Std. Error	t value	Pr(>|t|)
y2	1.249	0.106	11.748	< 2e-16 ***
x	0.418	0.039	10.630	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

	Estimate	Std. Error	t value	Pr(>|t|)
y2	0.896	0.204	4.400	1.082e-05 ***
r2	0.249	0.123	2.023	0.043 *
x	0.438	0.040	10.811	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
true alpha1 = 0.80, true rho1 = 0.20
naive          alpha1 = 1.2487 (SE 0.1063)
generalized CF alpha1 = 0.8962 (SE 0.2037), rho1 = 0.2492
```

With 50 units over 5 periods the standard error is large enough that this single draw cannot separate the two estimators; it is a template for the syntax, not evidence about the estimator. The naive coefficient is reported beside it so the direction of the correction is visible.

20.7.2. Stata (`ppmlhdfe`)

```
* ============================================================================
* Generalized-Residual Control Function in Stata using ppmlhdfe
* ============================================================================

* 1. Mundlak Device: Calculate unit-level means of time-varying variables
bysort id: egen mean_z = mean(z)
bysort id: egen mean_x = mean(x)

* 2. Step 1: Probit First Stage on pooled data with Mundlak means
probit y2 z x mean_z mean_x
predict double eta, xb
gen double r2 = cond(y2 == 1, normalden(eta)/normal(eta), -normalden(eta)/(1 - normal(eta)))

* (Alternatively, for a Logit first stage: raw response residual)
* logit y2 z x mean_z mean_x
* predict double phat, pr
* gen double r2 = y2 - phat

* 3. Step 2: Fixed-Effects Poisson Second Stage
ppmlhdfe y1 y2 r2 x, absorb(id time)

* 4. Inference: Panel Cluster Bootstrap Program
capture program drop boot_poisson_cf
program define boot_poisson_cf, rclass
    * Mundlak means must be rebuilt on the RESAMPLED clusters (new_id), not
    * carried in from the original data, or every replication reuses the
    * means of the original panel and the bootstrap understates the variance.
    quietly bysort new_id: egen double mz_b = mean(z)
    quietly bysort new_id: egen double mx_b = mean(x)

    * First stage
    quietly probit y2 z x mz_b mx_b
    quietly predict double eta_b, xb
    quietly gen double r2_b = cond(y2 == 1, normalden(eta_b)/normal(eta_b), -normalden(eta_b)/

    * Second stage
    quietly ppmlhdfe y1 y2 r2_b x, absorb(new_id time)
    return scalar alpha1 = _b[y2]
    return scalar rho1   = _b[r2_b]

    drop eta_b r2_b mz_b mx_b
end

bootstrap r(alpha1) r(rho1), reps(500) cluster(id) idcluster(new_id): boot_poisson_cf
```

20.8. Summary

In an exponential model the unobserved confounder enters multiplicatively, so the linear 2SLS logic does not carry over. A control function still works. The first-stage residual goes inside the exponential, and fixed-effects Poisson absorbs unit effects without an incidental-parameters problem. The price is a stronger assumption: the confounder's exponential moment must be constant given the instruments and the first-stage residual, which is more than zero correlation.

For a continuous endogenous regressor, the OLS residual is the control function. For a binary one, Terza's correction is exact under joint normality. The generalized residual is its first-order approximation, and for a logit first stage it is just the response residual. In short panels the Mundlak device handles the first stage. Multiplicative-moment GMM is an alternative that needs no first-stage model. Standard errors need a cluster bootstrap. We report the naive and the control-function estimates side by side, because choosing between them by an exogeneity pretest distorts inference.

Part IV.

Longitudinal Causal Inference

21. G-Methods for Time-Varying Treatments

```
using DataFrames
using Distributions
using Random
using Statistics
using LinearAlgebra
using GLM
using Printf
```

Most chapters so far use one treatment decision. Many real problems are longitudinal. Treatment changes over time, covariates respond to past treatment, and those covariates then affect future treatment. Standard regression adjustment can fail in this setting.

> **Related reading**: The g-estimation with time-varying covariates chapter of *Topics on Econometrics and Causal Inference* derives the identification result and applies the parametric g-formula to a worked binary-treatment example. The LMTP framework for continuous and modified-policy interventions is covered in Longitudinal modified treatment policy (LMTP) and in the Continuous Treatments chapter.

The main tools are:

1. **G-formula**: simulate outcomes under a treatment regime.
2. **IPTW**: reweight observations by the probability of their treatment history.
3. **Marginal structural models**: fit a treatment-history regression in the weighted population.

For longitudinal TMLE, use R's `ltmle` package; there is no Julia equivalent yet. The g-formula and IPTW/MSM translate directly to Julia with `GLM.jl`.

21.1. The time-varying confounding problem

Consider a two-period example:

- L_0: baseline covariate (e.g. health status at study entry)
- A_0: treatment at time 0 (e.g. medication)
- L_1: covariate at time 1, **influenced by** A_0 (e.g. health status after first dose)
- A_1: treatment at time 1, depends on L_1
- Y: outcome at the end of follow-up

L_1 is the problem variable. It confounds the effect of A_1 on Y, so we would like to adjust for it. But it is also affected by A_0, so adjusting for it blocks part of the effect of A_0. Regression without L_1 is confounded; regression with L_1 over-controls.

21.1.1. Simulating the failure of standard regression

```
Random.seed!(1)
n = 5000

L0 = randn(n)
A0 = Float64.(rand(n) .< @. 1 / (1 + exp(-(-0.5 + 0.8 * L0))))
L1 = @. 0.5 * L0 + 1.0 * A0 + randn()
A1 = Float64.(rand(n) .< @. 1 / (1 + exp(-(-0.5 + 0.8 * L1))))
Y  = @. 0.5 * A0 + 0.5 * A1 + 0.5 * L1 + 0.5 * L0 + randn()

df = DataFrame(L0 = L0, A0 = A0, L1 = L1, A1 = A1, Y = Y)

# A0 has a direct effect (0.5) AND an indirect effect through L1
# (A0 -> L1 -> Y contributes 1.0 * 0.5 = 0.5), so A0's total effect is 1.0;
# A1's total effect is 0.5; always-vs-never total = 1.5.
@printf("True total effect of always-treated vs never-treated: 1.500\n\n")

naive_no_L1   = lm(@formula(Y ~ A0 + A1 + L0),       df)
naive_with_L1 = lm(@formula(Y ~ A0 + A1 + L0 + L1), df)

@printf("Naive (no L1):   A0=%.3f, A1=%.3f, total=%.3f\n",
        coef(naive_no_L1)[2], coef(naive_no_L1)[3],
        coef(naive_no_L1)[2] + coef(naive_no_L1)[3])
@printf("Naive (with L1): A0=%.3f, A1=%.3f, total=%.3f\n",
        coef(naive_with_L1)[2], coef(naive_with_L1)[3],
        coef(naive_with_L1)[2] + coef(naive_with_L1)[3])
```

```
True total effect of always-treated vs never-treated: 1.500

Naive (no L1):   A0=0.944, A1=0.850, total=1.794
Naive (with L1): A0=0.474, A1=0.478, total=0.951
```

Neither regression recovers the true total effect:

- Excluding L_1 leaves $L_1 \to A_1$ confounding that biases A_1.
- Including L_1 blocks the $A_0 \to L_1 \to Y$ pathway, so the coefficient on A_0 now measures only the *direct* effect, not the total effect.

This is exactly the setting g-methods were designed for.

21.2. Parametric g-formula

The g-formula identifies the mean outcome under a fixed regime $\bar{a}$ under three assumptions. Consistency: a unit that follows $\bar{a}$ has outcome $Y(\bar{a})$. Sequential exchangeability: at each time t,

$Y(\bar{a}) \perp A_t \mid \bar{A}_{t-1} = \bar{a}_{t-1}, \bar{L}_t$. Positivity: every history that occurs under the regime has a positive probability of receiving the next treatment the regime assigns. In the simulation the first two hold by construction, since L_0 and L_1 are the only causes of treatment. The g-formula then writes the mean outcome under the regime as:

$$\mathbb{E}[Y(\bar{a})] = \sum_{\bar{l}} \mathbb{E}[Y \mid \bar{a}, \bar{l}] \prod_{t=0}^{K} f(l_t \mid \bar{a}_{t-1}, \bar{l}_{t-1}), \tag{21.1}$$

where $\bar{a} = (a_0, a_1, \ldots, a_K)$ is a treatment regime and $\bar{l}$ is the covariate history. In practice we fit models for the time-varying covariates and the outcome, then simulate the data under the treatment regime.

```
# Step 1: Fit nuisance models
mod_L1 = lm(@formula(L1 ~ L0 + A0), df)
mod_Y  = lm(@formula(Y  ~ L0 + A0 + L1 + A1), df)

# Step 2: Simulate counterfactual outcomes under a regime (a0, a1).
# The L1 distribution under A0 = a0 differs from the observed L1.
function g_compute(a0::Real, a1::Real; n_mc::Int = 100)
    sigma_L1 = sqrt(sum(abs2.(residuals(mod_L1))) / dof_residual(mod_L1))
    Y_sim = zeros(n)
    for _ in 1:n_mc
        df_a0 = DataFrame(L0 = L0, A0 = fill(a0, n))
        L1_sim = predict(mod_L1, df_a0) .+ sigma_L1 .* randn(n)
        df_sim = DataFrame(L0 = L0, A0 = fill(a0, n),
                           L1 = L1_sim, A1 = fill(a1, n))
        Y_sim .+= predict(mod_Y, df_sim)
    end
    Y_sim ./= n_mc
    return mean(Y_sim)
end

Random.seed!(99)
E_Y11 = g_compute(1.0, 1.0)
E_Y00 = g_compute(0.0, 0.0)
@printf("G-formula estimate (always vs never): %.3f  (true = 1.500)\n",
        E_Y11 - E_Y00)
```

```
G-formula estimate (always vs never): 1.483  (true = 1.500)
```

The important step is simulating L_1 under the counterfactual treatment. We do not condition on the observed L_1, because observed L_1 is partly a consequence of observed treatment.

The g-formula can handle other regimes too. For example, treat only at $t = 0$:

```
Random.seed!(99)
E_Y10 = g_compute(1.0, 0.0)
@printf("Effect of treating only at t=0: %.3f  (true = 1.0)\n",
        E_Y10 - E_Y00)
```

```
Effect of treating only at t=0: 1.005  (true = 1.0)
```

21.3. IPTW for time-varying treatments

IPTW reweights observations by the inverse probability of their observed treatment history. The stabilized weight is:

$$SW_i = \prod_{t=0}^{K} \frac{f(A_t \mid \bar{A}_{t-1})}{f(A_t \mid \bar{A}_{t-1}, \bar{L}_t)}. \tag{21.2}$$

The numerator uses treatment history only. The denominator also conditions on the covariate history up to and including L_t — the covariates measured just before A_t. (Skipping L_t would leave the $L_t \to A_t$ confounding intact in the pseudo-population.)

```
# Numerator: P(A0) and P(A1 | A0)
n0 = glm(@formula(A0 ~ 1),  df, Binomial(), LogitLink())
n1 = glm(@formula(A1 ~ A0), df, Binomial(), LogitLink())

# Denominator: P(A0 | L0) and P(A1 | A0, L0, L1)
d0 = glm(@formula(A0 ~ L0),           df, Binomial(), LogitLink())
d1 = glm(@formula(A1 ~ A0 + L0 + L1), df, Binomial(), LogitLink())

p_n0 = predict(n0); p_n1 = predict(n1)
p_d0 = predict(d0); p_d1 = predict(d1)

# Probability of the OBSERVED treatment at each time
w0_num = ifelse.(df.A0 .== 1, p_n0, 1 .- p_n0)
w0_den = ifelse.(df.A0 .== 1, p_d0, 1 .- p_d0)
w1_num = ifelse.(df.A1 .== 1, p_n1, 1 .- p_n1)
w1_den = ifelse.(df.A1 .== 1, p_d1, 1 .- p_d1)

sw = (w0_num .* w1_num) ./ (w0_den .* w1_den)
@printf("Weight summary: min=%.3f, mean=%.3f, max=%.3f\n",
        minimum(sw), mean(sw), maximum(sw))

# Trim extreme weights at 99th percentile. Semicolon: without it this is the
# cell's last expression and Jupyter dumps all 5000 weights into the page.
sw_trim = min.(sw, quantile(sw, 0.99));
@printf("After trimming at the 99th percentile: max=%.3f\n", maximum(sw_trim))
```

```
Weight summary: min=0.284, mean=1.000, max=13.489
After trimming at the 99th percentile: max=3.370
```

21.4. Marginal structural models

With stabilized weights, the marginal structural model is a weighted regression of Y on treatment history:

```
df.sw = sw_trim
msm_fit = lm(@formula(Y ~ A0 + A1), df, wts = df.sw)

# display(), not println() -- see the note in the heterogeneous-effects chapter:
# println() on a CoefTable prints the raw constructor, not the table.
display(coeftable(msm_fit))
@printf("\nMSM total effect (always vs never): %.3f  (true = 1.500)\n",
        coef(msm_fit)[2] + coef(msm_fit)[3])

# For comparison: the untrimmed weights
df.sw_u = sw
msm_untrimmed = lm(@formula(Y ~ A0 + A1), df, wts = df.sw_u)
@printf("Untrimmed MSM total effect:         %.3f\n",
        coef(msm_untrimmed)[2] + coef(msm_untrimmed)[3])
```

	Coef.	Std. Error	t	Pr(>	t	)
(Intercept)	-0.0498963	0.0288233	-1.73	0.0835	-0.106403	0.00661038
A0	1.04559	0.0399352	26.18	<1e-99	0.967303	1.12388
A1	0.547731	0.0391661	13.98	<1e-42	0.470948	0.624515

```
MSM total effect (always vs never): 1.593  (true = 1.500)
Untrimmed MSM total effect:         1.486
```

The weighted population is constructed so treatment is no longer associated with past covariates. That is why the MSM targets the total effect. Note the trimmed estimate sits a little above the truth while the untrimmed one is closer: the weight models here are correctly specified, so the untrimmed weights are consistent, and capping the top 1% of weights reintroduces a bit of confounding bias in exchange for lower variance. Trimming is a bias-variance trade, not a free lunch.

21.5. Hernán-style g-estimation: when to use which

Estimator	Strengths	Weaknesses
G-formula	Handles arbitrary regimes; transparent	Sensitive to model misspecification on L_t and Y
IPTW + MSM	Simple, intuitive; valid with correct propensity model	Variance large with extreme weights

In practice, reporting both is useful. Agreement is reassuring; disagreement suggests model misspecification in at least one of them.

For longitudinal TMLE, use R's `ltmle` package. A Julia port would be useful but does not exist yet.

21.6. A robustness check: g-formula with covariates only

Even without IPTW + MSM, the g-formula is a useful check on standard regression. If standard regression and the g-formula disagree, that gap is a warning about time-varying confounding.

```
Random.seed!(123)
g_total = E_Y11 - E_Y00
msm_total = coef(msm_fit)[2] + coef(msm_fit)[3]

@printf("%-30s %.3f\n", "True total effect:", 1.5)
@printf("%-30s %.3f  (wrong)\n", "Standard regression (no L1):",
        coef(naive_no_L1)[2] + coef(naive_no_L1)[3])
@printf("%-30s %.3f  (wrong)\n", "Standard regression (with L1):",
        coef(naive_with_L1)[2] + coef(naive_with_L1)[3])
@printf("%-30s %.3f  (correct)\n", "G-formula:", g_total)
@printf("%-30s %.3f  (correct)\n", "MSM (IPTW):", msm_total)
```

```
True total effect:             1.500
Standard regression (no L1):   1.794  (wrong)
Standard regression (with L1): 0.951  (wrong)
G-formula:                     1.483  (correct)
MSM (IPTW):                    1.593  (correct)
```

The standard regressions are wrong in opposite directions; the g-methods both recover the truth. When standard and g-formula estimates diverge, the divergence is the bias.

21.7. When to reach for these methods

Use g-methods when:

1. There is a time-varying confounder L_t that is affected by past treatment A_{t-1} and itself affects subsequent treatment A_t.

2. The research question concerns a *sustained* or *dynamic* treatment regime, not a single decision at a single time.

This includes many clinical follow-up studies and longitudinal studies of education or labor-market policies.

For a one-shot treatment, the methods from the Estimation and Nonparametric Causal Methods chapters are sufficient. The added complexity of g-methods is only justified when the time-varying confounder problem is present.

21.8. Summary

- Standard regression fails when a time-varying confounder is itself affected by past treatment.
- G-formula simulates outcomes under treatment regimes.
- IPTW + MSM reweights the population and then fits a weighted treatment history model.
- Longitudinal TMLE is best handled in R's `ltmle` package for now.
- Disagreement across estimators is useful information, not just a nuisance.

Part V.

Survival & Time-to-Event

22. Survival Analysis with Causal Inference

```
using DataFrames
using Distributions
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
using CSV
```

When the outcome is time-to-event, such as death, readmission, job exit, or machine failure, ordinary regression is not enough. Some observations are censored. We know the event had not happened by the end of follow-up, but we do not know the event time.

This Julia chapter implements Kaplan-Meier curves, IPW-adjusted survival curves, and RMST differences. For Cox models and doubly robust survival estimators, R's `survival` and `riskRegression` packages are still the more practical tools; see the R companion chapter.

22.1. The censoring problem

We observe $\min(T_i, C_i)$ and $\delta_i = \mathbb{1}\{T_i \leq C_i\}$, where T_i is the event time and C_i is the censoring time. The main quantity of interest is the survival function

$$S(t) = P(T > t) \tag{22.1}$$

the probability of surviving past time t. Causal contrasts are usually reported as:

- **Hazard ratio (HR)** — easy to estimate but hard to interpret causally (Hernán 2010).
- **Restricted mean survival difference (RMST)** — $\int_0^\tau [S_1(t) - S_0(t)]dt$ — interpretable as "years gained" up to time τ.
- **Survival probability difference at t** — direct probability statement, requires picking a follow-up time.

22.2. Simulated survival data

```
df = CSV.read("data/survival_sim.csv", DataFrame)
n  = nrow(df)
@printf("n = %d, events = %d (%.1f%%), censored = %d\n",
        n, Int(sum(df.event)), 100 * mean(df.event), Int(sum(1 .- df.event)))
```

```
n = 1500, events = 1010 (67.3%), censored = 490
```

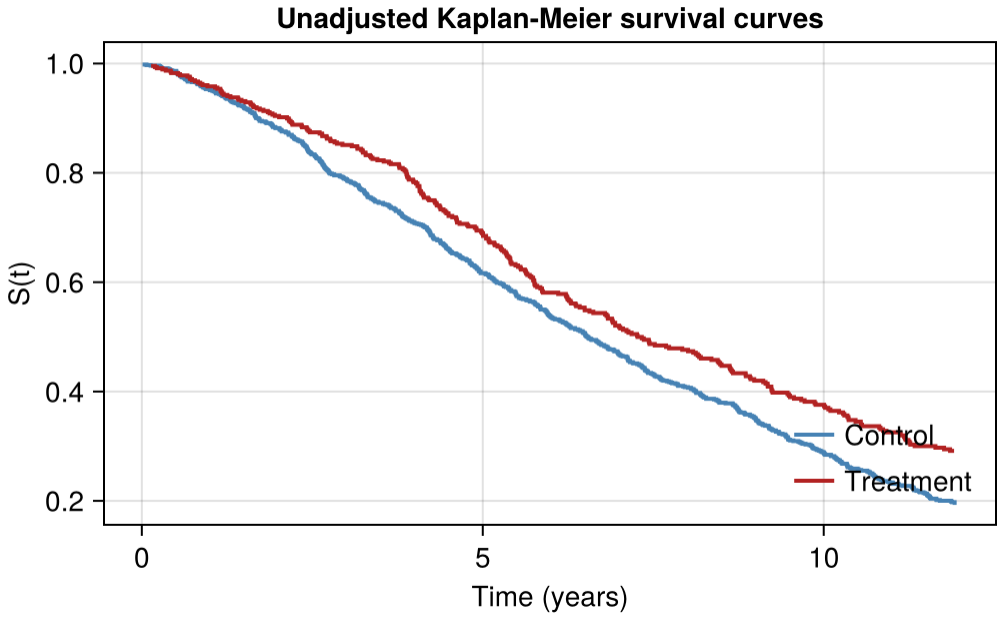
22.3. Kaplan-Meier estimator

The Kaplan-Meier estimator computes $\hat{S}(t)$ nonparametrically. It is short enough to implement directly:

```
"""
    kaplan_meier(time, event)
Compute Kaplan-Meier survival estimates at the unique event times.
Returns (times, survival).
"""
function kaplan_meier(time, event)
    df_sort = sort(DataFrame(t = time, d = event), :t)
    event_times = unique(df_sort.t[df_sort.d .== 1])
    survival = Float64[]
    s = 1.0
    for t in event_times
        n_at_risk = sum(df_sort.t .>= t)
        n_events  = sum((df_sort.t .== t) .& (df_sort.d .== 1))
        s *= 1 - n_events / n_at_risk
        push!(survival, s)
    end
    return event_times, survival
end

t_trt, s_trt = kaplan_meier(df.time[df.trt .== 1], df.event[df.trt .== 1])
t_ctl, s_ctl = kaplan_meier(df.time[df.trt .== 0], df.event[df.trt .== 0])

fig = Figure(size = (700, 400))
ax = Axis(fig[1, 1], xlabel = "Time (years)", ylabel = "S(t)",
          title = "Unadjusted Kaplan-Meier survival curves")
stairs!(ax, t_ctl, s_ctl, color = :steelblue, linewidth = 2,
        label = "Control")
stairs!(ax, t_trt, s_trt, color = :firebrick, linewidth = 2,
        label = "Treatment")
axislegend(ax, position = :rb, framevisible = false)
fig
```



The unadjusted KM curves are descriptive. They include both the treatment effect and baseline imbalance.

22.4. IPW-adjusted survival

To adjust for confounding, weight observations by the inverse propensity score before computing KM. These weights adjust for treatment assignment only. They do nothing about censoring, so the weighted KM curve is valid only if censoring is independent of the event time within each treatment arm. That holds here by construction: the simulated censoring is random dropout plus an administrative end of follow-up at 12 years. If censoring depended on prognosis, we would also need inverse-probability-of-censoring weights. The weights are capped at their 99th percentile, a choice that trades a little bias for less variance and that should be checked for sensitivity.

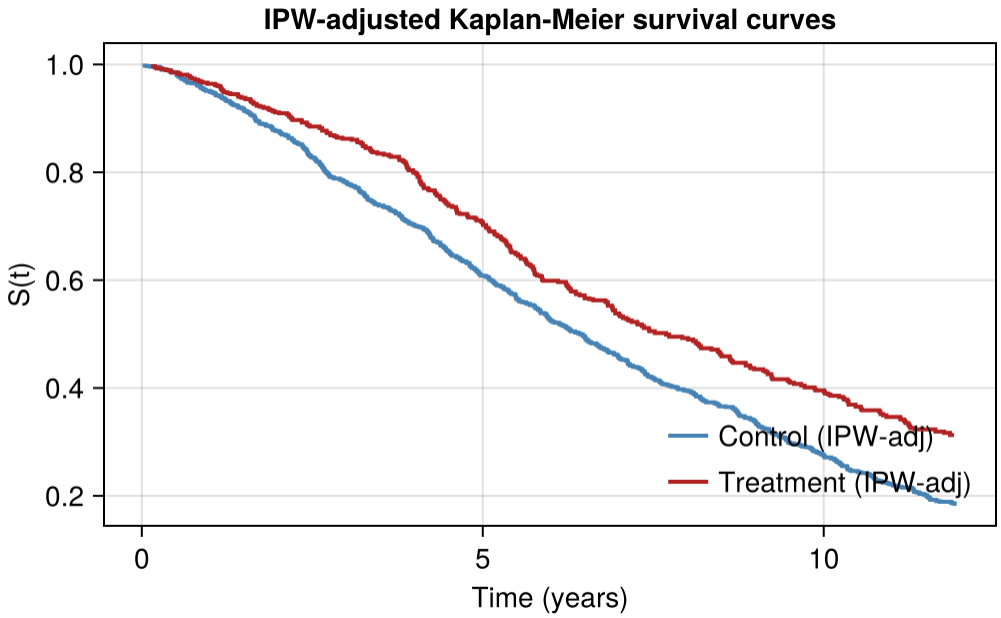
```
ps_fit = glm(@formula(trt ~ age + sex), df, Binomial(), LogitLink())
df.ps  = predict(ps_fit)
df.ipw = ifelse.(df.trt .== 1, 1 ./ df.ps, 1 ./ (1 .- df.ps))
df.ipw .= min.(df.ipw, quantile(df.ipw, 0.99))

"""
    weighted_kaplan_meier(time, event, weights)
KM estimate with sample weights - used for IPW-adjusted survival.
"""
function weighted_kaplan_meier(time, event, weights)
    df_sort = sort(DataFrame(t = time, d = event, w = weights), :t)
    event_times = unique(df_sort.t[df_sort.d .== 1])
    survival = Float64[]
    s = 1.0
```

```
    for t in event_times
        w_at_risk = sum(df_sort.w[df_sort.t .>= t])
        w_events  = sum(df_sort.w[(df_sort.t .== t) .& (df_sort.d .== 1)])
        s *= 1 - w_events / w_at_risk
        push!(survival, s)
    end
    return event_times, survival
end

t_trt_w, s_trt_w = weighted_kaplan_meier(df.time[df.trt .== 1],
                                         df.event[df.trt .== 1],
                                         df.ipw[df.trt .== 1])
t_ctl_w, s_ctl_w = weighted_kaplan_meier(df.time[df.trt .== 0],
                                         df.event[df.trt .== 0],
                                         df.ipw[df.trt .== 0])

fig2 = Figure(size = (700, 400))
ax2 = Axis(fig2[1, 1], xlabel = "Time (years)", ylabel = "S(t)",
           title = "IPW-adjusted Kaplan-Meier survival curves")
stairs!(ax2, t_ctl_w, s_ctl_w, color = :steelblue, linewidth = 2,
        label = "Control (IPW-adj)")
stairs!(ax2, t_trt_w, s_trt_w, color = :firebrick, linewidth = 2,
        label = "Treatment (IPW-adj)")
axislegend(ax2, position = :rb, framevisible = false)
fig2
```



The IPW-adjusted curves remove the age and sex imbalance that is in the propensity score model.

22.5. Restricted Mean Survival Time

The **RMST** up to time τ is

$$\text{RMST}(\tau) = E[\min(T, \tau)] = \int_0^\tau S(t)\, dt. \tag{22.2}$$

The contrast $\Delta_{\text{RMST}}(\tau) = \text{RMST}_1(\tau) - \text{RMST}_0(\tau)$ is interpretable as years gained up to τ. It does not require proportional hazards.

Because the KM curve is a step function, its integral is computed exactly by a left rectangle sum (width times the survival value at the left endpoint of each segment):

```
function rmst(times, survival, tau)
    # Add time 0 and the truncation point, with appropriate S values
    t_vec = [0.0; times; tau]
    s_vec = [1.0; survival; survival[end]]
    # Restrict to times  tau
    keep = t_vec .<= tau
    t_use = t_vec[keep]
    s_use = s_vec[keep]
    # Add the tau endpoint, unless it's already the last element (t_vec was
    # constructed with tau appended, so `keep` already retains it -- pushing
    # again would duplicate it and create a zero-width trailing interval).
    if t_use[end] != tau
        push!(t_use, tau)
        push!(s_use, s_use[end])
    end
    # Exact integral of the KM step function: width × left-height
    sum(diff(t_use) .* s_use[1:end-1])
end

  = 5.0
rmst_trt_unadj = rmst(t_trt, s_trt,  )
rmst_ctl_unadj = rmst(t_ctl, s_ctl,  )
rmst_trt_adj   = rmst(t_trt_w, s_trt_w,  )
rmst_ctl_adj   = rmst(t_ctl_w, s_ctl_w,  )

@printf("Unadjusted RMST( =5):\n")
@printf("  Treatment: %.3f years    Control: %.3f years    Diff: %.3f\n",
        rmst_trt_unadj, rmst_ctl_unadj, rmst_trt_unadj - rmst_ctl_unadj)
@printf("IPW-adjusted RMST( =5):\n")
@printf("  Treatment: %.3f years    Control: %.3f years    Diff: %.3f\n",
        rmst_trt_adj, rmst_ctl_adj, rmst_trt_adj - rmst_ctl_adj)
```

```
Unadjusted RMST( =5):
  Treatment: 4.351 years    Control: 4.147 years    Diff: 0.203
```

```
IPW-adjusted RMST(=5):
  Treatment: 4.397 years    Control: 4.122 years    Diff: 0.275
```

The IPW-adjusted RMST difference is the adjusted years-gained estimate up to 5 years.

22.6. Cox proportional hazards (regression adjustment)

Julia's survival-regression tools are less mature than R's `survival::coxph`. In Julia, the choices are:

- Use `Survival.jl` for basic Cox PH (limited adjustment capabilities).
- Use a parametric Weibull/exponential model via `Distributions` and `Optim` directly.
- Call out to R for serious survival modelling.

For applied work where the Cox model with covariate adjustment is the target, R remains the practical choice. See the companion R chapter for the full Cox + IPCW + doubly-robust workflow.

22.7. Doubly-robust survival estimation

R's `riskRegression::ate` combines a Cox outcome model with a propensity score and censoring model. Julia does not currently have an equivalent.

For now, the practical Julia workflow is:

1. **Description**: unadjusted KM curves (this chapter).
2. **Adjusted survival**: IPW-weighted KM (this chapter).
3. **Causal contrasts**: RMST differences from the IPW-adjusted KM (this chapter).
4. **Cox HR with adjustment + doubly-robust estimators**: call out to R via `RCall.jl` or use the R companion chapter.

22.8. When to use these methods

Use these methods when:

1. The outcome is time-to-event with censoring.
2. The treatment effect on survival probabilities or years-of-life is the causal question.

For non-censored outcomes, the standard Estimation and Heterogeneous Effects methods are sufficient.

22.9. Summary

- Survival outcomes require methods that handle censoring.
- Kaplan-Meier estimates the survival curve nonparametrically.
- IPW-adjusted KM handles baseline confounding through weights.
- RMST differences are often the clearest causal summary.
- For Cox regression and doubly robust survival estimators, use R for now.

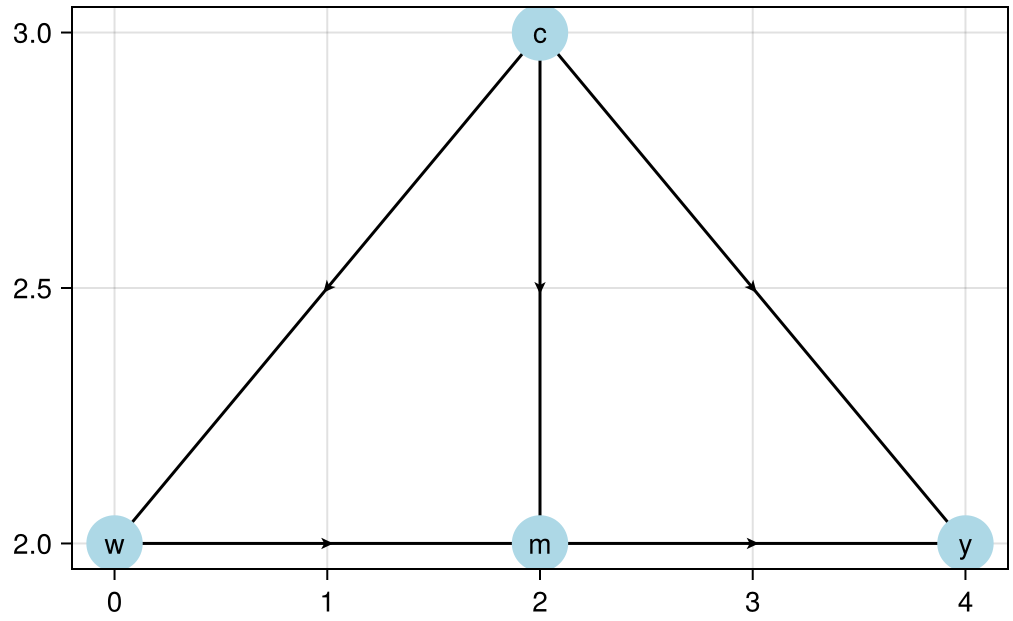
Part VI.

Mediation

23. Causal Mediation Analysis

```
using CairoMakie
CairoMakie.activate!(type = "png")
using GraphMakie
using Graphs
using GLM
using DataFrames
using StatsModels
using DataFramesMeta
using Random
using Distributions
using StatsFuns
using Statistics
using Lavaan
using Crumble
using Printf
```

Mediation asks how a treatment works, not only whether it works. The graph below is the setting: w is the treatment, m a mediator that w affects and that in turn affects the outcome y, and c a set of covariates confounding both the $w \to y$ and the $m \to y$ relationships. The effect of w on y splits into a path through m and a path around it.



23.1. Classical Mediation

Traditionally mediation model can be represented in the following equations:

$$Y = aW + bM + \epsilon_1 \tag{23.1}$$

$$M = cW + \epsilon_2 \tag{23.2}$$

That is, we'd like to study the effect of W on Y, and we see the effect can be a direct effect, and an indirect effect, through M.

Baron and Kenny's (http://davidakenny.net/cm/mediate.htm) method is done in four steps. Modern approach tends to use SEM (structural equation modeling) to model these two equations directly.

We simulate $n = 10{,}000$ observations from exactly this structure: $X \sim N(0, 1)$, $M = 0.5X + \varepsilon_2$, and $Y = 0.7M + 0.3X + \varepsilon_1$, with both errors standard normal. So the true direct effect is $a = 0.3$, the mediator coefficient is $b = 0.7$, the treatment-to-mediator coefficient is $c = 0.5$, the indirect effect is $bc = 0.35$, and the total effect is $0.3 + 0.35 = 0.65$. There is no confounding of any kind, which is what makes this the easy case.

```
using Random
using DataFrames
using StatsModels
using Lavaan

Random.seed!(1234)
n = 10000
X = randn(n)
M = 0.5 .* X .+ randn(n)
Y = 0.7 .* M .+ 0.3 .* X .+ randn(n)
Data = DataFrame(X = X, M = M, Y = Y)

model = """
   Y ~ a*X + b*M
   M ~ c*X
   # direct effect (a)
   # indirect effect (b*c)
   bc := b*c
   # total effect
   total := a + (b*c)
"""
```

```
"   Y ~ a*X + b*M\n   M ~ c*X\n   # direct effect (a)\n   # indirect effect (b*c)\n   bc := b*
```

```
fit = sem(model, Data)
println(fit)
parameterEstimates(fit)
```

```
lavaan 0.1.0-dev fit object
  Estimator:   ML
  Converged:   Yes
   ²(0) = 0.000  [p = NaN]
  Call summary() for full output.
```

	lhs	op	rhs	est	se	z	pvalue	ci_lower	ci_upper	
	String	String	String	Float64	Float64	Float64	Float64	Float64	Float64	
1	Y	~	X	0.295669	0.0111736	26.4614	0.0	0.273769	0.317569	...
2	Y	~	M	0.70038	0.0099825	70.1608	0.0	0.680814	0.719946	...
3	M	~	X	0.512129	0.00995286	51.4554	0.0	0.492621	0.531637	...
4	bc	:=	b*c	0.358685	0.00864452	41.4927	0.0	0.341742	0.375628	...
5	total	:=	a + (b*c)	0.654354	0.0121369	53.9143	0.0	0.630565	0.678142	...
6	M	~~	M	1.00917	0.0142719	70.7107	0.0	0.981199	1.03715	...
7	Y	~~	Y	1.00564	0.014222	70.7107	0.0	0.977769	1.03352	...
8	X	~~	X	1.01875	NaN	NaN	NaN	NaN	NaN	...

The estimates are $a = 0.296$, $b = 0.700$ and $c = 0.512$, against true values of 0.3, 0.7 and 0.5. The derived quantities are $bc = 0.359$ against a true 0.35 and a total of 0.654 against 0.65. Everything is recovered, and the model test statistic is 0 on 0 degrees of freedom because the model is saturated — it has as many parameters as moments, so it cannot fail to fit and the fit statistic carries no information.

23.1.1. Problems with Classical Mediation

- Lack of causal claim. We have to assume that there is no unmeasured confounder between M and Y. This is a strong assumption.
- Assumption of homogeneous effect.

23.2. Causal Mediation

23.2.1. CDE

Suppose we can set W and M at will to any (w, m), then we have the potential outcome $Y(w, m)$.

Controlled direct effect (CDE) is defined as

$$CDE(m) = E[Y(1, m) - Y(0, m)] \tag{23.3}$$

That is, setting M to m, what is the effect of W on Y?

23.2.2. Assumptions

- Conditional treatment randomization. Suppose we observe confounders X, which can be a joint set of confounders for the W to Y pathway, or the M to Y pathway.

$$Y(w, m) \perp W|X \tag{23.4}$$

This is the usual conditional independence (unconfoundedness, ignorability, etc.) assumption. This is saying the assignment of treatment, given covariates X, has nothing to do with potential outcomes.

- Conditional mediator randomization.

$$Y(w, m) \perp M|X, W = w \tag{23.5}$$

This is to say, within each strata of X, given treatment status, the assignment of mediator gives no information about potential outcome. This randomization is usually not implemented during many experiments (random trials). This is the assumption that makes a lot of mediation hard to make a causal claim.

- Positivity (overlap). There are two positivity assumptions:

$$P(W = w|X = x) > 0 \tag{23.6}$$

for all x. This is the usual positivity assumption.

$$P(M = m|X = x, W = w) > 0 \tag{23.7}$$

for all x, w, and m. This is the mediator positivity.

23.2.3. Estimation

CDE can be estimated using G-computation method, or IPW, or the doubly-robust methods, one of them is AIPW (augmented IPW).

23.2.4. G-computation

G-computation is to model the outcome equation:

$$\begin{aligned} CDE_G(m) = \sum_x [&E[Y \mid W = 1, M = m, X = x] \\ &- E[Y \mid W = 0, M = m, X = x]] \, P(X = x) \end{aligned} \tag{23.8}$$

23.2.5. IPW

IPW is to model the treatment assignment equation and the mediator assignment equation. The joint denominator factorizes *sequentially* – treatment given covariates, then mediator given treatment and covariates – which is the structure the estimator below relies on:

$$E[Y(w,m)] = E\left[\frac{I(W=w)\,I(M=m)}{g_W(w|X)\,g_M(m|w,X)}\,Y\right] \tag{23.9}$$

Therefore,

$$CDE_{ipw}(m) = E\left[\frac{I(W=1,M=m)}{g_M(m|1,X)\,g_W(1|X)}\,Y - \frac{I(W=0,M=m)}{g_M(m|0,X)\,g_W(0|X)}\,Y\right] \tag{23.10}$$

where g_M is the probability of the mediator and g_W the probability of treatment. **This indicator-based form is for a discrete (here binary) mediator.** For a continuous mediator the indicator $I(M=m)$ and the probability g_M must be replaced by a conditional density (or a kernel/stochastic-intervention formulation); the estimator above does not apply as written.

23.2.6. AIPW

$$CDE_{AIPW} = CDE_G(m) + B(\bar{Q}, g_M, g_W) \tag{23.11}$$

where $\bar{Q}$ is the mean outcome function.

$$\begin{aligned} B(\bar{Q}, g_M, g_W) = &\frac{1}{n}\sum_{i=1}^{n}\frac{I(M_i=m, W_i=1)}{g_M(m|1,X_i)g_W(1|X_i)}[Y_i - \bar{Q}(m,1,X_i)] \\ &- \frac{1}{n}\sum_{i=1}^{n}\frac{I(M_i=m, W_i=0)}{g_M(m|0,X_i)g_W(0|X_i)}[Y_i - \bar{Q}(m,0,X_i)] \end{aligned} \tag{23.12}$$

23.2.7. Example: G-computation

The three estimators below run on one simulated data set of $n = 5000$. There are two independent standard normal covariates, W_1 confounding the treatment-outcome relationship and W_2 confounding the mediator-outcome relationship. Treatment is $A \sim \text{Bernoulli}(\Lambda(-1+W_1/2))$, the mediator is $M \sim \text{Bernoulli}(\Lambda(-2+A/2+W_2/3))$, and the outcome is $Y \sim \text{Bernoulli}(\Lambda(-1+A-M/2+W_1/3+W_2/3))$. All three are binary.

The target is $CDE(0)$, the effect of treatment with the mediator held at 0 for everyone. Since the DGP is fully specified we can integrate it out: $CDE(0) = E_W[\Lambda(W_1/3+W_2/3)-\Lambda(-1+W_1/3+W_2/3)] = 0.222$. For comparison, $CDE(1) = 0.191$ — the effect is smaller when the mediator is held at 1, because the logistic link is flatter away from its centre.

```julia
using GLM
using Distributions
using StatsFuns
using Statistics

Random.seed!(1234)
n = 5000
# confounder of A/Y
W1 = randn(n)
# confounder of M/Y
W2 = randn(n)
# treatment
A = rand.(Bernoulli.(logistic.(-1 .+ W1 ./ 2)))
# binary mediator
M = rand.(Bernoulli.(logistic.(-2 .+ A ./ 2 .+ W2 ./ 3)))
# binary outcome
Y = rand.(Bernoulli.(logistic.(-1 .+ A .- M ./ 2 .+ W1 ./ 3 .+ W2 ./ 3)))
full_data = DataFrame(W1 = W1, W2 = W2, A = A, M = M, Y = Y)

# fit outcome regression
or_fit = glm(@formula(Y ~ A + M + W1 + W2), full_data, Binomial(), LogitLink())

# new data setting A and M
data_A1_M0 = copy(full_data)
data_A0_M0 = copy(full_data)
data_A1_M0.A .= 1; data_A1_M0.M .= 0
data_A0_M0.A .= 0; data_A0_M0.M .= 0

# predict on new data
Qbar_A1_M0 = predict(or_fit, data_A1_M0)
Qbar_A0_M0 = predict(or_fit, data_A0_M0)

# gcomp estimate of CDE(0)
cde_gcomp = mean(Qbar_A1_M0 .- Qbar_A0_M0)
@printf "CDE(0) G-computation = %.4g\n" cde_gcomp
```

```
CDE(0) G-computation = 0.2113
```

G-computation gives 0.2113.

23.2.8. Example: IPW

The same target through the treatment and mediator models instead of the outcome model.

```
# model for P(A = 1 | W)
ps_fit1 = glm(@formula(A ~ W1 + W2), full_data, Binomial(), LogitLink())
P_A1_W = predict(ps_fit1)
P_A0_W = 1 .- P_A1_W

# model for P(M = 0 | A, W)
ps_fit2 = glm(@formula(M ~ A + W1 + W2), full_data, Binomial(), LogitLink())

# P(M = 0 | A = 1, W)
data_A1 = copy(full_data); data_A1.A .= 1
P_M0_A1_W = 1 .- predict(ps_fit2, data_A1)

# P(M = 0 | A = 0, W)
data_A0 = copy(full_data); data_A0.A .= 0
P_M0_A0_W = 1 .- predict(ps_fit2, data_A0)

# ipw estimate of CDE(0)
cde_ipw = mean( (A .== 1) ./ P_A1_W .* (M .== 0) ./ P_M0_A1_W .* Y ) -
          mean( (A .== 0) ./ P_A0_W .* (M .== 0) ./ P_M0_A0_W .* Y )
@printf "CDE(0) IPW = %.4g\n" cde_ipw
```

```
CDE(0) IPW = 0.226
```

IPW gives 0.2260.

23.2.9. Example: AIPW

Combining both, so that either the outcome model or the two treatment models may be wrong.

```
# aipw estimate of E[Y(1,0)]
aiptw_EY_A1_M0 = mean(Qbar_A1_M0) +
mean( (A .== 1) ./ P_A1_W .* (M .== 0) ./ P_M0_A1_W .* (Y .- Qbar_A1_M0) )

# aipw estimate of E[Y(0,0)]
aiptw_EY_A0_M0 = mean(Qbar_A0_M0) +
mean( (A .== 0) ./ P_A0_W .* (M .== 0) ./ P_M0_A0_W .* (Y .- Qbar_A0_M0) )

# aipw estimate of CDE(0)
cde_aipw = aiptw_EY_A1_M0 - aiptw_EY_A0_M0
@printf "CDE(0) AIPW = %.4g\n" cde_aipw
```

```
CDE(0) AIPW = 0.2245
```

AIPW gives 0.2245.

The three estimates are 0.2113, 0.2260 and 0.2245 against a true $CDE(0)$ of 0.222. Every model here is correctly specified, and over 400 replications of this design the g-computation estimator averages 0.2216 with a sampling standard deviation of 0.0154, so this seed's 0.2113 is 0.7 standard deviations low. The three estimators share the same data and the same fitted nuisance models, so their agreement with each other is a check on implementation, not on sampling error.

23.3. NIE and NDE: Natural Direct and Indirect Effects

CDE is to study the effect of treatment, given the level of mediator. Instead, Natural Effect is to set mediator to its natural value with the value of treatment, that is, $M = M(w)$.

$$\begin{aligned} ATE &= NIE + NDE \\ &= (E[Y(1, M(1))] - E[Y(1, M(0))]) \\ &\quad + (E[Y(1, M(0))] - E[Y(0, M(0))]) \end{aligned} \tag{23.13}$$

The advantage of NDE and NIE comparing to CDE is that it's more "natural"; that is, you don't set the level of mediator deterministically. And it can decompose the ATE into direct and indirect effects.

However, there is an additional assumption needed to identify NDE and NIE.

23.3.1. Additional Assumption

$$Y(w, m) \perp M(w^*)|X \tag{23.14}$$

This is the "cross-world" condition: the outcome under (w, m) is independent of M under w^*. These two situations cannot happen in the same world; you cannot set W to both w and w^*. No experiment can implement it.

This cross-world independence is *additional to*, not a replacement for, the usual identifying assumptions. The standard mediation formula for natural effects also requires:

- treatment ignorability for the outcome (no unmeasured treatment-outcome confounding) given X;
- treatment ignorability for the mediator (no unmeasured treatment-mediator confounding), $M(w) \perp W \mid X$;
- mediator ignorability (no unmeasured mediator-outcome confounding) given W and X;
- positivity for both the treatment and the mediator; and
- **no treatment-induced confounding** of the mediator-outcome relationship – there must be no variable affected by W that confounds M and Y. (When such a confounder exists, natural effects are not identified by this formula and interventional direct/indirect effects are used instead.)

The cross-world condition by itself is not sufficient.

23.3.2. Estimation

We use the same simulated data. The outcome model now adds $A \times M$ and $M \times W_1$ interactions and the mediator model adds $A \times W_1$ and $W_1 \times W_2$ — both are richer than the DGP requires, which costs a little precision and no consistency. We then average $E[Y \mid A = a, M, W]$ over the distribution of M under $A = a'$, for each of the four (a, a') combinations.

The truths again follow from the DGP by integration: $E[Y(1, M(1))] = 0.478$, $E[Y(1, M(0))] = 0.486$ and $E[Y(0, M(0))] = 0.267$, so the true $NDE = 0.218$, $NIE = -0.008$ and $ATE = 0.211$. The indirect effect is slightly negative because treatment raises the chance of the mediator (from 0.124 to 0.188) while the mediator lowers the outcome — the $-M/2$ term. So this treatment helps directly and hurts a little through its mediator.

```
# fit outcome regression (include interaction because we can)
or_fit = glm(@formula(Y ~ A + M + W1 + W2 + A&M + M&W1), full_data, Binomial(), LogitLink())

# need E(Y | A = 0/1, M = 0/1, W1 = W1i, W2 = W2i)
function get_EY_a_m_Wi(full_data, or_fit, a, m)
    data_Aa_Mm_Wi = copy(full_data)
    data_Aa_Mm_Wi.A .= a
    data_Aa_Mm_Wi.M .= m
    predict(or_fit, data_Aa_Mm_Wi)
end

EY_A0_M0_Wi = get_EY_a_m_Wi(full_data, or_fit, 0, 0)
EY_A0_M1_Wi = get_EY_a_m_Wi(full_data, or_fit, 0, 1)
EY_A1_M0_Wi = get_EY_a_m_Wi(full_data, or_fit, 1, 0)
EY_A1_M1_Wi = get_EY_a_m_Wi(full_data, or_fit, 1, 1)

# include interactions -- why not? (NOTE: A*W1 expands to A + W1 + A&W1, so
# the MAIN effects are included. Writing only A&W1 would omit the main effect
# of A and force the estimated NIE to be essentially zero by construction.)
med_fit = glm(@formula(M ~ A*W1 + W1*W2), full_data, Binomial(), LogitLink())

# estimates of P(M = m | A = a, W = W_i)
function get_Pm_a_Wi(full_data, med_fit, a, m)
    data_Aa_Wi = copy(full_data)
    data_Aa_Wi.A .= a
    p = predict(med_fit, data_Aa_Wi)
    if m == 1
        return p
    else
        return 1 .- p
    end
end

PM0_A0_Wi = get_Pm_a_Wi(full_data, med_fit, 0, 0)
PM1_A0_Wi = get_Pm_a_Wi(full_data, med_fit, 0, 1)
```

```
PM0_A1_Wi = get_Pm_a_Wi(full_data, med_fit, 1, 0)
PM1_A1_Wi = get_Pm_a_Wi(full_data, med_fit, 1, 1)

# E(E(Y | A = 1, M, W) | A = 1, W)
EY1M1_Wi = EY_A1_M1_Wi .* PM1_A1_Wi .+ EY_A1_M0_Wi .* PM0_A1_Wi
# E(E(Y | A = 0, M, W) | A = 1, W)
EY0M1_Wi = EY_A0_M1_Wi .* PM1_A1_Wi .+ EY_A0_M0_Wi .* PM0_A1_Wi
# E(E(Y | A = 1, M, W) | A = 0, W)
EY1M0_Wi = EY_A1_M1_Wi .* PM1_A0_Wi .+ EY_A1_M0_Wi .* PM0_A0_Wi
# E(E(Y | A = 0, M, W) | A = 0, W)
EY0M0_Wi = EY_A0_M1_Wi .* PM1_A0_Wi .+ EY_A0_M0_Wi .* PM0_A0_Wi

# estimate of E[Y(1, M(1))]
E_Y1M1 = mean(EY1M1_Wi)
# estimate of E[Y(0, M(1))]
E_Y0M1 = mean(EY0M1_Wi)
# estimate of E[Y(1, M(0))]
E_Y1M0 = mean(EY1M0_Wi)
# estimate of E[Y(0, M(0))]
E_Y0M0 = mean(EY0M0_Wi)

# NDE = E[Y(1,M(0))] - E[Y(0,M(0))], NIE = E[Y(1,M(1))] - E[Y(1,M(0))]
@printf "NDE = %.4g\n" (E_Y1M0 - E_Y0M0)
@printf "NIE = %.4g\n" (E_Y1M1 - E_Y1M0)
@printf "ATE = %.4g\n" (E_Y1M1 - E_Y0M0)
```

```
NDE = 0.2093
NIE = -0.01266
ATE = 0.1966
```

The estimates are $NDE = 0.209$, $NIE = -0.013$ and $ATE = 0.197$, against truths of 0.218, -0.008 and 0.211. The decomposition holds exactly by construction, $0.2093 + (-0.0127) = 0.1966$, and the sign of the indirect effect is recovered. The direct and total effects sit about 0.01 below their truths, as the g-computation estimate of $CDE(0)$ did, for the same reason: it is one draw of the data, and every quantity in this section is computed from it.

Note how much smaller the indirect effect is than the direct one, -0.013 against 0.209. Reporting only the total effect of 0.197 would hide that the mediator works against the treatment.

23.4. IIE and IDE: Interventional Direct and Indirect Effects

People are not happy with the cross-world assumption in general. Interventional direct and indirect effects avoid it. The device is a random draw: let M_w^* denote a draw from the distribution of $M(w)$ given $X = x$, rather than the unit's own $M(w)$. The standard definitions (Vansteelandt and Daniel 2017; Díaz et al. 2021) use such a draw in every term:

$$\begin{aligned} IDE &= E[Y(1, M_0^*)] - E[Y(0, M_0^*)] \\ IIE &= E[Y(1, M_1^*)] - E[Y(1, M_0^*)] \end{aligned} \tag{23.15}$$

The direct effect holds the mediator draw fixed at its untreated distribution; the indirect effect moves only that distribution. The two add up to

$$IDE + IIE = E[Y(1, M_1^*)] - E[Y(0, M_0^*)], \tag{23.16}$$

the *overall interventional effect*. It need not equal the ATE: replacing the unit's own $M(w)$ by a random draw breaks the within-unit dependence between $M(w)$ and $Y(w, \cdot)$, and that dependence matters exactly when a treatment-induced confounder Z sits between the treatment and the mediator — the situation in the example below. The payoff is identification without the cross-world assumption, even in the presence of such a Z. These interventional effects are the estimands `Crumble.jl` estimates with `effect = "RI"`.

23.4.1. Example

This example uses the `medoutcon` vignette's own DGP, which is built to contain exactly the treatment-induced confounder that defeats natural effects. We draw $n = 1000$ observations. Three binary baseline covariates: $W_1 \sim \text{Bernoulli}(0.6)$, $W_2 \sim \text{Bernoulli}(0.3)$, and $W_3 \sim \text{Bernoulli}(\min(0.2 + (W_1 + W_2)/3,\, 1))$. Treatment is $A \sim \text{Bernoulli}(\Lambda(\sum_j W_j - 2))$.

Then the key variable: $Z \sim \text{Bernoulli}(\Lambda(-\log 2 + (W_1 + W_2 + W_3)/3 - A + 0.2))$ is a mediator-outcome confounder that depends on A. The mediator is $M \sim \text{Bernoulli}(\Lambda(\log 3(W_1 + W_2) + 2A - 2Z))$ and the outcome is $Y \sim \text{Bernoulli}(\Lambda(1/(\sum_j W_j - Z + A + M)))$.

Because Z is affected by A and confounds M and Y, natural direct and indirect effects are not identified here by the formula of the previous section. Interventional effects are, which is the reason for this example. Every variable is binary, so the truth follows by summing over the DGP: $IDE = -0.074$ and $IIE = -0.025$. Their sum, the overall interventional effect, is -0.099, while the ATE is -0.083. That is the gap Equation 23.16 warns about: with Z present, the two interventional effects do not add up to the total effect.

```
using DataFrames
using StatsModels
using Random
using StatsFuns
using Distributions
using Crumble

Random.seed!(1584)

# produces a simple data set based on a causal model with mediation
function make_example_data(n_obs = 1000)
    # baseline covariates
    w_1 = rand.(Bernoulli(0.6), n_obs)
```

```
    w_2 = rand.(Bernoulli(0.3), n_obs)
    w_3_prob = min.(0.2 .+ (w_1 .+ w_2) ./ 3, 1.0)
    w_3 = rand.(Bernoulli.(w_3_prob))

    # exposure
    a_prob = logistic.(w_1 .+ w_2 .+ w_3 .- 2)
    a = rand.(Bernoulli.(a_prob))

    # mediator-outcome confounder affected by treatment
    z_prob = logistic.(-log(2) .- a .+ (w_1 .+ w_2 .+ w_3) ./ 3 .+ 0.2)
    z = rand.(Bernoulli.(z_prob))

    # mediator -- could be multivariate
    m_prob = logistic.(log(3) .* (w_1 .+ w_2) .+ 2 .* a .- 2 .* z)
    m = rand.(Bernoulli.(m_prob))

    # outcome
    y_prob = logistic.(1 ./ (w_1 .+ w_2 .+ w_3 .- z .+ a .+ m))
    y = rand.(Bernoulli.(y_prob))

    # construct output
    dat = DataFrame(W_1 = w_1, W_2 = w_2, W_3 = w_3, A = a, Z = z, M = m, Y = y)
    return dat
end
```

```
make_example_data (generic function with 2 methods)
```

```
# set seed and simulate example data
example_data = make_example_data()
w_names = ["W_1", "W_2", "W_3"]
m_names = ["M"]

# quick look at the data
first(example_data, 6)
```

	W_1	W_2	W_3	A	Z	M	Y
	Bool	Bool	Bool	Bool	Bool	Bool	Bool
1	0	1	0	0	0	1	1
2	0	0	0	0	0	1	1
3	0	0	0	0	0	1	1
4	0	0	0	0	0	1	1
5	1	0	0	0	0	0	1
6	1	1	1	1	1	1	0

```
# one-step estimates of the interventional direct and indirect effects
# (effect = "RI"); the Riesz representers are bounded at 100, see below
result = crumble(example_data, ["A"];
                 outcome   = "Y",
                 mediators = m_names,
                 moc       = ["Z"],       # mediator-outcome confounder affected by A
                 covar     = w_names,
                 effect    = "RI",
                 learners  = ["glm"],
                 control   = crumble_control(alpha_cap = 100.0))
Crumble.tidy(result)
```

	estimand	estimate	std_error	conf_low	conf_high	p_value
	String	Float64	Float64	Float64	Float64	Float64
1	ride	0.0165558	0.124993	-0.22843	0.261541	0.894625
2	riie	-9.7107e-6	0.0967316	-0.189604	0.189584	0.99992

The one-step estimator gives an interventional direct effect of 0.017 with a 95% interval of $[-0.228, 0.262]$, against a true -0.074, and an indirect effect of 0.000 with an interval of $[-0.190, 0.190]$, against a true -0.025. Both intervals include zero and both truths.

`Crumble.jl` fits the Riesz representers — the weights in the one-step correction — with a neural network, and this design is hard on them. The true weight for the direct-effect functional reaches 45 in a covariate cell that holds about two observations per cross-fitting fold. When a fold misses that cell, the Riesz loss has no minimiser there and an unbounded fit can drift into the thousands. The call above bounds the weights at 100, well above the true maximum.

The flexible weights cost precision. The interval here is more than twice as wide as the one from `medoutcon` in the R companion chapter, which also compares the one-step estimator with TMLE; `Crumble.jl` implements only the one-step estimator.

23.5. Summary

Mediation splits the effect of a treatment into a part through a mediator and a part around it. The classical regression decomposition is causal only if nothing unmeasured confounds the mediator and the outcome. The controlled direct effect fixes the mediator at a level. It needs treatment and mediator randomization given covariates, and it can be estimated by g-computation, IPW or AIPW. Natural direct and indirect effects add up to the ATE. They need a cross-world independence that no experiment can implement, and no confounder of the mediator and outcome that is itself affected by treatment. Interventional effects drop the cross-world condition by using a random draw from the mediator's distribution. Their sum then need not equal the ATE.

In the simulated binary example, the three CDE estimators give 0.2113, 0.2260 and 0.2245 against a true 0.222. The natural effects show a small negative indirect effect beside a direct effect of 0.209. With a treatment-induced confounder, the one-step interventional direct effect is 0.017, with an interval from -0.228 to 0.262 that covers the true -0.074. The flexible Riesz-representer weights make it too imprecise at $n = 1000$ to be told apart from zero.

Part VII.

Causal Discovery

24. Causal Discovery: Observed Variables

```
using CausalInference
using RecursiveCausalDiscovery
using CausalGraphs
using Graphs
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using LinearAlgebra
using Random
using Printf
using DataFrames
using CSV
using Distributions: Normal, quantile
```

Earlier chapters assumed the graph was known, or at least that we could draw a defensible DAG from subject knowledge. Causal discovery asks what can be learned about the graph from observational data.

24.1. What can be recovered from the data?

Suppose we observe n independent and identically distributed observations of p variables $(X_1, \ldots, X_p)$. We would like to learn the directed acyclic graph G that generated them.

There is an important limit: observational data alone cannot distinguish between DAGs with the same conditional independence structure. The set of observationally equivalent DAGs is a **Markov equivalence class**. The data can identify this class, represented by a CPDAG, not necessarily the one true DAG.

A CPDAG has the same skeleton (undirected edges) as all DAGs in its MEC. Some edges can be oriented (they have the same direction in every member of the MEC); others remain undirected because both directions are consistent with the CI structure.

24.1.1. Why some edges cannot be oriented

Three DAGs are observationally equivalent — they produce the same set of conditional independences — and therefore indistinguishable from data:

$$X \to Y \to Z \qquad X \leftarrow Y \leftarrow Z \qquad X \leftarrow Y \to Z \tag{24.1}$$

All three imply $X \perp\!\!\!\perp Z \mid Y$ and no other CI. Their CPDAG shows $X - Y - Z$ with no arrowheads.

The structure $X \to Y \leftarrow Z$ (a **collider** at Y) is *not* equivalent: here $X \perp\!\!\!\perp Z$ but $X \not\perp\!\!\!\perp Z \mid Y$. Because it has a unique CI signature, the $\to Y \leftarrow$ orientation is identifiable from data. Algorithms orient these colliders; non-collider edges often remain undirected.

24.1.2. What a CPDAG tells you

A directed edge $X \to Y$ in the CPDAG means: in *every* DAG consistent with the data, X causes Y. An undirected edge $X - Y$ means some consistent DAGs have $X \to Y$ and others have $X \leftarrow Y$ — data alone cannot decide.

For causal estimation: once you have a CPDAG, you can use background knowledge (temporal ordering, experimental context) to orient the remaining undirected edges, and then apply identification strategies from earlier chapters.

24.2. Simulation

We generate a random Gaussian linear DAG using an Erdős–Rényi random graph with 8 nodes and edge probability 0.35. Each edge $j \to i$ generates a structural equation

$$X_i = \sum_{j \in \mathrm{pa}(i)} \beta_{ji} X_j + \varepsilon_i, \quad \varepsilon_i \sim N(0,1) \tag{24.2}$$

where coefficients β_{ji} are drawn uniformly from $[-1, -0.25] \cup [0.25, 1]$.

```
Random.seed!(2025)

n_vars    = 8
n_samples = 2000
edge_prob = 0.35
labels    = ["X$i" for i in 1:n_vars]

adj_mat   = gen_er_dag_adj_mat(n_vars, edge_prob)
data_mat  = gen_gaussian_data(adj_mat, n_samples)

true_dag  = SimpleDiGraph(adj_mat)
true_skel = SimpleGraph(true_dag)

@printf "Variables: %d   Samples: %d   True edges: %d\n" n_vars n_samples ne(true_skel)
```

```
Variables: 8   Samples: 2000   True edges: 9
```

```
draw_graph(adjmat_to_admg(adj_mat, labels); direction="TB")
```

```
[SVG graph - display in an HTML environment]
```

Figure 24.1.: True DAG used for simulation.

24.3. CI Test Infrastructure

Both algorithms use a **Fisher-Z conditional independence test** appropriate for Gaussian data. Given the partial correlation $\hat{\rho}_{XY|Z}$, the test statistic is

$$T = \sqrt{n - |Z| - 3} \cdot \operatorname{atanh}(\hat{\rho}_{XY|Z}) \;\sim\; N(0,1) \quad \text{under } H_0 : X \perp\!\!\!\perp Y \mid Z \tag{24.3}$$

Choosing the significance level α involves a bias-variance tradeoff on the skeleton:

- **Low α (e.g., 0.001)**: conservative test — it fails to reject independence more easily, so it removes more edges. Fewer spurious edges (false positives) but may discard real ones (false negatives). Produces a sparser skeleton.
- **High α (e.g., 0.05)**: liberal test — it rejects independence (declares dependence) more often, so it keeps more edges. Fewer false removals but may retain spurious edges. Produces a denser skeleton.

With $n = 2000$ and $p = 8$ we use $\alpha = 0.01$, a common default. With smaller n or larger p the CI tests lose power and true edges get dropped, so consider raising α to avoid over-pruning.

We wrap `fisher_z` in a counter to track exactly how many CI tests each algorithm calls:

```
function make_counted_ci(data, sig)
    count = Ref(0)
    function ci(x, y, S, d)
        count[] += 1
        fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig)
    end
    ci, count
end
```

```
make_counted_ci (generic function with 1 method)
```

24.4. PC Algorithm

The **PC algorithm** (Spirtes & Glymour, 1991) starts from a complete undirected graph and removes edges whenever a conditional independence is found. It searches for separating sets Z of increasing size, conditioning on subsets of the current neighbors. After skeleton discovery, Meek's rules orient as many edges as possible.

How it works, step by step:

1. Start with a complete undirected graph (every pair connected).
2. For each pair (X, Y), test $X \perp\!\!\!\perp Y \mid Z$ for all $Z \subseteq \text{neighbors}(X) \setminus \{Y\}$, starting with $|Z| = 0$. Remove the edge if any such test passes.
3. Repeat with conditioning sets of increasing size until no new edges are removed.
4. Identify colliders: if $X - Z - Y$ and X, Y non-adjacent, orient as $X \to Z \leftarrow Y$ only if Z was not in the separating set of X and Y.
5. Apply Meek's orientation rules to propagate orientations without creating new colliders or cycles.

```
sig_level = 0.01

ci_pc, ctr_pc = make_counted_ci(data_mat, sig_level)
cpdag_pc = pcalg(n_vars, ci_pc, data_mat)
skel_pc  = SimpleGraph(cpdag_pc)

f1_skel_pc  = f1_score(true_skel, skel_pc)
f1_cpdag_pc = f1_score(true_dag,  cpdag_pc)

@printf "PC:    skeleton F1 = %.3f   CPDAG F1 = %.3f   CI tests = %d\n" f1_skel_pc f1_cpdag_pc
```

```
PC:    skeleton F1 = 0.750   CPDAG F1 = 0.500   CI tests = 204
```

> **Note**
>
> **Reading the output: F1 scores**
> The **F1 score** is the harmonic mean of precision and recall, ranging from 0 (worst) to 1 (perfect).
>
> - **Skeleton F1** measures whether the algorithm found the right edges regardless of direction. An edge is a true positive if it exists in both the estimated skeleton and the true one.
> - **CPDAG F1** is stricter: it compares the two graphs' directed-edge sets, with an undirected CPDAG edge represented as *both* directions. A correctly-adjacent but unoriented edge therefore contributes one true positive (the direction matching the true DAG) and one false positive (the reverse) — a partial penalty — while a wrongly oriented edge contributes a false positive and a false negative.
>
> A high skeleton F1 with lower CPDAG F1 means the algorithm found the right adjacencies but left some edges unoriented — the typical case when the MEC contains many equivalent DAGs.

Computational complexity: In the worst case, PC tests all subsets of neighbors, giving exponential growth with the maximum degree. In sparse graphs this is manageable; in dense graphs it becomes prohibitive.

24.5. RSL-D Algorithm

RSL-D (Recursive Structure Learning — Diamond-free; Mokhtarian et al., JMLR 2025) works by *recursively removing* variables that satisfy a **removability** criterion.

Two sets are central to the criterion:

- **Markov boundary** $\mathrm{Mb}(X)$: the minimal set S such that $X \perp\!\!\!\perp$ (all other variables) $\mid S$. Conditioning on $\mathrm{Mb}(X)$ screens X off from everything else in the graph. In a DAG with no hidden variables and a faithful distribution, $\mathrm{Mb}(X)$ consists of X's parents, children, and co-parents (other parents of X's children). It is larger than the set of neighbours: in $X \to C \leftarrow Z$, Z is a co-parent in the boundary of X but is not adjacent to X.
- **Neighbourhood** $\mathrm{Ne}(X)$: the set of variables directly adjacent to X in the *current* skeleton (connected by an undirected edge at this stage of the algorithm).

So $\mathrm{Ne}(X) \subseteq \mathrm{Mb}(X)$, with equality only when every co-parent of X is also adjacent to it. RSL-D's guarantee also requires the graph to be diamond-free. The Erdős–Rényi DAG simulated above is not restricted to that class, so the simulation tests the algorithm beyond its guarantee.

Variable X is removable if, for every pair $Y \in \mathrm{Mb}(X)$, $Z \in \mathrm{Ne}(X)$:

$$\exists\, W \subseteq \mathrm{Mb}(X) \setminus \{Y, Z\} \text{ such that } Y \perp\!\!\!\perp Z \mid W \tag{24.4}$$

(Lemma 3 of the paper). Removing variables one by one and recording the local neighbourhood structure recovers the full CPDAG.

A variable is removable if its local neighborhood can be learned before the rest of the graph is fully known. The algorithm removes such variables one at a time and records the local structure.

How it works, step by step:

1. Estimate the Markov boundary $\mathrm{Mb}(X)$ of each variable (all variables that, conditioned on, make X independent of everything else).
2. Find a removable variable X — one whose neighbourhood satisfies the criterion above.
3. Record the local skeleton structure around X, then remove X from the graph.
4. Repeat on the reduced variable set until all variables have been processed.
5. Reconstruct the full CPDAG from the recorded local structures.

The key efficiency advantage: RSL-D uses at most $O(p \cdot d \cdot |\mathrm{MB}|^2)$ CI tests (where d is the max neighbourhood size). PC can require exponentially many in the worst case as the separator-set search grows.

```
ci_rsl, ctr_rsl = make_counted_ci(data_mat, sig_level)
cpdag_rsl = rsld(data_mat, ci_rsl)
skel_rsl  = SimpleGraph(cpdag_rsl)

f1_skel_rsl  = f1_score(true_skel, skel_rsl)
f1_cpdag_rsl = f1_score(true_dag,  cpdag_rsl)

@printf "RSL-D: skeleton F1 = %.3f   CPDAG F1 = %.3f   CI tests = %d\n" f1_skel_rsl f1_cpdag_rs
```

```
RSL-D: skeleton F1 = 1.000   CPDAG F1 = 0.947   CI tests = 42
```

24.6. Comparison

> **Note**
>
> **Reading the CPDAG figure**
> In the side-by-side graph display:
>
> - A **directed edge** $X_i \to X_j$ means this orientation is invariant across all equivalent DAGs — the algorithm is confident about the direction.
> - A **bidirected (red) edge** $X_i \leftrightarrow X_j$ in the CPDAG display represents an *unoriented* edge: both $X_i \to X_j$ and $X_i \leftarrow X_j$ are statistically equivalent. This is **not** a hidden common cause — it means the data cannot determine direction. (Hidden common causes are a latent-variable concept, covered in the next chapter.)
> - A **missing edge** means the algorithm found a conditional independence separating those two variables.
>
> Compare the estimated CPDAG to the true DAG: extra edges are false positives (the algorithm wrongly retained a spurious association); missing edges are false negatives (a true relationship was incorrectly removed by a CI test).

```
DataFrame(
    Algorithm   = ["PC",              "RSL-D"],
    Skeleton_F1 = round.([f1_skel_pc,  f1_skel_rsl],  digits=3),
    CPDAG_F1    = round.([f1_cpdag_pc, f1_cpdag_rsl], digits=3),
    CI_Tests    = [ctr_pc[],           ctr_rsl[]],
)
```

```
side_by_side([
    (adjmat_to_admg(adj_mat, labels),  "True DAG"),
    (cpdag_to_admg(cpdag_pc,  labels), "PC - estimated CPDAG"),
    (cpdag_to_admg(cpdag_rsl, labels), "RSL-D - estimated CPDAG"),
]; note="Red  edges in the CPDAGs are unoriented (not confounded) - both directions are statis
```

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">True DAG</p><svg width=\"22
4,4 -4,-256 217,-256 217,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"18\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"41\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X2->X1 -->\n<g id=\"edge1\" class=\"edge\">\n<title
215.67C23.63,-205.69 17.04,-192.65 14,-180 2.98,-134.08 8.24,-78.67 13.1,-
46.28\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"16.58,-46.65 14.71,-
36.22 9.67,-45.55 16.58,-46.65\"/>\n</g>\n<!-- X3 -->\n<g id=\"node3\" class=\"node\">\n<title>
anchor=\"middle\" x=\"95\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X3</text>\n</g>\n<!-- X2->X3 -->\n<g id=\"edge3\" class=\"edge\">\n<title
215.7C60.76,-207.39 68.56,-197.28 75.61,-188.14\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
190.16 81.81,-180.1 72.93,-185.88 78.47,-190.16\"/>\n</g>\n<!-- X5 -->\n<g id=\"node5\" class=
anchor=\"middle\" x=\"41\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge5\" class=\"edge\">\n<title
215.7C41,-207.98 41,-198.71 41,-190.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
190.1 41,-180.1 37.5,-190.1 44.5,-190.1\"/>\n</g>\n<!-- X6 -->\n<g id=\"node6\" class=\"node\">
anchor=\"middle\" x=\"137\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X6</text>\n</g>\n<!-- X3->X6 -->\n<g id=\"edge6\" class=\"edge\">\n<title
143.7C110.26,-135.56 116.19,-125.69 121.58,-116.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
118.48 126.74,-108.1 118.59,-114.88 124.59,-118.48\"/>\n</g>\n<!-- X4 -->\n<g id=\"node4\" clas
anchor=\"middle\" x=\"195\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X4</text>\n</g>\n<!-- X7 -->\n<g id=\"node7\" class=\"node\">\n<title>X7</title>
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X5->X7 -->\n<g id=\"edge8\" class=\"edge\">\n<title
143.7C41,-135.98 41,-126.71 41,-118.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
118.1 41,-108.1 37.5,-118.1 44.5,-118.1\"/>\n</g>\n<!-- X7->X1 -->\n<g id=\"edge2\" clas
71.7C32.75,-63.9 29.67,-54.51 26.82,-45.83\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=
44.51 23.62,-36.1 23.42,-46.7 30.07,-44.51\"/>\n</g>\n<!-- X8 -->\n<g id=\"node8\" class=\"node
anchor=\"middle\" x=\"141\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X8</text>\n</g>\n<!-- X8->X3 -->\n<g id=\"edge4\" class=\"edge\">\n<title
215.7C124.28,-207.56 117.8,-197.69 111.89,-188.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
186.54 106.24,-180.1 108.81,-190.38 114.66,-186.54\"/>\n</g>\n<!-- X8->X6 -->\n<g id=\"e
215.56C149.93,-205.33 153.43,-192.08 155,-180 157.06,-164.13 157.61,-159.79 155,-
144 153.56,-135.28 150.87,-126.06 148,-117.8\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points
116.46 144.48,-108.3 144.67,-118.89 151.23,-116.46\"/>\n</g>\n<!-- X8->X7 -->\n<g id=\"e
215.76C140.41,-196.63 136.83,-165.63 122,-144 108.93,-124.94 86.53,-
111.06 68.59,-102.3\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"69.72,-
98.96 59.17,-97.96 66.79,-105.32 69.72,-98.96\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">PC - estimated CPDAG</p><sv
4,4 -4,-184 240,-184 240,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"18\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X7 -->\n<g id=\"node7\" class=\"node\">\n<title>X7</title>
anchor=\"middle\" x=\"45\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge4\" class=\"edge\">\n<title
62.23C30.44,-56.75 32.66,-50.98 34.77,-45.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=
61.11 24.67,-71.7 31.54,-63.63 25.01,-61.11\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\
46.69 38.4,-36.1 31.54,-44.17 38.07,-46.69\"/>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node
anchor=\"middle\" x=\"118\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X3 -->\n<g id=\"node3\" class=\"node\">\n<title>X3</title>
anchor=\"middle\" x=\"164\" y=\"-14.3\" font-family=\"Times,serif\" font-

Table 24.1.: Algorithm comparison on the simulated 8-node DAG (n = 2000, significance 0.01)

	Algorithm	Skeleton_F1	CPDAG_F1	CI_Tests
	String	Float64	Float64	Int64
1	PC	0.75	0.5	204
2	RSL-D	1.0	0.947	42

> **Note**
>
> **Interpreting the comparison table**
>
> - **Skeleton F1** is the primary metric for comparing algorithms: it measures edge recovery independent of orientation, since orientation accuracy is bounded by the MEC.
> - **CPDAG F1** rewards correctly oriented edges and penalizes leaving orientable edges undirected.
> - **CI tests** is the computational cost metric. Fewer tests means faster runtime, especially important when n is large and each test involves inverting covariance matrices.
>
> On this dataset RSL-D typically matches or exceeds PC's accuracy while using substantially fewer CI tests — the advantage grows with p and graph density.

24.7. Monte Carlo Evaluation

A single dataset can be lucky. We average over 100 replications to get stable estimates.

What to look for in the MC results:

- The **mean skeleton F1** is the headline: how often does each algorithm recover the true adjacencies?
- The **distribution of CI test counts** (shown in Figure 24.3) shows whether the efficiency advantage is consistent or only appears on average.
- High variance in F1 across replications suggests the algorithms are sensitive to the particular random graph — denser or sparser graphs drawn from the same Erdős–Rényi model can be very different problems.

```
function evaluate_once(seed; n_vars=8, n_samples=2000, edge_prob=0.35, sig=0.01)
    Random.seed!(seed)
    adj  = gen_er_dag_adj_mat(n_vars, edge_prob)
    data = gen_gaussian_data(adj, n_samples)
    skel = SimpleGraph(SimpleDiGraph(adj))

    ci_pc,  ctr_pc  = make_counted_ci(data, sig)
    ci_rsl, ctr_rsl = make_counted_ci(data, sig)

    cp_pc  = pcalg(n_vars, ci_pc,  data)
    cp_rsl = rsld(data, ci_rsl)
```

```
    (
        f1_pc  = f1_score(skel, SimpleGraph(cp_pc)),
        f1_rsl = f1_score(skel, SimpleGraph(cp_rsl)),
        ci_pc  = ctr_pc[],
        ci_rsl = ctr_rsl[],
    )
end

mc = [evaluate_once(s) for s in 1:100]

@printf "\nMonte Carlo averages (100 replications, n=%d, p=%d)\n" n_samples n_vars
@printf "%-8s  Skeleton F1  CI tests\n" "Method"
@printf "%-8s  %10.3f  %8.1f\n" "PC"    mean(r.f1_pc  for r in mc) mean(r.ci_pc  for r in mc)
@printf "%-8s  %10.3f  %8.1f\n" "RSL-D" mean(r.f1_rsl for r in mc) mean(r.ci_rsl for r in mc)
```

```
Monte Carlo averages (100 replications, n=2000, p=8)
Method    Skeleton F1  CI tests
PC              0.894     233.4
RSL-D           0.939      56.2
```

```
let
    fig = Figure(size=(650, 380))
    ax  = Axis(fig[1,1];
        xlabel = "CI tests",
        ylabel = "Frequency",
        title  = "CI Test Counts: PC vs RSL-D (100 replications)")
    hist!(ax, [r.ci_pc  for r in mc]; bins=20, color=(:steelblue, 0.65), label="PC")
    hist!(ax, [r.ci_rsl for r in mc]; bins=20, color=(:tomato, 0.65),    label="RSL-D")
    axislegend(ax)
    fig
end
```

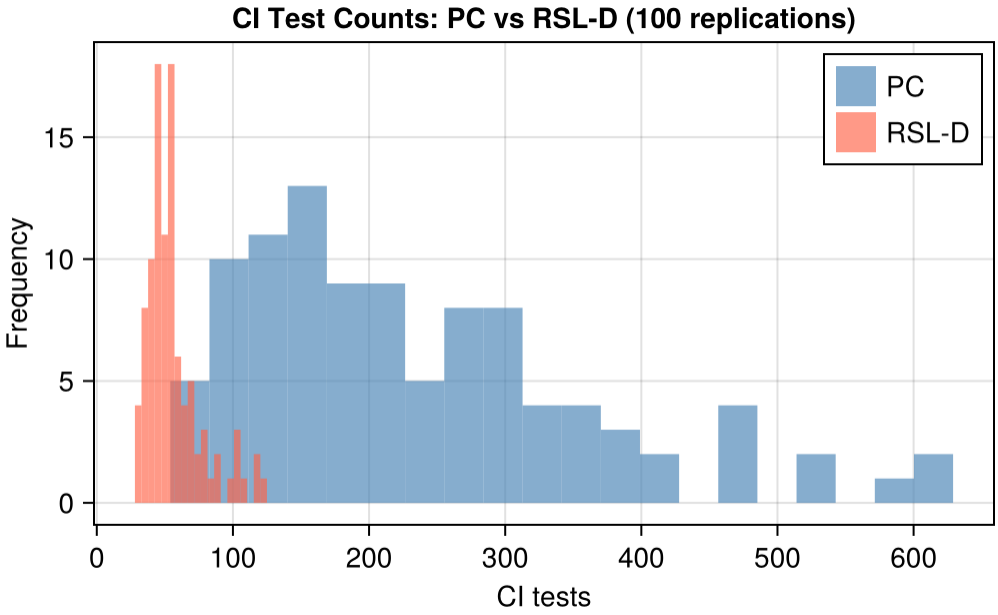


Figure 24.3.: Distribution of CI test counts over 100 replications. RSL-D consistently uses fewer CI tests.

24.8. Real Data: Student Performance in PISA

The simulation had a luxury real research never does: a known true DAG, so we could score recovery with F1. On real data there is **no ground truth**, so the questions change. Instead of "how close is the estimate to the truth?" we ask: do PC and RSL-D *agree*; what can *background knowledge* orient; and is the structure *stable* under resampling?

We illustrate on the OECD's **PISA 2022** assessment, restricted to **United States** students and six variables with a clear substantive ordering.

Node	Meaning	Role
`HISEI`	Highest parental occupational status (ISEI)	Family background
`HOMEPOS`	Home possessions index	Family background
`IMMIG`	Immigration status (1 native … 3 first-gen)	Demographic
`GRADE`	Grade relative to modal grade for age	Schooling
`GENDER`	1 = female, 0 = male	Demographic
`MATH`	Mean of the 10 mathematics plausible values	Outcome

```
pisa   = CSV.read("data/pisa_usa2022.csv", DataFrame)
plabs  = ["HISEI","HOMEPOS","IMMIG","GRADE","GENDER","MATH"]
Xp     = Matrix{Float64}(pisa[:, plabs])
wp     = Float64.(pisa.W)
np_, pp_ = size(Xp)
@printf "Students (complete cases): %d\n" np_
```

```
Students (complete cases): 3890
```

> **Note**
>
> **Three honesty caveats**, kept explicit:
>
> - **Plausible values.** Each student's ability is reported as 10 plausible values (posterior draws), not one score. We use their mean, understating the measurement variance; a full treatment runs discovery on each PV and pools.
> - **Categorical-as-Gaussian.** `IMMIG`, `GRADE`, `GENDER` are discrete; Fisher-Z assumes joint Gaussianity — a standard but imperfect approximation.
> - **One country, one wave.** Pooling would inject country/time structure that appears as spurious edges unless modelled as extra nodes.

24.8.1. Using the survey weights

PISA is not a simple random sample — students carry final weights `W_FSTUWT`. The CI tests are functions of the **covariance matrix** and a **sample size**, so we supply weighted inputs: a **weighted covariance/correlation** matrix, and **Kish's effective sample size** $n_{\text{eff}} = (\sum_i w_i)^2 / \sum_i w_i^2$, where w_i is student i's weight. This reproduces the weighted covariance, not the survey design. School clustering, stratification, the replicate-weight variance and plausible-value uncertainty are all left out, so the CI tests below are exploratory.

```
mu_p  = (Xp' * wp) ./ sum(wp)
Xc_p  = Xp .- mu_p'
Sig_p = (Xc_p' * (wp .* Xc_p)) ./ sum(wp)      # design-weighted covariance
sd_p  = sqrt.(diag(Sig_p)); Cw_p = Sig_p ./ (sd_p * sd_p')
neff  = sum(wp)^2 / sum(wp .^ 2)
@printf "Raw n = %d   Kish effective n = %.0f\n" np_ neff
```

```
Raw n = 3890   Kish effective n = 3481
```

```
let dfc = DataFrame(Variable = plabs)
    for (k, l) in enumerate(plabs); dfc[!, l] = round.(Cw_p[:, k], digits=2); end
    dfc
end
```

Table 24.3.: Design-weighted correlation matrix (PISA 2022, USA).

	Variable	HISEI	HOMEPOS	IMMIG	GRADE	GENDER	MATH
	String	Float64	Float64	Float64	Float64	Float64	Float64
1	HISEI	1.0	0.42	-0.17	0.05	-0.02	0.33
2	HOMEPOS	0.42	1.0	-0.18	0.05	0.03	0.4
3	IMMIG	-0.17	-0.18	1.0	0.07	0.01	-0.05
4	GRADE	0.05	0.05	0.07	1.0	0.08	0.17
5	GENDER	-0.02	0.03	0.01	0.08	1.0	-0.09
6	MATH	0.33	0.4	-0.05	0.17	-0.09	1.0

24.8.2. Two algorithms on the weighted data

The CI test recomputes correlations from a data matrix, so to honour the design we build a synthetic sample of n_{eff} rows whose empirical covariance equals the design-weighted Σ_w exactly (a Cholesky recolour). Both PC and RSL-D then run on these weighted inputs unchanged.

```
m_eff = round(Int, neff)
Random.seed!(1)
Z  = randn(m_eff, pp_); Z .-= mean(Z, dims=1)
Z  = Z * inv(cholesky(Symmetric(cov(Z))).U)     # whiten
Xw = Z * cholesky(Symmetric(Sig_p)).U .+ mu_p'  # recolour to weighted covariance
@printf "Synthetic covariance reproduces Σ_w (max abs diff = %.1e)\n" maximum(abs.(cov(Xw) .- S

sig_p = 0.01
ci_p(x, y, S, d) = fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig_p)

cpdag_pcp  = pcalg(pp_, ci_p, Xw)
cpdag_rslp = rsld(Xw, ci_p)
skel_pcp   = SimpleGraph(cpdag_pcp)
skel_rslp  = SimpleGraph(cpdag_rslp)

common = length(intersect(Set(Tuple.(sort.([ [e.src,e.dst] for e in edges(skel_pcp)]))),
                          Set(Tuple.(sort.([ [e.src,e.dst] for e in edges(skel_rslp)])))))
@printf "Skeleton edges - PC: %d   RSL-D: %d   in both: %d\n" ne(skel_pcp) ne(skel_rslp) commo
```

```
Synthetic covariance reproduces Σ_w (max abs diff = 9.1e-13)
Skeleton edges - PC: 9   RSL-D: 9   in both: 9
```

```
side_by_side([
    (cpdag_to_admg(cpdag_pcp,  plabs), "PC - estimated CPDAG"),
    (cpdag_to_admg(cpdag_rslp, plabs), "RSL-D - estimated CPDAG"),
]; note="Red  edges are unoriented (not confounded) - both directions are statistically equiva
```

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">PC - estimated CPDAG</p><sv
4,4 -4,-328 189.5,-328 189.5,4 -4,4\"/>\n<!-- HISEI -->\n<g id=\"node1\" class=\"node\">\n<titl
anchor=\"middle\" x=\"29.5\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HISEI</text>\n</g>\n<!-- HOMEPOS -->\n<g id=\"node2\" class=\"node\">\n<title>HO
anchor=\"middle\" x=\"70.5\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HOMEPOS</text>\n</g>\n<!-- HISEI->HOMEPOS -->\n<g id=\"edge1\" class=\"ed
215.73C35.94,-197.64 41.94,-168.52 49.5,-144 52.18,-135.3 55.65,-126.01 58.99,-
117.67\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"62.23,-119 62.8,-
108.42 55.76,-116.33 62.23,-119\"/>\n</g>\n<!-- IMMIG -->\n<g id=\"node3\" class=\"node\">\n<ti
anchor=\"middle\" x=\"70.5\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">IMMIG</text>\n</g>\n<!-- HISEI->IMMIG -->\n<g id=\"edge2\" class=\"edge\"
215.69C11.12,-185.23 -7.7,-120.67 13.5,-72 18.43,-60.68 27.11,-50.68 36.23,-
42.51\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"38.49,-45.19 43.92,-
36.09 34,-39.81 38.49,-45.19\"/>\n</g>\n<!-- MATH -->\n<g id=\"node6\" class=\"node\">\n<title>
anchor=\"middle\" x=\"89.5\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">MATH</text>\n</g>\n<!-- HISEI->MATH -->\n<g id=\"edge3\" class=\"edge\">\
215.7C51.52,-207.3 60.3,-197.07 68.19,-187.86\"/>\n<polygon fill=\"blue\" stroke=\"blue\" point
189.97 74.84,-180.1 65.67,-185.42 70.99,-189.97\"/>\n</g>\n<!-- HOMEPOS->IMMIG -->\n<g i
71.7C70.5,-63.98 70.5,-54.71 70.5,-46.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"
46.1 70.5,-36.1 67,-46.1 74,-46.1\"/>\n</g>\n<!-- GRADE -->\n<g id=\"node4\" class=\"node\">\n<
anchor=\"middle\" x=\"150.5\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">GRADE</text>\n</g>\n<!-- GRADE->IMMIG -->\n<g id=\"edge5\" class=\"edge\"
215.89C152.99,-185.2 152.36,-119.54 127.5,-72 121.68,-60.87 112.57,-
50.83 103.32,-42.56\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"105.5,-
39.82 95.6,-36.05 100.99,-45.17 105.5,-39.82\"/>\n</g>\n<!-- GRADE->MATH -->\n<g id=\"ed
215.7C128.11,-207.3 119.19,-197.07 111.16,-187.86\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
185.34 104.41,-180.1 108.34,-189.94 113.61,-185.34\"/>\n</g>\n<!-- GENDER -->\n<g id=\"node5\"
anchor=\"middle\" x=\"118.5\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">GENDER</text>\n</g>\n<!-- GENDER->GRADE -->\n<g id=\"edge7\" class=\"edge
287.7C130.05,-279.73 134.45,-270.1 138.49,-261.26\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
262.65 142.68,-252.1 135.34,-259.74 141.71,-262.65\"/>\n</g>\n<!-- GENDER->MATH -->\n<g
287.85C112.06,-277.49 109.01,-264.01 106.5,-252 102.19,-231.34 97.73,-
207.89 94.46,-190.25\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"97.84,-
189.3 92.59,-180.1 90.96,-190.56 97.84,-189.3\"/>\n</g>\n<!-- MATH->HOMEPOS -->\n<g id=
143.7C82.69,-135.9 80.14,-126.51 77.78,-117.83\"/>\n<polygon fill=\"blue\" stroke=\"blue\" poi
116.84 75.14,-108.1 74.38,-118.67 81.14,-116.84\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">RSL-
D - estimated CPDAG</p><svg width=\"220pt\" height=\"332pt\"\n viewBox=\"0.00 0.00 220.25 332.0
4,4 -4,-328 216.25,-328 216.25,4 -4,4\"/>\n<!-- HISEI -->\n<g id=\"node1\" class=\"node\">\n<ti
anchor=\"middle\" x=\"52.25\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HISEI</text>\n</g>\n<!-- HOMEPOS -->\n<g id=\"node2\" class=\"node\">\n<title>HO
anchor=\"middle\" x=\"52.25\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HOMEPOS</text>\n</g>\n<!-- HOMEPOS->HISEI -->\n<g id=\"edge1\" class=\"ed
287.86C23.68,-248.63 -18.31,-149.73 9.25,-72 12.92,-61.68 19.54,-51.9 26.51,-
43.61\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"29.13,-45.94 33.21,-
36.16 23.92,-41.26 29.13,-45.94\"/>\n</g>\n<!-- IMMIG -->\n<g id=\"node3\" class=\"node\">\n<ti
anchor=\"middle\" x=\"52.25\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">IMMIG</text>\n</g>\n<!-- HOMEPOS->IMMIG -->\n<g id=\"edge2\" class=\"edge
287.85C52.25,-250.83 52.25,-163.18 52.25,-118.39\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
118.23 52.25,-108.23 48.75,-118.23 55.75,-118.23\"/>\n</g>\n<!-- MATH -->\n<g id=\"node6\" clas
anchor=\"middle\" x=\"115.25\" y=\"-230.3\" font-family=\"Times,serif\" font-

> **!** Important
>
> **Read the disagreement, not just the agreement.** A purely data-driven method may orient an edge as `MATH → HISEI` — a child's test score causing their parent's occupational status. That is causally backwards, and the data cannot protect us: which direction this edge takes is not identifiable from observational data alone, so any orientation the algorithm prints reflects its (possibly mistaken) v-structure decisions under misspecification, not evidence about the true direction. This is exactly why background knowledge is not optional.

24.8.3. Orienting with background knowledge

Sex and immigration background are fixed at birth; family socioeconomic status is established long before the test; grade precedes the assessment; the score is the outcome. That gives a **tier ordering**

$$\{\text{GENDER}, \text{IMMIG}\} \prec \{\text{HISEI}, \text{HOMEPOS}\} \prec \text{GRADE} \prec \text{MATH}. \tag{24.5}$$

A cross-tier edge must point earlier→later; a within-tier edge the ordering cannot decide. We impose this on the discovered **skeleton**, because the ordering is knowledge we hold independently of how the data oriented things.

```
tier_p = Dict("GENDER"=>0,"IMMIG"=>0,"HISEI"=>1,"HOMEPOS"=>1,"GRADE"=>2,"MATH"=>3)
di_e = Tuple{Symbol,Symbol}[]; bi_e = Tuple{Symbol,Symbol}[]
for e in edges(skel_pcp)
    a, b = plabs[e.src], plabs[e.dst]
    if tier_p[a] == tier_p[b]
        push!(bi_e, (Symbol(a), Symbol(b)))          # within-tier: undirected
    else
        lo, hi = tier_p[a] < tier_p[b] ? (a, b) : (b, a)
        push!(di_e, (Symbol(lo), Symbol(hi)))        # cross-tier: earlier → later
    end
end
oriented_admg = make_graph(vertices=Symbol.(plabs), di_edges=di_e, bi_edges=bi_e)
```

```
ADMG([:HISEI, :HOMEPOS, :IMMIG, :GRADE, :GENDER, :MATH], [(:IMMIG, :HISEI), (:HISEI, :MATH), (
```

> **⚠** Warning
>
> **Where data-driven orientation disagreed with us.** Left to itself, an algorithm may orient `IMMIG-GRADE` as `GRADE → IMMIG` — schooling causing immigration status, which is impossible. Such mistakes come from v-structure and propagation decisions made under misspecification; the data is not informative about these directions. Imposing the tier order overrides them. The lesson: **never ship a discovered orientation you have prior reason to disbelieve**.

```
draw_graph(oriented_admg; direction="LR")
```

```
[SVG graph - display in an HTML environment]
```

Figure 24.5.: PC skeleton oriented by the temporal/logical tiers. Cross-tier edges point earlier-to-later; HISEI–HOMEPOS stays undirected (same tier, shown red).

Every edge into `MATH` is now a candidate causal parent of achievement — the bridge from discovery to estimation (back-door adjustment, e.g. on the parents of the chosen treatment variable). The only edge the ordering leaves undirected, `HISEI - HOMEPOS`, is two same-tier measures of family background that temporal logic cannot separate.

24.8.4. Stability: does the skeleton survive resampling?

A single fit can be lucky. We resample students with replacement, re-estimate the **weighted** PC skeleton, and record how often each edge appears.

```
function weighted_skel(Xb, wb, labs, sig)
    mu = (Xb' * wb) ./ sum(wb); Xc = Xb .- mu'
    Σ  = (Xc' * (wb .* Xc)) ./ sum(wb)
    m  = round(Int, sum(wb)^2 / sum(wb .^ 2))
    Z  = randn(m, length(labs)); Z .-= mean(Z, dims=1)
    Z  = Z * inv(cholesky(Symmetric(cov(Z))).U)
    Xs = Z * cholesky(Symmetric(Σ)).U .+ mu'
    ci(x, y, S, d) = fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig)
    SimpleGraph(pcalg(length(labs), ci, Xs))
end

B_pisa = 200
adj_b  = Vector{Matrix{Float64}}(undef, B_pisa)   # one slot per replicate (race-free)
Threads.@threads for b in 1:B_pisa
    rng = MersenneTwister(1000 + b)
    idx = rand(rng, 1:np_, np_)
    g   = weighted_skel(Xp[idx, :], wp[idx], plabs, sig_p)
    A   = zeros(pp_, pp_)
    for e in edges(g); A[e.src, e.dst] += 1; A[e.dst, e.src] += 1; end
    adj_b[b] = A
end
edge_freq = sum(adj_b) ./ B_pisa
```

```
6×6 Matrix{Float64}:
 0.0    1.0   1.0    0.015  0.0   1.0
 1.0    0.0   1.0    0.0    0.17  1.0
 1.0    1.0   0.0    0.895  0.02  0.0
 0.015  0.0   0.895  0.0    0.97  1.0
```

```
 0.0    0.17  0.02   0.97   0.0   0.98
 1.0    1.0   0.0    1.0    0.98  0.0
```

```
edge_names = String[]; edge_vals = Float64[]
for i in 1:pp_-1, j in i+1:pp_
    if edge_freq[i, j] > 0
        push!(edge_names, "$(plabs[i]) - $(plabs[j])")
        push!(edge_vals,  edge_freq[i, j])
    end
end
ord = sortperm(edge_vals)
let
    fig = Figure(size=(680, 380))
    ax  = Axis(fig[1,1]; xlabel="Selection frequency across bootstrap resamples",
               yticks=(1:length(ord), edge_names[ord]), title="Bootstrap edge stability (weight
    barplot!(ax, 1:length(ord), edge_vals[ord]; direction=:x, color=(:steelblue, 0.8))
    vlines!(ax, [0.5]; linestyle=:dash, color=:gray)
    xlims!(ax, 0, 1)
    fig
end
```

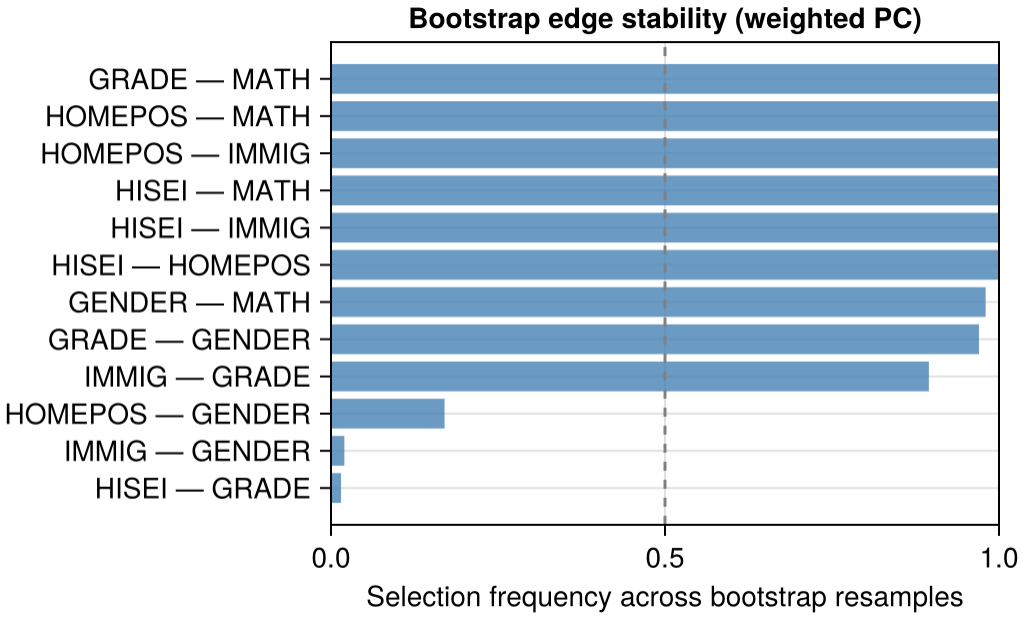


Figure 24.6.: Bootstrap edge-selection frequency (weighted PC, 200 resamples). Edges near 1.0 are robust; near 0.5 are sample-dependent.

The family-background→math edges (`HISEI`, `HOMEPOS`) are the most stable features: the SES gradient in achievement is not an artefact. On real data, causal discovery is best read as **hypothesis generation** — it proposes a skeleton and the orientations the data support, which you then reconcile with subject-matter knowledge before estimating effects.

24.9. Summary

Both PC and RSL-D target the Markov equivalence class. The practical difference is computational: RSL-D usually uses fewer CI tests, especially as the graph gets denser.

From CPDAG to causal estimation: Once a CPDAG is in hand, the workflow continues as follows:

1. **Use background knowledge to orient undirected edges.** Temporal ordering is the most common source: if X is measured before Y, the edge must be $X \to Y$. Domain expertise, experimental context, or exclusion restrictions can orient others.
2. **Check identifiability.** With a fully oriented DAG, apply the backdoor, front-door, or ID algorithm (Chapter 3) to determine whether the effect of interest is identifiable.
3. **Estimate.** Use the identified functional with the estimators from Chapter 4 (RA, IPW, AIPW).

If some edges remain undirected after exhausting background knowledge, the bound over the equivalence class is not automatic. Something has to enumerate what the class permits. For total effects under a linear Gaussian model that procedure is IDA (intervention calculus when the DAG is absent): walk the DAGs consistent with the CPDAG, compute the effect in each, and report the range of the resulting multiset. `CausalInference.jl` does not implement it – it gives you adjustment-set search (`find_backdoor_adjustment`, `list_covariate_adjustment` and relatives) rather than effect enumeration – so in Julia you either enumerate the consistent DAGs yourself with `Graphs.jl` and apply the identified functional to each, or call R's `pcalg::ida` through `RCall.jl`. The alternative is to collect interventional data and break the remaining equivalences.

> **Note**
>
> **What cannot be recovered from observational data alone**
> Both algorithms identify the CPDAG, not necessarily the true DAG. Some edge directions require extra assumptions, temporal ordering, or interventional data.
> The next chapter addresses a harder problem: recovering the causal skeleton when some variables are *unobserved*.

25. Causal Discovery: Latent Variables

```
using CausalInference
using RecursiveCausalDiscovery
using CausalGraphs
using Graphs
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using LinearAlgebra
using Random
using Printf
using DataFrames
using Combinatorics: powerset
using Distributions: Normal, quantile
```

The previous chapter assumed all causally relevant variables were observed. That is a strong assumption. In economics, ability, demand shocks, and peer effects are often unobserved.

When latent confounders exist, PC and RSL-D can give the wrong graph. Here I use algorithms designed for latent variables.

25.0.1. Why PC fails with latent variables

Suppose the true graph is $X \leftarrow U \rightarrow Y$ where U is unobserved. In the observed data, X and Y are marginally correlated (through U) and no observed conditioning set separates them. PC will incorrectly draw an edge $X - Y$ and may orient it as $X \rightarrow Y$ or $X \leftarrow Y$, neither of which exists in the truth.

With even one hidden common cause, the conditional independences among observed variables may not correspond to any DAG on the observed variables alone. We need a MAG/PAG representation.

25.1. Maximal Ancestral Graphs and PAGs

With latent variables, the appropriate representation is a **Maximal Ancestral Graph (MAG)**. Over a set of *observed* variables $\mathbf{O}$, a MAG encodes:

- $X \rightarrow Y$: X is an ancestor of Y in the full DAG
- $X \leftrightarrow Y$: X and Y have a common hidden ancestor (hidden confounder)
- No edge: $X \perp\!\!\!\perp Y \mid Z$ for some $Z \subseteq \mathbf{O} \setminus \{X, Y\}$

The MAG over observed variables summarizes the causal and confounding relationships visible in the observed data without naming the latent variables.

Just as DAGs have Markov equivalence classes represented by CPDAGs, MAGs have equivalence classes represented by **Partial Ancestral Graphs (PAGs)**. In a PAG, the mark $\circ$ on an edge endpoint means "could be arrowhead or tail in some member of the equivalence class."

PAG mark	Meaning	Economic interpretation
$X \to Y$	X causes Y in every equivalent MAG	Robust causal direction
$X \circ\!\!\to Y$	Some MAGs have $X \to Y$, others $X \leftrightarrow Y$ (the arrowhead at Y is invariant)	Direction uncertain
$X \leftrightarrow Y$	Hidden common cause in every equivalent MAG	Definite unmeasured confounder
$X \circ\!\!\circ Y$	Could be $\to$, $\leftarrow$, or $\leftrightarrow$	Maximally uncertain

Reading a PAG for policy purposes:

- A definite $X \to Y$ edge is the most useful finding: it means any intervention on X propagates to Y regardless of which specific MAG the data came from.
- A definite $X \leftrightarrow Y$ edge is a warning: before using regression of Y on X to estimate a causal effect, you must address the hidden confounder — through an instrument, a proxy, or a natural experiment.
- $\circ$ marks indicate what you *don't* know. Additional data, temporal ordering, or experimental variation can resolve them.

25.2. Simulation with Hidden Variables

We reuse the same 8-node Gaussian linear DAG from the previous chapter and hide 2 nodes, treating them as unobserved confounders.

```
Random.seed!(2025)

n_vars    = 8
n_samples = 2000
edge_prob = 0.35
latent    = [3, 6]
obs_idx   = setdiff(1:n_vars, latent)
all_labels = ["X$i" for i in 1:n_vars]
obs_labels = ["X$(obs_idx[i])" for i in 1:length(obs_idx)]

adj_mat   = gen_er_dag_adj_mat(n_vars, edge_prob)
data_full = gen_gaussian_data(adj_mat, n_samples)
data_obs  = data_full[:, obs_idx]
n_obs     = length(obs_idx)
```

```
@printf "Total variables: %d   Hidden: %s   Observed: %d\n" n_vars latent n_obs
```

```
Total variables: 8   Hidden: [3, 6]   Observed: 6
```

25.2.1. True structure over observed variables

Two observed variables are adjacent in the MAG iff no subset of the *observed* variables d-separates them in the full DAG.

```
true_dag = SimpleDiGraph(adj_mat)

function true_mag_skeleton(dag, observed)
    p = length(observed)
    skel = SimpleGraph(p)
    for i in 1:(p-1), j in (i+1):p
        u, v = observed[i], observed[j]
        other_obs = setdiff(observed, [u, v])
        separated = any(dsep(dag, u, v, S) for S in powerset(other_obs))
        !separated && add_edge!(skel, i, j)
    end
    return skel
end

true_obs_skel = true_mag_skeleton(true_dag, obs_idx)
@printf "True MAG skeleton edges (over %d observed nodes): %d\n" n_obs ne(true_obs_skel)
```

```
True MAG skeleton edges (over 6 observed nodes): 5
```

```
side_by_side([
    (admg_with_latent(adj_mat, all_labels, latent), "Full DAG (square = hidden)"),
    (skeleton_to_admg(true_obs_skel, obs_labels),   "True MAG skeleton (observed)"),
]; note="Square nodes are unobserved (latent). Plaintext nodes are observed.")
```

25.3. FCI Algorithm

FCI is the extension of PC for latent variables. It starts from a complete graph, removes edges with CI tests, and then uses orientation rules that can produce bidirected edges for hidden common causes.

How FCI extends PC:

1. **Phase 1 (skeleton):** Same as PC — remove edges via CI tests with growing conditioning sets.

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">Full DAG (square = hidden)<
4,4 -4,-256 217,-256 217,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"18\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"41\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X2->X1 -->\n<g id=\"edge1\" class=\"edge\">\n<title
215.67C23.63,-205.69 17.04,-192.65 14,-180 2.98,-134.08 8.24,-78.67 13.1,-
46.28\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"16.58,-46.65 14.71,-
36.22 9.67,-45.55 16.58,-46.65\"/>\n</g>\n<!-- X3 -->\n<g id=\"node3\" class=\"node\">\n<title>
180 77,-180 77,-144 113,-144 113,-180\"/>\n<text text-anchor=\"middle\" x=\"95\" y=\"-
158.3\" font-family=\"Times,serif\" font-size=\"14.00\">X3</text>\n</g>\n<!-- X2->X3 -->
215.7C60.76,-207.39 68.56,-197.28 75.61,-188.14\"/>\n<polygon fill=\"blue\" stroke=\"blue\" poi
190.16 81.81,-180.1 72.93,-185.88 78.47,-190.16\"/>\n</g>\n<!-- X5 -->\n<g id=\"node5\" class=\
anchor=\"middle\" x=\"41\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge5\" class=\"edge\">\n<title
215.7C41,-207.98 41,-198.71 41,-190.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
190.1 41,-180.1 37.5,-190.1 44.5,-190.1\"/>\n</g>\n<!-- X6 -->\n<g id=\"node6\" class=\"node\">
108 119,-108 119,-72 155,-72 155,-108\"/>\n<text text-anchor=\"middle\" x=\"137\" y=\"-
86.3\" font-family=\"Times,serif\" font-size=\"14.00\">X6</text>\n</g>\n<!-- X3->X6 -->
143.7C110.26,-135.56 116.19,-125.69 121.58,-116.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
118.48 126.74,-108.1 118.59,-114.88 124.59,-118.48\"/>\n</g>\n<!-- X4 -->\n<g id=\"node4\" clas
anchor=\"middle\" x=\"195\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X4</text>\n</g>\n<!-- X7 -->\n<g id=\"node7\" class=\"node\">\n<title>X7</title>
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X5->X7 -->\n<g id=\"edge8\" class=\"edge\">\n<title
143.7C41,-135.98 41,-126.71 41,-118.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
118.1 41,-108.1 37.5,-118.1 44.5,-118.1\"/>\n</g>\n<!-- X7->X1 -->\n<g id=\"edge2\" clas
71.7C32.75,-63.9 29.67,-54.51 26.82,-45.83\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\
44.51 23.62,-36.1 23.42,-46.7 30.07,-44.51\"/>\n</g>\n<!-- X8 -->\n<g id=\"node8\" class=\"node
anchor=\"middle\" x=\"141\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X8</text>\n</g>\n<!-- X8->X3 -->\n<g id=\"edge4\" class=\"edge\">\n<title
215.7C124.28,-207.56 117.8,-197.69 111.89,-188.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
186.54 106.24,-180.1 108.81,-190.38 114.66,-186.54\"/>\n</g>\n<!-- X8->X6 -->\n<g id=\"e
215.56C149.93,-205.33 153.43,-192.08 155,-180 157.06,-164.13 157.61,-159.79 155,-
144 153.56,-135.28 150.87,-126.06 148,-117.8\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points
116.46 144.48,-108.3 144.67,-118.89 151.23,-116.46\"/>\n</g>\n<!-- X8->X7 -->\n<g id=\"e
215.76C140.41,-196.63 136.83,-165.63 122,-144 108.93,-124.94 86.53,-
111.06 68.59,-102.3\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"69.72,-
98.96 59.17,-97.96 66.79,-105.32 69.72,-98.96\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">True MAG skeleton (observed
4,4 -4,-328 117,-328 117,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"41\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"18\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X1->X2 -->\n<g id=\"edge1\" class=\"edge\">\n<title
277.95C30.37,-272.65 28.55,-267.1 26.81,-261.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
279.29 35.31,-287.7 35.52,-277.1 28.87,-279.29\"/>\n<polygon fill=\"red\" stroke=\"red\" points
260.51 23.62,-252.1 23.42,-262.7 30.07,-260.51\"/>\n</g>\n<!-- X7 -->\n<g id=\"node5\" class=\
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge2\" class=\"edge\">\n<title
277.62C44.09,-269.39 44.69,-260.33 45,-252 46.81,-204.03 46.81,-191.97 45,-

2. **Phase 2 (initial orientation):** Mark all edge endpoints as ∘ (uncertain); orient unshielded colliders as in PC.
3. **Phase 3 (Possible-D-SEP pruning):** For each remaining edge $X - Y$, compute the **Possible-D-SEP** sets and run *additional* CI tests conditioning on their subsets, removing further edges. With latent variables, two non-adjacent observed nodes need not be separable by a subset of their neighbors — this stage is what makes FCI's skeleton differ from PC's, and it is the computationally expensive part. Colliders are then re-oriented on the pruned skeleton.
4. **Phase 4 (rule propagation):** Apply the 10 orientation rules (Zhang 2008) that propagate known marks without creating contradictions, including rules that can produce definite → and ↔ edges. These rules perform no CI tests and are cheap.

The extra marks let FCI say what is known and what remains uncertain, but the output is harder to read than a DAG.

```
sig_level = 0.01

count_fci = Ref(0)
ci_fci = (x, y, S, d) -> begin
    count_fci[] += 1
    fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig_level)
end

pag_fci  = fcialg(n_obs, ci_fci, data_obs)
skel_fci = SimpleGraph(pag_fci)

f1_fci = f1_score(true_obs_skel, skel_fci)
@printf "FCI:     skeleton F1 = %.3f   CI tests = %d\n" f1_fci count_fci[]
```

```
FCI:     skeleton F1 = 0.889   CI tests = 76
```

```
# No `println` here: plot_fci_graph_text prints the edge list as a side effect and
# returns nothing, so wrapping it in println() appends a literal "nothing" to the
# output -- which is what the rendered page used to show.
plot_fci_graph_text(pag_fci, ["X$(obs_idx[i])" for i in 1:n_obs])
```

```
X1<-X2   X1<-X7   X2o-oX5  X5o-oX7
```

Figure 25.2.

> **Note**
>
> **Reading the FCI text output**
> The text representation uses ASCII edge marks:
>
> - `X1 --> X2`: definite cause — X_1 is an ancestor of X_2 in every equivalent MAG (not

necessarily a *direct* effect: the path may run through latent mediators)
- `X1 <-> X2`: definite hidden common cause ($X_1 \leftrightarrow X_2$)
- `X1 o-> X2`: the X_1 endpoint is uncertain; X_2 endpoint is definitely an arrowhead
- `X1 o-o X2`: both endpoints uncertain; could be any of $\rightarrow$, $\leftarrow$, $\leftrightarrow$

In practice for economic research: focus first on `<->` edges — these flag definite confounding and tell you where IV or proxy strategies are needed. Then examine `-->` edges — these are causal claims that survive across all statistically equivalent structures.

When FCI gives wrong answers: FCI assumes the CI tests are perfectly accurate (no finite-sample error). In practice, with small n or many variables, some false CI decisions propagate through the orientation rules. The skeleton quality (F1) is generally more reliable than the orientation quality.

25.4. L-MARVEL Algorithm

L-MARVEL (Latent Markov Boundary Variable Elimination; Mokhtarian et al., JMLR 2025) extends the recursive MARVEL approach to handle latent confounders. Instead of exhaustive CI-test-based Markov boundary discovery, L-MARVEL uses a **Gaussian precision matrix** estimator:

$$\widehat{\mathrm{Mb}}(X) = \left\{ Y : \frac{|\hat{P}_{XY}|}{\sqrt{\hat{P}_{XX}\hat{P}_{YY}}} > \tau \right\} \tag{25.1}$$

where $\hat{P} = \hat{\Sigma}^{-1}$ is the precision matrix and τ is a threshold from the Fisher-Z distribution.

In a Gaussian linear model, a zero in the precision matrix means conditional independence given all other variables. L-MARVEL uses this fact to estimate Markov boundaries with fewer CI tests.

The removability criterion changes for the latent case (Theorem 32). Recall: $\mathrm{Mb}(X)$ is the Markov boundary of X — the minimal conditioning set that screens X off from all other variables (estimated here via the precision matrix above rather than by exhaustive CI tests) — and $\mathrm{Ne}(X)$ is the set of variables directly adjacent to X in the current skeleton. Variable X is removable iff for every $Y \in \mathrm{Mb}(X)$ and $Z \in \mathrm{Ne}(X)$:

$$\underbrace{\exists\, W \subseteq \mathrm{Mb}(X) \setminus \{Y, Z\} : Y \perp\!\!\!\perp Z \mid W}_{\text{Condition 1}} \quad \text{or} \quad \underbrace{\forall\, W \subseteq \mathrm{Mb}(X) \setminus \{Y, Z\} : Y \not\perp\!\!\!\perp Z \mid W \cup \{X\}}_{\text{Condition 2}} \tag{25.2}$$

Intuition for the two conditions:

- **Condition 1** is the same as the observed-variable (RSL-D) removability criterion: Y and Z can be separated by some subset of the Markov boundary.

- **Condition 2** is new and handles latent variables: it captures latent-induced dependencies through the theorem's removable-variable criterion. Loosely, when Y and Z are connected through a hidden common cause, no subset of $\mathrm{Mb}(X)$ separates them, yet conditioning on X changes the dependence. **This is not a generic d-separation rule** — it should not be read as "conditioning on X blocks hidden confounding," which would be false in general MAG/PAG reasoning. It is a theorem-specific condition (Theorem 32) on when X can be removed without losing latent-structure information; the formal statement above, not the intuition, is what the algorithm uses.

Together, the two conditions ensure that removing X does not destroy information about the latent structure connecting its neighbours.

```
count_lm = Ref(0)
ci_lm = (x, y, S, d) -> begin
    count_lm[] += 1
    fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig_level)
end

lm_skel = l_marvel(data_obs, ci_lm)

f1_lm = f1_score(true_obs_skel, lm_skel)
@printf "L-MARVEL: skeleton F1 = %.3f   CI tests = %d\n" f1_lm count_lm[]
```

```
L-MARVEL: skeleton F1 = 1.000   CI tests = 14
```

> **Note**
>
> **L-MARVEL output: skeleton only**
> Unlike FCI, L-MARVEL returns only the **skeleton** — undirected edges with no arrowhead or bidirected marks. This is a deliberate design choice: the recursive removal procedure identifies *adjacencies* efficiently but does not run the full PAG orientation phase.
> This is still valuable. Knowing which pairs of variables are adjacent tells you which potential causal relationships to investigate further (with instruments, experiments, or domain knowledge). Variables with no edge are conditionally independent given some observed set — no causal relationship need be posited between them.
> If you need orientations as well as skeleton, run FCI. If you only need to screen out non-adjacent pairs (the most common use case in large-p problems), L-MARVEL's precision matrix approach is substantially faster.

25.5. Comparison

```
DataFrame(
    Algorithm   = ["FCI", "L-MARVEL"],
    Output      = ["PAG (orientations + skeleton)", "Skeleton only"],
```

Table 25.2.: Algorithm comparison on the latent-variable scenario (2 hidden nodes, n = 2000)

	Algorithm	Output	Skeleton_F1	CI_Tests
	String	String	Float64	Int64
1	FCI	PAG (orientations + skeleton)	0.889	76
2	L-MARVEL	Skeleton only	1.0	14

```
    Skeleton_F1 = round.([f1_fci, f1_lm], digits=3),
    CI_Tests    = [count_fci[], count_lm[]],
)
```

```
side_by_side([
    (skeleton_to_admg(true_obs_skel, obs_labels), "True MAG skeleton"),
    (skeleton_to_admg(skel_fci,      obs_labels), "FCI - estimated skeleton"),
    (skeleton_to_admg(lm_skel,       obs_labels), "L-MARVEL - estimated skeleton"),
]; note="All edges shown as undirected (skeleton comparison only).")
```

25.6. Monte Carlo Evaluation

```
function evaluate_latent(seed; n_vars=8, n_samples=2000, edge_prob=0.35, n_latent=2, sig=0.01)
    Random.seed!(seed)
    adj       = gen_er_dag_adj_mat(n_vars, edge_prob)
    data_full = gen_gaussian_data(adj, n_samples)
    dag       = SimpleDiGraph(adj)

    latent_vars = sort(randperm(n_vars)[1:n_latent])
    obs_vars    = setdiff(1:n_vars, latent_vars)
    data_obs    = data_full[:, obs_vars]
    n_obs       = length(obs_vars)

    true_skel = true_mag_skeleton(dag, obs_vars)

    cnt_fci = Ref(0)
    ci_fci  = (x, y, S, d) -> begin cnt_fci[] += 1; fisher_z(x, y, collect(Int, S), Matrix{Floa
    pag     = fcialg(n_obs, ci_fci, data_obs)
    f1f     = f1_score(true_skel, SimpleGraph(pag))

    cnt_lm  = Ref(0)
    ci_lm   = (x, y, S, d) -> begin cnt_lm[] += 1; fisher_z(x, y, collect(Int, S), Matrix{Float
    skel_lm = l_marvel(data_obs, ci_lm)
    f1l     = f1_score(true_skel, skel_lm)

    (f1_fci=f1f, f1_lm=f1l, ci_fci=cnt_fci[], ci_lm=cnt_lm[])
```

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">True MAG skeleton</p><svg w
4,4 -4,-328 117,-328 117,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"41\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"18\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X1->X2 -->\n<g id=\"edge1\" class=\"edge\">\n<title
277.95C30.37,-272.65 28.55,-267.1 26.81,-261.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
279.29 35.31,-287.7 35.52,-277.1 28.87,-279.29\"/>\n<polygon fill=\"red\" stroke=\"red\" points
260.51 23.62,-252.1 23.42,-262.7 30.07,-260.51\"/>\n</g>\n<!-- X7 -->\n<g id=\"node5\" class=\"
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge2\" class=\"edge\">\n<title
277.62C44.09,-269.39 44.69,-260.33 45,-252 46.81,-204.03 46.81,-191.97 45,-
144 44.69,-135.67 44.09,-126.61 43.46,-118.38\"/>\n<polygon fill=\"red\" stroke=\"red\" points=
277.58 42.62,-287.83 46.93,-278.15 39.95,-277.58\"/>\n<polygon fill=\"red\" stroke=\"red\" poin
117.85 42.62,-108.17 39.95,-118.42 46.93,-117.85\"/>\n</g>\n<!-- X5 -->\n<g id=\"node4\" class=
anchor=\"middle\" x=\"18\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge3\" class=\"edge\">\n<title
205.67C18,-200.69 18,-195.49 18,-190.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"14.5
205.7 18,-215.7 21.5,-205.7 14.5,-205.7\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"21.5
190.1 18,-180.1 14.5,-190.1 21.5,-190.1\"/>\n</g>\n<!-- X4 -->\n<g id=\"node3\" class=\"node\">
anchor=\"middle\" x=\"95\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X4</text>\n</g>\n<!-- X5->X7 -->\n<g id=\"edge4\" class=\"edge\">\n<title
133.95C28.63,-128.65 30.45,-123.1 32.19,-117.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
133.1 23.69,-143.7 30.13,-135.29 23.48,-133.1\"/>\n<polygon fill=\"red\" stroke=\"red\" points=
118.7 35.38,-108.1 28.93,-116.51 35.58,-118.7\"/>\n</g>\n<!-- X8 -->\n<g id=\"node6\" class=\"n
anchor=\"middle\" x=\"41\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X8</text>\n</g>\n<!-- X7->X8 -->\n<g id=\"edge5\" class=\"edge\">\n<title
61.67C41,-56.69 41,-51.49 41,-46.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"37.5,-
61.7 41,-71.7 44.5,-61.7 37.5,-61.7\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"44.5,-
46.1 41,-36.1 37.5,-46.1 44.5,-46.1\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">FCI - estimated skeleton</p
4,4 -4,-256 171,-256 171,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"41\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"18\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X1->X2 -->\n<g id=\"edge1\" class=\"edge\">\n<title
205.95C30.37,-200.65 28.55,-195.1 26.81,-189.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
207.29 35.31,-215.7 35.52,-205.1 28.87,-207.29\"/>\n<polygon fill=\"red\" stroke=\"red\" points
188.51 23.62,-180.1 23.42,-190.7 30.07,-188.51\"/>\n</g>\n<!-- X7 -->\n<g id=\"node5\" class=\"
anchor=\"middle\" x=\"41\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge2\" class=\"edge\">\n<title
205.62C44.09,-197.39 44.69,-188.33 45,-180 46.81,-132.03 46.81,-119.97 45,-
72 44.69,-63.67 44.09,-54.61 43.46,-46.38\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"39
205.58 42.62,-215.83 46.93,-206.15 39.95,-205.58\"/>\n<polygon fill=\"red\" stroke=\"red\" poin
45.85 42.62,-36.17 39.95,-46.42 46.93,-45.85\"/>\n</g>\n<!-- X5 -->\n<g id=\"node4\" class=\"no
anchor=\"middle\" x=\"18\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge3\" class=\"edge\">\n<title
133.67C18,-128.69 18,-123.49 18,-118.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"14.5
133.7 18,-143.7 21.5,-133.7 14.5,-133.7\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"21.5
118.1 18,-108.1 14.5,-118.1 21.5,-118.1\"/>\n</g>\n<!-- X4 -->\n<g id=\"node3\" class=\"node\">
anchor=\"middle\" x=\"95\" y=\"-230.3\" font-family=\"Times,serif\" font-

```
end

mc = [evaluate_latent(s) for s in 1:100]

@printf "\nMonte Carlo averages (100 replications, n=%d, p=%d, %d latent)\n" n_samples n_vars 2
@printf "%-10s  Skeleton F1  CI tests\n" "Method"
@printf "%-10s  %10.3f  %8.1f\n" "FCI"      mean(r.f1_fci for r in mc) mean(r.ci_fci for r in m
@printf "%-10s  %10.3f  %8.1f\n" "L-MARVEL" mean(r.f1_lm  for r in mc) mean(r.ci_lm  for r in m
```

```
Monte Carlo averages (100 replications, n=2000, p=8, 2 latent)
Method      Skeleton F1  CI tests
FCI               0.912     170.5
L-MARVEL          0.934      80.9
```

```
let
    fig = Figure(size=(650, 380))
    ax  = Axis(fig[1,1];
        xlabel = "CI tests",
        ylabel = "Frequency",
        title  = "CI Test Counts: FCI vs L-MARVEL (100 replications)")
    hist!(ax, [r.ci_fci for r in mc]; bins=20, color=(:steelblue, 0.65), label="FCI")
    hist!(ax, [r.ci_lm  for r in mc]; bins=20, color=(:tomato, 0.65),    label="L-MARVEL")
    axislegend(ax)
    fig
end
```

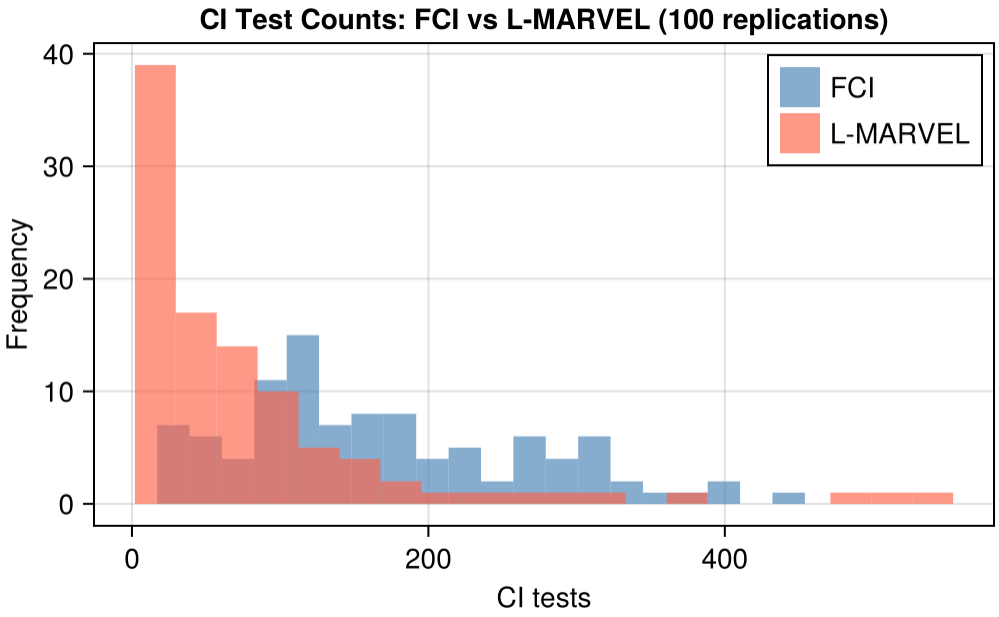


Figure 25.4.: CI test count distributions across 100 replications.

25.7. Summary

Property	PC / RSL-D	FCI	L-MARVEL
Latent confounders	assumes none	handles	handles
Output	CPDAG	PAG	Skeleton
Orientations			
MB estimation	CI tests	CI tests	Precision matrix
CI complexity	PC: $O(p^2 2^p)$ worst case; RSL-D: $O(p \cdot d \cdot \lvert\text{MB}\rvert^2)$	Same as PC	Fewer (Gaussian MB pre-filters)

FCI and L-MARVEL serve different purposes. Use FCI when orientations matter. Use L-MARVEL when the goal is a faster skeleton screen.

25.7.1. Practical decision guide

Use PC or RSL-D when: - You are confident in causal sufficiency (all relevant variables are measured) - You want a CPDAG that you can orient further with background knowledge - RSL-D is preferred over PC when $p > 10$ or the graph may be moderately dense

Use FCI when: - You suspect hidden confounders but don't know where - You need edge orientations and $\leftrightarrow$ marks to guide IV or proxy strategies - Sample size is large enough that CI-test errors don't cascade badly through orientation rules

Use L-MARVEL when: - You suspect latent confounders and have many variables ($p > 20$) - The goal is to prune the adjacency graph before applying domain knowledge or estimation - Gaussian data is appropriate (precision matrix estimator requires this)

25.7.2. From discovery to estimation

Causal discovery is a first step, not a final answer. A reasonable workflow is:

1. **Discover** — run FCI or L-MARVEL to get candidate adjacencies and, if using FCI, some edge marks
2. **Refine** — apply temporal ordering, institutional knowledge, and exclusion restrictions to orient remaining edges
3. **Identify** — check whether the effect of interest is identified given the refined graph (backdoor, front-door, ID algorithm)
4. **Estimate** — use TMLE, AIPW, or the estimators in earlier chapters

Causal discovery narrows the space of possible structures. Domain knowledge still does the hard work.

> ⚠ Warning
>
> **Sample size requirements**
> L-MARVEL's Gaussian MB estimator requires $n > p + 1$ samples. With many variables and limited data, fall back to CI-test-based MB estimation by passing a custom `mb_estimator` to `l_marvel`.

> ℹ Note
>
> **Going further: orientation in the latent case**
> FCI's PAG output distinguishes definite causes ($\rightarrow$), definite hidden common causes ($\leftrightarrow$), and uncertain cases ($\circ$). The PAG can be combined with background knowledge such as temporal ordering to further orient edges before estimation.

25.8. How Much Does Causal Discovery Help in Economics?

Recovering the MAG skeleton is still far from recovering the full DAG. Much is still missing:

What is recovered	What is still missing
L-MARVEL skeleton	Edge directions; direct cause vs. hidden common cause; latent nodes
FCI PAG	Unique MAG; latent nodes; most edges still carry $\circ$ marks
True MAG	Latent nodes and their connections
Full DAG	Nothing — this is the goal

The core econometric question is almost always "what is the causal effect of X on Y?" Causal discovery with latent variables does not answer this directly. A definite $X \leftrightarrow Y$ in a PAG tells you confounding exists — but not how to remove it. Without oriented edges you cannot even check whether the effect is identified, let alone estimate it.

Where these methods do add value in economics:

1. **Falsifying structural models.** If your model implies $X \perp\!\!\!\perp Z \mid W$, test it. A rejection means the model's independence assumptions are inconsistent with the data — discovery tells you *which* assumptions fail before you commit to a full structural estimation.
2. **Flagging where instruments are needed.** A definite $X \leftrightarrow Y$ in the PAG is a data-driven diagnostic: OLS of Y on X is biased, and you need an instrument, proxy, or natural experiment. Discovery does not supply the instrument but tells you where to look for one.
3. **High-dimensional variable selection.** With many candidate controls, knowing which pairs are conditionally independent prunes the problem before applying identification strategies. This is most useful in settings with $p > 20$ variables where theory does not specify the full graph.

4. **Hypothesis generation when theory is silent.** For new economic phenomena — platform markets, fintech, peer effects in novel settings — where theory does not give a strong causal ordering, discovery algorithms generate hypotheses worth investigating with better-powered research designs.

Causal discovery is a complement to IV, RD, DiD, and synthetic control, not a substitute. It is most useful early in a project, as a diagnostic and hypothesis-generating tool.

26. From Graph to Estimate

```
using CausalGraphs
using DataFrames
using Random
using Statistics
using CairoMakie
CairoMakie.activate!(type = "png")
```

The previous chapters covered discovery: algorithms that learn a graph from data. But a graph is not an estimate. After we have a graph, we still need to ask two questions:

1. **Identification**: is the target effect a function of the observed-data distribution?
2. **Estimation**: once the identifying formula is known, estimate it from the data.

`CausalGraphs.jl` implements both steps for **acyclic directed mixed graphs (ADMGs)** — directed graphs that may also have bidirected edges representing unmeasured common causes. ADMGs are the natural output of discovery algorithms like FCI when there are latent confounders. They are also the natural way to encode researcher hypotheses about which arrows are causal and which open paths represent hidden confounding.

Here are three cases: backdoor identification, front-door identification, and non-identification.

26.1. Backdoor: a-fixable effects

Start with the simple case. X is a common cause of treatment A and outcome Y, and X is observed. Backdoor adjustment applies. In the package terminology this is `a_fixable`.

```
# Build the graph: X is a common cause of A and Y
g_backdoor = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)

id_backdoor = identify(g_backdoor, :A, :Y)
@printf("Identification strategy: %s\n", id_backdoor.strategy)
```

```
Identification strategy: a_fixable
```

The package recognises the backdoor structure: `X` blocks the only open backdoor path from A to Y, so adjusting on X identifies the ACE — the average causal effect, which is the ATE of the earlier chapters under a name `CausalEstimate.jl` uses for the field it returns.

```
Random.seed!(1)
n = 2000
X = randn(n)
A = Float64.(rand(n) .< 1 ./ (1 .+ exp.(-X)))      # treatment depends on X
Y = 2 .* A .+ X .+ 0.5 .* randn(n)                  # true ACE = 2
data_bd = DataFrame(X = X, A = A, Y = Y)

result_bd = estimate_causal(
    a = [1, 0],
    data      = data_bd,
    graph     = g_backdoor,
    treatment = :A,
    outcome   = :Y,
)

@printf("\nBackdoor estimation (true ACE = 2.0):\n")
@printf("  TMLE  ACE: %.3f  [95%% CI: %.3f, %.3f]\n",
        result_bd[:TMLE].ACE, result_bd[:TMLE].lower_ci, result_bd[:TMLE].upper_ci)
@printf("  AIPW  ACE: %.3f  [95%% CI: %.3f, %.3f]\n",
        result_bd[:Onestep].ACE, result_bd[:Onestep].lower_ci, result_bd[:Onestep].upper_ci)
@printf("  IPW   ACE: %.3f\n", result_bd[:IPW].ACE)
@printf("  GCOMP ACE: %.3f\n", result_bd[:Gcomp].ACE)
```

```
Backdoor estimation (true ACE = 2.0):
  TMLE  ACE: 2.018  [95% CI: 1.972, 2.065]
  AIPW  ACE: 2.018  [95% CI: 1.972, 2.065]
  IPW   ACE: 2.041
  GCOMP ACE: 2.018
```

All four estimators recover the true ACE in this simple example. TMLE and AIPW also return confidence intervals.

26.2. Front-door: p-fixable effects

Now suppose A and Y have an unmeasured common cause U, but there is a measured mediator M on the path $A \to M \to Y$. Backdoor adjustment fails because U is unobserved. The front-door formula can still identify the effect.

In ADMG notation, the unmeasured confounder appears as a bidirected edge between A and Y.

```
# A → M → Y, with unmeasured confounding between A and Y (bidirected edge)
g_frontdoor = make_graph(
    vertices = [:A, :M, :Y],
    di_edges = [(:A, :M), (:M, :Y)],
    bi_edges = [(:A, :Y)],
)

id_frontdoor = identify(g_frontdoor, :A, :Y)
@printf("Identification strategy: %s\n", id_frontdoor.strategy)
```

```
Identification strategy: p_fixable
```

The package returns `p_fixable`, recognizing the front-door structure. This uses a different functional than backdoor adjustment and must model M too.

```
Random.seed!(2)
n = 3000
U = randn(n)                                          # unmeasured confounder
A = Float64.(rand(n) .< 1 ./ (1 .+ exp.(-U)))         # A depends on U
M = Float64.(rand(n) .< 1 ./ (1 .+ exp.(-(0.5 .+ 2.0 .* A))))   # M depends on A only
Y = 1.5 .* M .+ 2.0 .* U .+ 0.5 .* randn(n)           # Y depends on M and U
data_fd = DataFrame(A = A, M = M, Y = Y)

# True ACE: effect of setting A=1 vs A=0
# E[Y|do(A=1)] - E[Y|do(A=0)] = 1.5 * (E[M|A=1] - E[M|A=0])
true_M_a1 = 1 / (1 + exp(-(0.5 + 2.0)))   # scalar P(M=1 | A=1)
true_M_a0 = 1 / (1 + exp(-0.5))           # scalar P(M=1 | A=0)
true_ace_fd = 1.5 * (true_M_a1 - true_M_a0)
@printf("True front-door ACE = %.3f\n\n", true_ace_fd)

result_fd = estimate_causal(
    a = [1, 0],
    data      = data_fd,
    graph     = g_frontdoor,
    treatment = :A,
    outcome   = :Y,
)

@printf("Front-door estimation:\n")
@printf("  TMLE  ACE: %.3f  [95%% CI: %.3f, %.3f]\n",
        result_fd[:TMLE].ACE, result_fd[:TMLE].lower_ci, result_fd[:TMLE].upper_ci)
```

```
True front-door ACE = 0.453

Front-door estimation:
  TMLE  ACE: 0.484  [95% CI: 0.401, 0.567]
```

The front-door functional pins down the effect even though U is unobserved. A naive mean difference is badly biased:

```
naive_ace = mean(Y[A .== 1]) - mean(Y[A .== 0])
@printf("Naïve mean difference (Y|A=1) - (Y|A=0): %.3f  (biased!)\n", naive_ace)
@printf("Front-door TMLE estimate:                 %.3f\n", result_fd[:TMLE].ACE)
@printf("True ACE:                                 %.3f\n", true_ace_fd)
```

```
Naïve mean difference (Y|A=1) - (Y|A=0): 2.132  (biased!)
Front-door TMLE estimate:                 0.484
True ACE:                                 0.453
```

The naive difference is biased by the path through U.

26.3. When identification fails

Not every graph identifies the effect. If A and Y have a hidden common cause and there is no front-door variable, the effect is not identified.

```
# A → Y with unmeasured confounding and no mediator
g_unident = make_graph(
    vertices = [:A, :Y],
    di_edges = [(:A, :Y)],
    bi_edges = [(:A, :Y)],
)

try
    id_unident = identify(g_unident, :A, :Y)
    @printf("Strategy: %s\n", id_unident.strategy)
catch e
    @printf("Identification failed: %s\n", sprint(showerror, e))
end
```

```
Strategy: not_identified
```

The package returns `not_identified` (or raises an informative error, depending on the version). The honest response in applied work is to report the non-identifiability, then either:

1. Find additional variables that close the unmeasured-confounding paths
2. Apply sensitivity analysis to bound the effect under maintained assumptions
3. Find an instrumental variable for a LATE-style identification (covered in the IV chapter)

Saying that the hidden confounding is probably small is sensitivity analysis, not identification.

26.4. End-to-end: discover, identify, estimate

The previous chapters' discovery algorithms (PC, FCI, RSL-D, L-MARVEL) return equivalence classes, not single graphs. A CPDAG's undirected edge means the orientation is unresolved, not that a hidden confounder is present, and a PAG is a set of compatible MAGs, not one ADMG. Before `identify()` can run we must either identify the effect in every graph of the class, or state the substantive restrictions that pick one graph. The sketch below shows where that choice sits:

```
using CausalInference   # for PC algorithm

# Step 1: discover the graph (chapter 11)
graph_pc = pcalg(data, 0.05, gausscitest)

# Step 2: PC returns a CPDAG. Choose one member DAG using stated background
# knowledge (or loop over every member), and build graph_admg from that choice.
# Treating undirected edges as bidirected would add confounding assumptions.

# Step 3: identify and estimate
id  = identify(graph_admg, :A, :Y)
result = estimate_causal(
    a = [1, 0],
    data = data,
    graph = graph_admg,
    treatment = :A,
    outcome = :Y,
)
```

In applied work, I would not trust the discovered graph blindly. A more reasonable workflow is:

1. **Hypothesise** a graph based on subject-matter knowledge
2. **Discover** a data-driven graph using PC/FCI/RSL-D
3. **Compare** the two — disagreement points to substantive questions
4. **Identify** under both graphs separately
5. **Estimate** under both, and report the range as part of the result

If the estimate changes a lot across plausible graphs, the data alone is not settling the question.

26.5. What changes after identification

Backdoor adjustment is not always available because many realistic DAGs have hidden confounders. Front-door, nested, and general ID algorithms may still identify the effect, although their formulas quickly become difficult to derive by hand.

The identification result also determines the estimator. TMLE for a backdoor adjustment is different from TMLE for a front-door formula. Automating this choice helps us avoid applying an estimator that does not match the graph. It also forces us to state which variables support an unconfoundedness claim instead of leaving the claim implicit.

26.6. Summary

- A graph is not an estimate. Identification is the bridge.
- `CausalGraphs.jl` routes ADMGs to backdoor, front-door, nested, or non-identified cases.
- `estimate_causal()` then runs the estimator that matches the identification strategy.
- In applied work, hypothesize a graph, use discovery as a check, identify, estimate, and report sensitivity to plausible graph choices.

Part VIII.

Appendix

27. Julia Packages Used in This Book

This appendix lists the Julia packages used in the book. Several were written alongside the examples and live under `~/projects/software`. They fill the gaps needed for the workflows here: fixed effects, semiparametric causal estimators, staggered DiD, synthetic control, TASC, regression discontinuity, SEM, and causal mediation.

The packages are regular Julia projects, and the manifest reaches them by path rather than through the registry. Three entries are relative – `Panelest`, `RDRobust` and `SynthDiD` resolve through `../../software/`. The other nine are absolute paths under `/home/xao/projects`, seven in `software/` and two, `CausalInference` and `RecursiveCausalDiscovery`, in `repo_cloned/` where they are local clones of registered packages. The manifest is therefore machine-specific: `Pkg.instantiate()` on another machine will fail until those paths exist or are re-pointed.

`LinearAlgebra`, `Random`, `Statistics`, `Printf`, `DelimitedFiles`, `Downloads` and `Logging` are Julia standard libraries. They sit on the default load path, so they need no `Project.toml` entry and no installation.

27.1. Working With The Environment

The reproducible workflow is:

```
cd ~/projects/books/causal_econometrics_julia
julia --project -e 'using Pkg; Pkg.instantiate()'
julia --project -e 'using CausalGraphs, CausalEstimate, Panelest, DiD, SynthDiD, RDRobust, Lava
quarto render
```

The standard `julia-1.12` Jupyter kernel uses `--project=@.`, so Quarto picks up the book environment when rendering from this directory. A custom sysimage can still be useful for speed, but it is optional and should be rebuilt whenever the local packages change.

27.2. Package Map

Package	Used For	Main Book Chapters
`Panelest.jl`	Fixed-effects OLS, GLM-style panel models, clustered covariance, IV/2SLS (`feiv`), and Wooldridge's ETWFE (`etwfe()`, `emfx()`)	Estimation, DiD, IV/RDD
`CausalGraphs.jl`	DAG/ADMG construction, graph-based identification, Pearl-Shpitser ID, and estimator routing	Identification, DAGs/ADMGs, Smoking cessation
`CausalEstimate.jl`	Unified TMLE and AIPW estimation: `estimate(ATE/ATT(...), TMLE/AIPW(...), df)` and graph-implied backdoor via `GraphID`	Estimation, Nonparametric, Smoking cessation
`CausalInference.jl` (local clone)	PC and FCI algorithms, Fisher-Z CI tests	Causal Discovery (both chapters)
`RecursiveCausalDiscovery.jl` (local clone)	RSL-D and L-MARVEL recursive discovery, F1 scoring	Causal Discovery (both chapters)
`SynthDiD.jl`	Synthetic control, standard DiD, and synthetic DiD estimators	DiD
`TASC.jl`	Time-Aware Synthetic Control with Kalman filtering, RTS smoothing, EM fitting, SC/RSC utilities, and TASC plot recipes	Synthetic control, DiD, and TASC
`RDRobust.jl`	Local-polynomial RD estimation, bandwidth selection, RD plot data, and fuzzy RD	IV/RDD
`Lavaan.jl`	SEM, CFA, defined parameters, fit measures, and lavaan-style mediation syntax	Mediation
`Crumble.jl`	Flexible causal mediation using cross-fitting, Riesz representers, and neural-network nuisance estimation	Mediation
`ShiftShareIV.jl`	Bartik/shift-share IV, Rotemberg weights, BHJ shock-level collapse	Shift-share IV
`DidInterference.jl`	DiD with spatial interference between units (`eval: false`)	DiD
`Endid.jl`	Distributional difference-in-differences (`eval: false`)	Distributional effects

Package	Used For	Main Book Chapters
`TVHTE.jl`	Time-varying heterogeneous event-study effects (`eval: false`)	DiD
`CairoMakie.jl` (registry)	Every figure in the book, via Makie's Cairo backend	Almost every chapter
`GraphMakie.jl` (registry)	Graph layouts for DAGs and mediation diagrams	Identification, IV/RDD, Mediation
`GeometryBasics.jl` (registry)	Geometric primitives for the identification figure	Identification
`CSV.jl` (registry)	Reading the example data files	Estimation, DiD, IV/RDD, Shift-share IV, Survival, Causal Discovery
`ReadStatTables.jl` (registry)	Reading Stata `.dta` example data	Estimation, DiD
`DataFramesMeta.jl` (registry)	`@chain`/`@transform` data wrangling	DiD, IV/RDD, Mediation, Poisson-IV
`RegressionTables.jl` (registry)	Formatted regression output	Estimation, DiD, IV/RDD
`Distributions.jl` (registry)	Draws and distributional calculations in the DGPs	Most chapters with a simulation
`StatsBase.jl` (registry)	Descriptive statistics and sampling helpers	Poisson-IV
`StatsFuns.jl` (registry)	Log/exp and distribution helper functions	Mediation
`MLJ.jl` (registry)	Machine-learning interface for nuisance estimation	Estimation, Nonparametric, Heterogeneous effects, IV/RDD
`MLJLinearModels.jl` (registry)	Linear and logistic learners for MLJ	Nonparametric
`MLJDecisionTreeInterface.jl` (registry)	Decision-tree learners for MLJ	Heterogeneous effects
`DecisionTree.jl` (registry)	Decision trees and random forests	Estimation, Nonparametric, IV/RDD
`EvoTrees.jl` (registry)	Gradient-boosted trees as a nuisance learner	Nonparametric
`GLMNet.jl` (registry)	Lasso and elastic net for high-dimensional nuisance models	Estimation

27.3. Panelest.jl

`Panelest.jl` handles the regression models with high-dimensional fixed effects. Its API stays close to the formula style used throughout the book:

```julia
using Panelest, Vcov

feols(df, @formula(y ~ x + fe(id) + fe(year)))
fepois(df, @formula(y ~ x + fe(id) + fe(year)))
feols(df, @formula(y ~ x + fe(id) + fe(year)), vcov = Vcov.cluster(:id))
```

The package exports `feols`, `fepois`, `felogit`, `feprobit`, `feglm`, `clogit`, `cre`, `feiv`, and `fe`. The fixed-effect term `fe(...)` is the central convenience feature. In the DiD chapter, it lets ETWFE specifications stay readable:

```julia
feols(
    mpdta,
    @formula(lemp ~ lpop + first_treat_str & year_str + fe(countyreal) + fe(year)),
    vcov = Vcov.cluster(:countyreal),
)
```

The current package also includes `feiv` for 2SLS-style workflows:

```julia
feiv(df, @formula(y ~ x_exo + fe(id)), endo = :x_endo, inst = :z)
```

27.4. CausalEstimate.jl

`CausalEstimate.jl` is the unified estimation front-end used in the estimation, nonparametric, and applied graph chapters. It combines TMLE and AIPW under one `estimate(...)` call, with cross-fitting and doubly robust inference built in.

```julia
using CausalEstimate, DataFrames

n = 1000
df = DataFrame(
    Y = randn(n),
    A = rand([0, 1], n),
    W = randn(n),
)

# AIPW for ATE
result = estimate(ATE(outcome=:Y, treatment=:A, confounders=[:W]), AIPW(crossfit=5), df)
estimate(result)       # point estimate
confint(result)        # (lower, upper) 95% CI
pvalue(result)         # two-sided p-value

# TMLE for ATE
result_tmle = estimate(ATE(outcome=:Y, treatment=:A, confounders=[:W]), TMLE(crossfit=5), df)

# ATT
result_att = estimate(ATT(outcome=:Y, treatment=:A, confounders=[:W]), AIPW(crossfit=5), df)
```

For graph-identified backdoor (a-fixable) effects, pass a `GraphID` layer:

```
using CausalEstimate, CausalGraphs

g = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)

result = estimate(
    ATE(outcome = :Y, treatment = :A),
    GraphID(graph = g),
    AIPW(crossfit = 5),
    df,
)
```

`GraphID` reads the Markov pillow of the treatment from the graph and uses it as the adjustment set. For p-fixable (front-door), nested-fixable, and general ID-algorithm effects, use `CausalGraphs.estimate_causal(...)` directly.

27.5. CausalGraphs.jl

`CausalGraphs.jl` is the graph layer for the DAG/ADMG chapters. It constructs graphs, checks graph properties, runs `identify()`, and exposes the general Pearl-Shpitser `ID_algorithm()`:

```
using CausalGraphs

g = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)

identify(g, :A, :Y).strategy
```

For non-backdoor effects (p-fixable, nested-fixable, ID plug-in), `CausalGraphs.estimate_causal(...)` provides the full estimation pipeline:

```
res = CausalGraphs.estimate_causal(
    a = [1.0, 0.0],
    data = df,
    graph = g,
    treatment = :A,
    outcome = :Y,
)
```

```
res[:TMLE].ACE          # front-door NPS TMLE
res[:IDPlugin].ACE      # finite-support ID plug-in
```

27.6. ETWFE in Panelest

`Panelest` fits Wooldridge's extended TWFE and aggregates its cell effects:

```
using Panelest, Vcov

model = etwfe(df, @formula(y ~ x); gvar = :first_treat, tvar = :year,
              vcov = Vcov.cluster(:id))
att         = emfx(model)
event_study = emfx(model, type = "event")
calendar    = emfx(model, type = "calendar")
```

`etwfe()` has the same specification as R's `etwfe`: cohort and year effects, one dummy per treatment cell, and Wooldridge's interactions for each control. It takes `family = "poisson"` and `cgroup = "never"`. `emfx()` averages the cell effects over treated observations into an overall ATT, event-time effects or calendar-time effects, on the count scale for Poisson. `dataset("mpdta")` returns a synthetic balanced panel for examples.

27.7. SynthDiD.jl

`SynthDiD.jl` implements the comparison among standard DiD, synthetic control, and synthetic DiD. The package exposes a matrix-oriented workflow:

```
using SynthDiD

df = california_prop99()
setup = panel_matrices(df, :State, :Year, :PacksPerCapita, :treated)

_sdid = synthdid_estimate(setup.Y, setup.N0, setup.T0)
_sc   = sc_estimate(setup.Y, setup.N0, setup.T0)
_did  = did_estimate(setup.Y, setup.N0, setup.T0)
```

The most important functions are `panel_matrices`, `synthdid_estimate`, `sc_estimate`, `did_estimate`, `effect_curve`, `placebo`, `vcov`, `se`, `california_prop99`, and `synthdid_controls`.

27.8. TASC.jl

`TASC.jl` implements Time-Aware Synthetic Control. It keeps the same panel-data spirit as synthetic control, but models the untreated outcome panel with a low-rank linear Gaussian state-space model:

```
using TASC

model = fit_tasc(Y; d = 2, T0 = 19)
pred = predict_counterfactual(model, Y)
```

The package exports `fit_tasc`, `predict_counterfactual`, `predict_post_intervention`, `tasc_plot`, classical synthetic-control baselines, HSVT/RSC-style denoising helpers, and synthetic data generators. It is registered in the environment's `Project.toml` as a developed dependency.

27.9. RDRobust.jl

`RDRobust.jl` is the RD layer used in the IV/RDD chapter. It mirrors the familiar `rdrobust` workflow:

```
using RDRobust

fit = rdrobust(y, x; c = 0.0)
bw = rdbwselect(y, x; c = 0.0)
plot_data = rdplot(y, x; c = 0.0)
```

It supports local-polynomial RD estimation, robust bias-corrected intervals, bandwidth selectors, covariate adjustment, clustered inference, multiple kernels, and fuzzy RD designs. In the book, `rdrobust` is used both for a sharp RD example and for a fuzzy RD comparison with a manual 2SLS specification.

27.10. Lavaan.jl

`Lavaan.jl` ports the R `lavaan` modeling style to Julia. It is used for classical SEM-based mediation:

```
using Lavaan

model = """
  m ~ a * x
  y ~ b * m + c * x
  indirect := a * b
  total := c + indirect
"""
```

```
fit = sem(model, df)
parameterEstimates(fit)
```

The public API includes `lavaan`, `cfa`, `sem`, `growth`, `sam`, `parameterEstimates`, `fitMeasures`, `standardizedSolution`, `lavInspect`, `lavTestLRT`, `modindices`, `rsquare`, `lavPredict`, and `simulateData`. The package supports the familiar lavaan operators `=~`, `~`, `~~`, `~1`, and `:=`.

27.11. Crumble.jl

`Crumble.jl` is a port of the R package `crumble`. It estimates natural (`"N"`), organic (`"O"`), randomized interventional (`"RI"`) and recanting-twin (`"RT"`) effects by cross-fitted one-step estimation, combining sequential outcome regressions with sequentially fitted Riesz representers. The mediation chapter uses `"RI"`, the interventional effects that remain identified when a mediator-outcome confounder is affected by treatment:

```
using Crumble

result = crumble(
    df,
    ["A"];
    outcome = "Y",
    mediators = ["M"],
    moc = ["Z"],
    covar = ["W1", "W2"],
    effect = "RI",
    control = crumble_control(alpha_cap = 100.0),
)

Crumble.tidy(result)
```

Versions before 2026-09-28 did not implement the estimator, and their output should not be used. The current version is validated against the exact truth of the mediation chapter's DGP. `alpha_cap` bounds the fitted representers: under weak overlap a cross-fitting fold can miss a covariate pattern the shift needs, and an unbounded fit then drifts far above the true weight.

27.12. Practical Advice

These packages are best treated as a coherent local research stack. When changing package code, rerun a direct load check from the book directory:

```
julia --project -e 'using CausalEstimate, CausalGraphs, Panelest, SynthDiD, RDRobust, Lavaan, C
```

For content changes, render the affected chapter first. For package API changes, render the full book after deleting the relevant `_freeze` chapter directory so Quarto does not reuse stale cached results.

References

Each chapter lists the works it cites at the bottom of its own page; the complete bibliography lives in `references.bib` in the book repository. Selected entries appear below. The main background references are Imbens and Rubin (2015) and Hernán and Robins (2020).

Abadie, Alberto. 2021. "Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects." *Journal of Economic Literature* 59 (2): 391–425.

Abadie, Alberto, Alexis Diamond, and Jens Hainmueller. 2010. "Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program." *Journal of the American Statistical Association* 105 (490): 493–505.

Angrist, Joshua D., and Jörn-Steffen Pischke. 2009. *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

Blandhol, Christine, John Bonney, Magne Mogstad, and Alexander Torgovitsky. 2025. "When Is TSLS Actually LATE?"

Borusyak, Kirill, and Peter Hull. 2023. "Nonrandom Exposure to Exogenous Shocks." *Econometrica* 91 (6): 2155–85. https://doi.org/10.3982/ECTA19367.

———. 2026. "Optimal Formula Instruments."

Borusyak, Kirill, Peter Hull, and Xavier Jaravel. 2022. "Quasi-Experimental Shift-Share Research Designs." *Review of Economic Studies* 89 (1): 181–213.

Borusyak, Kirill, Xavier Jaravel, and Jann Spiess. 2024. "Revisiting Event Study Designs: Robust and Efficient Estimation." *Review of Economic Studies* 91 (6): 3253–85.

Botosaru, Irene, and Laura Liu. 2025. "Time-Varying Heterogeneous Treatment Effects in Event Studies." *arXiv Preprint arXiv:2509.13698*. https://arxiv.org/abs/2509.13698.

———. 2026. "Event Studies with Feedback." *AEA Papers and Proceedings* 116: 70–74. https://doi.org/10.1257/pandp.20261110.

Brown, Nicholas, and Kyle Butts. 2025. "Dynamic Treatment Effect Estimation with Interactive Fixed Effects and Short Panels." *Journal of Econometrics* 250: 106013.

Callaway, Brantly, Andrew Goodman-Bacon, and Pedro H. C. Sant'Anna. 2024. "Difference-in-Differences with a Continuous Treatment." NBER Working Paper 32117. National Bureau of Economic Research. https://doi.org/10.3386/w32117.

Callaway, Brantly, and Pedro H. C. Sant'Anna. 2021. "Difference-in-Differences with Multiple Time Periods." *Journal of Econometrics* 225 (2): 200–230.

Chamberlain, Gary. 1992. "Comment: Sequential Moment Restrictions in Panel Data." In *Journal of Business & Economic Statistics*, 10:20–26. 1. https://doi.org/10.1080/07350015.1992.10509984.

Chernozhukov, Victor, Mert Demirer, Esther Duflo, and Iván Fernández-Val. 2018. "Generic Machine Learning Inference on Heterogeneous Treatment Effects in Randomized Experiments." *NBER Working Paper 24678*.

Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. https://doi.org/10.1177/1536867X20909691.

Currie, Janet, and Jonathan Gruber. 1996. "Health Insurance Eligibility, Utilization of Medical Care, and Child Health." *Quarterly Journal of Economics* 111 (2): 431–66. https://doi.org/10.2307/2946684.

Díaz, Iván, Nima S. Hejazi, Kara E. Rudolph, and Mark J. van der Laan. 2021. "Nonparametric Efficient Causal Mediation with Intermediate Confounders." *Biometrika* 108 (3): 627–41.

Ferman, Bruno, and Cristine Pinto. 2017. "Placebo Tests for Synthetic Controls." MPRA Paper 78079. Munich Personal RePEc Archive. https://mpra.ub.uni-muenchen.de/78079/1/MPRA_paper_78079.pdf.

Goldsmith-Pinkham, Paul, Isaac Sorkin, and Henry Swift. 2020. "Bartik Instruments: What, When, Why, and How." *American Economic Review* 110 (8): 2586–2624.

Gourieroux, Christian, Alain Monfort, Eric Renault, and Alain Trognon. 1987. "Generalized Residuals." *Journal of Econometrics* 34 (1–2): 5–32.

Guggenberger, Patrik. 2010a. "The Impact of a Hausman Pretest on the Asymptotic Size of a Hypothesis Test." *Econometric Theory* 26 (2): 369–82. https://doi.org/10.1017/S0266466609100014.

———. 2010b. "The Impact of a Hausman Pretest on the Size of a Hypothesis Test: The Panel Data Case." *Journal of Econometrics* 156 (2): 337–43. https://doi.org/10.1016/j.jeconom.2009.11.003.

Hernán, Miguel A. 2010. "The Hazards of Hazard Ratios." *Epidemiology* 21 (1): 13–15.

Hernán, Miguel A., and James M. Robins. 2020. *Causal Inference: What If*. Chapman & Hall/CRC.

Hirano, Keisuke, and Guido W. Imbens. 2004. "The Propensity Score with Continuous Treatments." In *Applied Bayesian Modeling and Causal Inference from Incomplete-Data Perspectives*, 73–84. Wiley.

Imbens, Guido W., and Donald B. Rubin. 2015. *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press.

Imbens, Guido W., and Jeffrey M. Wooldridge. 2007. "What's New in Econometrics? Lecture 6: Control Functions and Related Methods." National Bureau of Economic Research Summer Institute. https://users.nber.org/~confer/2007/si2007/WNE/lect_6_controlfuncs.pdf.

Kennedy, Edward H. 2020. "Towards Optimal Doubly Robust Estimation of Heterogeneous Causal Effects." *arXiv Preprint arXiv:2004.14497*.

Kennedy, Edward H., Zongming Ma, Matthew D. McHugh, and Dylan S. Small. 2017. "Non-Parametric Methods for Doubly Robust Estimation of Continuous Treatment Effects." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 79 (4): 1229–45.

Künzel, Sören R., Jasjeet S. Sekhon, Peter J. Bickel, and Bin Yu. 2019. "Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning." *Proceedings of the National Academy of Sciences* 116 (10): 4156–65.

Lee, Yu-Chin, and Jeffrey M. Wooldridge. 2025. "Distributional Difference-in-Differences via Engression." *Working Paper*.

Lin, Wei, and Jeffrey M. Wooldridge. 2019. "Testing and Correcting for Endogeneity in Nonlinear Unobserved Effects Models." In *Panel Data Econometrics: Theory*, edited by Mike Tsionas, 21–43. Academic Press. https://doi.org/10.1016/B978-0-12-814367-4.00002-2.

Miguel, Edward, and Michael Kremer. 2004. "Worms: Identifying Impacts on Education and Health in the Presence of Treatment Externalities." *Econometrica* 72 (1): 159–217. https://doi.org/10.1111/j.1468-0262.2004.00481.x.

Mullahy, John. 1997. "Instrumental-Variable Estimation of Count Data Models: Applications to Models of Cigarette Smoking Behavior." *Review of Economics and Statistics* 79 (4): 586–93.

Nie, Xinkun, and Stefan Wager. 2021. "Quasi-Oracle Estimation of Heterogeneous Treatment Effects." *Biometrika* 108 (2): 299–319.

Ramsey, J. B. 1969. "Tests for Specification Errors in Classical Linear Least-Squares Regression

Analysis." *Journal of the Royal Statistical Society, Series B* 31 (2): 350–71.

Rho, Saeyoung, Cyrus Illick, Samhitha Narasipura, Alberto Abadie, Daniel Hsu, and Vishal Misra. 2026. "Time-Aware Synthetic Control." *arXiv Preprint arXiv:2601.03099*. https://arxiv.org/abs/2601.03099.

Rubin, Donald B. 1980. "Randomization Analysis of Experimental Data: The Fisher Randomization Test Comment." *Journal of the American Statistical Association* 75 (371): 591–93.

Słoczyński, Tymon. 2022. "Interpreting OLS Estimands When Treatment Effects Are Heterogeneous: Smaller Groups Get Larger Weights." *The Review of Economics and Statistics* 104 (3): 501–9.

———. 2024. "When Should We (Not) Interpret Linear IV Estimands as LATE?" *Quantitative Economics*.

Terza, Joseph V. 1998. "Estimating Count Models with Endogenous Switching: Sample Selection and Endogenous Treatment Effects." *Journal of Econometrics* 84: 129–54.

Vansteelandt, Stijn, and Rhian M. Daniel. 2017. "Interventional Effects for Mediation Analysis with Multiple Mediators." *Epidemiology* 28 (2): 258–65.

Windmeijer, Frank. 2000. "Moment Conditions for Fixed Effects Count Data Models with Endogenous Regressors." *Economics Letters* 68 (1): 21–24. https://doi.org/10.1016/S0165-1765(00)00228-7.

Wooldridge, Jeffrey M. 1999. "Distribution-Free Estimation of Some Nonlinear Panel Data Models." *Journal of Econometrics* 90 (1): 77–97. https://doi.org/10.1016/S0304-4076(98)00033-5.

———. 2010. *Econometric Analysis of Cross Section and Panel Data*. 2nd ed. MIT Press.

———. 2014. "Quasi-Maximum Likelihood Estimation and Testing for Nonlinear Models with Endogenous Explanatory Variables." *Journal of Econometrics* 182 (1): 226–34.

———. 2026. "A Heterogeneous Slopes Approach to Difference-in-Differences with Non-Binary Treatments." SSRN Working Paper 7530698.

Xu, Ruonan. 2023. "Difference-in-Differences with Interference." *arXiv Preprint arXiv:2306.12003*. https://arxiv.org/abs/2306.12003.

———. 2026. "Dynamic Difference-in-Differences with Interference." *AEA Papers and Proceedings* 116: 58–63. https://doi.org/10.1257/pandp.20261108.